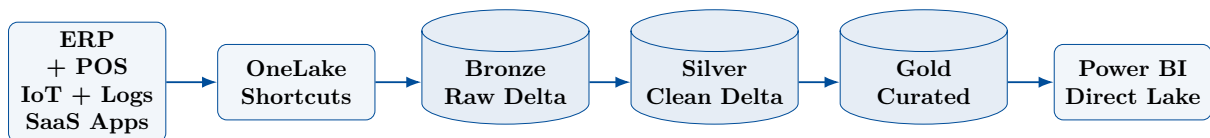

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

Microsoft Fabric Master Data Engineer



OneLake medallion pipeline without unnecessary copies



From OneLake foundations to governed, production-grade analytics products.

PhD Aned Esquerra Arguelles

A 24-week Microsoft Fabric data engineering program

August 31, 2026

Preface

Welcome to *Microsoft Fabric Master Data Engineer*. This book is a rigorous, hands-on academic guide for learning how modern data engineering changes when storage, compute, governance, orchestration, streaming, machine learning, and business intelligence are delivered through the unified Microsoft Fabric software-as-a-service platform.

The goal is not to memorize product screens. The goal is to develop the architecture judgment expected of a master data engineer: when to use OneLake shortcuts instead of copying data, when to model data as Delta tables versus warehouse tables, how to organize medallion layers, how to reason about capacity units, and how to publish governed analytics products that Power BI users can trust.

Who This Book Is For

This book is written for data engineers, analytics engineers, BI developers, cloud architects, database professionals, and technical leaders who need to design and operate data platforms on Microsoft Fabric. It assumes basic SQL literacy and general familiarity with cloud storage, notebooks, and batch pipelines. Python and PySpark are introduced through practical listings, and the exercises gradually move from guided operations to architecture design.

How This Book Is Organized

The program is organized as **6 Parts**, matching the six Fabric milestones of the training plan, and **24 chapters**, matching 24 instructional weeks plus appendices:

- **Part I: Core Data Infrastructure and OneLake** establishes workspaces, capacities, OneLake, lakehouses, Delta-Parquet, and medallion architecture.
- **Part II: Data Warehousing and Analytics** builds SQL warehouses, semantic models, Direct Lake datasets, and performance-aware reporting paths.
- **Part III: Data Engineering and ETL Orchestration** develops Spark notebooks, Dataflows Gen2, Data Factory pipelines, environments, deployment patterns, and production ETL.
- **Part IV: Real-Time Intelligence and Streaming** introduces Eventstreams, KQL databases, real-time dashboards, and operational telemetry scenarios.
- **Part V: Data Science and Machine Learning** applies Fabric notebooks, feature engineering, MLflow, experiments, and model operationalization to data products.
- **Part VI: Governance, Security, and Exam Prep** covers domains, Purview integration, sensitivity, lineage, access control, monitoring, cost management, and certification-style review.

Reading Paths

- **New to Fabric:** read Part I in order, complete every hands-on lab, and draw each architecture before implementing it.
- **Experienced Azure data engineer:** focus on OneLake, shortcuts, Direct Lake, capacity management, and Fabric governance differences from separate Azure services.
- **BI-first professional:** read Chapters 1, 5–8, and 21–24 to connect semantic models and Power BI delivery to governed lakehouse design.
- **Platform architect:** read each Friday use case first, then work backward through the week’s concepts to validate design trade-offs.
- **Exam preparation:** use each chapter’s Key Takeaways, Interview Q&A, and Exercises as spaced-repetition checkpoints.

Conventions

Code identifiers, workspace names, lakehouse paths, and commands appear in `monospace`. PySpark listings use the shared `pystyle` listing style. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations point to product documentation and data-management research in the bibliography. Hands-on steps are written for the Microsoft Fabric web experience and PowerShell-friendly Windows workflows unless a cloud service explicitly requires a browser action.

Contents

0	Getting Started: Your Microsoft Fabric Trial and Workspace	1
0.1	Why Chapter 0 Exists	2
0.2	Prerequisites: Identity Before Infrastructure	3
0.2.1	Free Ways to Obtain an Organizational Account	3
0.2.2	Minimum Access You Need	4
0.3	Starting the Free Fabric Trial	4
0.3.1	What the Trial Includes	4
0.3.2	Trial Capacity Versus F SKUs	5
0.4	Enabling Fabric at the Tenant	5
0.4.1	Delegating to Capacity Administrators	5
0.5	Creating Your Workspace	6
0.5.1	Workspace Roles	6
0.5.2	Creating Your First Lakehouse	6
0.6	Tooling: Browser First, Optional Desktop Tools	7
0.6.1	A First PySpark Read and Delta Write	7
0.6.2	REST and JSON Awareness	8
0.7	Cost and Quota Guardrails	8
0.7.1	Trial Expiry	8
0.7.2	Bursting and Throttling	9
0.7.3	Paid Capacity Later	9
0.8	Verify the Setup	9
0.8.1	Step 1: Create the Workspace and Lakehouse	9
0.8.2	Step 2: Upload a Sample File to OneLake	9
0.8.3	Step 3: Run a One-Cell Notebook	10
0.8.4	Troubleshooting Checklist	11
0.9	How This Setup Differs from Raw IaaS Clouds	11
0.10	Further Reading	11
0.11	Exercises	12
0.12	Curated Community Resources	13
I	Core Data Infrastructure and OneLake	15
1	OneLake and Microsoft Fabric Foundations	16
1.1	Week 1 Roadmap	17
1.2	Monday: Introduction to Microsoft Fabric	18
1.2.1	The Workload Map	18
1.2.2	Capacities, F SKUs, and Capacity Units	19
1.2.3	Workspaces and Domains	20
1.2.4	Monday Architecture Lens	20

1.3	Tuesday: OneLake Concepts	20
1.3.1	Workspaces, Items, Folders, and Paths	21
1.3.2	Delta-Parquet as the Native Open Format	21
1.3.3	Medallion Thinking in OneLake	21
1.3.4	Tuesday Engineering Lens	21
1.4	Wednesday: OneLake Shortcuts	22
1.4.1	Internal and External Shortcuts	22
1.4.2	When Virtualization Beats Copying	22
1.4.3	Risks and Controls	23
1.4.4	Shortcut Decision Matrix	23
1.5	Thursday: Hands-on Project	23
1.5.1	Goal and Architecture	23
1.5.2	Step-by-Step Actions	24
1.5.3	Representing the Shortcut as Configuration	25
1.5.4	Optional SQL Check	25
1.5.5	Validation Checklist	25
1.6	Friday: Use Case and Architecture	26
1.6.1	Architecture Strategy	26
1.6.2	Design Decisions	26
1.7	Operational Guidance for Week 1	27
1.7.1	Governance Checklist	27
1.7.2	Quality and Observability	27
1.7.3	Interview Q&A	28
1.8	Further Reading	28
1.9	Exercises	29
1.10	Capstone Project: A Multi-Cloud Retail Landing Zone with OneLake Shortcuts	29
2	The Medallion Architecture	35
2.1	Week 2 Roadmap	36
2.2	Monday: Medallion Principles	37
2.2.1	Bronze: Raw Landing and Source Fidelity	37
2.2.2	Silver: Validation, Standardization, and Conformance	37
2.2.3	Gold: Enriched, Curated, and Consumption Ready	38
2.2.4	Layer Contracts	38
2.3	Tuesday: Lakehouse vs Data Warehouse in Fabric	38
2.3.1	When to Choose a Lakehouse	39
2.3.2	When to Choose a Warehouse	39
2.3.3	Decision Aid	39
2.4	Wednesday: Organizing Workspaces and Lakehouses at Scale	40
2.4.1	Single Lakehouse Pattern	40
2.4.2	Three Lakehouses in One Workspace	40
2.4.3	Multi-Workspace Pattern	40
2.4.4	Naming, Ownership, and Promotion	40
2.5	Thursday: Hands-on Project	41
2.5.1	Goal and Architecture	41
2.5.2	Step-by-Step Actions	41
2.5.3	Optional Spark SQL Validation	43
2.5.4	Validation Checklist	43
2.6	Friday: Use Case and Architecture	43
2.6.1	Architecture Strategy	44
2.6.2	Design Decisions	44

2.7	Operational Guidance for Week 2	45
2.7.1	Quality and Observability	45
2.7.2	Security Checklist	45
2.8	Interview Q&A	46
2.9	Further Reading	46
2.10	Exercises	47
2.11	Capstone Project: Financial Medallion with Workspace Isolation	47
3	Apache Spark in Fabric	52
3.1	Week 3 Roadmap	53
3.2	Monday: Spark Compute in Fabric	54
3.2.1	Starter Pools	54
3.2.2	Custom Pools	54
3.2.3	Spark Environments	55
3.3	Tuesday: Fabric Notebooks and PySpark Basics	55
3.3.1	The PySpark DataFrame Model	55
3.3.2	Reading Files from OneLake	56
3.3.3	Writing Delta Tables	56
3.4	Wednesday: Spark SQL and Hybrid Notebooks	56
3.4.1	Temporary Views	56
3.4.2	Choosing DataFrame or SQL Syntax	57
3.5	Thursday: Hands-on Project	57
3.5.1	Goal and Architecture	57
3.5.2	Step-by-Step Actions	57
3.5.3	Validation Checklist	59
3.6	Friday: Use Case and Architecture	59
3.6.1	Architecture Strategy	60
3.6.2	Design Decisions	60
3.7	Operational Guidance for Week 3	60
3.7.1	Performance and Reliability	60
3.7.2	Security Checklist	61
3.8	Interview Q&A	61
3.9	Further Reading	61
3.10	Exercises	62
3.11	Capstone Project: Spark-Based Telemetry Ingestion at 100 GB per Day	63
4	Delta Lake Optimization	68
4.1	Week 4 Roadmap	69
4.2	Monday: Delta Lake Internals	70
4.2.1	Parquet Data Files	70
4.2.2	The JSON Transaction Log	70
4.2.3	Why Internals Affect Optimization	71
4.3	Tuesday: Optimizing Delta Tables	71
4.3.1	V-Order in Fabric	71
4.3.2	OPTIMIZE for Compaction	72
4.3.3	Z-Ordering	72
4.4	Wednesday: Managing Delta Tables	72
4.4.1	VACUUM and Retention	72
4.4.2	Time Travel	73
4.4.3	Schema Evolution	73
4.5	Thursday: Hands-on Project	73
4.5.1	Goal and Architecture	73

4.5.2	Step-by-Step Actions	73
4.5.3	Validation Checklist	75
4.6	Friday: Use Case and Architecture	75
4.6.1	Symptoms	75
4.6.2	Remediation Strategy	75
4.6.3	Operational Runbook	76
4.7	Operational Guidance for Week 4	76
4.7.1	Performance and Reliability	76
4.7.2	Security and Governance Checklist	77
4.8	Interview Q&A	77
4.9	Further Reading	77
4.10	Exercises	78
4.11	Milestone 1 Capstone: Spark-Driven Medallion Lakehouse	78
II	Data Warehousing and Analytics	83
5	Synapse Data Warehouse in Fabric	84
5.1	Week 5 Roadmap	85
5.2	Monday: Warehouse Architecture	85
5.2.1	Why Separation Matters	86
5.2.2	The Warehouse and Its Neighbors	86
5.3	Tuesday: T-SQL in Fabric	86
5.3.1	COPY INTO	87
5.3.2	Cross-Database Querying	87
5.4	Wednesday: Ingestion Patterns	87
5.4.1	Idempotent Load Design	88
5.5	Thursday: Hands-on Project	88
5.5.1	Goal and Architecture	88
5.5.2	Step-by-Step Actions	89
5.5.3	Validation Checklist	89
5.6	Friday: Use Case and Architecture	90
5.6.1	Capacity and Cost Reasoning	90
5.6.2	Lakehouse or Warehouse for the Migration	90
5.7	Operational Guidance for Week 5	91
5.7.1	Performance and Reliability	91
5.7.2	Security and Governance Checklist	91
5.8	Interview Q&A	91
5.9	Further Reading	92
5.10	Exercises	92
5.11	Capstone Project: Idempotent Warehouse Load Framework	93
6	SQL Analytics Endpoint	96
6.1	Week 6 Roadmap	97
6.2	Monday: The Automatic Endpoint	97
6.3	Tuesday: Views, TVFs, and Stored Procedures	98
6.3.1	Views as Reusable Joins	98
6.3.2	Table-Valued Functions for Parameterized Access	98
6.3.3	Stored Procedures for Operational Routines	99
6.4	Wednesday: Security in the SQL Endpoint	99
6.4.1	Object-Level Security	99
6.4.2	Row-Level Security	99

6.5	Thursday: Hands-on Project	100
6.5.1	Goal and Architecture	100
6.5.2	Step-by-Step Actions	100
6.5.3	Validation Checklist	101
6.6	Friday: Use Case and Architecture	101
6.6.1	The Pattern	101
6.6.2	Trade-Offs	102
6.7	Operational Guidance for Week 6	102
6.7.1	Performance and Reliability	102
6.7.2	Security and Governance Checklist	102
6.8	Interview Q&A	103
6.9	Further Reading	103
6.10	Exercises	104
6.11	Capstone Project: A Governed SQL Consumption Layer	104
7	Data Modeling for Power BI	106
7.1	Week 7 Roadmap	107
7.2	Monday: Kimball Dimensional Modeling	107
7.2.1	The Three Fact Table Types	107
7.2.2	Conformed Dimensions	108
7.2.3	Slowly Changing Dimensions	108
7.3	Tuesday: The Default Semantic Model	108
7.3.1	When the Default Model Is Enough	109
7.4	Wednesday: Custom Semantic Models and Relationships	109
7.4.1	Relationships	109
7.4.2	DAX Measures	109
7.5	Thursday: Hands-on Project	109
7.5.1	Goal and Architecture	110
7.5.2	Step-by-Step Actions	110
7.5.3	Validation Checklist	111
7.6	Friday: Use Case and Architecture	111
7.6.1	Diagnosing the Cycle	111
7.6.2	The Bridge Table Pattern	111
7.7	Operational Guidance for Week 7	112
7.7.1	Performance and Reliability	112
7.7.2	Security and Governance Checklist	112
7.8	Interview Q&A	112
7.9	Further Reading	113
7.10	Exercises	113
7.11	Capstone Project: Conformed Star Schema with SCD2 History	114
8	Power BI Direct Lake Mode	117
8.1	Week 8 Roadmap	118
8.2	Monday: The Direct Lake Concept	118
8.2.1	When Import Still Wins	119
8.3	Tuesday: Framing and Fallback	119
8.3.1	What Triggers Fallback	119
8.4	Wednesday: Optimizing Models for Direct Lake	120
8.5	Thursday: Hands-on Project	120
8.5.1	Goal and Architecture	120
8.5.2	Step-by-Step Actions	120
8.5.3	Validation Checklist	121

8.6	Friday: Use Case and Architecture	121
8.6.1	The Architecture	121
8.6.2	The Guardrail Conversation	121
8.7	Operational Guidance for Week 8	122
8.7.1	Performance and Reliability	122
8.7.2	Security and Governance Checklist	122
8.8	Interview Q&A	122
8.9	Further Reading	123
8.10	Exercises	123
8.11	Milestone 2 Capstone: End-to-End Analytics Workflow	124
III	Data Engineering and ETL Orchestration	127
9	Data Factory Pipelines	128
9.1	Week 9 Roadmap	129
9.2	Monday: Pipelines, Copy, Lookup, and Get Metadata	129
9.3	Tuesday: Control Flow	130
9.4	Wednesday: Parameters, Variables, and Data Contracts	130
9.4.1	Parameters and Variables	130
9.4.2	Data Contracts	130
9.5	Thursday: Hands-on Project	131
9.5.1	Goal and Architecture	131
9.5.2	Step-by-Step Actions	131
9.5.3	Validation Checklist	132
9.6	Friday: Use Case and Architecture	132
9.6.1	The Framework Anatomy	133
9.7	Operational Guidance for Week 9	133
9.7.1	Performance and Reliability	133
9.7.2	Security and Governance Checklist	134
9.8	Interview Q&A	134
9.9	Further Reading	134
9.10	Exercises	135
9.11	Capstone Project: Config-Driven Multi-Source Ingestion	135
10	Dataflows Gen2	138
10.1	Week 10 Roadmap	139
10.2	Monday: Power Query Online	139
10.3	Tuesday: Output Destinations	140
10.4	Wednesday: Staging, Incremental Refresh, and Data Quality	140
10.4.1	Staging versus Destinations	140
10.4.2	Incremental Refresh	140
10.4.3	Data-Quality Gates	141
10.5	Thursday: Hands-on Project	141
10.5.1	Goal and Architecture	141
10.5.2	Step-by-Step Actions	141
10.5.3	Validation Checklist	142
10.6	Friday: Use Case and Architecture	142
10.6.1	Recommendation	143
10.7	Operational Guidance for Week 10	143
10.7.1	Performance and Reliability	143
10.7.2	Security and Governance Checklist	143

10.8 Interview Q&A	144
10.9 Further Reading	144
10.10 Exercises	145
10.11 Capstone Project: Analyst-Grade API Ingestion with Quality Gates	145
11 Data Integration and Connectivity	148
11.1 Week 11 Roadmap	149
11.2 Monday: On-Premises versus VNet Data Gateways	149
11.3 Tuesday: Connecting Hybrid Sources	150
11.3.1 SQL Server	150
11.3.2 Oracle and SAP	150
11.3.3 Connection Hygiene	150
11.4 Wednesday: REST APIs and Webhooks	150
11.4.1 REST APIs	150
11.4.2 Webhooks	151
11.5 Thursday: Hands-on Project	151
11.5.1 Goal and Architecture	151
11.5.2 Step-by-Step Actions	151
11.5.3 Validation Checklist	152
11.6 Friday: Use Case and Architecture	152
11.6.1 The Pattern	152
11.6.2 Hardening the Pattern	152
11.7 Operational Guidance for Week 11	153
11.7.1 Performance and Reliability	153
11.7.2 Security and Governance Checklist	153
11.8 Interview Q&A	153
11.9 Further Reading	154
11.10 Exercises	154
11.11 Capstone Project: Clustered Hybrid Extraction with Failover Evidence	155
12 Orchestration and Monitoring	157
12.1 Week 12 Roadmap	158
12.2 Monday: Invoking Notebooks and Dataflows from Pipelines	158
12.3 Tuesday: Dependency Management	159
12.4 Wednesday: Monitoring Hub and Alert Routing	159
12.5 Thursday: Hands-on Project	159
12.5.1 Goal and Architecture	160
12.5.2 Step-by-Step Actions	160
12.5.3 Validation Checklist	160
12.6 Friday: Use Case and Architecture	160
12.6.1 The Design	160
12.7 Operational Guidance for Week 12	161
12.7.1 Performance and Reliability	161
12.7.2 Security and Governance Checklist	161
12.8 Interview Q&A	162
12.9 Further Reading	162
12.10 Exercises	163
12.11 Milestone 3 Capstone: Automated Batch ELT Pipeline	163

IV Real-Time Intelligence and Streaming	166
13 Eventstreams and Real-Time Ingestion	167
13.1 Week 13 Roadmap	168
13.2 Monday: Real-Time Intelligence Architecture	168
13.3 Tuesday: Eventstream Sources	169
13.4 Wednesday: Eventstream Destinations	169
13.4.1 Delivery Semantics and Idempotency	170
13.5 Thursday: Hands-on Project	170
13.5.1 Goal and Architecture	170
13.5.2 Step-by-Step Actions	170
13.5.3 Validation Checklist	171
13.6 Friday: Use Case and Architecture	171
13.6.1 The Architecture	171
13.7 Operational Guidance for Week 13	172
13.7.1 Performance and Reliability	172
13.7.2 Security and Governance Checklist	172
13.8 Interview Q&A	172
13.9 Further Reading	173
13.10 Exercises	173
13.11 Capstone Project: Replay-Safe Telemetry Pipeline	174
14 KQL Databases and Eventhouses	176
14.1 Week 14 Roadmap	177
14.2 Monday: Eventhouses and KQL Databases	177
14.3 Tuesday: KQL Basics, Materialized Views, and Dedup	178
14.3.1 The Core Verbs	178
14.3.2 Materialized Views as Incremental State	178
14.3.3 Dedup with <code>arg_max</code>	178
14.4 Wednesday: Time Series, Joins, and Late Arrivals	178
14.4.1 Time-Series Analysis	178
14.4.2 Stream Joins with Time-Windowed Keys	179
14.4.3 Late Arrivals and Window Policy	179
14.5 Thursday: Hands-on Project	179
14.5.1 Goal and Architecture	179
14.5.2 Step-by-Step Actions	179
14.5.3 Validation Checklist	180
14.6 Friday: Use Case and Architecture	180
14.6.1 Diagnosis	180
14.6.2 The Fix: End-to-End Exactly-Once as a Discipline	180
14.7 Operational Guidance for Week 14	181
14.7.1 Performance and Reliability	181
14.7.2 Security and Governance Checklist	181
14.8 Interview Q&A	181
14.9 Further Reading	182
14.10 Exercises	182
14.11 Capstone Project: Windowed Analytics over a Replay-Safe Stream	183
15 Data Activator (Reflex)	185
15.1 Week 15 Roadmap	186
15.2 Monday: Triggers, Conditions, and Actions	186
15.3 Tuesday: Reflex from Power BI and Eventstreams	186

15.3.1	From Power BI Visuals	187
15.3.2	From Eventstreams and KQL	187
15.4	Wednesday: Actions and Custom Workflows	187
15.5	Thursday: Hands-on Project	187
15.5.1	Goal and Architecture	187
15.5.2	Step-by-Step Actions	188
15.5.3	Validation Checklist	188
15.6	Friday: Use Case and Architecture	188
15.6.1	The Design	188
15.7	Operational Guidance for Week 15	189
15.7.1	Performance and Reliability	189
15.7.2	Security and Governance Checklist	189
15.8	Interview Q&A	189
15.9	Further Reading	190
15.10	Exercises	190
15.11	Capstone Project: Fleet Alerting with Noise Discipline	191
16	Real-Time Dashboards	193
16.1	Week 16 Roadmap	194
16.2	Monday: KQL Querysets versus Real-Time Dashboards	194
16.3	Tuesday: Building Dashboards on KQL	194
16.4	Wednesday: Automatic Page Refresh and Reverse ETL	195
16.4.1	Power BI Automatic Page Refresh	195
16.4.2	Reverse ETL	195
16.5	Thursday: Hands-on Project	196
16.5.1	Goal and Architecture	196
16.5.2	Step-by-Step Actions	196
16.5.3	Validation Checklist	196
16.6	Friday: Use Case and Architecture	197
16.6.1	The Design	197
16.7	Operational Guidance for Week 16	197
16.7.1	Performance and Reliability	197
16.7.2	Security and Governance Checklist	197
16.8	Interview Q&A	198
16.9	Further Reading	198
16.10	Exercises	199
16.11	Milestone 4 Capstone: Real-Time Fraud Detection	199
V	Data Science and Machine Learning	202
17	Data Science Foundations in Fabric	203
17.1	Week 17 Roadmap	204
17.2	Monday: The Data Science Experience	204
17.2.1	Data Wrangler	204
17.3	Tuesday: EDA with Pandas and PySpark	205
17.4	Wednesday: Feature Engineering and Leakage	205
17.4.1	The Leakage Taxonomy	205
17.5	Thursday: Hands-on Project	206
17.5.1	Goal and Architecture	206
17.5.2	Step-by-Step Actions	206
17.5.3	Validation Checklist	207

17.6 Friday: Use Case and Architecture	207
17.6.1 The Architecture	207
17.7 Operational Guidance for Week 17	208
17.7.1 Performance and Reliability	208
17.7.2 Security and Governance Checklist	208
17.8 Interview Q&A	208
17.9 Further Reading	209
17.10 Exercises	209
17.11 Capstone Project: Leakage-Safe Feature Store Slice	210
18 Model Training and MLflow	212
18.1 Week 18 Roadmap	213
18.2 Monday: Experiments and Models in Fabric	213
18.3 Tuesday: Tracking with MLflow	213
18.3.1 Autolog	213
18.3.2 Explicit Logging	214
18.4 Wednesday: The Model Registry	214
18.5 Thursday: Hands-on Project	215
18.5.1 Goal and Architecture	215
18.5.2 Step-by-Step Actions	215
18.5.3 Validation Checklist	215
18.6 Friday: Use Case and Architecture	216
18.6.1 The Workflow	216
18.7 Operational Guidance for Week 18	216
18.7.1 Performance and Reliability	216
18.7.2 Security and Governance Checklist	217
18.8 Interview Q&A	217
18.9 Further Reading	217
18.10 Exercises	218
18.11 Capstone Project: Governed Model Search and Registration	218
19 SynapseML and AI Integration	220
19.1 Week 19 Roadmap	221
19.2 Monday: Introduction to SynapseML	221
19.3 Tuesday: Azure OpenAI, Cognitive Services, and Secrets	222
19.3.1 Pre-Built AI in the Pipeline	222
19.3.2 Secret Management	222
19.4 Wednesday: Copilot in Fabric	222
19.5 Thursday: Hands-on Project	222
19.5.1 Goal and Architecture	222
19.5.2 Step-by-Step Actions	223
19.5.3 Validation Checklist	223
19.6 Friday: Use Case and Architecture	224
19.6.1 The Pipeline	224
19.7 Operational Guidance for Week 19	224
19.7.1 Performance and Reliability	224
19.7.2 Security and Governance Checklist	224
19.8 Interview Q&A	225
19.9 Further Reading	225
19.10 Exercises	226
19.11 Capstone Project: Budgeted AI Enrichment Pipeline	226

20 MLOps and Batch Scoring	228
20.1 Week 20 Roadmap	229
20.2 Monday: Batch Scoring with PREDICT	229
20.3 Tuesday: Applying Registered Models to Delta Tables	229
20.4 Wednesday: Automating Inference via Pipelines	230
20.5 Thursday: Hands-on Project	230
20.5.1 Goal and Architecture	231
20.5.2 Step-by-Step Actions	231
20.5.3 Validation Checklist	231
20.6 Friday: Use Case and Architecture	231
20.6.1 Drift Detection	231
20.6.2 The Retraining Trigger	231
20.7 Operational Guidance for Week 20	232
20.7.1 Performance and Reliability	232
20.7.2 Security and Governance Checklist	232
20.8 Interview Q&A	232
20.9 Further Reading	233
20.10 Exercises	233
20.11 Milestone 5 Capstone: End-to-End MLOps Pipeline	234
 VI Governance, Security, and Exam Prep	 237
21 Security and Access Control	238
21.1 Week 21 Roadmap	239
21.2 Monday: Workspace Roles	239
21.3 Tuesday: Item-Level versus Workspace-Level Permissions	239
21.4 Wednesday: OneLake Data Access Roles and SQL RLS	240
21.4.1 OneLake Data Access Roles	240
21.4.2 SQL Row-Level Security	240
21.5 Thursday: Hands-on Project	240
21.5.1 Goal and Architecture	241
21.5.2 Step-by-Step Actions	241
21.5.3 Validation Checklist	241
21.6 Friday: Use Case and Architecture	241
21.6.1 The Perimeter Design	241
21.7 Operational Guidance for Week 21	242
21.7.1 Performance and Reliability	242
21.7.2 Security and Governance Checklist	242
21.8 Interview Q&A	242
21.9 Further Reading	243
21.10 Exercises	243
21.11 Capstone Project: Layered Access Model with Audit Evidence	244
 22 Governance, Purview, and GDPR Erasure	 246
22.1 Week 22 Roadmap	247
22.2 Monday: Purview Hub and Tenant Administration	247
22.3 Tuesday: Sensitivity Labels and Inheritance	248
22.4 Wednesday: Lineage, Endorsement, and the Erasure Path	248
22.4.1 Lineage and Impact Analysis	248
22.4.2 The GDPR Erasure Propagation Path	248
22.4.3 Crypto-Shredding versus Row-Level DELETE	248

22.5 Thursday: Hands-on Project	249
22.5.1 Goal and Architecture	249
22.5.2 Step-by-Step Actions	249
22.5.3 Validation Checklist	250
22.6 Friday: Use Case and Architecture	250
22.6.1 The Mesh Design	250
22.7 Operational Guidance for Week 22	251
22.7.1 Performance and Reliability	251
22.7.2 Security and Governance Checklist	251
22.8 Interview Q&A	252
22.9 Further Reading	252
22.10 Exercises	253
22.11 Capstone Project: Auditable Erasure Pipeline	253
23 Administration and CI/CD	255
23.1 Week 23 Roadmap	256
23.2 Monday: Capacity Management	256
23.2.1 The Metrics App and Throttling	256
23.2.2 Pause, Resume, and Surge Protection	256
23.3 Tuesday: Git Integration	257
23.4 Wednesday: Deployment Pipelines, BCDR, and Zero-Downtime Schema Change	257
23.4.1 Dev → Test → Prod	257
23.4.2 Multi-Region BCDR	257
23.4.3 Zero-Downtime Schema Evolution	257
23.5 Thursday: Hands-on Project	258
23.5.1 Goal and Architecture	258
23.5.2 Step-by-Step Actions	258
23.5.3 Validation Checklist	258
23.6 Friday: Use Case and Architecture	259
23.6.1 The Design	259
23.7 Operational Guidance for Week 23	260
23.7.1 Performance and Reliability	260
23.7.2 Security and Governance Checklist	260
23.8 Interview Q&A	260
23.9 Further Reading	261
23.10 Exercises	261
23.11 Capstone Project: Git-Native Staged Delivery	262
24 DP-600 Certification Preparation	264
24.1 Week 24 Roadmap	265
24.2 Monday: The DP-600 Objectives	265
24.3 Tuesday: Deep Dive—Direct Lake versus Import versus DirectQuery	266
24.4 Wednesday: Mock Exam 1	266
24.5 Thursday: Targeted Remediation	267
24.6 Friday: Mock Exam 2 and Readiness	267
24.7 Operational Guidance for Week 24	267
24.7.1 Performance and Reliability	267
24.7.2 Security and Governance Checklist	268
24.8 Interview Q&A	268
24.9 Further Reading	268
24.10 Exercises	269
24.11 Milestone 6 Capstone: Secure, Governed Data Platform	269

A	Environment Setup and Sample Data	272
A.1	Accounts and Tenant	272
A.2	Local Tooling	272
A.3	Repository and Credential Conventions	273
A.4	Workspace Naming Conventions	273
A.5	Sample Datasets	273
A.6	Troubleshooting Checklist	274
B	CLI and API Quick Reference	275
B.1	Azure CLI Essentials	275
B.2	OneLake Paths and Endpoints	275
B.3	Fabric REST API	276
B.4	T-SQL Quick Reference	276
B.5	KQL Quick Reference	277
B.6	PySpark and Delta Quick Reference	277
B.7	DAX Quick Reference	278
C	Lab Solutions and Acceptance Guidance	279
C.1	Part I: Core Infrastructure (Weeks 1–4)	279
C.1.1	Week 1: Workspace, Lakehouse, and Shortcut	279
C.1.2	Week 2: Medallion Transformations	279
C.1.3	Week 3: Spark Notebook Transformations	279
C.1.4	Week 4: OPTIMIZE, V-Order, and Time Travel	280
C.1.5	Milestone 1 Capstone	280
C.2	Part II: Warehousing and BI (Weeks 5–8)	280
C.2.1	Week 5: COPY INTO Five Million Rows	280
C.2.2	Week 6: SSMS and the Silver-to-Gold View	280
C.2.3	Week 7: Relationship, Measure, and SCD2 Merge	280
C.2.4	Week 8: Direct Lake versus DirectQuery Benchmark	280
C.2.5	Milestone 2 Capstone	280
C.3	Part III: ETL Orchestration (Weeks 9–12)	281
C.3.1	Week 9: Metadata-Driven Pipeline	281
C.3.2	Week 10: Dataflow Gen2 with Quality Gate	281
C.3.3	Week 11: Gateway Extraction	281
C.3.4	Week 12: Master Orchestration Pipeline	281
C.3.5	Milestone 3 Capstone	281
C.4	Part IV: Real-Time Intelligence (Weeks 13–16)	281
C.4.1	Week 13: Eventstream and Producer	281
C.4.2	Week 14: Rolling Average and Anomalies	281
C.4.3	Week 15: Sustained-Threshold Alert	281
C.4.4	Week 16: Dashboard and Reverse ETL	282
C.4.5	Milestone 4 Capstone	282
C.5	Part V: Data Science and ML (Weeks 17–20)	282
C.5.1	Week 17: Data Wrangler Export	282
C.5.2	Week 18: Autologged Random Forest	282
C.5.3	Week 19: SynapseML Sentiment	282
C.5.4	Week 20: Batch Scoring at 100k	282
C.5.5	Milestone 5 Capstone	282
C.6	Part VI: Governance and Exam Prep (Weeks 21–24)	282
C.6.1	Week 21: Roles and RLS	282
C.6.2	Week 22: Labels, Lineage, and Crypto-Shredding	283
C.6.3	Week 23: Git and Deployment Pipeline	283

C.6.4	Week 24: Mock Exams and Remediation	283
C.6.5	Milestone 6 Capstone	283
D	Production-Readiness and Governance Checklists	284
D.1	OneLake and Lakehouse	284
D.2	Warehouse	285
D.3	Pipelines and Dataflows	285
D.4	Real-Time Intelligence	285
D.5	Power BI and Semantic Models	286
D.6	Machine Learning	286
D.7	Purview, Security, and Compliance	287
E	DP-600 Exam Mapping and Study Guide	289
E.1	Exam Profile	289
E.2	Domain-to-Chapter Map	289
E.3	Objective-Level Mapping	290
E.3.1	Plan, Implement, and Manage an Analytics Solution	290
E.3.2	Prepare and Serve Data	290
E.3.3	Implement and Manage Semantic Models	290
E.3.4	Real-Time Intelligence	291
E.3.5	Data Science and ML Touchpoints	291
E.4	Study Strategy	291
E.5	Self-Assessment Checklist	291
F	Interview Question Bank	293
F.1	Core Infrastructure: OneLake, Medallion, Spark, Delta (Chapters 1–4)	293
F.2	Warehousing and BI: Warehouse, SQL Endpoint, Modeling, Direct Lake (Chapters 5–8)	294
F.3	ETL and Orchestration: Pipelines, Dataflows, Connectivity, Monitoring (Chapters 9–12)	295
F.4	Real-Time Intelligence: Eventstreams, KQL, Reflex, Dashboards (Chapters 13–16)	297
F.5	Data Science and ML: Foundations, MLflow, SynapseML, MLOps (Chapters 17–20)	298
F.6	Governance, Security, Administration, and Certification (Chapters 21–24)	299
G	Cross-Cloud Service Equivalence	301
G.1	Object and Data-Lake Storage	301
G.2	Managed Relational Databases	302
G.3	Elastic Compute and Analytical Warehouses	302
G.4	Data Organization and the Medallion Pattern	302
G.5	Document Data Modeling	303
G.6	Globally Distributed and NoSQL Databases	303
G.7	Managed Apache Spark	304
G.8	Query Acceleration and Materialized Views	304
G.9	In-Memory Caching and Hybrid Ingestion	304
G.10	Wide-Column and Key-Value NoSQL	305
G.11	Data Protection: Cloning, Time Travel, and Migration	305
G.12	Cloud Data Warehouses	305
G.13	SQL over the Open Lake	306
G.14	Dimensional Modeling and the Semantic Layer	306
G.15	BI Serving Modes: Import, Live Query, and Direct Lake	306
G.16	Visual Orchestration and Bulk Ingestion	307
G.17	Low-Code Transformation	307

G.18 Hybrid Connectivity	308
G.19 Orchestration Discipline and Monitoring	308
G.20 Streaming Ingestion	308
G.21 Time-Series and Log Analytics Engines	309
G.22 Data-Driven Alerting	309
G.23 Live Dashboards and Reverse ETL	310
G.24 Data Science Environments	310
G.25 Experiment Tracking and Model Registries	311
G.26 AI Enrichment at Scale	311
G.27 The MLOps Loop	311
G.28 Layered Access Control	312
G.29 Metadata Governance and Erasure	312
G.30 CI/CD, Capacity, and Disaster Recovery	313
G.31 Certification Landscape	313
G.32 The Databricks Column: A Consolidated Map	314
Glossary	315

List of Figures

1	Chapter 0 setup flow from organizational identity to a verified Fabric lakehouse.	3
1.1	A OneLake-centered medallion pattern for Microsoft Fabric.	21
1.2	Multi-cloud retail virtualization through OneLake shortcuts and medallion curation.	26
1.3	Capstone architecture for a multi-cloud retail landing zone using OneLake shortcuts.	31
2.1	Bronze-to-Silver-to-Gold flow for a Fabric medallion architecture.	38
2.2	Multi-workspace isolation pattern for financial medallion architecture.	44
2.3	Reference architecture for the financial medallion capstone.	48
3.1	Spark driver and executors on a Fabric pool reading OneLake and writing Delta.	55
3.2	Spark notebook ingestion pipeline for 100 GB per day of telemetry JSON.	60
3.3	Reference architecture for the Spark telemetry ingestion capstone.	64
4.1	Delta table internals: Parquet data files plus a transaction log.	71
4.2	Small-file remediation through compaction and optimized layout.	76
4.3	Milestone 1 reference architecture from Bronze shortcuts to V-Order optimized Gold tables.	79
5.1	Week 5 capstone: manifest-driven warehouse load framework.	94
6.1	Logical data warehouse: files to governed SQL objects without duplication.	102
7.1	Resolving a circular dependency with a bridge table.	111
7.2	Week 7 capstone: conformed star schema feeding one semantic model.	115
8.1	Milestone 2 reference architecture: warehouse-to-Direct Lake serving layer.	125
9.1	Metadata-driven ingestion framework.	133
11.1	Bank-grade hybrid pattern: outbound-only gateway cluster.	152
12.1	Self-healing reconciliation orchestration across 15 systems.	161
12.2	Milestone 3 reference architecture: orchestrated batch ELT.	164
15.1	Logistics alerting: telemetry to driver notification in seconds.	189
16.1	Milestone 4 reference architecture: real-time fraud detection.	200
17.1	Spark-native inventory forecasting for 1,000 stores.	207
19.1	Terabyte-scale sentiment enrichment feeding BI and alerting.	224

20.1	The self-aware MLOps lifecycle.	232
20.2	Milestone 5 reference architecture: the MLflow-driven MLOps lifecycle.	235
22.1	Five-domain data mesh with centralized Purview governance.	251
23.1	Multi-team CI/CD over one shared platform.	259
24.1	Milestone 6 reference architecture: the governed CI/CD platform.	270

List of Tables

1	Common Fabric workspace roles for a learning environment.	6
2	Common Chapter 0 setup symptoms and first checks.	11
1.1	Major Fabric workloads and their relationship to the shared analytics platform. .	19
1.2	Shortcut types and architecture questions.	22
1.3	Shortcut versus materialization decision matrix.	23
1.4	Retail architecture decisions for shortcut-first modernization.	27
2.1	Layer contracts in a medallion design.	38
2.2	Lakehouse-versus-Warehouse decision aid for Fabric medallion design.	39
2.3	Workspace and lakehouse organization patterns for medallion scale.	41
2.4	Financial institution decisions for strict medallion isolation.	44
3.1	Compute choices for Spark development in Fabric.	55
3.2	Choosing PySpark or Spark SQL inside Fabric notebooks.	57
3.3	Design decisions for a 100 GB/day telemetry ingestion layer.	60
4.1	Optimization choices for Delta tables in Fabric.	72
4.2	Diagnosing and remediating a degraded streaming Delta table.	76
5.1	The warehouse versus its read-only lakehouse sibling.	86
5.2	The three warehouse ingestion patterns and when each earns its place.	88
5.3	Fabric licensing shapes for warehouse workloads.	90
6.1	The endpoint complements the warehouse; it does not replace it.	98
7.1	Choosing between SCD Type 1 and Type 2.	108
8.1	The three Power BI storage modes compared.	119
8.2	Benchmark evidence template: fill with your measured values.	121
9.1	Control-flow activities and their canonical uses.	130
9.2	Classifying schema changes against a data contract.	131
10.1	Dataflow Gen2 output destinations.	140
10.2	Three transformation tools for an Excel-migrating analyst team.	143
11.1	Choosing between the two gateway types.	149
12.1	Dependency conditions and their canonical uses.	159
13.1	Eventstream source types.	169
13.2	Global telemetry architecture decisions.	172

16.1	Querysets explore; dashboards operate.	194
16.2	Manufacturing-floor dashboard design, mapped to Part IV disciplines.	197
17.1	Sample-first EDA: which tool answers which question.	205
18.1	Collaborative tracking conventions.	216
21.1	Workspace roles and their intended holders.	239
22.1	Two erasure strategies, honestly compared.	249
24.1	DP-600 domains mapped to chapters. Verify current weights against the live skills outline.	265
24.2	The storage-mode decision drill.	266
24.3	The program's certification-readiness standard.	267
A.1	Standard datasets and where they are used.	273
E.1	DP-600 domains mapped to chapters (from Chapter 24). Verify current weights against the live skills outline [33].	290

Getting Started: Your Microsoft Fabric Trial and Workspace

Learning Objectives

- Explain why Microsoft Fabric requires an organizational Microsoft 365 or Power BI identity rather than a personal consumer account.
- Start a free Fabric trial capacity and distinguish the trial from paid Fabric F SKUs and capacity units.
- Verify the tenant-level Fabric switch that allows users to create Fabric items, and identify who can enable it.
- Create a workspace, assign it to the Trial capacity, and apply the appropriate workspace role for each collaborator.
- Create a first lakehouse in OneLake and understand how lakehouse **Files** and **Tables** become the foundation for later chapters.
- Use the browser-based Fabric experience, optional Windows tooling, and Fabric notebooks to read and write a Delta table.
- Establish cost, quota, and validation guardrails before beginning Chapter 1.

Cross-Cloud Equivalence

The first practical difference between Microsoft Fabric and raw infrastructure clouds appears before you create any data: Fabric starts from an organization tenant and a SaaS analytics capacity, not from a general-purpose cloud subscription.

Setup step	AWS	Azure	GCP
Free offer	Free Tier (12 mo + always free)	Free account (\$200/30 days + always free)	Free trial (\$300/90 days + Always Free)
Sign-up guard	Card + root email	Card + Microsoft account	Card + Google account
Identity model	IAM / IAM Identity Center	Microsoft Entra ID + RBAC	Cloud IAM
Admin isolation	Never use root daily	Avoid Global Admin daily	Avoid Owner; least privilege
CLI	AWS CLI v2 (aws configure)	Azure CLI (az login)	gcloud CLI (gcloud init)
Browser shell	AWS CloudShell	Azure Cloud Shell	Cloud Shell
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts
Console	AWS Management Console	Azure Portal	Google Cloud Console

THIS book’s platform, Microsoft Fabric, uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card, org tenant) rather than an IaaS subscription; Snowflake uses a 30-day/\$400 trial (no card) guarded by Resource Monitors; MongoDB Atlas offers a forever-free M0 cluster (no card) secured by database users and an IP allowlist.

Key Terms

Organizational account: a Microsoft Entra ID-backed work or school account, normally written as `name@organization.com`. **Tenant:** the Microsoft 365 and Entra ID boundary that owns users, groups, admin settings, and Fabric’s OneLake namespace. **Fabric trial capacity:** a temporary Fabric capacity that lets an eligible user evaluate Fabric workloads. **Capacity unit:** a normalized measure of Fabric compute used across Spark, SQL, pipelines, real-time workloads, and Power BI. **Workspace:** the collaboration, permission, and capacity-assignment container for Fabric items. **Lakehouse:** a Fabric data engineering item that exposes OneLake files, managed Delta tables, Spark, and SQL-oriented access.

0.1 Why Chapter 0 Exists

Chapter 1 begins with OneLake architecture. Before that architecture is useful, you need a working Fabric tenant, an enabled Fabric experience, an active capacity, a workspace assigned to that capacity, and a lakehouse where notebooks can run. This chapter builds that foundation deliberately in a Windows and PowerShell learning environment, while keeping the important distinction clear: Microsoft Fabric is a SaaS analytics platform, not an IaaS cloud account where you provision virtual networks, storage accounts, virtual machines, and Spark clusters by hand.

Fabric unifies data engineering, data integration, data warehousing, real-time analytics, data science, and Power BI in one browser-based experience [59]. The platform hides many infrastructure chores, but it does not remove setup decisions. Identity, tenant settings, capacity assignment, workspace roles, cost controls, and verification still matter. A learner who skips these details often reaches Chapter 1 with a confusing symptom: the **New** button is missing, a lakehouse cannot be created, a notebook will not attach, or a workspace is quietly using the wrong capacity.

Note

This chapter assumes you are using Windows with PowerShell for local commands and a browser for Fabric. Fabric itself runs in the Microsoft cloud; the local machine is mostly a client for documentation, sample files, optional desktop tools, and book builds.

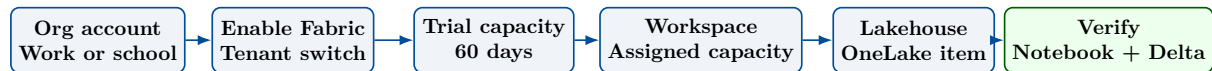


Figure 1: Chapter 0 setup flow from organizational identity to a verified Fabric lakehouse.

0.2 Prerequisites: Identity Before Infrastructure

The first prerequisite is not a credit card. It is an organizational identity. Fabric is attached to a Microsoft 365 and Power BI tenant backed by Microsoft Entra ID. The account must be a work or school account, such as `student@contoso.onmicrosoft.com` or `analyst@yourcompany.com`. That identity places the user inside a tenant where administrators can control Fabric settings, create or allow capacities, manage licenses, configure governance, and own OneLake.

A personal Microsoft account, such as a consumer Outlook.com account, is not enough to enable Fabric by itself. A Gmail address used as a Microsoft account has the same limitation. Those accounts can sign in to consumer Microsoft services, but they do not create the organizational tenant boundary that Fabric needs for tenant settings, workspace governance, Power BI administration, and OneLake’s tenant-wide namespace. In practical terms, a personal account has no Fabric tenant administrator who can turn on Fabric item creation, no organizational users or groups to assign workspace roles to, and no tenant-level OneLake home for the analytics estate.

Definition

An **organizational account** is an account managed by Microsoft Entra ID in a work, school, developer, or Power BI tenant. Fabric depends on that tenant because the platform’s security, administration, workspaces, capacities, and OneLake namespace are tenant-scoped.

0.2.1 Free Ways to Obtain an Organizational Account

For training, the most common path is a Microsoft 365 Developer Program tenant. If you qualify, this gives you a developer Microsoft 365 tenant with administrator control for learning and app development. The exact availability and renewal rules can change, so treat it as a learning tenant rather than a permanent production environment. After it exists, you sign in with an account such as `admin@yourdevtenant.onmicrosoft.com`, open Fabric, and evaluate whether the Fabric trial option is available.

A second path is a Power BI or Fabric free sign-up using a work email address. This path is appropriate when your organization allows self-service sign-up and the email domain can be associated with an organizational tenant. It is less suitable when a company blocks self-service trials or centrally manages Power BI. In that case, ask the Microsoft 365, Power BI, or Fabric administrator to enable the trial or provide a training workspace.

Common Pitfall

Trying to start Fabric from a personal account. If you sign in with a consumer Outlook.com, Hotmail, Live.com, or Gmail-backed Microsoft account, you may reach Microsoft pages but still be unable to create Fabric items. Switch to an organizational work or school account before troubleshooting capacities or workspaces.

0.2.2 Minimum Access You Need

For the hands-on work in this book, you need enough permission to do five things:

1. Sign in to <https://app.fabric.microsoft.com> with an organizational account.
2. Use or start a Fabric trial capacity, or have a workspace assigned to an existing Fabric capacity.
3. Create a workspace, or be added to a workspace as Admin, Member, or Contributor.
4. Create Fabric items in that workspace, including a lakehouse and notebook.
5. Upload or create a small sample file in the lakehouse and run a notebook attached to it.

If you are in a managed enterprise tenant, you may not personally control all five. That is normal. The setup is still valid if an administrator creates the capacity and workspace for you, then adds you as a workspace Member or Contributor.

0.3 Starting the Free Fabric Trial

Open a browser and go to <https://app.fabric.microsoft.com>. Sign in with your organizational account. If your tenant allows trials and your account is eligible, the Fabric interface provides an option to start a trial. Microsoft documents Fabric trial behavior separately from paid capacity licensing [37]. The trial is commonly described as a 60-day evaluation capacity that lets you create and run Fabric items without first purchasing a paid F SKU.

The trial does not mean Fabric has become a personal local tool. It creates or assigns a trial capacity in the tenant. Workspaces must use a Fabric capacity to run Fabric workloads such as lakehouses, notebooks, Data Factory pipelines, warehouses, and other Fabric items. The trial capacity is the temporary compute pool you will attach to your learning workspace.

0.3.1 What the Trial Includes

The trial capacity is meant for evaluation and hands-on learning. It enables Fabric experiences that otherwise require capacity. In this book, that means you can create a workspace, assign it to the Trial capacity, create a lakehouse, run notebooks, write Delta tables, and inspect OneLake storage behavior. The trial is shared through the tenant model: administrators and capacity settings can determine who can use it and which workspaces are assigned.

A trial is not a production capacity. It expires, can be disabled by tenant policy, and may have limits that are inappropriate for real workloads. Use it to learn concepts, validate book exercises, and understand how jobs consume capacity. Do not treat it as a durable home for business data.

Definition

A **Fabric capacity** is a pool of compute resources used by Fabric workloads. Paid capacities are sold as F SKUs with different amounts of compute. Capacity units, or CUs, are the

normalized way to reason about consumption across different Fabric engines [35].

0.3.2 Trial Capacity Versus F SKUs

Paid Fabric capacities are normally referred to by F SKU names, such as smaller and larger F capacities. A larger SKU provides more capacity units and can sustain more concurrent or heavier work. A trial capacity gives learners a temporary capacity so they can experience Fabric without committing to a paid SKU first. The engineering model is the same: your workspace is assigned to a capacity, workloads consume capacity, and overloaded capacity can affect responsiveness.

The practical takeaway is simple. You do not buy, size, or pause the trial the way you would a paid Azure Fabric capacity, but you should still think like an engineer. A poorly designed notebook, runaway loop, accidental full scan, or repeated refresh can consume capacity and make the trial feel slow. The habits you learn on the trial carry forward to paid F SKUs.

0.4 Enabling Fabric at the Tenant

Starting a trial and creating items depends on tenant settings. In the Fabric or Power BI Admin portal, administrators can control whether users can create Fabric items. The setting is often described as **Users can create Fabric items** under Fabric tenant settings [36]. If the switch is off for your whole organization or for your security group, the user interface may hide Fabric creation commands or show errors when you try to create a lakehouse, notebook, warehouse, or other Fabric item.

Who can enable it? In a managed tenant, a Power BI administrator, Fabric administrator, or Microsoft 365 Global Administrator usually controls the relevant Admin portal setting. A Global Administrator should not be used for daily analytics work, but that role may be needed to assign the first Fabric or Power BI admin roles. After the tenant setting is enabled, day-to-day management can be delegated through Fabric administrators, capacity administrators, workspace admins, and security groups.

Common Pitfall

The tenant switch is off. If you can sign in but cannot create a lakehouse, do not immediately blame your browser, license, or notebook. Check whether the Admin portal setting **Users can create Fabric items** is enabled for your user or group. In many enterprise tenants, this is the first blocked setup step.

0.4.1 Delegating to Capacity Administrators

Tenant administrators should not personally manage every workspace. A cleaner model is to enable Fabric for approved security groups, create or allow capacities, then delegate capacity administration to the platform or data engineering team. Capacity admins can manage capacity assignment and monitor usage without needing full tenant administrator privileges. Workspace admins then decide who can collaborate inside each workspace.

This separation matters for least privilege. A training instructor may be a capacity admin for a trial or paid learning capacity. Students may be workspace Contributors. A platform owner may be a Fabric admin. A business analyst may be a Viewer on a curated workspace. These roles are intentionally different.

0.5 Creating Your Workspace

After the tenant setting and capacity are available, create the workspace that will hold the book exercises. In the Fabric portal, select **Workspaces**, choose **New workspace**, and use a name that makes the environment clear. For this book, a useful personal training name is `fabric-book-dev`. In a classroom, include a short cohort or lab identifier, such as `fabric-de-july-lab01`. Avoid names that imply production unless the workspace is truly governed as production.

During or after workspace creation, assign the workspace to the Trial capacity. In the workspace settings, find the license or capacity section and choose the Trial capacity. If you do not see the trial, it may not have started, it may belong to another user, your tenant may restrict capacity assignment, or you may lack workspace admin permission. Microsoft describes workspaces as collaboration containers for Fabric items and permissions [61].

0.5.1 Workspace Roles

Workspace roles determine what collaborators can do. Use the smallest role that supports the learning task.

Role	Typical permissions	Training use
Admin	Manage workspace settings, access, and items	Instructor, lab owner, or learner in a personal tenant
Member	Create and manage content, share many items, collaborate broadly	Senior learner or team lead in a shared workspace
Contributor	Create and edit content but with less administrative control	Default role for hands-on builders in a controlled lab
Viewer	Read and interact with published content without editing it	Reviewers, stakeholders, and learners observing demos

Table 1: Common Fabric workspace roles for a learning environment.

Tip

For book exercises, use one dedicated development workspace. Do not mix training lakehouses with enterprise reports, certified semantic models, or production pipelines. Separation makes cleanup and troubleshooting much safer.

0.5.2 Creating Your First Lakehouse

Inside the workspace, select **New item** and create a **Lakehouse**. Name it `lh_getting_started`. Fabric creates a lakehouse item with a file area and a table area. The file area is useful for raw uploads and folders. The table area contains managed Delta tables that Fabric can expose to Spark and SQL experiences.

This lakehouse lives in OneLake. OneLake is the tenant-wide logical store for Fabric [60]. You are not choosing a storage account, container, region-specific endpoint, or access key in the way you would with raw object storage. Fabric projects the lakehouse into OneLake under the tenant and workspace context. This is why the organizational account and tenant setup mattered at the beginning of the chapter.

Note

OneLake is tenant-wide, but access is not automatically tenant-wide. Users still need permission through workspaces, items, sharing, data access roles, semantic models, or other Fabric governance controls. The namespace is unified; authorization remains explicit.

0.6 Tooling: Browser First, Optional Desktop Tools

The primary tool for this chapter is the Fabric web experience. You do not need to install Spark, Hadoop, Python, Java, the Azure CLI, or a local database to create the first lakehouse. The browser provides the workspace, lakehouse explorer, notebook editor, Spark session attachment, and item management surface. This is one of the most important differences between Fabric and a self-managed data platform.

Optional tools can improve the experience on Windows:

- **Power BI Desktop** is useful later when you build semantic models and reports locally before publishing. It is not required for creating a lakehouse.
- **OneLake File Explorer for Windows** can show OneLake in File Explorer so authorized users can browse and copy files through a familiar interface. Use it carefully; it is convenient, but it does not replace data engineering discipline.
- **Fabric notebooks** run inside Fabric and support PySpark for reading files, transforming data, and writing Delta tables. This book uses notebooks for many data engineering labs.
- **Fabric REST APIs and command-line automation** are useful for repeatable administration and deployment where your organization supports them. For Chapter 0, the browser is enough; automation becomes more relevant after you understand the manual path.

0.6.1 A First PySpark Read and Delta Write

After creating `lh_getting_started`, create a new notebook in the same workspace and attach the lakehouse. The following cell creates a tiny in-memory DataFrame, writes it as a managed Delta table in the lakehouse, then reads it back. It intentionally avoids external files so that Spark itself is the only moving part.

```

1 from pyspark.sql import functions as F
2
3 rows = [
4     (1, "bronze", "raw file landing"),
5     (2, "silver", "clean conformed tables"),
6     (3, "gold", "business-ready metrics"),
7 ]
8
9 layers_df = spark.createDataFrame(rows, ["layer_id", "layer_name", "purpose"])
10
11 (
12     layers_df
13     .withColumn("loaded_at_utc", F.current_timestamp())
14     .write
15     .format("delta")
16     .mode("overwrite")
17     .saveAsTable("setup_layers")
18 )
19

```

```
20 spark.read.table("setup_layers").orderBy("layer_id").show(truncate=False)
```

Listing 1: PySpark one-cell smoke test for a Fabric lakehouse.

The expected result is a three-row table containing **bronze**, **silver**, and **gold**, plus a timestamp column. If this works, the notebook can start Spark, attach to the lakehouse, write Delta, register a table, and read the table back.

0.6.2 REST and JSON Awareness

You do not need REST automation for the first lab, but it is useful to recognize that Fabric items and administrative operations can be represented as structured requests. The exact endpoint and API version may change, so the next listing is a conceptual JSON body for documenting a workspace setup request rather than a command to paste blindly.

```
1 {
2   "workspaceName": "fabric-book-dev",
3   "capacity": {
4     "assignment": "Trial",
5     "purpose": "Microsoft Fabric book exercises"
6   },
7   "items": [
8     {
9       "type": "Lakehouse",
10      "displayName": "lh_getting_started"
11    },
12    {
13      "type": "Notebook",
14      "displayName": "nb_setup_smoke_test",
15      "defaultLakehouse": "lh_getting_started"
16    }
17  ],
18  "roles": [
19    {
20      "principal": "learner group",
21      "role": "Contributor"
22    }
23  ]
24 }
```

Listing 2: Conceptual REST/JSON setup record for a Fabric training workspace.

0.7 Cost and Quota Guardrails

Fabric's trial removes the initial purchase step, but it does not remove finite capacity. A capacity can become busy, throttled, or temporarily burst depending on workload shape and product behavior. Interactive report usage, Spark notebooks, warehouse queries, Data Factory activities, semantic model refreshes, and real-time workloads all consume from the same capacity model. The lesson for a data engineer is to measure consumption early, even when the direct bill is zero.

0.7.1 Trial Expiry

A trial is temporary. Before relying on a workspace for more than a lab, record when the trial started, who owns it, and what should happen when it expires. If the trial ends and the workspace is not reassigned to a paid capacity, Fabric item creation and execution may stop working as expected. Export notebooks or keep source-controlled copies of important code before the trial ends.

0.7.2 Bursting and Throttling

Fabric capacities can smooth bursts, but sustained overuse eventually produces delays or throttling. A single learner can trigger this with repeated full-table writes, an accidental Cartesian join, or too many notebook reruns. In shared classes, twenty learners running the same Spark cell at once can expose capacity limits quickly. That is not a failure of Fabric; it is an operating reality of shared compute.

0.7.3 Paid Capacity Later

If you later move to paid Fabric capacity, you can scale, pause, resume, or delete capacity depending on the purchasing model and administrative controls. Pausing a paid capacity is a cost guardrail, but it also stops workloads assigned to it. Plan environment separation so development capacities can be paused safely while production capacities remain governed and monitored.

Tip

Measure it. Install or ask for access to the Microsoft Fabric Capacity Metrics app when you move beyond a solo trial. Use it to identify which workspace, item, user, or workload is consuming capacity before increasing the SKU or blaming the platform [34].

0.8 Verify the Setup

A setup is complete only when it produces observable evidence. The verification path below proves four things: the workspace exists, the lakehouse exists, OneLake accepts a sample file, and Spark can read and write data.

0.8.1 Step 1: Create the Workspace and Lakehouse

Confirm that the workspace appears in the Fabric portal and is assigned to the Trial capacity. Confirm that `lh_getting_started` appears as a lakehouse item in that workspace. Open the lakehouse and verify that the explorer shows **Files** and **Tables**.

0.8.2 Step 2: Upload a Sample File to OneLake

Create a small CSV file on Windows using PowerShell, then upload it through the lakehouse explorer to **Files/setup**. The command below creates the local sample file in the current directory. It is local preparation only; the upload happens in the browser unless you choose an approved automation method later.

```
1 @'
2 order_id,customer_id,order_date,sales_amount
3 1001,C001,2026-07-01,42.50
4 1002,C002,2026-07-02,99.00
5 1003,C001,2026-07-03,18.25
6 '@ | Set-Content -Path .\fabric_setup_orders.csv -Encoding utf8
```

Listing 3: PowerShell command to create a small CSV sample file on Windows.

In the Fabric lakehouse, choose **Files**, create a folder named `setup`, and upload `fabric_setup_orders.csv`. The path should appear as `Files/setup/fabric_setup_orders.csv`. If you use OneLake File Explorer instead, copy the file only into the intended lakehouse folder and wait for the Fabric interface to refresh.

0.8.3 Step 3: Run a One-Cell Notebook

Create or open `nb_setup_smoke_test`, attach `lh_getting_started`, and run the following cell.

```

1 from pyspark.sql import functions as F
2
3 source_path = "Files/setup/fabric_setup_orders.csv"
4
5 orders_df = (
6     spark.read
7     .option("header", True)
8     .option("inferSchema", True)
9     .csv(source_path)
10 )
11
12 verified_df = (
13     orders_df
14     .withColumn("order_date", F.to_date("order_date"))
15     .withColumn("sales_amount", F.col("sales_amount").cast("decimal(18,2)"))
16 )
17
18 verified_df.write.format("delta").mode("overwrite").saveAsTable("setup_orders")
19
20 result_df = spark.sql("""
21 SELECT
22     COUNT(*) AS row_count,
23     COUNT(DISTINCT customer_id) AS customer_count,
24     CAST(SUM(sales_amount) AS DECIMAL(18,2)) AS total_sales_amount
25 FROM setup_orders
26 """)
27
28 result_df.show(truncate=False)

```

Listing 4: PySpark verification cell that reads an uploaded CSV and writes a Delta table.

Expected result:

```

1 +-----+-----+-----+
2 |row_count|customer_count|total_sales_amount|
3 +-----+-----+-----+
4 |3        |2            |159.75            |
5 +-----+-----+-----+

```

Listing 5: Expected verification output.

If the output matches, Chapter 0 is successful. Spark read a OneLake file, converted types, wrote a managed Delta table, registered it in the lakehouse, and queried it back.

0.8.4 Troubleshooting Checklist

Symptom	Likely cause	Action
Cannot create Fabric item	Tenant setting is disabled or restricted	Ask Fabric or Power BI admin to enable Users can create Fabric items for your group
No Trial capacity option	Trial not started, blocked, expired, or unavailable to user	Start trial if eligible or ask admin for capacity assignment
Notebook will not attach	Lakehouse missing, capacity unavailable, or workspace not assigned	Recheck workspace capacity and default lakehouse
CSV path not found	File uploaded to wrong folder or name differs	Confirm exact path under Files/setup
Delta write fails	Spark session not ready or table permission issue	Restart session and confirm workspace role is Contributor or higher
Slow or queued execution	Capacity is busy or throttled	Wait, stop unnecessary sessions, and inspect capacity metrics if available

Table 2: Common Chapter 0 setup symptoms and first checks.

0.9 How This Setup Differs from Raw IaaS Clouds

In AWS, Azure, or GCP, a first data engineering lab often begins by creating a cloud subscription or project, selecting a region, enabling APIs, provisioning object storage, creating IAM roles, configuring command-line credentials, and deciding which compute service will run Spark. Fabric intentionally compresses many of those choices into a SaaS experience. You still make architecture decisions, but the first units are tenant, capacity, workspace, item, and OneLake path.

That difference is useful pedagogically. You can begin with data architecture before building a platform from primitives. However, it can also mislead learners into thinking there is no infrastructure. There is infrastructure, but Microsoft operates much of it behind the SaaS boundary. Your engineering responsibilities move upward: capacity usage, workspace design, item ownership, data contracts, governance, quality, and consumption paths.

Note

When this book compares Fabric with Azure services, remember that Fabric is not simply “Synapse plus Power BI.” It is a SaaS analytics platform with shared OneLake storage, capacities, workspaces, governance, and integrated workloads. Some Azure concepts reappear, but the operating model is different.

0.10 Further Reading

Start with Microsoft’s overview of Fabric [59], the Fabric trial documentation [37], Fabric concepts and licenses [35], tenant settings for Fabric item creation [36], workspaces [61], OneLake [60], lakehouses [53], and the Capacity Metrics app [34]. Return to these pages whenever a user-interface label changes; the concepts in this chapter are stable, but the portal wording and placement can evolve.

Key Takeaways

- Microsoft Fabric requires an organizational work or school account because tenant settings, administration, workspaces, capacities, and OneLake are tenant-scoped.
- A personal Outlook.com, Hotmail, Live.com, or Gmail-backed Microsoft account cannot by itself enable a Fabric tenant for hands-on data engineering.
- The Fabric trial provides a temporary capacity for evaluation; it is not a production capacity and should be monitored like shared compute.
- The Admin portal setting **Users can create Fabric items** must be enabled for the relevant users or groups before lakehouses and notebooks can be created.
- Workspaces are collaboration and permission containers, and they must be assigned to the Trial or another Fabric capacity for Fabric workloads.
- A first lakehouse proves that OneLake is available and gives later chapters a place to store files, managed Delta tables, and notebook outputs.
- Verification is complete only after you upload a sample file, run a PySpark cell, write a Delta table, and confirm the expected result.

0.11 Exercises

1. **(Identity)** Write down the account you used for Fabric. Is it an organizational work or school account? Explain how you know without including private information.
2. **(Admin)** Find the tenant setting that controls whether users can create Fabric items. If you cannot access the Admin portal, document who in your organization owns that setting.
3. **(Capacity)** Start or identify the Fabric trial capacity. Record the trial start date, expected expiry date, and the workspace assigned to it.
4. **(Workspace)** Create `fabric-book-dev` or an equivalent training workspace. Assign it to the Trial capacity and add any collaborators with the least role they need.
5. **(Lakehouse)** Create `lh_getting_started`. Identify where Files and Tables appear in the lakehouse explorer.
6. **(Upload)** Create `fabric_setup_orders.csv` with PowerShell and upload it to Files/setup. Confirm the path in the Fabric portal.
7. **(Notebook)** Run the verification PySpark cell and confirm the expected result: three rows, two customers, and total sales of 159.75.
8. **(Cost)** If you have access, open the Capacity Metrics app or ask your administrator how capacity usage is monitored. Note where you would look if notebooks became slow.
9. **(Setup checklist before Chapter 1)** Confirm all of the following: organizational account works; Fabric item creation is enabled; trial or paid capacity is available; workspace is assigned to capacity; lakehouse exists; sample file is uploaded; notebook smoke test passes; cleanup owner is known.
10. **(Interview)** In five sentences, explain why Fabric setup begins with tenant identity and capacity assignment rather than with a storage account and virtual machine.

0.12 Curated Community Resources

Beyond this book and the vendor documentation, the following community-curated resources are worth your time. Do not try to consume everything at once: pick one resource per category, practice it with real data, and build something small before moving on.

Practical Case Studies (Video Walkthroughs)

Fifteen end-to-end builds that show how the tools work together:

1. SQL Data Warehouse from Scratch — SQL, ETL, data modeling, star schemas (<https://lnkd.in/gcrJ2qde>)
2. End-to-End Data Analytics Project — Python, Pandas, SQL Server, data cleaning (<https://lnkd.in/gHxTN-B2>)
3. Netflix Data Engineering Project — Python, SQL, ELT, exploratory analysis (<https://lnkd.in/gMkbpWmh>)
4. DuckDB and Python Data Engineering Project — API ingestion, pipeline development (<https://lnkd.in/gEPnubGA>)
5. Airflow Podcast Data Pipeline — Airflow, APIs, scheduling (<https://lnkd.in/gmy5rS4Z>)
6. Automated Python ETL Pipeline with Airflow — SQL Server, DAG development (<https://lnkd.in/ghrBc8BE>)
7. PySpark and dbt Data Engineering Project — incremental loading, dimensional modeling (<https://lnkd.in/gVZjvyR7>)
8. Apache Spark Data Engineering Project — PySpark, Databricks (<https://lnkd.in/gVNkqk5A>)
9. Airbnb Data Engineering Project — dbt, Snowflake, AWS, testing (<https://lnkd.in/g2f4TxPW>)
10. AWS ETL Pipeline Project — S3, Lambda, Glue, Athena, serverless ETL (<https://lnkd.in/gFC3NkNs>)
11. AWS Data Warehouse Project — PySpark, Glue, Redshift (<https://lnkd.in/gyZtHMEF>)
12. End-to-End Azure Data Engineering Project — Data Factory, Databricks, ADLS (<https://lnkd.in/gUpbRjzE>)
13. Spotify Azure Data Engineering Project — streaming, Delta Lake (<https://lnkd.in/gUGKuZ8N>)
14. Uber Real-Time Data Engineering Project — Event Hubs, structured streaming (<https://lnkd.in/gu7Tpg7D>)
15. End-to-End GCP Data Engineering Project — Cloud Storage, Dataproc, BigQuery (<https://lnkd.in/gv8Yedfu>)

Free Video Courses

Twenty videos covering the essential skill path — recommended learning order: Python, SQL, Linux, Git, Docker, data modeling, data warehousing, Spark, Kafka, Airflow, dbt, cloud, portfolio projects.

- **Foundations:** Data Engineering Course for Beginners (<https://lnkd.in/gprfkqnp>); Big Data Engineer Masterclass (<https://lnkd.in/gqPkrG63>)
- **Core tools:** Python for Data Engineers (<https://lnkd.in/gZeKJEi4>); SQL for Big Data Engineering (<https://lnkd.in/gt5DZXwQ>); PostgreSQL Full Course (<https://lnkd.in/gAU5n7gM>); Linux Command Line (<https://lnkd.in/gA-7zFaQ>); Git and GitHub Tutorial (<https://lnkd.in/gS73kHau>); Docker Tutorial (<https://lnkd.in/g59TCWPD>)
- **Warehousing and modeling:** Data Warehousing Beginner's Guide (https://lnkd.in/gS_-iJAb); Data Modeling, Star Schema and Kimball (<https://lnkd.in/g4it9vpy>)
- **Batch processing:** PySpark Tutorial for Beginners (<https://lnkd.in/gre86dH6>); PySpark Full Course (<https://lnkd.in/gZXg38hk>)
- **Streaming:** Apache Kafka Full Course (https://lnkd.in/gnnXc9_d); Kafka Crash Course with Project (<https://lnkd.in/gP6DFCem>)
- **Pipelines:** Apache Airflow Tutorial (<https://lnkd.in/gbFzzReZ>); dbt Tutorial with Netflix Project (<https://lnkd.in/gHvmdjNX>); ELT with dbt, Snowflake and Airflow (https://lnkd.in/g5ZP_xC3)
- **Cloud:** AWS for Data Engineers (<https://lnkd.in/g8vRZmkp>)
- **End-to-end projects:** Airbnb Project with Snowflake, dbt and AWS (<https://lnkd.in/g2f4TxPW>); Real-Time E-Commerce with Kafka and Snowflake (<https://lnkd.in/g58V6A3P>)

GitHub Repositories

Twenty free repositories to learn from, practice with, and build upon. Choose one roadmap, complete one project, study the code structure, then break it, fix it, and document what you learned.

- **Roadmaps and guides:** Data Engineering Zoomcamp (<https://lnkd.in/gxKmbbHc>); Data Engineer Handbook (<https://lnkd.in/gstev9z7>); Data Engineering Cookbook (<https://lnkd.in/gABzJ-vE>); Complete Data Engineer Roadmap (<https://lnkd.in/gmwQrmvm>)
- **Curated lists:** Awesome Data Engineering (<https://lnkd.in/gh4Ky273>); Awesome Big Data (https://lnkd.in/g_y-n3zm); Awesome Open-Source Data Engineering (<https://lnkd.in/ggDTw66U>)
- **Portfolio and analytics engineering:** Data Engineering Project Template (<https://lnkd.in/g-QKrMXV>); Jaffle Shop dbt Practice Project (<https://lnkd.in/gmXVhYKX>); dbt Core (<https://lnkd.in/gnCXxNbB>)
- **Ingestion and orchestration:** Apache Airflow (<https://lnkd.in/gqvG5F6N>); Airbyte (<https://lnkd.in/g2B9AVwM>); dlt (<https://lnkd.in/g3A8BMp3>)
- **Processing:** Apache Spark (<https://lnkd.in/g5CPE3hh>); Apache Kafka (<https://lnkd.in/gdH5T4xX>)
- **Storage and lakehouses:** DuckDB (<https://lnkd.in/gehQd42W>); Apache Iceberg (<https://lnkd.in/gVweVV8y>)
- **Quality, lineage and governance:** Great Expectations (<https://lnkd.in/g8TPyAy8>); OpenLineage (https://lnkd.in/gz3_MiAP); DataHub (<https://lnkd.in/gEcNMRkJ>)

Part I

Core Data Infrastructure and OneLake

OneLake and Microsoft Fabric Foundations

Learning Objectives

- Explain Microsoft Fabric as a unified SaaS analytics platform and distinguish its major workloads.
- Reason about Fabric capacities, F SKUs, capacity units, workspaces, domains, and tenant-level organization.
- Define OneLake as a single logical data lake per tenant and describe how workspaces, folders, and lakehouses project into it.
- Describe Delta-Parquet as the native open storage pattern for reliable lakehouse tables.
- Create and evaluate OneLake shortcuts for internal and external data virtualization scenarios.
- Build a first lakehouse lab that reads shortcut data with PySpark without copying the underlying files.
- Architect a multi-cloud retail data virtualization pattern that unifies AWS S3 history and Dataverse CRM data through OneLake.

Cross-Cloud Equivalence

Although this book teaches **Microsoft Fabric**, OneLake is built directly on the Azure column below, and the Week 1 storage foundations map almost one-to-one across the other clouds.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool / Cold	Nearline
Archive (instant)	Glacier Instant Retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication (GRS)	Dual/multi-region + Transfer
Online transfer	DataSync	AzCopy	Storage Transfer Service
Offline transfer	Snow Family	Data Box	Transfer Appliance
Design durability	11 nines	11 nines (LRS) / 16 (GRS)	11 nines
Availability SLA	99.9% (Standard)	99.9% (RA-GRS read 99.99%)	99.95% (multi-region)
Encryption at rest	SSE-S3 / SSE-KMS / SSE-C	SSE + CMK via Key Vault	Google-managed / CMEK / CSEK
Access control	IAM & bucket policies, ACLs	Azure RBAC, SAS, POSIX ACLs	IAM, ACLs, signed URLs
Pricing levers	Storage + requests + egress + retrieval	Storage + operations + egress + retrieval	Storage + operations + egress + retrieval
Consistency	Strong read-after-write	Strong read-after-write	Strong (global)

The same storage layer reappears as a managed abstraction across the stack: **Fabric OneLake** is a tenant-wide lake over ADLS Gen2 (Delta-Parquet), **Snowflake** persists data as immutable micro-partitions on the provider's object store, and **MongoDB's** WiredTiger is a node-local engine rather than object storage—the contrast is itself instructive.

Key Terms

Fabric capacity: a pool of compute resources, measured through capacity units, that powers Fabric workloads. **OneLake:** the tenant-wide logical data lake for Microsoft Fabric. **Lakehouse:** a Fabric item that combines data lake storage, Delta tables, SQL access, and Spark access. **Shortcut:** a OneLake pointer to data stored elsewhere, exposed as if it were local. **Delta-Parquet:** Parquet files plus Delta Lake transaction logs that support ACID table semantics.

1.1 Week 1 Roadmap

This chapter is the first week of the Microsoft Fabric Master Data Engineer program. Monday introduces the platform. Tuesday establishes OneLake and the lakehouse storage contract. Wednesday focuses on shortcuts and data virtualization. Thursday is a guided project in which you create a workspace, provision a lakehouse, create a shortcut to public blob data, and read the data from a notebook. Friday closes with a global retailer architecture pattern that uses OneLake as the unifying namespace for multi-cloud analytics.

The week deliberately begins with foundations rather than ingestion tools. In Fabric, storage architecture choices influence every later decision: Spark jobs write Delta tables, warehouses can query lakehouse data through SQL endpoints, Power BI can use Direct Lake over curated

models, domains organize data products, and governance metadata depends on workspace and item boundaries. Microsoft describes Fabric as an end-to-end analytics platform that brings data engineering, data integration, warehousing, real-time intelligence, data science, and business intelligence into a single SaaS experience [59]. OneLake is the storage foundation of that experience [60].

Note

This chapter intentionally uses the term *data product*. In Fabric, a useful data product is not only a table. It is a governed bundle of storage, transformations, ownership, quality expectations, lineage, security, and consumption paths.

1.2 Monday: Introduction to Microsoft Fabric

Microsoft Fabric is best understood as a unified SaaS analytics operating environment rather than a single data tool. It combines experiences that historically lived in separate products: Data Factory for integration, Synapse-style data engineering and warehousing, Real-Time Intelligence, Data Science, and Power BI. The unification matters because the platform gives these workloads shared identity, shared capacity, shared governance, shared workspace organization, and shared OneLake storage.

Definition

Microsoft Fabric is a unified SaaS analytics platform that integrates data movement, data engineering, data warehousing, real-time analytics, data science, governance, and Power BI over a common tenant, workspace, capacity, and OneLake foundation.

A traditional Azure analytics estate often contains independent services: storage accounts, Spark pools, SQL pools, Data Factory instances, Key Vaults, Power BI workspaces, monitoring resources, and security assignments. Fabric does not remove the need for architecture. It changes the default unit of architecture. Workspaces and Fabric items become the visible work surface; capacities become the shared compute boundary; OneLake becomes the default data lake; and Power BI semantic delivery is integrated instead of bolted on at the end.

1.2.1 The Workload Map

Fabric workloads are specialized experiences over a shared platform. A data engineer should know their purpose and their storage relationship.

Workload		Primary role	Storage and serving pattern
Data Engineering		Build lakehouses, notebooks, Spark jobs, and Delta tables	Writes files and tables in OneLake, commonly in medallion layers
Data Factory		Ingest, transform, and orchestrate data pipelines	Moves or transforms data into lakehouses, warehouses, or external systems
Data Warehouse		Serve relational analytics with SQL	Uses warehouse storage and can integrate with lakehouse data through SQL experiences
Real-Time Intelligence	Intelligence	Process event streams, logs, and telemetry	Eventstreams and KQL databases support streaming and operational analytics
Data Science		Explore, train, track, and operationalize models	Uses notebooks, experiments, models, and lakehouse data
Power BI		Model and visualize business data	Consumes lakehouse and warehouse data through import, DirectQuery, or Direct Lake

Table 1.1: Major Fabric workloads and their relationship to the shared analytics platform.

Note

Do not treat workloads as independent silos. The same customer table may be landed by Data Factory, cleaned by Spark, served by a warehouse, certified in a semantic model, secured by domain governance, and visualized in Power BI. Fabric’s value is the shared operating model across those steps.

1.2.2 Capacities, F SKUs, and Capacity Units

A Fabric capacity is the compute pool assigned to one or more workspaces. Microsoft licenses Fabric through capacity SKUs and concepts that determine how much work the platform can execute [35]. F SKUs represent Fabric capacities with different amounts of available compute. Capacity units, often abbreviated as CUs, are the practical abstraction used to reason about consumption across workloads.

Definition

A **capacity unit** is an abstract measure of Fabric compute throughput. It lets the platform meter diverse work—Spark sessions, warehouse queries, Data Factory activities, Power BI interactions, and real-time workloads—against a shared capacity pool.

Capacity planning is not only a procurement exercise. It is an engineering feedback loop. The same capacity can be affected by long Spark notebooks, expensive SQL queries, concurrent refreshes, pipeline bursts, and report traffic. A lakehouse design that creates many small files may slow downstream reads and burn capacity. A semantic model that unnecessarily imports data may duplicate storage and refresh work. A pipeline that copies external data that could have been virtualized through a shortcut may increase both latency and compute cost.

Tip

Start capacity design by classifying workloads by shape: interactive reports, scheduled batch jobs, ad hoc notebooks, streaming ingestion, and administrative operations. Then test concurrency, not just single-job performance.

1.2.3 Workspaces and Domains

Workspaces are collaboration and item containers. They hold lakehouses, warehouses, notebooks, pipelines, semantic models, reports, dataflows, eventstreams, KQL databases, and related artifacts. A workspace is also the practical boundary for assigning capacity and permissions. Domains provide a higher-level governance organization, allowing related workspaces and data products to be grouped by business area such as Finance, Retail Operations, Supply Chain, or Customer Experience [25].

A master data engineer treats workspace design as information architecture. A single shared workspace for every team becomes ungovernable. A workspace per developer fragments lineage and reuse. A useful middle ground aligns workspaces to data products, environments, or bounded domains. For example, `retail-sales-dev`, `retail-sales-test`, and `retail-sales-prod` can live under a Retail domain, while customer support analytics lives in a Customer domain.

Common Pitfall

Using workspaces as folders. Workspaces carry permissions, capacity assignment, lineage, deployment, and operational meaning. If you create workspaces only to make a navigation tree look tidy, you will eventually fight inconsistent access and duplicated data products.

1.2.4 Monday Architecture Lens

The platform view produces three questions that recur throughout the book. First, what is the compute boundary? In Fabric, the answer is normally a capacity assigned to workspaces. Second, what is the collaboration and security boundary? The answer is normally a workspace, refined by item permissions and downstream semantic model security. Third, what is the data boundary? The answer is OneLake plus the source systems linked through shortcuts or pipelines. Good Fabric architecture keeps those three boundaries aligned enough to operate, but not so tightly coupled that every change requires a tenant redesign.

1.3 Tuesday: OneLake Concepts

Microsoft presents OneLake as the “OneDrive for Data” for an organization [60]. The phrase is useful because it emphasizes a single logical namespace across the tenant. Users do not need to choose a storage account every time they create a lakehouse. Fabric items place data into OneLake paths managed by the platform, and authorized tools can access those files through Fabric experiences and supported APIs.

Definition

OneLake is the single logical data lake for a Fabric tenant. It stores Fabric data items and exposes a unified namespace so workloads can share data without every team provisioning separate storage accounts.

The single-lake idea does not mean every team sees every file. Authorization still matters. Workspaces, item permissions, data access roles, sharing, endorsement, and governance controls determine who can discover and use data. The value of OneLake is that storage becomes a shared substrate rather than a collection of disconnected lakes.

1.3.1 Workspaces, Items, Folders, and Paths

A Fabric lakehouse item has a storage layout that separates files and tables. The **Files** area is for arbitrary folders and files, including raw landing data. The **Tables** area is for managed Delta tables that Fabric can expose through Spark, SQL analytics endpoints, and Power BI-oriented experiences. This distinction becomes central when you design medallion layers.

Note

A folder of Parquet files in **Files** is not automatically the same thing as a managed lakehouse table. A Delta table has a transaction log and table metadata that enable reliable reads, writes, schema evolution, and query interoperability.

1.3.2 Delta-Parquet as the Native Open Format

Fabric lakehouses use open data formats. Parquet provides columnar file storage with efficient compression and predicate pushdown [2]. Delta Lake adds a transaction log over Parquet files to provide ACID table behavior on object storage [4, 9]. The lakehouse architecture pattern unifies warehouse-style analytics and data-science workloads over open storage [5]. Fabric adopts this direction by making Delta tables a first-class lakehouse construct [53].

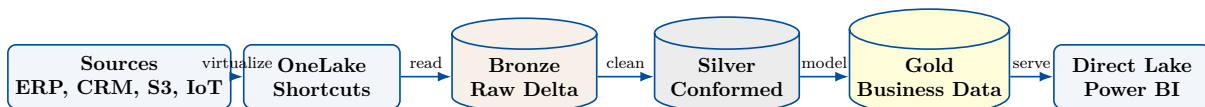
Definition

Delta-Parquet means table data is stored as Parquet files and table state is managed by a Delta transaction log. The combination supports atomic commits, time-aware metadata, schema enforcement, and consistent reads by multiple engines.

Delta matters because distributed engines do not write one neat database file. Spark jobs write many files, and concurrent jobs can overlap. Without a transaction log, readers may see partial results, duplicate files, or inconsistent schemas. With Delta, a table commit is a versioned metadata operation over a set of Parquet files.

1.3.3 Medallion Thinking in OneLake

The medallion architecture organizes data into Bronze, Silver, and Gold layers. Bronze stores raw or lightly standardized data. Silver stores cleaned, conformed, deduplicated, and enriched data. Gold stores business-ready aggregates and dimensional models for consumption [6]. OneLake does not force this layout, but Fabric lakehouses make it natural to model those layers through lakehouses, folders, schemas, or tables.



Open Delta-Parquet tables in OneLake preserve interoperability across Spark, SQL, and BI workloads.

Figure 1.1: A OneLake-centered medallion pattern for Microsoft Fabric.

1.3.4 Tuesday Engineering Lens

OneLake encourages teams to separate logical data access from physical storage administration. That separation is powerful, but it can hide operational reality. A table still has files. Files still have sizes, partitions, formats, schemas, and update patterns. A shortcut still depends

on a remote system. A report still depends on a semantic model and refresh behavior. The mature Fabric engineer uses OneLake to simplify access while continuing to reason about storage contracts.

1.4 Wednesday: OneLake Shortcuts

A shortcut is a pointer inside OneLake to data stored somewhere else. Instead of copying files from an external source into OneLake, Fabric can expose the remote data through the OneLake namespace. Microsoft documents shortcuts for sources such as ADLS Gen2, Amazon S3, Dataverse, and other Fabric locations [40]. For a data engineer, shortcuts are one of Fabric’s most important design primitives because they separate logical access from physical movement.

Definition

A **OneLake shortcut** is a virtual folder reference that appears inside OneLake while the underlying files remain in their source location. The shortcut lets Fabric workloads read data through the OneLake namespace without first copying it.

1.4.1 Internal and External Shortcuts

Internal shortcuts point to data already in OneLake, often in another workspace or item. They support reuse without duplicating files. External shortcuts point to data outside OneLake, such as ADLS Gen2, Amazon S3, or Dataverse. Both patterns can simplify architecture, but they have different governance and operational implications.

Shortcut type	Typical use	Engineering questions
Internal OneLake	Share curated data between workspaces without copying	Who owns the source? What happens if the source table schema changes? How is lineage communicated?
ADLS Gen2	Reuse Azure lake data during migration or coexistence	Which identity accesses storage? Are firewall, region, and folder conventions compatible?
Amazon S3	Virtualize historical or partner data in AWS	What are latency, egress, credentials, and consistency implications?
Dataverse	Bring business application data into analytical pipelines	Which tables are needed? How are privacy, row-level access, and refresh expectations handled?

Table 1.2: Shortcut types and architecture questions.

1.4.2 When Virtualization Beats Copying

Shortcut virtualization is valuable when data is large, shared, governed by another platform, updated by another team, or too expensive to duplicate. It is also useful during migrations: a team can build Fabric transformation logic against external paths, then later decide whether selected data should be physically materialized into OneLake Delta tables.

Tip

Use shortcuts for access and exploration; use managed Delta tables for reliable curated products. A shortcut to raw external data is not a substitute for Silver and Gold tables with clear ownership, quality checks, and contracts.

1.4.3 Risks and Controls

Shortcuts do not remove physics. Remote data can be slower than local data. External cloud access can involve network latency, availability dependencies, credential management, and possible data-transfer costs. External schemas can change without Fabric knowing in advance. The source system remains the system of record, so deletion or restructuring at the source may break downstream Fabric jobs.

Common Pitfall

Treating virtual data as governed data. A shortcut makes external data visible through OneLake, but it does not automatically certify quality, freeze schema, apply business definitions, or guarantee service-level agreements. Curate important shortcut data into managed Delta tables before exposing it broadly.

1.4.4 Shortcut Decision Matrix

Shortcut decisions should be documented explicitly. The following matrix is a practical review aid.

Question	Shortcut-friendly answer	Materialize-first answer
How large is the source?	Too large or costly to copy before proving value	Small enough to copy cheaply and repeatedly
Who owns source quality?	Another team owns it and publishes a clear contract	Your team must repair quality before any consumption
How stable is schema?	Stable or source publishes change notifications	Volatile and likely to break downstream code
What is the consumption path?	Exploration, profiling, or controlled engineering jobs	Certified BI, finance reporting, or audited outputs
What is the latency target?	Batch or exploratory access tolerates remote reads	Interactive dashboards require predictable local performance

Table 1.3: Shortcut versus materialization decision matrix.

1.5 Thursday: Hands-on Project

The project for Week 1 creates the minimum useful Fabric data engineering environment: a workspace, a lakehouse, a shortcut, and a notebook that reads the shortcut. The scenario assumes you have access to a Fabric tenant, an available capacity or trial capacity, and permissions to create a workspace and lakehouse.

1.5.1 Goal and Architecture

You will create a workspace named `fabric-de-week01`, add a lakehouse named `lh_onelake_foundations`, create a shortcut named `public_samples`, and read files exposed by that shortcut with PySpark. The shortcut target can be any public Azure Blob container that your tenant can access. If your instructor provides a specific URL, use that URL. If not, use a public sample container approved for your training environment.

Note

Fabric user interface labels change over time. When a button label differs slightly, follow the action intent: create a workspace, create a lakehouse item, open the lakehouse explorer,

create a shortcut under **Files**, select the Azure Blob or ADLS-compatible source, and validate that files appear without uploading them.

1.5.2 Step-by-Step Actions

1. Open the Microsoft Fabric portal and confirm that the Fabric experience is enabled for your tenant.
2. Select **Workspaces**, then create a new workspace named `fabric-de-week01`. Assign it to a Fabric capacity if your organization requires explicit capacity selection.
3. Inside the workspace, choose **New item**, then create a **Lakehouse** named `lh_onelake_foundations`.
4. In the lakehouse explorer, expand **Files**. Choose the command to create a new shortcut.
5. Select the external source type for the public sample container. For an Azure Blob public container, choose the source option your tenant exposes for Blob or ADLS-compatible access.
6. Enter shortcut name `public_samples`. Enter the account, container, and folder information supplied by your instructor or the public training dataset.
7. Complete the connection step. If authentication is requested for a public source, choose the anonymous or organizational method that matches the published source and your tenant policy.
8. Confirm that `Files/public_samples` appears in the lakehouse and that its folders or files are visible. You have virtualized data; you have not copied it into the lakehouse.
9. Create a new notebook attached to the lakehouse. Name it `nb_week01_shortcut_read`.
10. Run the PySpark listing below, adjusting the relative path and file format to match the public dataset.

```

1 from pyspark.sql import functions as F
2
3 # In a Fabric notebook attached to lh_onelake_foundations, Files paths are
4 # reachable relative to the lakehouse mount. Adjust the child folder and file
5 # extension to match the public sample container used by your instructor.
6 shortcut_path = "Files/public_samples/sales/*.parquet"
7
8 sales_df = spark.read.parquet(shortcut_path)
9
10 profile_df = (
11     sales_df
12     .select(
13         F.count("*").alias("row_count"),
14         F.countDistinct("CustomerId").alias("customer_count"),
15         F.min("OrderDate").alias("first_order_date"),
16         F.max("OrderDate").alias("last_order_date"),
17         F.sum("SalesAmount").alias("total_sales_amount"),
18     )
19 )
20
21 profile_df.show(truncate=False)
22
23 # Persist a curated Delta table in the lakehouse. The source files remain in
24 # the external container; this table is a managed Fabric lakehouse asset.
25 clean_df = (
26     sales_df
27     .withColumn("OrderDate", F.to_date("OrderDate"))

```

```

28     .withColumn("SalesAmount", F.col("SalesAmount").cast("decimal(18,2)"))
29     .dropDuplicates(["OrderId"])
30 )
31
32 clean_df.write.format("delta").mode("overwrite").saveAsTable("
    silver_sales_orders")

```

Listing 1.1: PySpark notebook snippet for reading data exposed through a OneLake shortcut.

1.5.3 Representing the Shortcut as Configuration

In production, connection details should be managed through deployment processes, environment-specific parameters, and secure credentials. The following JSON is not a secret file. It is a conceptual deployment record that captures the intent of a shortcut so it can be reviewed consistently.

```

1 {
2   "workspace": "fabric-de-week01",
3   "lakehouse": "lh_onelake_foundations",
4   "shortcut": {
5     "name": "public_samples",
6     "location": "Files/public_samples",
7     "sourceType": "AzureBlobPublicContainer",
8     "sourceUri": "https://examplepublicdata.blob.core.windows.net/sales",
9     "copyData": false,
10    "intendedLayer": "Bronze exploration",
11    "owner": "data-engineering-training",
12    "qualityContract": "not certified; curate before BI consumption"
13  }
14 }

```

Listing 1.2: Conceptual JSON record for documenting a OneLake shortcut target.

1.5.4 Optional SQL Check

After the notebook writes a managed Delta table, use the SQL analytics endpoint or a SQL query cell to check record counts and date ranges. This listing is intentionally simple: Week 1 validates access and table creation, not warehouse performance tuning.

```

1 SELECT
2   COUNT(*) AS row_count,
3   COUNT(DISTINCT CustomerId) AS customer_count,
4   MIN(OrderDate) AS first_order_date,
5   MAX(OrderDate) AS last_order_date,
6   SUM(SalesAmount) AS total_sales_amount
7 FROM silver_sales_orders;

```

Listing 1.3: SQL validation after writing a managed Delta table from shortcut data.

Tip

After a successful shortcut read, immediately write down three facts: the owner of the external data, the expected refresh or update pattern, and the downstream table that will become the certified copy. These facts prevent shortcut sprawl.

1.5.5 Validation Checklist

A successful lab has observable evidence:

- The workspace exists and is assigned to the expected capacity.

- The lakehouse appears in the workspace and opens without errors.
- The shortcut folder appears under **Files** and exposes external files.
- The notebook reads the shortcut path without uploading files manually.
- A managed Delta table such as `silver_sales_orders` is created from the shortcut data.
- You can explain which data remains external and which data is now managed in the lakehouse.

1.6 Friday: Use Case and Architecture

A global retailer has ten years of historical sales data stored in Amazon S3 by region. Current customer and loyalty activity lives in Dataverse because front-office teams use Dynamics-based applications. Executives want a single Power BI view of customer lifetime value, store performance, and campaign effectiveness. The data engineering team wants to avoid a large one-time migration before proving value.

1.6.1 Architecture Strategy

The design uses OneLake shortcuts to virtualize both historical S3 data and current Dataverse data into a Fabric lakehouse. Bronze reads remain virtualized at first. Silver tables standardize customer, order, product, calendar, and region fields into managed Delta tables. Gold tables publish conformed metrics for Power BI through a semantic model using Direct Lake where appropriate [24].

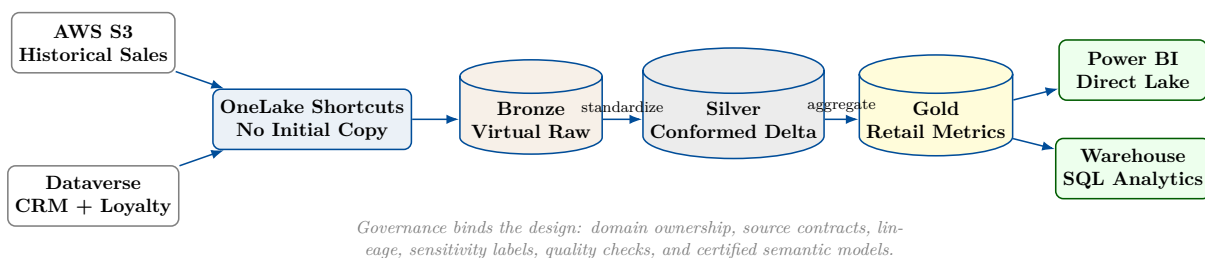


Figure 1.2: Multi-cloud retail virtualization through OneLake shortcuts and medallion curation.

1.6.2 Design Decisions

The first decision is to avoid copying all S3 history on day one. A shortcut makes the data visible to Fabric jobs, which lets engineers profile schema, quality, and partitioning before committing to a migration. The second decision is to materialize curated Silver and Gold tables in OneLake. Business users should not query raw external folders directly. The third decision is to represent Dataverse as a governed source with privacy-aware curation: customer identifiers, consent fields, and loyalty attributes require explicit access rules.

Concern	Shortcut-first decision	Curation decision
Historical sales	Use S3 shortcut for profiling and incremental value	Convert validated partitions into Silver Delta tables
Current CRM	Use Dataverse shortcut for governed analytical access	Mask or restrict sensitive fields before Gold publication
Cost and latency	Avoid bulk migration before value is proven	Optimize high-value queries into local Delta and semantic models
Ownership	Source teams retain operational systems	Domain data owners certify curated tables and metrics
BI consumption	Do not point reports at raw virtual folders	Publish Gold metrics through Direct Lake semantic models

Table 1.4: Retail architecture decisions for shortcut-first modernization.

Common Pitfall

Skipping the Silver contract. Multi-cloud virtualization can make data look unified before it is semantically unified. Customer keys, time zones, currencies, returns, store hierarchies, and consent fields must be standardized before executives trust a global report.

1.7 Operational Guidance for Week 1

A Week 1 Fabric environment can already fail in production-like ways. A workspace can be created in the wrong capacity. A shortcut can point to the wrong folder. A notebook can read external files slowly because partitioning is poor. A user can assume a shortcut means the data has been certified. The data engineer's job is to make these assumptions visible.

1.7.1 Governance Checklist

- Assign every workspace to a domain or document why it is temporary.
- Name lakehouses, shortcuts, notebooks, and tables consistently.
- Document source ownership and update frequency for every shortcut.
- Separate raw shortcut exploration from certified Silver and Gold tables.
- Record sensitivity and privacy expectations before publishing semantic models.
- Test whether intended users can access the data and whether unintended users cannot.

1.7.2 Quality and Observability

Even in the first lab, build the habit of recording quality observations. Count rows, inspect null rates, check duplicate keys, record source file paths, and save the notebook output that proves the shortcut worked. In later chapters these checks become formal data quality gates. Week 1 is where the discipline begins.

```

1 from pyspark.sql import functions as F
2
3 required_columns = ["OrderId", "CustomerId", "OrderDate", "SalesAmount"]
4 missing_columns = sorted(set(required_columns) - set(sales_df.columns))
5 if missing_columns:
6     raise ValueError(f"Missing required columns: {missing_columns}")
7
8 quality_df = sales_df.select(
9     F.count("*").alias("rows"),

```

```

10 F.sum(F.col("OrderId").isNull().cast("int")).alias("null_order_ids"),
11 F.sum(F.col("CustomerId").isNull().cast("int")).alias("null_customer_ids"),
12 F.countDistinct("OrderId").alias("distinct_orders"),
13 )
14
15 quality_df.show(truncate=False)

```

Listing 1.4: Small data quality profile for a newly read shortcut data frame.

1.7.3 Interview Q&A

1. **Q: Why is OneLake described as OneDrive for Data?**

A: Because it provides a single logical tenant-wide namespace where Fabric data items can be discovered and accessed through shared platform controls, rather than forcing every team to manage separate storage accounts.

2. **Q: When would you use a shortcut instead of copying data?**

A: Use a shortcut when the source is large, externally owned, frequently reused, or still being evaluated. Copy or materialize when you need curated quality, predictable performance, stable schemas, and certified consumption.

3. **Q: What is the relationship between Parquet and Delta Lake?**

A: Parquet stores columnar files; Delta Lake adds the transaction log and metadata that make those files behave like reliable tables for concurrent engines.

4. **Q: Why do capacities matter to data engineers?**

A: Capacity is the shared compute pool. Inefficient Spark jobs, warehouse queries, refreshes, and report usage compete for it, so engineering choices affect performance and cost.

5. **Q: Why should a BI report not point directly to a raw external shortcut?**

A: Raw shortcut data usually lacks semantic standardization, quality checks, privacy treatment, and performance guarantees. A report should consume certified Gold tables or semantic models.

6. **Q: How do domains change the operating model?**

A: Domains group related workspaces and data products under business ownership. They make governance and discovery align with organizational accountability rather than with accidental workspace lists.

1.8 Further Reading

Begin with Microsoft's overview of Fabric [59], OneLake [60], OneLake shortcuts [40], lakehouses [53], capacities and licenses [35], domains [25], and Direct Lake [24]. For storage foundations, read the Delta Lake paper [4], the lakehouse architecture paper [5], Apache Parquet documentation [2], and Delta Lake documentation [9]. For medallion design vocabulary, compare Fabric implementations with the medallion architecture description [6].

Key Takeaways

- Fabric is a unified SaaS analytics platform, not merely a collection of renamed Azure services.
- Capacities and capacity units turn compute into a shared engineering constraint across Spark, SQL, pipelines, real-time workloads, and Power BI.

- OneLake provides a single logical tenant-wide data lake, while workspaces and items provide practical collaboration and security boundaries.
- Fabric lakehouses use open Delta-Parquet patterns so Spark, SQL, and BI workloads can interoperate over reliable tables.
- Shortcuts virtualize data from internal OneLake locations and external systems such as ADLS Gen2, Amazon S3, and Dataverse.
- Virtualization avoids unnecessary copying, but curated Silver and Gold Delta tables are still required for trusted business consumption.
- A sound Week 1 architecture already includes ownership, naming, quality, privacy, capacity, and consumption decisions.

1.9 Exercises

1. **(Conceptual)** Explain the difference between a Fabric workspace, a domain, a capacity, and a lakehouse. Give one governance decision associated with each.
2. **(Conceptual)** Why is a shortcut not the same thing as a certified table? Describe at least three risks that remain after a shortcut is created.
3. **(Hands-on)** Create a development workspace and lakehouse. Add a shortcut under **Files**, then take a screenshot or write a short observation proving that the data was not uploaded manually.
4. **(Hands-on)** Modify the PySpark listing to read CSV or JSON data from your shortcut, infer or provide a schema, and write a managed Delta table named `bronze_source_profile`.
5. **(Design)** Design a workspace naming convention for development, test, and production Fabric environments in a retail organization. Include capacity assignment and domain placement.
6. **(Design)** For the global retailer, decide which data should remain virtualized and which should be materialized into OneLake. Justify your decision using cost, latency, ownership, and BI trust.
7. **(Interview)** In five sentences, defend Delta-Parquet as the native lakehouse format for Fabric data engineering.
8. **(Operational)** Create a Week 1 runbook entry that documents shortcut owner, source URI, authentication method, expected update frequency, downstream tables, and escalation contact.

1.10 Capstone Project: A Multi-Cloud Retail Landing Zone with OneLake Shortcuts

Capstone Project

Mission. Design and implement a Week 1 retail landing zone that uses Microsoft Fabric as the SaaS control plane, a trial-backed workspace as the collaboration boundary, OneLake as the analytical namespace, and shortcuts as the virtualization layer. Your goal is to let analysts query historical sales in Amazon S3 and current CRM data from Dataverse without first performing a bulk migration. You will then write one curated Silver Delta table that

becomes the trusted starting point for downstream BI.

Estimated time. 3–5 hours, depending on tenant permissions, source availability, and whether your class uses real S3 and Dataverse sources or instructor-provided public Blob and simulated CRM files.

What you'll practice.

- Creating a Fabric workspace on trial capacity and assigning work to a Lakehouse item.
- Using OneLake shortcuts to expose external data in place instead of copying raw files.
- Organizing a Bronze zone that separates virtual raw folders from managed curated tables.
- Reading shortcut paths with PySpark and producing a unified Delta-Parquet Silver table.
- Explaining the difference between virtualized source access and certified analytical data products.

Scenario

A global retailer operates stores and e-commerce channels in North America, Europe, and Asia-Pacific. Ten years of historical order, return, and store sales files are stored by region in AWS S3 because the original point-of-sale modernization program ran on AWS. Current customer profile, loyalty status, and campaign consent data lives in Dataverse because the customer service and marketing teams use Dynamics-based applications.

The executive request is simple: create a single Fabric landing zone that can answer customer lifetime value, regional sales, and loyalty conversion questions without waiting for a large data migration. The engineering constraint is equally important: historical S3 files and operational CRM exports must remain in their source systems during Week 1. Fabric should provide unified access, profiling, and curation while avoiding unnecessary duplication of raw data. This is the Friday architecture scenario made concrete as a hands-on capstone.

Requirements

Build the smallest production-shaped solution that proves the architecture:

- A Fabric workspace named `fabric-de-week01-retail` assigned to a Fabric trial capacity or another instructor-approved capacity [37].
- A Lakehouse named `lh_retail_landing` in that workspace.
- A Bronze folder structure under `Files/bronze` with shortcut locations for historical sales and CRM data.
- OneLake shortcuts to an Amazon S3 source for historical sales, or to an instructor-approved public Blob source that simulates S3 if S3 credentials are unavailable.
- A OneLake shortcut to Dataverse CRM data, or to an instructor-approved simulated CRM folder if Dataverse permissions are unavailable.
- A PySpark notebook named `nb_capstone_retail_silver` attached to the Lakehouse.
- Notebook logic that reads both shortcut sources in place, standardizes schemas, joins or unions data as appropriate, and writes a managed Silver Delta table named `silver_retail_customer_sales`.
- No bulk copy of raw S3, Blob, or Dataverse data into OneLake. Only the curated Silver Delta output is materialized in the Lakehouse.

Note

If your tenant does not allow S3 or Dataverse shortcuts, use public Blob or instructor-provided files as stand-ins. Keep the design language honest: the substitute source represents the external system, while the shortcut still proves the no-copy access pattern described in the OneLake shortcut documentation [40].

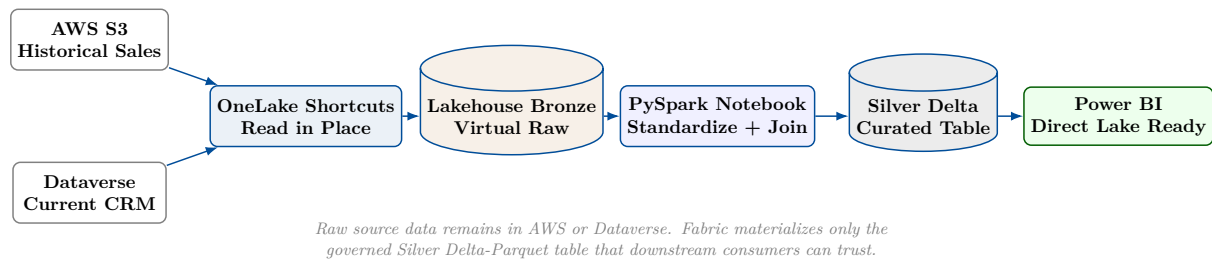
Reference Architecture

Figure 1.3: Capstone architecture for a multi-cloud retail landing zone using OneLake shortcuts.

Implementation Steps

1. **Create the workspace.** In Fabric, create `fabric-de-week01-retail`. Assign trial capacity or the instructor-approved capacity. Record the capacity name and whether the workspace is intended for development only.
2. **Create the Lakehouse.** Add `lh_retail_landing`. Confirm that the Lakehouse exposes **Files** and **Tables**; the Silver output will become a managed Delta table, not a loose folder of files [53].
3. **Create the Bronze folders.** Under **Files**, use `bronze/s3_sales` for historical sales and `bronze/dataverse_crm` for current CRM. These names describe source origin and medallion intent.
4. **Create the sales shortcut.** Add a OneLake shortcut at `Files/bronze/s3_sales`. Use Amazon S3 when credentials are available. Otherwise, point the shortcut to a public Blob folder that simulates historical sales. Do not upload the sales files manually.
5. **Create the CRM shortcut.** Add a shortcut at `Files/bronze/dataverse_crm`. Use Dataverse when permitted. Otherwise, point to simulated CRM profile and loyalty files.
6. **Validate virtualization.** Open each shortcut folder and confirm that files are visible through OneLake. Document the remote source, authentication method, owner, and expected refresh pattern.
7. **Create the notebook.** Add `nb_capstone_retail_silver`, attach it to `lh_retail_landing`, and run a small row-count profile before transformation.
8. **Write the Silver table.** Use the notebook pattern in listing 1.5 and adjust column names to match your source files.

```

1 from pyspark.sql import functions as F
2
3 sales_path = "Files/bronze/s3_sales/**/*.parquet"
4 crm_path = "Files/bronze/dataverse_crm/**/*.parquet"
5

```

```

6 sales_raw_df = spark.read.parquet(sales_path)
7 crm_raw_df = spark.read.parquet(crm_path)
8
9 sales_df = (
10     sales_raw_df
11     .select(
12         F.col("OrderId").cast("string").alias("order_id"),
13         F.col("CustomerId").cast("string").alias("customer_id"),
14         F.to_date("OrderDate").alias("order_date"),
15         F.col("Region").cast("string").alias("region"),
16         F.col("SalesAmount").cast("decimal(18,2)").alias("sales_amount"),
17         F.col("CurrencyCode").cast("string").alias("currency_code"),
18     )
19     .where(F.col("order_id").isNotNull())
20     .dropDuplicates(["order_id"])
21 )
22
23 crm_df = (
24     crm_raw_df
25     .select(
26         F.col("CustomerId").cast("string").alias("customer_id"),
27         F.col("LoyaltyTier").cast("string").alias("loyalty_tier"),
28         F.col("EmailOptIn").cast("boolean").alias("email_opt_in"),
29         F.to_timestamp("ModifiedOn").alias("crm_modified_on"),
30     )
31     .where(F.col("customer_id").isNotNull())
32     .dropDuplicates(["customer_id"])
33 )
34
35 silver_df = (
36     sales_df
37     .join(crm_df, on="customer_id", how="left")
38     .withColumn("sales_year", F.year("order_date"))
39     .withColumn("silver_loaded_at", F.current_timestamp())
40 )
41
42 silver_df.write.format("delta").mode("overwrite").saveAsTable(
43     "silver_retail_customer_sales"
44 )

```

Listing 1.5: PySpark notebook pattern for reading shortcut data in place and writing a Silver Delta table.

The second listing is a documentation artifact you can store with your runbook or deployment notes. It records the intended shortcut layout without placing secrets in source control. Real deployments should use secure connection objects, tenant-approved credentials, and environment-specific automation.

```

1 {
2     "workspace": "fabric-de-week01-retail",
3     "lakehouse": "lh_retail_landing",
4     "bronzeShortcuts": [
5         {
6             "name": "s3_sales",
7             "oneLakePath": "Files/bronze/s3_sales",
8             "sourceType": "AmazonS3",
9             "sourceUri": "s3://retail-history-example/sales/",
10            "copyData": false,
11            "owner": "retail-platform-history"
12        },
13        {
14            "name": "dataverse_crm",
15            "oneLakePath": "Files/bronze/dataverse_crm",

```

```

16     "sourceType": "Dataverse",
17     "sourceUri": "environment://crm/customers",
18     "copyData": false,
19     "owner": "customer-experience-platform"
20   }
21 ],
22 "silverOutput": {
23   "table": "silver_retail_customer_sales",
24   "format": "Delta",
25   "materializedInOneLake": true
26 }
27 }

```

Listing 1.6: Shortcut intent record for the capstone landing zone.

If your class prefers SQL validation, run a SQL cell or SQL analytics endpoint query after the write:

```

1 SELECT
2   COUNT(*) AS row_count,
3   COUNT(DISTINCT customer_id) AS customer_count,
4   MIN(order_date) AS first_order_date,
5   MAX(order_date) AS last_order_date,
6   SUM(sales_amount) AS total_sales_amount
7 FROM silver_retail_customer_sales;

```

Listing 1.7: SQL validation for the Silver customer sales table.

Deliverables

Submit a concise package that proves the architecture and implementation:

- Workspace name, capacity assignment, Lakehouse name, and notebook name.
- Screenshot or exported evidence showing both shortcut folders under **Files/bronze**.
- A short runbook entry for each shortcut: source owner, source URI pattern, authentication method, update frequency, and escalation contact.
- Notebook output showing row counts for historical sales, CRM profiles, and the final Silver table.
- The schema of `silver_retail_customer_sales`, including keys, dates, measures, and CRM enrichment fields.
- A paragraph explaining which data was virtualized, which data was materialized, and why that split is appropriate for Week 1.

Acceptance Criteria

Use this rubric before declaring the capstone complete:

- ☐ The workspace exists on trial or approved Fabric capacity and contains the required Lakehouse.
- ☐ Both external sources are represented through OneLake shortcuts under the Bronze zone.
- ☐ Historical sales data is read in place and not bulk copied into OneLake.
- ☐ CRM or simulated CRM data is read in place and not bulk copied into OneLake.
- ☐ The PySpark notebook reads both shortcut paths successfully and includes basic row-count or null-count validation.

- ☐ The notebook writes `silver_retail_customer_sales` as a managed Delta table.
- ☐ The Silver table has standardized customer keys, dates, currency or amount fields, and CRM enrichment fields.
- ☐ The design explicitly distinguishes virtual Bronze access from materialized Silver curation.
- ☐ The final explanation states that raw source data remains in the external systems and that only curated output is stored as Delta-Parquet in the Lakehouse.
- ☐ No secrets, connection strings, access keys, or personal data are pasted into notebook markdown, listings, screenshots, or source-controlled files.

Stretch Goals

If you finish early, extend the capstone without changing the core no-copy promise:

- Add a Gold aggregate table such as `gold_regional_loyalty_sales` grouped by region, sales year, and loyalty tier.
- Build a small Power BI semantic model over the Silver or Gold table and test whether Direct Lake is appropriate for the table shape.
- Add an additional shortcut to ADLS Gen2 to represent a supplier, inventory, or product catalog feed.
- Convert the notebook into a repeatable medallion job with parameters for source paths, target table name, and write mode.
- Add data quality assertions for duplicate order identifiers, invalid dates, negative sales amounts, missing customer keys, and unsupported currency codes.
- Document a promotion plan from development to test to production workspaces, including who owns shortcut credentials and who certifies the Gold semantic model.

The Medallion Architecture

Learning Objectives

- Explain the Bronze, Silver, and Gold responsibilities in a medallion architecture.
- Distinguish raw landing, validated conformance, and enriched analytical curation in Microsoft Fabric.
- Decide when a Fabric Lakehouse is a better serving target than a Fabric Data Warehouse, and when the warehouse is the safer choice.
- Organize workspaces and lakehouses so medallion layers scale across teams, environments, domains, and compliance boundaries.
- Deploy Bronze, Silver, and Gold lakehouses and prepare folder conventions for future transformations.
- Use PySpark to write Delta tables across medallion layers and inspect OneLake paths.
- Design a multi-workspace financial architecture that strictly isolates raw ingested logs from curated financial reports.

Cross-Cloud Equivalence

The medallion pattern is not a Microsoft-only vocabulary. It is a durable data-organization idea: separate raw facts from validated facts, then publish curated business products. The implementation changes by platform, but the engineering intent stays consistent.

Concept	Fabric	AWS/Azure/GCP lake	data	Snowflake
Bronze raw zone	OneLake such as <code>lh_bronze</code> ; raw files under <code>Files/bronze</code> and optional raw Delta tables	Object prefixes such as <code>s3://lake/raw/</code> , <code>abfss://lake/raw/</code> , or <code>gs://lake/raw/</code>		RAW database or schema, often loaded by Snowpipe, COPY, or external tables
Silver validated zone	Lakehouse or schema holding Delta tables with types, keys, deduplication, and quality checks	Prefixes such as <code>/validated/</code> or <code>/staging/</code> with Delta, Iceberg, or Parquet contracts		STAGING, CLEAN, or CONFORMED schemas with transformed tables
Gold curated zone	Lakehouse tables, warehouse tables, semantic models, and certified Power BI datasets	Prefixes such as <code>/curated/</code> or <code>/analytics/</code> optimized for BI and data products		ANALYTICS, MART, or presentation schemas consumed by BI tools
Layer boundary	Workspace, lakehouse, folder, table, and item permissions in OneLake	Bucket, account, container, prefix, IAM, ACL, and catalog boundaries		Database, schema, role, warehouse, and masking-policy boundaries
Promotion path	Dev/test/prod Fabric workspaces and deployment pipelines	Environment-specific buckets or prefixes with CI/CD and catalogs		Environment databases plus controlled role grants and cloning

MongoDB can express a similar progression with collections or databases such as `raw_events`, `validated_events`, and `analytics_summaries`; however, document databases usually emphasize application access patterns rather than object-lake folder zones.

Key Terms

Bronze: the raw or minimally standardized layer that preserves source fidelity. **Silver:** the validated layer that enforces schema, removes duplicates, standardizes keys, and records quality. **Gold:** the enriched and curated layer that serves business metrics, dimensional models, semantic models, and reports. **Lakehouse:** the Fabric item for Spark-first open Delta-Parquet engineering. **Warehouse:** the Fabric item for SQL-first relational modeling and serving.

2.1 Week 2 Roadmap

Week 2 turns the storage foundation from Chapter 1 into an operating architecture. Monday defines the medallion layers. Tuesday compares Fabric Lakehouses and Fabric Data Warehouses. Wednesday explains how to organize workspaces and lakehouses at scale. Thursday is a guided hands-on project that creates separate Bronze, Silver, and Gold lakehouses and prepares OneLake paths. Friday applies the design to a financial institution that must isolate raw ingested logs from curated financial reports.

The medallion architecture is valuable because it gives data teams a shared language for trust. Bronze answers, “What did the source send?” Silver answers, “What has engineering validated and conformed?” Gold answers, “What should the business consume?” Databricks popularized the Bronze, Silver, Gold vocabulary for lakehouse pipelines [6], and Fabric makes the pattern practical with OneLake, lakehouses, Delta tables, Spark notebooks, warehouses, and Power BI [53, 60].

Note

Medallion is not a law of nature. It is a contract. A small team may implement all three layers in one lakehouse; a regulated enterprise may use separate workspaces, lakehouses, capacities, domains, and security groups for each layer.

2.2 Monday: Medallion Principles

The simplest medallion definition is a progression from raw to trusted to useful. Each layer exists because downstream consumers need a different balance between fidelity, quality, and usability. A Bronze layer should preserve evidence. A Silver layer should enforce engineering truth. A Gold layer should publish business truth.

Definition

A **medallion architecture** is a layered data design that incrementally improves raw source data into validated, conformed, and curated analytical products. In Fabric, medallion layers are commonly implemented with OneLake Lakehouses, Delta tables, notebooks, pipelines, warehouses, and semantic models.

2.2.1 Bronze: Raw Landing and Source Fidelity

Bronze is the first analytical landing zone. It should answer audit and replay questions: which file arrived, when did it arrive, what source produced it, what schema did it have, and can the pipeline reprocess it if transformation logic changes? Bronze may contain CSV, JSON, Avro, Parquet, or raw Delta tables. The key design principle is that Bronze should be close to the source contract and should avoid premature business interpretation.

A useful Bronze table often adds ingestion metadata without changing the business payload. Examples include source file path, ingestion timestamp, batch identifier, hash values, and source system name. These fields make later debugging possible. They also preserve lineage when a Silver validation fails.

Tip

For regulated data, treat Bronze as evidence. Avoid overwriting raw data unless the retention policy explicitly requires it. Prefer append-only ingestion, immutable landing folders, and metadata that proves when and how records arrived.

2.2.2 Silver: Validation, Standardization, and Conformance

Silver is where engineering earns trust. Raw strings become typed dates, decimals, booleans, and identifiers. Duplicates are removed according to deterministic rules. Nullability expectations are checked. Source-specific codes are mapped to enterprise reference values. Personally identifiable or sensitive fields are masked, tokenized, or excluded when required by policy. Delta Lake's transaction log helps make those transformations reliable over object storage [4, 9].

Silver tables should be boring in the best sense. They are not usually the final executive dashboard tables, but they should be reusable across multiple Gold products. Customer, account, product, branch, employee, calendar, and transaction tables often become Silver building blocks.

Common Pitfall

Skipping Silver to save time. A direct Bronze-to-report path can look fast during a demo, but it pushes schema drift, duplicate handling, invalid dates, privacy rules, and business definitions into every downstream report.

2.2.3 Gold: Enriched, Curated, and Consumption Ready

Gold is optimized for use. It may contain star schemas, aggregate tables, metric marts, feature sets, executive reporting tables, or semantic models. A Gold product should have an owner, definition, refresh expectation, access model, and quality signal. In Fabric, Gold may be represented by Delta tables in a lakehouse, relational tables in a warehouse, or a certified Power BI semantic model that uses Direct Lake where appropriate [24].

Gold is where business language matters most. The table name `gold_daily_loan_risk` means little unless the organization agrees on loan status, delinquency bucket, exposure amount, currency conversion, calendar, and risk score timing. The architecture is technical, but the value is semantic.

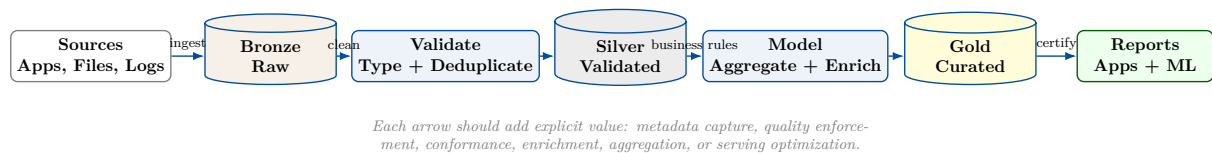


Figure 2.1: Bronze-to-Silver-to-Gold flow for a Fabric medallion architecture.

2.2.4 Layer Contracts

A layer contract states what consumers may assume. Bronze consumers may assume source fidelity and ingestion metadata, but not business correctness. Silver consumers may assume typed, validated, deduplicated, and conformed records. Gold consumers may assume business definitions, curated dimensions, certified metrics, and an intended serving path.

Layer	Primary promise	Typical checks	Common consumers
Bronze	Preserve source payload and ingestion evidence	File count, row count, schema snapshot, corrupt-record handling	Data engineers, auditors, reprocessing jobs
Silver	Provide reliable conformed records	Required columns, types, duplicate keys, referential checks, privacy treatment	Engineering teams, analysts, Gold pipelines
Gold	Publish business-ready products	Metric reconciliation, dimensional completeness, performance, endorsement	Power BI, finance teams, applications, executives

Table 2.1: Layer contracts in a medallion design.

2.3 Tuesday: Lakehouse vs Data Warehouse in Fabric

Fabric includes both Lakehouse and Data Warehouse items because analytical work has different shapes. The Lakehouse is Spark-first and file-aware. The Warehouse is SQL-first and relational. Both can participate in a medallion architecture, and neither is automatically superior.

Definition

A **Fabric Lakehouse** is best for open Delta-Parquet engineering, notebooks, Spark transformations, files, data science, and mixed structured or semi-structured ingestion. A **Fabric Data Warehouse** is best for SQL-first relational modeling, familiar warehouse development, controlled schemas, and serving teams that primarily use T-SQL.

2.3.1 When to Choose a Lakehouse

Choose a Lakehouse when the work begins with files, semi-structured data, data engineering notebooks, or Delta table transformations. Bronze and Silver are frequently lakehouse-centered because engineers need to inspect files, manage folders, process large datasets with Spark, and preserve open formats. Lakehouses are also natural when data scientists need feature exploration from the same Delta tables used by data engineers.

A lakehouse does not mean ungoverned data. It simply means the primary abstraction is open files and Delta tables. Fabric's lakehouse item still participates in workspaces, OneLake, SQL analytics endpoints, semantic models, and governance metadata [53].

2.3.2 When to Choose a Warehouse

Choose a Warehouse when the work is primarily relational, SQL-authored, and consumption oriented. Finance reporting, regulatory aggregates, dimensional marts, and teams with strong T-SQL practices may prefer warehouse development. A warehouse can be a strong Gold layer target when consumers expect SQL schemas, constraints of a relational serving model, and controlled access patterns.

The warehouse choice can also reduce cognitive load for teams that do not need Spark. If the source is already well-modeled relational data and the transformation logic is SQL-centric, moving everything through notebooks may be unnecessary.

2.3.3 Decision Aid

Decision question	Prefer Lakehouse	Prefer Warehouse
Primary authoring style	PySpark, Spark SQL, notebooks, file operations	T-SQL, stored procedures, relational modeling
Data shape	Raw files, JSON, CSV, Parquet, Delta, semi-structured feeds	Structured tables, facts, dimensions, SQL serving
Medallion layer	Bronze and Silver by default; Gold when Direct Lake and open Delta are enough	Gold marts, finance reporting, SQL-serving products
Consumer expectation	Data engineers, data scientists, mixed engines, Power BI Direct Lake	SQL analysts, BI engineers, governed relational users
Change pattern	Schema discovery and iterative engineering	Stable schemas and curated consumption contracts
Optimization focus	File layout, partitions, Delta transactions, Spark jobs	SQL query patterns, dimensional models, relational security

Table 2.2: Lakehouse-versus-Warehouse decision aid for Fabric medallion design.

Tip

A common enterprise pattern is Lakehouse for Bronze and Silver, then either Lakehouse Gold for Direct Lake semantic models or Warehouse Gold for SQL-first finance and regulatory marts. Let consumption requirements decide the Gold serving technology.

Common Pitfall

Choosing by team preference only. A Spark team may overuse notebooks for relational marts, while a SQL team may force raw file engineering into warehouse-shaped workflows. Choose the item type that best fits data shape, transformation style, and consumer contract.

2.4 Wednesday: Organizing Workspaces and Lakehouses at Scale

Medallion layers can be implemented inside one lakehouse, across multiple lakehouses in one workspace, or across multiple workspaces. The right boundary depends on security, team ownership, environment promotion, capacity management, and operational blast radius.

2.4.1 Single Lakehouse Pattern

A single lakehouse can contain `Files/bronze`, `Tables/silver_*`, and `Tables/gold_*`. This pattern is useful for training, prototypes, and small teams. It minimizes setup and makes lineage easy to see. Its weakness is that layer permissions and operational ownership are harder to separate.

2.4.2 Three Lakehouses in One Workspace

A workspace can contain `lh_bronze`, `lh_silver`, and `lh_gold`. This pattern makes layer intent explicit while keeping collaboration simple. It is a good default for a domain team that shares one security boundary but wants clean medallion separation.

2.4.3 Multi-Workspace Pattern

A regulated organization may use separate workspaces such as `fin-raw-prod`, `fin-curated-prod`, and `fin-reporting-prod`. Raw ingestion teams receive access to the raw workspace. Transformation services receive controlled access to read Bronze and write Silver. Report builders receive access only to curated Gold or semantic models. This pattern has more operational overhead, but it creates stronger isolation and auditability [61, 25].

Note

Workspace boundaries are not just visual organization. They affect permissions, item sharing, deployment pipelines, capacity assignment, lineage review, and incident blast radius. Use them deliberately.

2.4.4 Naming, Ownership, and Promotion

Names should reveal layer, domain, environment, and purpose. A practical convention is `<domain>-<layer>-<env>` for workspaces and `lh_<domain>-<layer>` for lakehouses. Examples include `finance-bronze-dev`, `finance-silver-test`, and `finance-gold-prod`. Tables can use prefixes such as `bronze_`, `silver_`, and `gold_` when multiple layers share a catalog.

Ownership should be equally explicit. Bronze often belongs to ingestion or platform teams, Silver to data engineering domain teams, and Gold to business data product owners. Promotion should move definitions from development to test to production without granting developers broad write access to production raw zones.

Scale pattern	Best fit	Strength	Trade-off
One lakehouse	Training, proof of concept, small team	Fast setup and simple navigation	Weak layer isolation
Three lakehouses	Domain team with shared security boundary	Clear medallion ownership	Requires cross-lakehouse references
Multi-workspace	Regulated or multi-team enterprise	Strong isolation and blast-radius control	More governance and deployment work

Table 2.3: Workspace and lakehouse organization patterns for medallion scale.

2.5 Thursday: Hands-on Project

The Week 2 lab creates separate Bronze, Silver, and Gold lakehouses. You will upload a raw CSV to Bronze, prepare folder conventions, and run PySpark patterns that write Delta data to each layer. The lab is intentionally small: the goal is to practice medallion shape before adding orchestration and production data quality gates in later weeks.

2.5.1 Goal and Architecture

Create a workspace named `fabric-de-week02`. Add three lakehouses: `lh_bronze`, `lh_silver`, and `lh_gold`. Upload a raw CSV file to `lh_bronze` under `Files/bronze/raw_csv/finance_transactions/`. Then use a notebook attached to the lakehouses to write Bronze, Silver, and Gold Delta outputs.

Note

Fabric notebook path syntax can vary depending on attached lakehouses and workspace context. In a training tenant, first use the lakehouse explorer and notebook file browser to confirm paths before parameterizing production jobs.

2.5.2 Step-by-Step Actions

1. Open Microsoft Fabric and create a workspace named `fabric-de-week02`. Assign the expected training capacity.
2. Create lakehouses named `lh_bronze`, `lh_silver`, and `lh_gold`.
3. In `lh_bronze`, create folders `Files/bronze/raw_csv/finance_transactions/`, `Files/bronze/quarant` and `Files/bronze/metadata/`.
4. Upload a small raw CSV with columns such as `transaction_id`, `account_id`, `transaction_date`, `amount`, `currency`, and `source_system`.
5. In `lh_silver`, reserve table names such as `silver_finance_transactions` and `silver_account_daily`.
6. In `lh_gold`, reserve table names such as `gold_daily_cash_movement` and `gold_currency_exposure`.
7. Create a notebook named `nb_week02_medallion_setup`. Attach the three lakehouses if your Fabric experience supports multiple attachments; otherwise run one layer at a time with the appropriate default lakehouse.
8. Run the following PySpark patterns, adjusting paths for your tenant and sample file.

```

1 from pyspark.sql import functions as F
2
3 raw_path = "Files/bronze/raw_csv/finance_transactions/*.csv"
4

```

```

5 raw_df = (
6     spark.read
7     .option("header", "true")
8     .option("inferSchema", "false")
9     .csv(raw_path)
10 )
11
12 bronze_df = (
13     raw_df
14     .withColumn("ingested_at", F.current_timestamp())
15     .withColumn("source_file", F.input_file_name())
16 )
17
18 bronze_df.write.format("delta").mode("overwrite").saveAsTable(
19     "bronze_finance_transactions_raw"
20 )
21
22 silver_df = (
23     bronze_df
24     .select(
25         F.col("transaction_id").cast("string").alias("transaction_id"),
26         F.col("account_id").cast("string").alias("account_id"),
27         F.to_date("transaction_date").alias("transaction_date"),
28         F.col("amount").cast("decimal(18,2)").alias("amount"),
29         F.upper(F.col("currency")).alias("currency"),
30         F.col("source_system").cast("string").alias("source_system"),
31         "ingested_at",
32         "source_file",
33     )
34     .where(F.col("transaction_id").isNotNull())
35     .where(F.col("account_id").isNotNull())
36     .where(F.col("transaction_date").isNotNull())
37     .dropDuplicates(["transaction_id"])
38 )
39
40 silver_df.write.format("delta").mode("overwrite").saveAsTable(
41     "silver_finance_transactions"
42 )
43
44 gold_df = (
45     silver_df
46     .groupBy("transaction_date", "currency")
47     .agg(
48         F.count("*").alias("transaction_count"),
49         F.sum("amount").alias("net_amount"),
50         F.sum(F.when(F.col("amount") >= 0, F.col("amount")).otherwise(F.lit(0)))
51         .alias("credits"),
52         F.sum(F.when(F.col("amount") < 0, F.col("amount")).otherwise(F.lit(0)))
53         .alias("debits"),
54     )
55     .withColumn("gold_loaded_at", F.current_timestamp())
56 )
57
58 gold_df.write.format("delta").mode("overwrite").saveAsTable(
59     "gold_daily_cash_movement"
60 )

```

Listing 2.1: PySpark pattern for reading raw CSV and writing Delta outputs to Bronze, Silver, and Gold.

The next listing records folder and OneLake path conventions. Use it as a notebook cell, deployment note, or runbook fragment. It keeps names explicit before automation is introduced.


```

1 workspace = "fabric-de-week02"
2 lakehouses = {
3     "bronze": "lh_bronze",
4     "silver": "lh_silver",
5     "gold": "lh_gold",
6 }
7
8 path_conventions = [
9     "Files/bronze/raw_csv/finance_transactions/",
10    "Files/bronze/quarantine/",
11    "Files/bronze/metadata/",
12    "Tables/bronze_finance_transactions_raw",
13    "Tables/silver_finance_transactions",
14    "Tables/gold_daily_cash_movement",
15 ]
16
17 for layer, lakehouse in lakehouses.items():
18     print(f"{layer}: {workspace}/{lakehouse}")
19
20 for path in path_conventions:
21     print(path)

```

Listing 2.2: Folder and OneLake path convention helper for Week 2.

2.5.3 Optional Spark SQL Validation

After the writes complete, validate row counts and simple totals. These checks do not prove financial correctness, but they confirm that each layer exists and is queryable.

```

1 SELECT 'bronze' AS layer, COUNT(*) AS row_count
2 FROM bronze_finance_transactions_raw
3 UNION ALL
4 SELECT 'silver' AS layer, COUNT(*) AS row_count
5 FROM silver_finance_transactions
6 UNION ALL
7 SELECT 'gold' AS layer, COUNT(*) AS row_count
8 FROM gold_daily_cash_movement;

```

Listing 2.3: Spark SQL validation for Week 2 medallion tables.

2.5.4 Validation Checklist

- The workspace exists and contains `lh_bronze`, `lh_silver`, and `lh_gold`.
- A raw CSV is uploaded only to the Bronze raw folder.
- Bronze output preserves source fields plus ingestion metadata.
- Silver output casts types, filters invalid keys, removes duplicate transaction identifiers, and writes Delta.
- Gold output summarizes daily cash movement by date and currency.
- Folder conventions are documented without secrets or connection strings.

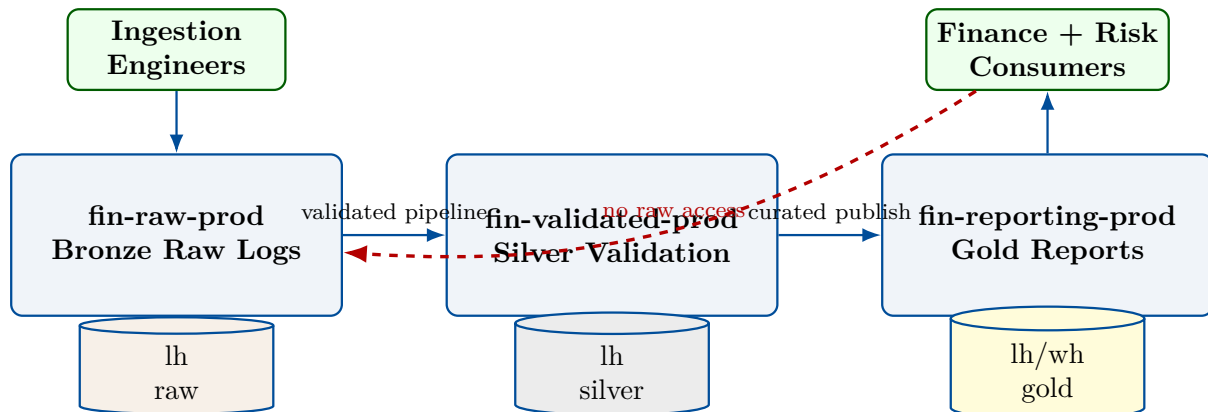
2.6 Friday: Use Case and Architecture

A financial institution ingests raw application logs, payment events, customer interaction events, and operational telemetry. The raw logs may contain sensitive identifiers, operational secrets, IP addresses, device identifiers, and unfiltered message text. Finance and risk teams need curated

reports about transaction volume, exception rates, settlement timeliness, and control failures. They must not receive broad access to raw logs.

2.6.1 Architecture Strategy

The design separates raw ingestion from curated reporting through workspaces. **fin-raw-prod** contains Bronze lakehouses and ingestion notebooks. **fin-validated-prod** contains Silver validation tables and quality outcomes. **fin-reporting-prod** contains Gold reporting tables, a warehouse or semantic model, and certified reports. Access flows forward through controlled jobs and item sharing; business users never receive direct access to the raw workspace.



Workspace isolation prevents report users from browsing raw logs while still allowing governed, forward-only medallion publication.

Figure 2.2: Multi-workspace isolation pattern for financial medallion architecture.

2.6.2 Design Decisions

Raw logs remain in the Bronze workspace with restricted permissions because they may contain sensitive or irrelevant operational detail. Silver jobs parse, classify, mask, deduplicate, and validate events. Gold tables expose only approved metrics, dimensions, and report-ready aggregates. The organization can certify Gold semantic models without granting report builders access to raw messages.

Concern	Isolation decision	Medallion impact
Raw log sensitivity	Restrict fin-raw-prod to ingestion operators and controlled service principals	Bronze stores evidence but is not broadly browsable
Validation ownership	Data engineering owns fin-validated-prod and publishes quality results	Silver becomes the trusted reuse layer
Reporting access	Finance and risk users access fin-reporting-prod only	Gold contains approved metrics and semantic models
Incident response	Raw workspace access can be audited separately	Investigations do not require report workspace permissions

Table 2.4: Financial institution decisions for strict medallion isolation.

2.7 Operational Guidance for Week 2

A medallion design becomes real only when ownership and controls match the diagrams. Every layer should have an owner, retention expectation, quality expectation, and consumer policy. A missing policy will eventually become a production incident: either raw data leaks too widely, Silver quality is assumed without evidence, or Gold reports drift from definitions.

2.7.1 Quality and Observability

Quality checks should become more strict as data moves forward. Bronze checks prove ingestion completeness. Silver checks prove validity and conformance. Gold checks prove business reconciliation. Fabric capacity monitoring also matters because inefficient transformations can consume shared capacity units [34].

```

1 from pyspark.sql import functions as F
2
3 required_columns = [
4     "transaction_id",
5     "account_id",
6     "transaction_date",
7     "amount",
8     "currency",
9 ]
10
11 missing_columns = sorted(set(required_columns) - set(silver_df.columns))
12 if missing_columns:
13     raise ValueError(f"Missing required columns: {missing_columns}")
14
15 quality_df = silver_df.select(
16     F.count("*").alias("rows"),
17     F.sum(F.col("transaction_id").isNull().cast("int")).alias("
18 null_transaction_ids"),
19     F.countDistinct("transaction_id").alias("distinct_transactions"),
20     F.sum(F.col("amount").isNull().cast("int")).alias("null_amounts"),
21 )
22 quality_df.show(truncate=False)

```

Listing 2.4: Small quality gate for Silver transaction data.

2.7.2 Security Checklist

- Limit Bronze workspace membership to ingestion operators and approved service principals.
- Store curated outputs in a separate workspace when raw data contains sensitive or unfiltered fields.
- Use item sharing, semantic model permissions, and workspace roles intentionally rather than relying on naming conventions.
- Remove raw message text, secrets, tokens, and unnecessary identifiers before Gold publication.
- Record lineage from Gold reports back to Silver validation tables and Bronze ingestion batches.
- Audit who can create shortcuts or copy raw folders into less restricted workspaces.

2.8 Interview Q&A

1. **Q: What problem does the medallion architecture solve?**

A: It separates source fidelity, engineering validation, and business consumption so teams know which data is raw, which data is trusted, and which data is certified for decisions.

2. **Q: What belongs in Bronze?**

A: Raw or minimally standardized source data, ingestion metadata, source file paths, batch identifiers, and enough evidence to replay or audit ingestion.

3. **Q: What makes Silver different from Bronze?**

A: Silver applies schema enforcement, type casting, deduplication, key standardization, privacy treatment, and quality checks.

4. **Q: What makes Gold different from Silver?**

A: Gold adds business definitions, aggregations, dimensional models, semantic models, performance tuning, and certification for consumption.

5. **Q: When would you choose a Fabric Warehouse for Gold?**

A: Choose it when the Gold product is SQL-first, relational, finance-oriented, or consumed by teams that expect controlled warehouse schemas.

6. **Q: Why might a financial institution split medallion layers across workspaces?**

A: Separate workspaces enforce stronger access boundaries, limit raw data exposure, reduce blast radius, and support audited promotion from raw logs to curated reports.

2.9 Further Reading

Review Microsoft’s Fabric overview [59], OneLake overview [60], lakehouse documentation [53], workspace guidance [61], domains [25], Direct Lake [24], and capacity metrics [34]. For lakehouse theory and storage reliability, read the medallion architecture overview [6], the Delta Lake paper [4], Delta Lake documentation [9], and the lakehouse architecture paper [5].

Key Takeaways

- Bronze preserves source fidelity and ingestion evidence; it should not be treated as business-ready data.
- Silver validates, conforms, deduplicates, types, and secures data so it can be reused safely.
- Gold publishes curated business products through lakehouse tables, warehouses, semantic models, and reports.
- Lakehouses are best for Spark-first, file-aware, open Delta-Parquet engineering; warehouses are best for SQL-first relational serving.
- Workspace and lakehouse boundaries should reflect ownership, security, environment promotion, and operational blast radius.
- Regulated scenarios often require multi-workspace isolation so raw logs and curated reports have different access models.
- Every medallion layer needs an owner, contract, quality expectation, and documented consumer policy.

2.10 Exercises

1. **(Conceptual)** Define Bronze, Silver, and Gold in your own words. Include one example table for each layer.
2. **(Conceptual)** Explain why preserving source fidelity in Bronze can be more important than cleaning data immediately.
3. **(Design)** Choose Lakehouse or Warehouse for a Gold monthly finance mart. Justify the decision using authoring style, consumers, and security.
4. **(Design)** Draft a workspace naming convention for development, test, and production medallion workspaces in a bank.
5. **(Hands-on)** Create the three Week 2 lakehouses and upload a sample CSV to the Bronze raw folder.
6. **(Hands-on)** Modify listing 2.1 to quarantine rows with missing `transaction_id` instead of dropping them silently.
7. **(Operational)** Write a runbook checklist for promoting a Silver table to Gold, including quality, owner approval, and access review.
8. **(Interview)** In four sentences, defend why raw financial logs should not be exposed directly to report authors.

2.11 Capstone Project: Financial Medallion with Workspace Isolation

Capstone Project

Mission. Build an end-to-end medallion architecture for a financial institution. The architecture must isolate raw ingested logs from curated financial reports by using separate Fabric workspaces and medallion lakehouses. Your implementation should prove that raw events can be landed, validated, curated, and reported without giving report consumers direct access to raw log folders.

Estimated time. 4–6 hours, depending on tenant permissions, available sample data, and whether your class uses separate development and production workspaces.

What you'll practice.

- Creating medallion workspaces for raw, validated, and reporting layers.
- Landing raw financial logs in a restricted Bronze lakehouse.
- Validating and masking data into a Silver Delta table.
- Publishing a Gold aggregate for finance and risk reporting.
- Documenting access boundaries, lineage, and acceptance criteria for a regulated data product.

Scenario

A financial institution operates digital banking, card processing, loan servicing, and fraud-monitoring platforms. Each platform emits operational logs and transaction events. Raw files may include account identifiers, user agents, IP addresses, exception text, internal service names, and operational metadata. Finance and risk teams need curated metrics about transaction volume, rejected payments, settlement timeliness, high-risk exception categories,

and daily exposure. They must not browse raw logs because raw events may contain sensitive or irrelevant operational details.

Your job is to design and implement the smallest production-shaped medallion that proves separation of duties. The ingestion team owns raw landing. The data engineering team owns validation. Finance owns curated reporting definitions. Access must flow forward through governed transformations, not through broad workspace membership.

Requirements

Build a working Fabric design with these minimum elements:

- A restricted workspace named `fin-raw-dev` or `fin-raw-prod` with a Bronze lakehouse named `lh_fin_bronze`.
- A validation workspace named `fin-validated-dev` or `fin-validated-prod` with a Silver lakehouse named `lh_fin_silver`.
- A reporting workspace named `fin-reporting-dev` or `fin-reporting-prod` with a Gold lakehouse or warehouse named `lh_fin_gold` or `wh_fin_gold`.
- A raw folder path `Files/bronze/raw_logs/payment_events/` that contains sample CSV or JSON events.
- A Silver Delta table named `silver_payment_events` with typed fields, duplicate removal, and sensitive-field treatment.
- A Gold table named `gold_daily_payment_controls` with daily metrics suitable for finance and risk reports.
- Documentation proving that report consumers need access only to the reporting workspace or semantic model.
- No secrets, keys, tokens, connection strings, or real personal data in notebooks, screenshots, or exported files.

Reference Architecture

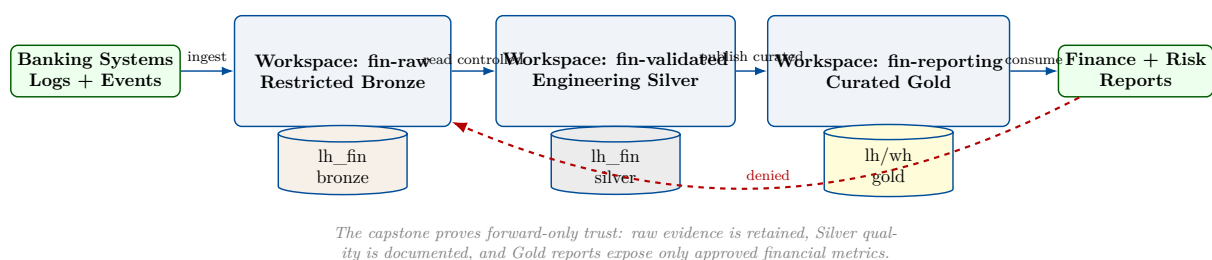


Figure 2.3: Reference architecture for the financial medallion capstone.

Implementation Steps

1. **Create workspaces.** Create `fin-raw-dev`, `fin-validated-dev`, and `fin-reporting-dev`. If your tenant permits only one workspace, simulate the boundary with three lakehouses and document the production workspace split.
2. **Create lakehouses.** Add `lh_fin_bronze`, `lh_fin_silver`, and `lh_fin_gold`. Use a warehouse for Gold if your team chooses SQL-first reporting.

3. **Create raw folders.** In Bronze, create Files/bronze/raw_logs/payment_events/, Files/bronze/quarantine/, and Files/bronze/metadata/.
4. **Upload sample events.** Use instructor-provided synthetic logs or create a small CSV with payment event identifiers, timestamps, account identifiers, amount, currency, status, exception category, and channel.
5. **Write Bronze and Silver.** Run the first notebook listing to preserve raw ingestion metadata and create `silver_payment_events`.
6. **Write Gold.** Run the second listing to create `gold_daily_payment_controls` for reporting.
7. **Document access.** Record which personas can access each workspace and explain why report consumers do not need raw workspace membership.
8. **Validate lineage.** Capture row counts for raw, Silver, and Gold outputs and explain any rejected or quarantined records.

```

1 from pyspark.sql import functions as F
2
3 raw_path = "Files/bronze/raw_logs/payment_events/*.csv"
4
5 raw_events_df = (
6     spark.read
7     .option("header", "true")
8     .option("inferSchema", "false")
9     .csv(raw_path)
10 )
11
12 bronze_events_df = (
13     raw_events_df
14     .withColumn("ingested_at", F.current_timestamp())
15     .withColumn("source_file", F.input_file_name())
16 )
17
18 bronze_events_df.write.format("delta").mode("overwrite").saveAsTable(
19     "bronze_payment_events_raw"
20 )
21
22 silver_payment_events_df = (
23     bronze_events_df
24     .select(
25         F.col("event_id").cast("string").alias("event_id"),
26         F.col("payment_id").cast("string").alias("payment_id"),
27         F.sha2(F.col("account_id").cast("string"), 256).alias("account_hash"),
28         F.to_timestamp("event_timestamp").alias("event_timestamp"),
29         F.to_date("event_timestamp").alias("event_date"),
30         F.col("amount").cast("decimal(18,2)").alias("amount"),
31         F.upper(F.col("currency")).alias("currency"),
32         F.upper(F.col("status")).alias("status"),
33         F.col("exception_category").cast("string").alias("exception_category"),
34         F.col("channel").cast("string").alias("channel"),
35         "ingested_at",
36         "source_file",
37     )
38     .where(F.col("event_id").isNotNull())
39     .where(F.col("payment_id").isNotNull())
40     .where(F.col("event_timestamp").isNotNull())
41     .dropDuplicates(["event_id"])
42 )
43
44 silver_payment_events_df.write.format("delta").mode("overwrite").saveAsTable(

```

```

45     "silver_payment_events"
46 )

```

Listing 2.5: Capstone Bronze-to-Silver PySpark transformation with sensitive-field treatment.

```

1  from pyspark.sql import functions as F
2
3  gold_daily_controls_df = (
4      silver_payment_events_df
5      .groupBy("event_date", "currency", "channel")
6      .agg(
7          F.count("*").alias("event_count"),
8          F.countDistinct("payment_id").alias("payment_count"),
9          F.sum("amount").alias("gross_amount"),
10         F.sum(F.when(F.col("status") == "REJECTED", 1).otherwise(0)).alias("
rejected_count"),
11         F.sum(F.when(F.col("exception_category").isNotNull(), 1).otherwise(0)).
alias("exception_count"),
12     )
13     .withColumn(
14         "rejection_rate",
15         F.col("rejected_count") / F.when(F.col("event_count") == 0, None).
otherwise(F.col("event_count")),
16     )
17     .withColumn("gold_loaded_at", F.current_timestamp())
18 )
19
20 gold_daily_controls_df.write.format("delta").mode("overwrite").saveAsTable(
21     "gold_daily_payment_controls"
22 )
23
24 gold_daily_controls_df.orderBy("event_date", "currency", "channel").show(50,
truncate=False)

```

Listing 2.6: Capstone Silver-to-Gold PySpark aggregate for financial controls.

If Gold is implemented as a warehouse table, use a SQL validation query after loading or shortcutting the curated data into the SQL-serving layer.

```

1  SELECT
2      event_date,
3      currency,
4      SUM(event_count) AS event_count,
5      SUM(rejected_count) AS rejected_count,
6      SUM(exception_count) AS exception_count
7  FROM gold_daily_payment_controls
8  GROUP BY event_date, currency
9  ORDER BY event_date, currency;

```

Listing 2.7: Capstone SQL validation for curated financial control metrics.

Deliverables

Submit evidence that the architecture and implementation are complete:

- Workspace names, lakehouse or warehouse names, and capacity assignment notes.
- Screenshot or exported evidence of the Bronze raw folder and the Silver and Gold tables.
- Notebook output showing row counts for Bronze, Silver, and Gold.
- Schema for `silver_payment_events`, including which sensitive fields were hashed, removed, or retained.

- Schema for `gold_daily_payment_controls`, including metric definitions.
- Access matrix listing ingestion engineers, data engineers, finance report authors, risk consumers, and administrators.
- A short explanation of why report consumers do not need access to `fin-raw-dev` or `fin-raw-prod`.

Acceptance Criteria

Use this rubric before declaring the capstone complete:

- ☐ Raw files are stored only in the Bronze raw path.
- ☐ Bronze data includes ingestion metadata such as load timestamp and source file.
- ☐ Silver data casts types, removes duplicate event identifiers, and handles sensitive account identifiers.
- ☐ Gold data contains only curated metrics required for finance and risk reporting.
- ☐ Report consumers can use Gold outputs without direct raw workspace access.
- ☐ The access matrix shows different personas for raw ingestion, validation, and reporting.
- ☐ Row counts are reconciled or rejected records are explained.
- ☐ No secrets, credentials, tokens, real account identifiers, or unnecessary personal data appear in submitted artifacts.
- ☐ The final architecture explains how data moves forward from Bronze to Silver to Gold.
- ☐ The design states whether Gold is served from a Lakehouse, Warehouse, semantic model, or a combination.

Stretch Goals

If you finish early, extend the capstone while preserving strict isolation:

- Add a quarantine table for invalid payment events and report rejection reasons by source file.
- Add a reference table for currency validation and reject unsupported currency codes.
- Build a small Power BI semantic model over `gold_daily_payment_controls` and document whether Direct Lake is appropriate.
- Split development and production into separate workspace sets and describe a deployment-pipeline promotion path.
- Add sensitivity labels or item-level notes that identify raw, validated, and curated assets.
- Add a reconciliation check that compares total Bronze payment identifiers with Silver accepted plus quarantined identifiers.

Apache Spark in Fabric

Learning Objectives

- Explain how Microsoft Fabric provides managed Apache Spark compute through Starter pools and Custom pools.
- Configure Spark environments so notebooks use consistent libraries, runtime versions, and session settings.
- Use Fabric Notebooks to write PySpark that reads and writes files in OneLake.
- Combine PySpark DataFrame logic and Spark SQL in the same notebook for practical lakehouse engineering.
- Build a Bronze-to-Silver transformation that reads raw CSV, removes duplicates, drops invalid null records, and writes Delta tables.
- Design a scalable Spark notebook ingestion layer for telemetry systems that process 100 GB of JSON logs daily.
- Implement a capstone telemetry pipeline from Bronze JSON to cleaned Silver Delta with operational checks.

Cross-Cloud Equivalence

Managed Spark exists on every major analytical cloud, but the operating model differs. Fabric hides much of the cluster administration behind pools, lakehouses, environments, and notebooks; other platforms expose more choices around clusters, jobs, and object storage.

Concept	Fabric Spark	AWS	Azure	GCP	
Cluster or pool model	Starter pools for fast managed sessions; Custom pools for controlled node family, size, and scale settings	AWS Glue jobs, EMR clusters, and EMR Serverless applications	Synapse Spark pools and Azure Databricks clusters or serverless compute	Dataprocs, Dataprocs batches	clusters and Serverless
Autoscaling	Pool-level autoscale and dynamic Spark allocation are managed through Fabric capacity and pool settings	Glue workers scale by job configuration; EMR supports managed scaling; EMR Serverless scales per application	Synapse pools and Databricks support autoscale settings by pool or cluster	Dataprocs autoscaling policies and serverless automatic resource allocation	
Notebook experience	Fabric Notebooks attach to lakehouses, use Spark sessions, support PySpark, Spark SQL, and collaborative authoring	Glue Studio notebooks, EMR notebooks, SageMaker Studio, or managed Jupyter patterns	Synapse notebooks and Databricks notebooks with rich job integration	Vertex AI, Dataprocs, and notebook-connected batches	Workbench, Jupyter, and connected
Delta and lakehouse integration	Native Fabric Lakehouse tables in OneLake use Delta-Parquet and integrate with SQL analytics endpoints and Direct Lake	Delta Lake on S3 through EMR, Glue, or Databricks; catalog integration depends on Glue Data Catalog or Unity Catalog	Delta Lake through Synapse or Databricks on ADLS; strong lakehouse integration in Databricks	Delta Lake on Cloud Storage through Dataprocs or Databricks on GCP	
Serverless option	Starter pools feel serverless to notebook authors; Fabric capacity still governs consumption	AWS Glue and EMR Serverless	Synapse serverless patterns for SQL, and Databricks serverless where available	Dataprocs for Spark batches	Serverless batches
Snowflake approaches distributed data engineering through Snowpark rather than a user-managed Spark pool. MongoDB commonly participates through the MongoDB Spark Connector, where Spark reads operational collections and writes curated analytical outputs to a lakehouse or warehouse.					

Key Terms

Driver: the Spark process that plans work and coordinates executors. **Executor:** a worker process that runs tasks and stores shuffle or cached data. **Starter pool:** Fabric-managed Spark compute intended for quick notebook startup and low-friction development. **Custom pool:** a Fabric Spark pool with explicit sizing and scale settings. **Spark environment:** a reusable Fabric item that pins runtime, libraries, and Spark properties.

3.1 Week 3 Roadmap

Week 3 moves from storage architecture into distributed processing. Monday introduces Spark compute in Fabric, including Starter pools, Custom pools, and Spark environments. Tuesday focuses on Fabric Notebooks, PySpark basics, and reading or writing files in OneLake. Wednesday adds Spark SQL and shows how to combine SQL cells with PySpark DataFrames in the same notebook. Thursday is a hands-on transformation from raw Bronze CSV to cleaned Silver Delta tables. Friday applies the pattern to a telemetry platform that must process 100 GB of JSON logs every day.

Apache Spark is the distributed execution engine behind many modern lakehouse workloads. It splits large file scans and transformations into tasks, runs them across executors, and writes outputs in formats such as Delta and Parquet. Fabric packages this capability as a managed

notebook and lakehouse experience so data engineers can focus on ingestion, validation, and table contracts rather than cluster plumbing [3, 11].

Note

Spark is not required for every Fabric workload. Use it when the data shape, volume, file format, transformation logic, or semi-structured parsing requirement benefits from distributed processing. For small relational transformations, Warehouse SQL or Dataflows Gen2 may be simpler.

3.2 Monday: Spark Compute in Fabric

Fabric Spark compute is organized around managed sessions. A notebook requests a Spark session from a pool, the session starts with a runtime and environment, and Spark creates a driver plus executors to process work. The user experience is intentionally simpler than administering open-source clusters, but the engineering decisions still matter: startup latency, concurrency, scale limits, library versions, and capacity consumption.

Definition

Apache Spark in Fabric is a managed Spark service for notebooks, lakehouse transformations, machine learning preparation, and file-scale data engineering. Spark sessions run on Fabric capacity, read and write OneLake, and commonly store trusted outputs as Delta tables.

3.2.1 Starter Pools

Starter pools are the fastest way to begin Spark development in Fabric. They are managed for convenience and optimized for quick session startup. A training notebook, exploratory data profiling task, or small Bronze-to-Silver prototype is a good fit when the default runtime and general sizing are enough.

Starter pools reduce friction. New engineers can open a notebook, attach a lakehouse, run a PySpark cell, and inspect results without waiting for a separately administered cluster. The trade-off is control: production workloads may need more explicit sizing, library pinning, and concurrency planning than a starter configuration provides.

Tip

Use Starter pools while discovering schemas, writing the first notebook cells, and teaching the programming model. Move repeatable production jobs to a controlled pool and environment once workload size, dependencies, and service-level expectations are known.

3.2.2 Custom Pools

Custom pools give teams more control over Spark resources. A Custom pool can specify node families, node sizes, autoscale boundaries, and other settings that affect throughput and cost. The goal is not to create the largest possible cluster; the goal is to provide enough parallelism for the workload while keeping predictable capacity consumption.

Custom pools are useful when notebooks become scheduled jobs, when multiple pipelines compete for the same capacity, or when a large transformation repeatedly reads many files. For the telemetry scenario later in this chapter, a Custom pool is easier to reason about because daily data volume, expected duration, and concurrency can be estimated.

Common Pitfall

Treating pool size as the only performance lever. Larger pools may hide inefficient code for a while, but file layout, partition sizing, schema inference, skew, shuffle volume, and Delta write patterns often matter more than raw executor count.

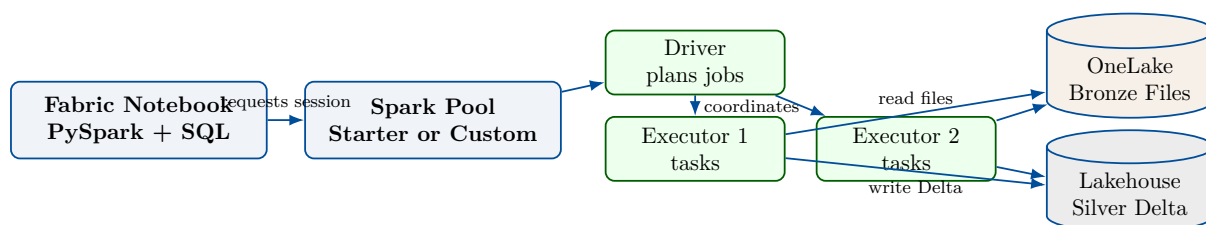
3.2.3 Spark Environments

Spark environments package runtime choices, libraries, and Spark properties so notebooks start consistently [16]. Without an environment, two notebooks can drift: one may install a library interactively, another may use a different runtime, and a scheduled job may fail because a dependency was not available. Environments make notebook behavior repeatable.

A practical environment should specify only what the workload needs. Common examples include JSON parsing helpers, validation packages approved by the organization, or Spark properties for shuffle partitions. Avoid turning the environment into a dumping ground for every library used by every team.

Choice	Best fit	Engineering concern
Starter pool	Training, exploration, short notebooks, early prototypes	Limited control and less workload-specific sizing
Custom pool	Scheduled transformations, larger files, predictable operations	Requires sizing, autoscale, and capacity planning
Spark environment	Repeatable notebooks and dependency governance	Must be versioned and kept small enough to maintain

Table 3.1: Compute choices for Spark development in Fabric.



Fabric manages session startup and pool resources, while Spark still uses the standard driver-and-executor execution model over OneLake files and Delta tables.

Figure 3.1: Spark driver and executors on a Fabric pool reading OneLake and writing Delta.

3.3 Tuesday: Fabric Notebooks and PySpark Basics

Fabric Notebooks are interactive development items for Spark workloads. A notebook can attach to a lakehouse, run PySpark cells, use Spark SQL, display DataFrames, and be scheduled or orchestrated later. The notebook is both a development surface and a repeatable job artifact, so it should be written with production habits even during learning [27].

3.3.1 The PySpark DataFrame Model

PySpark code usually describes transformations before Spark executes them. Reading a file creates a DataFrame. Selecting columns, casting types, filtering rows, and joining tables create

a logical plan. Spark executes the plan when an action such as `count()`, `show()`, `write`, or `collect()` is called.

This lazy execution model is powerful because Spark can optimize the plan. It can also surprise beginners. A cell that defines ten transformations may appear to succeed even when the source path is wrong; the error appears only when an action triggers a scan.

Definition

A **DataFrame** is a distributed table-like abstraction with named columns and a schema. In PySpark, DataFrames are transformed with methods such as `select`, `where`, `groupBy`, and `join`; Spark translates those operations into distributed execution.

3.3.2 Reading Files from OneLake

A Fabric notebook attached to a lakehouse can read relative paths under `Files/`. For example, raw CSV might live under `Files/bronze/raw_csv/orders/`, and raw JSON telemetry might live under `Files/bronze/raw_json/telemetry/`. In production, use explicit path conventions, avoid hardcoded personal workspace names, and validate that files exist before starting expensive transformations.

Tip

For CSV, specify whether headers exist and decide whether schema inference is acceptable. For large or scheduled jobs, prefer explicit schemas so a single malformed file cannot silently change column types.

3.3.3 Writing Delta Tables

Fabric lakehouse tables are commonly written as Delta tables. A DataFrame write using `format("delta")` and `saveAsTable(...)` creates a table that can be read by Spark and exposed through the lakehouse experience. Delta transactions provide a safer table abstraction than unmanaged folders because readers see consistent versions and writers record metadata in the transaction log [4, 9].

Common Pitfall

Using `overwrite` casually. `mode("overwrite")` is convenient in labs, but production jobs should specify overwrite scope carefully, preserve history when required, and avoid replacing an entire table because one daily batch arrived late.

3.4 Wednesday: Spark SQL and Hybrid Notebooks

Spark SQL lets engineers express transformations in SQL while Spark executes them on the same distributed engine. In Fabric notebooks, a team can use PySpark for file parsing and schema logic, then use SQL for validation queries, aggregates, or analyst-friendly transformations. This hybrid style is practical because data engineering teams often include both Python-first and SQL-first contributors.

3.4.1 Temporary Views

A PySpark DataFrame can be registered as a temporary view with `createOrReplaceTempView`. After that, Spark SQL can query it in the same session. Temporary views are not durable lakehouse tables; they are session-scoped names that help organize notebook logic.

Note

Use temporary views to bridge cells, not as production storage. Durable outputs should be written as Delta tables with clear names, owners, and refresh expectations.

3.4.2 Choosing DataFrame or SQL Syntax

DataFrame syntax works well when logic is programmatic: dynamic column lists, loops over quality rules, reusable functions, or complex parsing. SQL syntax works well for joins, aggregates, window functions, and validation queries that analysts can read. Both compile to Spark plans, so the choice is often about maintainability.

Need	Prefer PySpark	Prefer Spark SQL
File parsing	Explicit schemas, file metadata, dynamic paths	Simple external table queries after files are tabled
Data quality	Programmatic checks across many columns	Clear count and exception queries
Aggregations	Reusable functions and conditional expressions	Grouping, joins, and window calculations readable by analysts
Notebook collaboration	Python-first engineering team	SQL-first analytics team

Table 3.2: Choosing PySpark or Spark SQL inside Fabric notebooks.

3.5 Thursday: Hands-on Project

The Week 3 lab implements a small but production-shaped Silver transformation. A raw CSV arrives in the Bronze Lakehouse. The notebook reads the file, captures ingestion metadata, removes duplicate business keys, drops rows missing required fields, and writes the accepted records to the Silver Lakehouse as a Delta table. The same notebook then mixes PySpark with Spark SQL for validation and summary checks.

3.5.1 Goal and Architecture

Create or reuse a workspace named `fabric-de-week03`. Add lakehouses named `lh_bronze` and `lh_silver`. Upload raw CSV files to `lh_bronze` under `Files/bronze/raw_csv/telemetry_devices/`. Create a notebook named `nb_week03_spark_csv_to_silver` and attach the required lakehouses.

Note

If your tenant permits only one default lakehouse per notebook, first run the lab in one lakehouse with explicit layer prefixes. Then repeat in a multi-lakehouse setup when attachments and shortcuts are available.

3.5.2 Step-by-Step Actions

1. Create `fabric-de-week03`, `lh_bronze`, and `lh_silver` if they do not already exist.
2. In Bronze, create `Files/bronze/raw_csv/telemetry_devices/` and upload a CSV with columns such as `event_id`, `device_id`, `event_timestamp`, `temperature_c`, `status`, and `source_system`.

3. Create a notebook attached to the Bronze and Silver lakehouses, or attach one lakehouse and adjust paths for your training tenant.
4. Run the PySpark transformation in listing 3.1.
5. Run the hybrid Spark SQL validation in listing 3.2.
6. Confirm that `silver_telemetry_device_events` appears as a Delta table and that duplicate and null records are excluded.

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.types import StructType, StructField, StringType
3
4 raw_schema = StructType([
5     StructField("event_id", StringType(), True),
6     StructField("device_id", StringType(), True),
7     StructField("event_timestamp", StringType(), True),
8     StructField("temperature_c", StringType(), True),
9     StructField("status", StringType(), True),
10    StructField("source_system", StringType(), True),
11 ])
12
13 raw_path = "Files/bronze/raw_csv/telemetry_devices/*.csv"
14
15 raw_df = (
16     spark.read
17     .option("header", "true")
18     .schema(raw_schema)
19     .csv(raw_path)
20 )
21
22 bronze_with_metadata_df = (
23     raw_df
24     .withColumn("ingested_at", F.current_timestamp())
25     .withColumn("source_file", F.input_file_name())
26 )
27
28 required_columns = ["event_id", "device_id", "event_timestamp"]
29
30 silver_df = (
31     bronze_with_metadata_df
32     .dropDuplicates(["event_id"])
33     .dropna(subset=required_columns)
34     .select(
35         F.col("event_id").cast("string").alias("event_id"),
36         F.col("device_id").cast("string").alias("device_id"),
37         F.to_timestamp("event_timestamp").alias("event_timestamp"),
38         F.to_date("event_timestamp").alias("event_date"),
39         F.col("temperature_c").cast("double").alias("temperature_c"),
40         F.upper(F.col("status")).alias("status"),
41         F.col("source_system").cast("string").alias("source_system"),
42         "ingested_at",
43         "source_file",
44     )
45     .where(F.col("event_timestamp").isNotNull())
46 )
47
48 silver_df.write.format("delta").mode("overwrite").saveAsTable(
49     "silver_telemetry_device_events"
50 )

```

Listing 3.1: PySpark Bronze CSV to cleaned Silver Delta table.

The second listing registers the Silver DataFrame as a temporary view and uses Spark SQL

from Python. This is useful when a notebook cell needs to calculate validation metrics or feed a SQL-first aggregate without leaving the PySpark session.

```

1 silver_df.createOrReplaceTempView("v_silver_telemetry_device_events")
2
3 quality_summary_df = spark.sql("""
4 SELECT
5     COUNT(*) AS accepted_rows,
6     COUNT(DISTINCT event_id) AS distinct_events,
7     SUM(CASE WHEN temperature_c IS NULL THEN 1 ELSE 0 END) AS
8     missing_temperature_rows,
9     MIN(event_timestamp) AS first_event_timestamp,
10    MAX(event_timestamp) AS last_event_timestamp
11 FROM v_silver_telemetry_device_events
12 """)
13 quality_summary_df.show(truncate=False)
14
15 daily_status_df = spark.sql("""
16 SELECT
17     event_date,
18     status,
19     COUNT(*) AS event_count,
20     AVG(temperature_c) AS average_temperature_c
21 FROM v_silver_telemetry_device_events
22 GROUP BY event_date, status
23 ORDER BY event_date, status
24 """)
25
26 daily_status_df.write.format("delta").mode("overwrite").saveAsTable(
27     "silver_telemetry_daily_status_summary"
28 )

```

Listing 3.2: Mixing Spark SQL with PySpark in the same notebook.

3.5.3 Validation Checklist

- The notebook uses an explicit schema instead of relying on CSV inference.
- Duplicate `event_id` values are removed before the Silver table is written.
- Records with null `event_id`, `device_id`, or `event_timestamp` are excluded from Silver.
- The Silver table is written in Delta format and appears in the lakehouse table list.
- A Spark SQL query validates row counts, distinct keys, and basic time range.
- No secret values, personal workspace identifiers, or real device identifiers are hardcoded in the notebook.

3.6 Friday: Use Case and Architecture

A telemetry platform receives approximately 100 GB of JSON logs per day from applications, devices, gateways, and operational services. The raw stream is valuable for debugging and audit, but analytics teams need reliable Silver Delta tables with typed timestamps, normalized severities, required identifiers, and duplicate handling. Spark Notebooks are a strong fit because JSON parsing, nested fields, and large daily files benefit from distributed processing.

3.6.1 Architecture Strategy

The design lands raw JSON in a Bronze Lakehouse under date-partitioned folders, then runs a scheduled Spark Notebook that reads one processing date, applies an explicit schema, drops malformed or incomplete records, deduplicates event identifiers, and writes Silver Delta partitioned by event date. The pool should be sized from observed throughput, not guessed permanently. Start with a Custom pool for predictable daily processing, measure duration, then tune file sizes, shuffle partitions, and schedule windows.

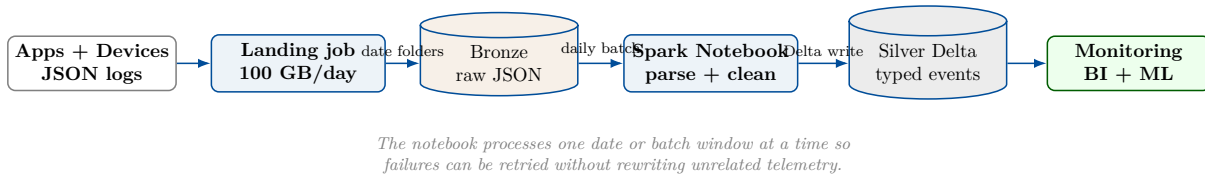


Figure 3.2: Spark notebook ingestion pipeline for 100 GB per day of telemetry JSON.

3.6.2 Design Decisions

Date folders make incremental processing observable. Explicit JSON schemas reduce expensive inference and prevent schema drift from surprising downstream tables. Silver Delta tables provide durable, queryable outputs for BI, alerting, and machine learning features. Quality metrics should be written after every run so operators know how many raw records were accepted, rejected, duplicated, or malformed.

Concern	Decision	Reason
Daily volume	Process by date or batch window	Limits retry scope and supports predictable scheduling
JSON parsing	Use explicit schema with a corrupt-record strategy	Avoids inference cost and captures malformed data
Compute	Use a Custom pool after prototype validation	Provides repeatable sizing and autoscale boundaries
Storage	Write Silver as Delta partitioned by event date	Improves query pruning and table reliability
Operations	Record quality metrics per run	Makes acceptance, rejection, and duplicate rates visible

Table 3.3: Design decisions for a 100 GB/day telemetry ingestion layer.

3.7 Operational Guidance for Week 3

Spark operations should focus on repeatability and evidence. Every scheduled notebook should identify its processing window, input path, output table, pool, environment, and quality metrics. A failed daily load should be retryable without guessing which files were read or which table partitions were overwritten.

3.7.1 Performance and Reliability

Avoid using `inferSchema` for large CSV or JSON jobs. Avoid tiny files when possible because thousands of small files can create scheduling overhead. Prefer explicit schemas, partition-aware reads, and deterministic output paths. Monitor duration, executor utilization, data skew, and failed tasks before increasing pool size.

Tip

Tune with measurements. Capture input bytes, input file count, accepted rows, rejected rows, duplicate count, write duration, and pool used. These numbers make capacity conversations factual.

3.7.2 Security Checklist

- Store raw telemetry in Bronze with least-privilege workspace access.
- Remove tokens, secrets, request headers, and unnecessary payload text before Silver publication.
- Do not print full raw JSON records in notebook output.
- Pin required libraries in a Spark environment rather than installing packages interactively in production cells.
- Use parameterized processing dates and avoid hardcoded personal paths.
- Keep Silver Delta tables documented with owner, refresh cadence, schema, and quality expectations.

3.8 Interview Q&A

1. **Q: What is the difference between a Starter pool and a Custom pool in Fabric Spark?**
A: A Starter pool is optimized for quick, managed notebook startup with minimal configuration. A Custom pool gives teams explicit control over sizing and scale settings for repeatable workloads.
2. **Q: Why are Spark environments important?**
A: They make notebooks repeatable by pinning runtime choices, libraries, and Spark properties instead of relying on interactive installs or hidden session state.
3. **Q: When should you avoid schema inference?**
A: Avoid it for large or scheduled jobs because it adds scan cost and can let unexpected files change inferred column types.
4. **Q: How can PySpark and Spark SQL work together?**
A: PySpark can create a DataFrame and register it as a temporary view; Spark SQL can then query that view in the same session.
5. **Q: Why write Silver outputs as Delta tables?**
A: Delta tables provide transactional metadata, consistent reads, schema management options, and strong integration with Fabric lakehouses.
6. **Q: What is the safest retry unit for a daily telemetry pipeline?**
A: A specific processing date or batch window, because the job can reread the same Bronze files and replace only the corresponding Silver output scope.

3.9 Further Reading

Review Microsoft documentation for Fabric Spark compute [11], Fabric Notebooks [27], Spark environments [16], Fabric Lakehouses [53], and OneLake [60]. For distributed execution and

table reliability, read the Apache Spark overview [3], Delta Lake documentation [9], and the Delta Lake paper [4].

Key Takeaways

- Fabric Spark provides managed distributed processing through notebook sessions that run on Starter or Custom pools.
- Starter pools are convenient for learning and exploration; Custom pools are better for repeatable production workloads.
- Spark environments keep notebooks consistent by controlling runtime, dependencies, and Spark properties.
- PySpark DataFrames are lazily evaluated, so errors often appear when an action reads, displays, or writes data.
- Fabric notebooks can combine PySpark and Spark SQL through temporary views and shared Spark sessions.
- Bronze-to-Silver Spark jobs should use explicit schemas, deterministic duplicate rules, null handling, and Delta writes.
- A 100 GB/day telemetry ingestion layer should process bounded windows, record quality metrics, and tune from observed performance.

3.10 Exercises

1. **(Conceptual)** Explain the driver and executor roles in a Spark application running on a Fabric pool.
2. **(Conceptual)** Compare Starter pools and Custom pools for a training notebook, a scheduled daily job, and an urgent debugging notebook.
3. **(Design)** Define a Spark environment for a telemetry ingestion team. Include runtime, approved libraries, and one Spark property you would discuss before changing.
4. **(Hands-on)** Create a Fabric notebook that reads a CSV from `Files/bronze/raw_csv/` and displays the schema without writing a table.
5. **(Hands-on)** Modify listing 3.1 to write rejected rows with missing required fields to a quarantine Delta table.
6. **(SQL)** Register a DataFrame as a temporary view and write a Spark SQL query that counts records by processing date and status.
7. **(Operational)** Draft a runbook for rerunning a failed daily telemetry batch without corrupting other dates.
8. **(Interview)** In five sentences, explain why explicit schemas matter for 100 GB/day JSON processing.

3.11 Capstone Project: Spark-Based Telemetry Ingestion at 100 GB per Day

Capstone Project

Mission. Build a Spark-based telemetry ingestion pipeline in Microsoft Fabric. The pipeline must read raw JSON logs from a Bronze Lakehouse, clean and deduplicate the records, drop invalid null-key rows, and write typed Silver Delta tables that can support a 100 GB/day operating target.

Estimated time. 4–6 hours, depending on tenant permissions, available sample data, and whether the class simulates 100 GB with smaller generated files.

What you'll practice.

- Designing a bounded daily Spark processing window.
- Reading nested JSON from OneLake with an explicit schema.
- Applying duplicate and null handling before Silver publication.
- Writing partitioned Delta tables for queryable telemetry.
- Recording run metrics that prove operational readiness.

Scenario

A software company operates web applications, device gateways, and background services. Each component emits JSON telemetry containing event identifiers, device identifiers, timestamps, severity, service name, region, latency, and optional diagnostic payloads. The platform lands about 100 GB per day in Bronze. Engineering, reliability, BI, and machine learning teams need a cleaned Silver table for trend analysis, incident review, anomaly detection, and service-level reporting.

Your job is to build the first production-shaped notebook. It does not need to ingest a real 100 GB during training, but the design must show how it would process one daily folder, validate records, remove duplicates, and write partitioned Delta output without exposing raw payloads unnecessarily.

Requirements

Build a working Fabric design with these minimum elements:

- A workspace named `telemetry-de-dev` with a Bronze lakehouse `lh_telemetry_bronze` and a Silver lakehouse `lh_telemetry_silver`, or one training lakehouse with clearly separated layer paths.
- A raw JSON path such as `Files/bronze/raw_json/telemetry/event_date=2026-07-31/`.
- A notebook named `nb_telemetry_bronze_to_silver` that accepts a processing date variable.
- An explicit JSON schema with required fields `event_id`, `device_id`, and `event_timestamp`.
- Duplicate removal by `event_id` and null dropping for required keys.
- A Silver Delta table named `silver_telemetry_events` partitioned by `event_date`.
- A run metrics table named `silver_telemetry_ingestion_metrics`.
- No secrets, tokens, full request headers, or real customer identifiers in notebook output or deliverables.

Reference Architecture

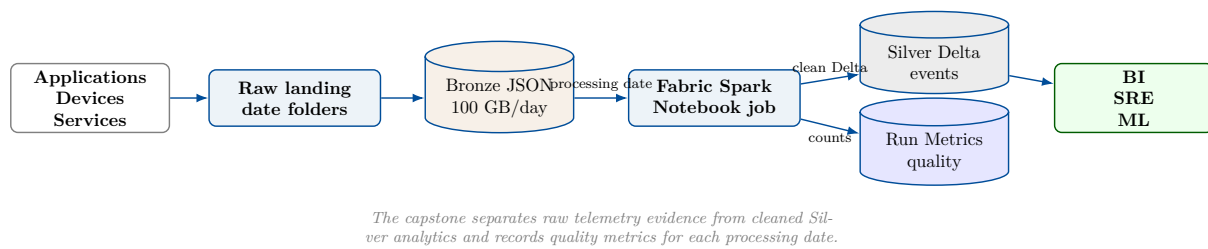


Figure 3.3: Reference architecture for the Spark telemetry ingestion capstone.

Implementation Steps

1. **Create lakehouses.** Create `lh_telemetry_bronze` and `lh_telemetry_silver`. If your training tenant supports one lakehouse only, create paths and table prefixes that preserve the same boundary.
2. **Prepare raw JSON.** Upload instructor-provided synthetic telemetry JSON to `Files/bronze/raw_json`. Use smaller files for training if needed.
3. **Configure compute.** Start with a Starter pool for the first run. Document the Custom pool settings you would use for scheduled 100 GB/day processing.
4. **Create an environment.** Pin approved runtime and library choices in a Spark environment when your tenant allows it.
5. **Run Bronze to Silver.** Execute listing 3.3 for one processing date.
6. **Record metrics.** Execute listing 3.4 to write run-level quality metrics.
7. **Validate output.** Query the Silver table by `event_date`, `severity`, and `service_name`.
8. **Document operations.** Describe retry behavior, expected schedule, pool choice, and how rejected records would be investigated.

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.types import (
3     StructType, StructField, StringType, DoubleType, LongType
4 )
5
6 processing_date = "2026-07-31"
7 raw_path = f"Files/bronze/raw_json/telemetry/event_date={processing_date}/*.
8     json"
9
10 telemetry_schema = StructType([
11     StructField("event_id", StringType(), True),
12     StructField("device_id", StringType(), True),
13     StructField("event_timestamp", StringType(), True),
14     StructField("severity", StringType(), True),
15     StructField("service_name", StringType(), True),
16     StructField("region", StringType(), True),
17     StructField("latency_ms", DoubleType(), True),
18     StructField("bytes_in", LongType(), True),
19     StructField("bytes_out", LongType(), True),
20     StructField("payload", StringType(), True),
21 ])
22 raw_json_df = spark.read.schema(telemetry_schema).json(raw_path)

```

```

23 raw_count = raw_json_df.count()
24
25 accepted_df = (
26     raw_json_df
27     .withColumn("ingested_at", F.current_timestamp())
28     .withColumn("source_file", F.input_file_name())
29     .dropDuplicates(["event_id"])
30     .dropna(subset=["event_id", "device_id", "event_timestamp"])
31     .select(
32         F.col("event_id").cast("string").alias("event_id"),
33         F.sha2(F.col("device_id").cast("string"), 256).alias("device_hash"),
34         F.to_timestamp("event_timestamp").alias("event_timestamp"),
35         F.to_date("event_timestamp").alias("event_date"),
36         F.upper(F.col("severity")).alias("severity"),
37         F.col("service_name").cast("string").alias("service_name"),
38         F.col("region").cast("string").alias("region"),
39         F.col("latency_ms").cast("double").alias("latency_ms"),
40         F.col("bytes_in").cast("long").alias("bytes_in"),
41         F.col("bytes_out").cast("long").alias("bytes_out"),
42         "ingested_at",
43         "source_file",
44     )
45     .where(F.col("event_timestamp").isNotNull())
46 )
47
48 accepted_df.write.format("delta").mode("overwrite").partitionBy("event_date").
49     saveAsTable(
50     "silver_telemetry_events"
51 )

```

Listing 3.3: Capstone Bronze JSON to cleaned Silver Delta transformation.

```

1 accepted_df.createOrReplaceTempView("v_accepted_telemetry")
2 accepted_count = accepted_df.count()
3
4 duplicate_count = raw_count - raw_json_df.dropDuplicates(["event_id"]).count()
5 required_non_null_count = (
6     raw_json_df
7     .dropDuplicates(["event_id"])
8     .dropna(subset=["event_id", "device_id", "event_timestamp"])
9     .count()
10 )
11 rejected_count = raw_count - accepted_count
12
13 metrics_rows = [(
14     processing_date,
15     raw_count,
16     accepted_count,
17     rejected_count,
18     duplicate_count,
19     required_non_null_count,
20 )]
21 metrics_columns = [
22     "processing_date",
23     "raw_rows",
24     "accepted_rows",
25     "rejected_rows",
26     "duplicate_rows",
27     "required_non_null_rows",
28 ]
29
30 metrics_df = spark.createDataFrame(metrics_rows, metrics_columns).withColumn(
31     "metrics_recorded_at", F.current_timestamp()
32 )

```

```

33
34 metrics_df.write.format("delta").mode("append").saveAsTable(
35     "silver_telemetry_ingestion_metrics"
36 )
37
38 spark.sql("""
39 SELECT
40     event_date,
41     severity,
42     service_name,
43     COUNT(*) AS event_count,
44     AVG(latency_ms) AS average_latency_ms,
45     SUM(bytes_in + bytes_out) AS total_bytes
46 FROM v_accepted_telemetry
47 GROUP BY event_date, severity, service_name
48 ORDER BY event_date, severity, service_name
49 """).show(100, truncate=False)

```

Listing 3.4: Capstone run metrics and Spark SQL validation.

Deliverables

Submit evidence that the architecture and implementation are complete:

- Workspace, lakehouse, notebook, pool, and environment names.
- Screenshot or exported evidence of the Bronze JSON folder and Silver Delta tables.
- Notebook output showing raw, accepted, rejected, duplicate, and required non-null counts.
- Schema for `silver_telemetry_events`, including hashed device identifiers and removed raw payload fields.
- Description of partitioning strategy and retry unit for daily processing.
- A short operational note explaining how the design scales from training files to 100 GB/day.
- Confirmation that no secrets, request headers, tokens, or real customer identifiers were printed or submitted.

Acceptance Criteria

Use this rubric before declaring the capstone complete:

- ☐ Raw JSON files are stored in a Bronze path by processing date.
- ☐ The notebook reads only the requested processing date or batch window.
- ☐ The JSON reader uses an explicit schema.
- ☐ Silver records remove duplicate `event_id` values.
- ☐ Records missing `event_id`, `device_id`, or `event_timestamp` are excluded or quarantined.
- ☐ The Silver Delta table is partitioned by `event_date`.
- ☐ Run metrics capture raw, accepted, rejected, and duplicate counts.
- ☐ Raw payloads, tokens, and full diagnostic text are not exposed in Silver outputs.
- ☐ The design states when a Custom pool would replace a Starter pool.
- ☐ Retry behavior is bounded to one processing date or batch window.

Stretch Goals

If you finish early, extend the capstone while preserving the Bronze-to-Silver contract:

- Add a quarantine Delta table that stores rejected records and rejection reasons.
- Add schema evolution handling for a new optional field such as `app_version`.
- Add a small Gold aggregate for daily service health by region and severity.
- Compare Starter pool and Custom pool execution time on the same sample data.
- Add a validation that fails the notebook when rejected records exceed an agreed threshold.
- Document a deployment path from development to production workspaces.

Delta Lake Optimization

Learning Objectives

- Explain how a Delta table combines Parquet data files with a JSON transaction log.
- Describe how Fabric optimization features such as V-Order, OPTIMIZE, and Z-Ordering improve analytical reads.
- Manage Delta table history, retention, and schema changes with Time Travel, VACUUM, and schema evolution.
- Run OPTIMIZE with V-Order on a large Silver table and use Time Travel after an accidental deletion.
- Diagnose the small-file problem and design a remediation plan for streaming tables degraded over months.
- Integrate Weeks 1–4 by building a Spark-driven Bronze, Silver, and Gold lakehouse with OneLake shortcuts and optimized Delta tables.

Cross-Cloud Equivalence

Open table formats solve a shared problem: many cloud object stores are excellent for durable files, but analytical tables also need transactions, metadata, compaction, clustering, retention, and historical access. Fabric centers this experience on Delta tables in OneLake, while other platforms expose similar ideas through Delta, Iceberg, Hudi, managed clustering, or compaction services.

Optimization need	Fabric Delta	AWS	Azure/Databricks	Snowflake
Compaction and small-file handling	<code>OPTIMIZE</code> rewrites many small Delta files into fewer larger files; V-Order can be enabled for Fabric read efficiency	Delta, Iceberg, or Hudi tables on S3 use compaction through EMR, Glue, Athena, or table-maintenance jobs	Delta <code>OPTIMIZE</code> on Databricks and Synapse/Fabric-style lakehouse patterns compact object-storage files	Micro-partitions are managed by the service; clustering maintenance is automatic or configured through clustering features
Data skipping and clustering	Z-Ordering colocates related values; Delta statistics and V-Order improve selective scans	Iceberg partition specs, Hudi clustering, Delta Z-Order, and Glue catalog metadata help prune S3 reads	Databricks <code>OPTIMIZE ZORDER BY</code> and data-skipping statistics are common tuning levers	Automatic clustering and pruning reduce scanned micro-partitions
Time travel	Delta history supports <code>VERSION AS OF</code> and timestamp-based reads before retention cleanup	Delta, Iceberg, and Hudi support snapshot or commit-based reads when retained	Delta Time Travel is a core Databricks and Delta Lake capability	Built-in Time Travel exposes previous table states within the configured retention window
Vacuum and retention	<code>VACUUM</code> removes old unreferenced files after the safety retention period	Table formats require expire-snapshot, clean, or compaction jobs to remove obsolete files from S3	Delta <code>VACUUM</code> and retention settings control cleanup after history is no longer needed	Fail-safe and Time Travel retention are managed service features rather than user file deletion

GCP commonly expresses the same pattern through BigLake, BigQuery external tables, Dataproc, and Iceberg or Delta tables on Cloud Storage. MongoDB is not an open-table-format lakehouse engine; it typically feeds optimized Delta, Iceberg, or warehouse tables through connectors, change streams, or batch exports.

Key Terms

Delta table: a table abstraction that stores data as Parquet files and records table actions in a transaction log. **Transaction log:** ordered metadata files that describe adds, removes, schema changes, and commits. **V-Order:** Fabric’s write-time layout optimization for faster analytical reads. **Z-Ordering:** a clustering technique that colocates related values to improve data skipping. **Small-file problem:** degraded performance caused by many tiny files that create excessive listing, scheduling, and metadata overhead.

4.1 Week 4 Roadmap

Week 4 closes Part I by turning lakehouse storage into an operational discipline. Monday explains Delta internals: Parquet data files plus the JSON transaction log. Tuesday covers table optimization with V-Order, `OPTIMIZE`, and Z-Ordering. Wednesday covers table management with `VACUUM`, Time Travel, and schema evolution. Thursday is a hands-on recovery and optimization lab. Friday applies the concepts to a streaming table that accumulated millions of small files over six months.

Delta Lake matters because a lakehouse is only useful when tables remain reliable and fast as data changes. Fabric Lakehouses store managed tables in Delta format, which means Spark, SQL analytics endpoints, Direct Lake semantic models, and other engines can reason about consistent table versions rather than loose folders of files [53, 9]. Optimization does not replace good modeling, but it keeps Silver and Gold tables usable when ingestion patterns, streaming frequency, and analytical filters evolve.

Note

This chapter focuses on data-engineering operations, not one-time performance tricks. A healthy Delta table has a clear owner, expected write pattern, retention policy, optimization cadence, recovery plan, and evidence that reads improved after maintenance.

4.2 Monday: Delta Lake Internals

A Delta table is not a single file. It is a directory containing Parquet data files plus a special `_delta_log` folder. The Parquet files store columnar data. The transaction log stores ordered JSON actions that describe which files belong to each table version, which files were removed, what schema is active, and what operation produced the commit [4, 2].

Definition

Delta Lake is an open table format that adds ACID transactions, scalable metadata, schema enforcement, schema evolution, and Time Travel over Parquet files in object storage or lakehouse storage.

4.2.1 Parquet Data Files

Parquet is a columnar file format. Values for the same column are stored together, which improves compression and lets analytical engines read only the columns required by a query. For example, a query that groups sales by `order_date` and `region` does not need to scan every descriptive column in the table.

Parquet files are immutable from the practical perspective of table writes. Updating, deleting, or compacting records usually creates new files and marks old files as no longer active in the table log. This immutability is why the transaction log is essential: it tells readers which Parquet files make up the current table state.

Tip

Think of Parquet files as efficient pages and the Delta log as the table of contents plus audit trail. Query engines need both to produce a consistent result.

4.2.2 The JSON Transaction Log

Every Delta commit creates a new version. Early versions are commonly represented as JSON log files under `_delta_log`; checkpoint files may later summarize metadata for efficient reads. A commit can add files, remove files, change metadata, update protocol requirements, or record operation metrics.

Because the log is ordered, a reader can reconstruct version 0, version 1, version 2, and so on. This is the foundation for ACID behavior and Time Travel. When one job writes a new Silver table version, concurrent readers can continue using a consistent previous version until they refresh.

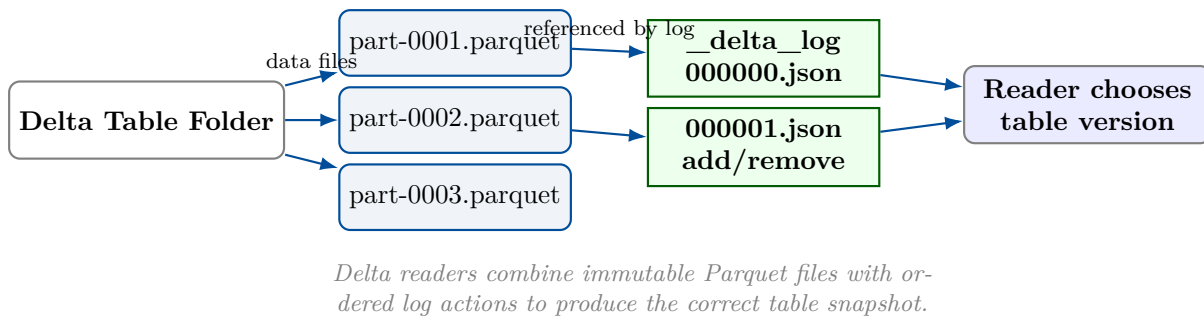


Figure 4.1: Delta table internals: Parquet data files plus a transaction log.

4.2.3 Why Internals Affect Optimization

Optimization rewrites files, but it should not change the logical table result. Compaction creates new larger Parquet files and records removal actions for the older smaller files. Z-Ordering rewrites data into a layout that improves skipping for common filters. Time Travel remains possible because previous versions still reference older files until retention cleanup removes them.

Common Pitfall

Deleting files manually. Removing Parquet files outside Delta operations can corrupt historical versions and break readers. Use Delta commands and retention-aware maintenance instead of manual file deletion.

4.3 Tuesday: Optimizing Delta Tables

Optimization starts with the workload. A table read mostly by date, customer, and product needs a different layout than a table read by device, event type, and timestamp. Fabric supports Delta table optimization patterns that reduce file overhead and improve query pruning for common analytics [22, 41].

4.3.1 V-Order in Fabric

V-Order is a Fabric optimization for Parquet files that improves compression and read performance for Fabric analytical engines. It is especially relevant when a Delta table supports SQL analytics endpoint queries, Direct Lake semantic models, and repeated reporting scans. V-Order is not a substitute for medallion modeling; it makes well-designed tables more efficient to read.

In Fabric, V-Order can be applied through table write settings or optimization commands depending on the workload and runtime support. The important operational habit is to document whether critical Silver and Gold tables are expected to be V-Order optimized and when the optimization runs.

Note

V-Order is a physical layout decision. It should not change business results, table names, schemas, or downstream contracts.

4.3.2 OPTIMIZE for Compaction

OPTIMIZE rewrites many small files into fewer larger files. This reduces file-listing overhead, Spark task scheduling overhead, and metadata pressure. It is most valuable after frequent appends, streaming micro-batches, or partitioned writes that produce many tiny files.

Compaction has a cost because it reads and rewrites data. Schedule it according to the table's write pattern and consumption needs. A table loaded once nightly may need optimization after each load. A streaming table may need scheduled maintenance during a low-traffic window.

Tip

Measure file count before and after compaction. A successful optimization should reduce active file count, improve scan time for representative queries, and preserve row counts.

4.3.3 Z-Ordering

Z-Ordering colocates records with similar values across one or more columns so that filters can skip more files. It is useful when queries often filter on high-value columns such as `customer_id`, `device_id`, `order_date`, or `region`. Do not Z-Order every column. Choose columns from real query patterns and avoid changing layout constantly.

Technique	Best fit	Watch for
V-Order	Fabric analytical reads over Delta-Parquet tables	Requires write or optimize support and clear table policy
OPTIMIZE	Tables with many small files from streaming or frequent appends	Rewrite cost and maintenance scheduling
Z-Ordering	Selective queries that repeatedly filter by known columns	Poor column choice can waste compute without improving reads
Partitioning	Large tables commonly filtered by low-cardinality date or business period	Over-partitioning can create the small-file problem

Table 4.1: Optimization choices for Delta tables in Fabric.

4.4 Wednesday: Managing Delta Tables

Optimization makes reads faster; management keeps tables recoverable and sustainable. The three core management topics are retention cleanup, historical reads, and safe schema changes.

4.4.1 VACUUM and Retention

VACUUM deletes data files that are no longer referenced by the current table and are older than the retention threshold [52]. Without cleanup, old files can accumulate indefinitely after overwrites, deletes, merges, and compaction. With aggressive cleanup, Time Travel and rollback options can disappear too quickly.

The retention window should reflect business requirements. A development table may keep less history. A regulated Silver table may require longer retention and separate archival controls. In any case, record the policy before automating cleanup.

Common Pitfall

Vacuuming too soon after a mistake. If an accidental delete is discovered after obsolete files are vacuumed, Time Travel may no longer be able to read the pre-delete version.

4.4.2 Time Travel

Time Travel lets readers query a previous version or timestamp of a Delta table [9]. It is useful for debugging, audit comparison, reproducible experiments, and recovery after a bad write. A data engineer can compare row counts between versions, recover deleted records, or prove when a schema changed.

Time Travel depends on retained log and data files. It should be part of an incident playbook: identify the bad version, query the last known good version, validate it, and restore or rewrite only after approval.

4.4.3 Schema Evolution

Schema enforcement protects tables from unexpected columns and type changes. Schema evolution allows intentional changes, such as adding a nullable column when a source system begins sending `promotion_code`. The difference is governance: unplanned drift should fail or be quarantined; planned evolution should be reviewed, documented, and tested.

Definition

Schema evolution is the controlled process of changing a Delta table schema, commonly by adding nullable columns, while preserving existing data and downstream compatibility.

4.5 Thursday: Hands-on Project

The Week 4 lab uses a large Silver table such as `silver_sales_orders`. You will compact it with `OPTIMIZE` and V-Order, simulate an accidental deletion in a safe training copy, and query the previous version with Time Travel. Use synthetic or instructor-provided data; never run delete experiments on a production table.

4.5.1 Goal and Architecture

Create or reuse a workspace named `fabric-de-week04`. Add a lakehouse named `lh_silver`. Prepare a Delta table named `silver_sales_orders` with enough rows and files to make optimization visible. The table should include columns such as `order_id`, `customer_id`, `order_date`, `region`, `product_id`, and `net_amount`.

Note

If your tenant does not support every optimization syntax shown here, keep the learning outcome: compact the Delta table, record file-count and query-time evidence, and demonstrate a versioned read before cleanup.

4.5.2 Step-by-Step Actions

1. Create a training Silver Delta table with realistic columns and many files.
2. Record a baseline row count, active file count if available, and query duration for a representative filter.

3. Run OPTIMIZE with V-Order and optional Z-Ordering for the common query columns.
4. Query Delta history and note the latest version number.
5. In a training copy, simulate an accidental delete for one region or date range.
6. Use VERSION AS OF to read the previous version and validate the missing rows.
7. Restore the affected rows from the previous version or rebuild the training table.
8. Run VACUUM only after confirming the retention policy and recovery needs.

```

1 -- Capture a baseline for representative analytical filters.
2 SELECT region, COUNT(*) AS order_count, SUM(net_amount) AS total_net_amount
3 FROM silver_sales_orders
4 WHERE order_date >= DATE '2026-07-01'
5 GROUP BY region
6 ORDER BY region;
7
8 -- Compact files and apply Fabric V-Order where supported.
9 OPTIMIZE silver_sales_orders VORDER;
10
11 -- Add clustering for common selective filters when supported by the runtime.
12 OPTIMIZE silver_sales_orders
13 ZORDER BY (customer_id, order_date);
14
15 DESCRIBE HISTORY silver_sales_orders;

```

Listing 4.1: Optimizing a Silver Delta table with V-Order and Z-Ordering.

After optimization, rerun the same baseline query. The row count and business totals must match. The expected improvement is fewer active files and faster or more predictable reads, not changed data.

```

1 -- Simulate a mistake only in a training copy.
2 CREATE TABLE silver_sales_orders_training AS
3 SELECT * FROM silver_sales_orders;
4
5 DELETE FROM silver_sales_orders_training
6 WHERE region = 'West'
7 AND order_date >= DATE '2026-07-01';
8
9 DESCRIBE HISTORY silver_sales_orders_training;
10
11 -- Replace 12 with the last known good version from DESCRIBE HISTORY.
12 SELECT COUNT(*) AS west_orders_before_delete
13 FROM silver_sales_orders_training VERSION AS OF 12
14 WHERE region = 'West'
15 AND order_date >= DATE '2026-07-01';

```

Listing 4.2: Using Time Travel after an accidental training-table deletion.

Some recovery workflows are easier in PySpark because you can read the older version into a DataFrame, filter the missing slice, and append it back after validation.

```

1 from pyspark.sql import functions as F
2
3 table_name = "silver_sales_orders_training"
4 last_good_version = 12
5
6 previous_df = (
7     spark.read
8     .format("delta")
9     .option("versionAsOf", last_good_version)
10    .table(table_name)

```



```

11 )
12
13 missing_west_df = (
14     previous_df
15     .where((F.col("region") == "West") & (F.col("order_date") >= F.lit("
2026-07-01"))))
16 )
17
18 missing_west_df.createOrReplaceTempView("v_recovered_west_orders")
19
20 spark.sql("""
21 SELECT COUNT(*) AS rows_to_restore, SUM(net_amount) AS amount_to_restore
22 FROM v_recovered_west_orders
23 """).show(truncate=False)

```

Listing 4.3: Reading a prior Delta version in PySpark for targeted recovery.

```

1 -- Run only after recovery is complete and retention requirements are satisfied
2 VACUUM silver_sales_orders_training RETAIN 168 HOURS;

```

Listing 4.4: Retention-aware cleanup after recovery validation.

4.5.3 Validation Checklist

- Baseline and post-optimization row counts match.
- Representative query filters are documented before choosing Z-Order columns.
- DESCRIBE HISTORY shows optimization and delete operations.
- Time Travel reads the last known good version before retention cleanup.
- Recovery logic is tested on a training copy, not a production table.
- VACUUM is delayed until the recovery window and retention policy are understood.

4.6 Friday: Use Case and Architecture

A streaming pipeline has appended micro-batches to a Silver events table for six months. Each micro-batch wrote many tiny files into date partitions. Analysts now report that simple filters take minutes, capacity consumption spikes during business hours, and Direct Lake reports refresh unpredictably. The table contains correct data, but the physical layout has degraded.

4.6.1 Symptoms

The small-file problem shows up in several ways. Spark jobs spend too much time listing files and scheduling tasks. SQL queries scan many file footers before reading useful rows. Metadata operations slow down. Optimization attempts may compete with streaming writes. Downstream users experience unstable latency even when daily row volume has not changed.

4.6.2 Remediation Strategy

The remediation plan has two parts: fix the existing table and prevent recurrence. First, pause or coordinate streaming writes, snapshot table history, and run partition-aware compaction for the worst partitions. Second, adjust streaming trigger intervals, batch sizing, partition strategy, and scheduled OPTIMIZE cadence so new tiny files do not accumulate unchecked.

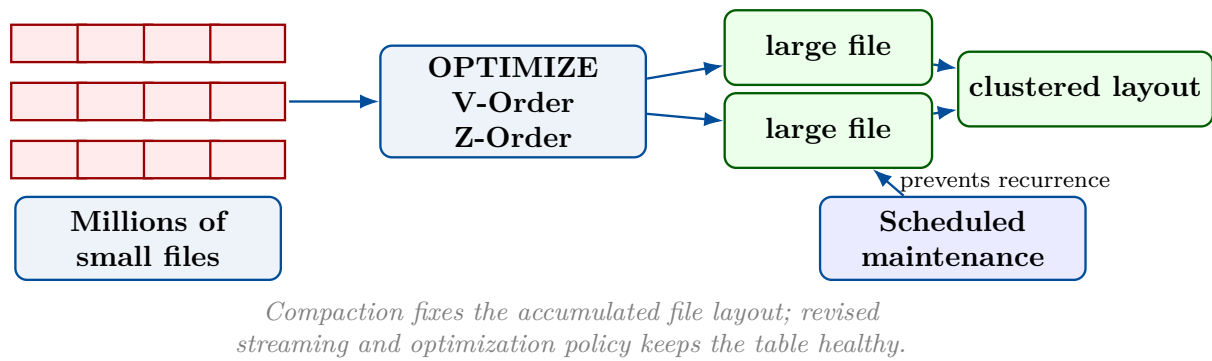


Figure 4.2: Small-file remediation through compaction and optimized layout.

4.6.3 Operational Runbook

1. Identify affected tables, partitions, file counts, average file sizes, and top user queries.
2. Confirm retention requirements and capture the current Delta version.
3. Coordinate a maintenance window or use an approach compatible with active streaming writes.
4. Compact the oldest and worst partitions first; validate row counts after each batch.
5. Apply V-Order and targeted Z-Ordering for documented query filters.
6. Tune streaming triggers or batch sizes so future writes produce healthier files.
7. Schedule recurring optimization and record metrics after every maintenance run.
8. Delay `VACUUM` until rollback confidence and retention requirements are satisfied.

Evidence	Interpretation	Action
Millions of active files	Metadata and scheduling overhead dominate reads	Run partition-aware <code>OPTIMIZE</code> and prevent tiny future writes
Queries filter by date and device	Layout should support data skipping on those columns	Consider date partitioning plus Z-Order on <code>device_id</code>
Streaming writes every few seconds	Micro-batches may create too many files	Increase trigger interval or use available output coalescing patterns
History needed for audit	Cleanup cannot be aggressive	Align <code>VACUUM</code> retention with audit and restore policy

Table 4.2: Diagnosing and remediating a degraded streaming Delta table.

4.7 Operational Guidance for Week 4

Delta operations should be automated only after they are understood manually. The first maintenance run should capture baseline query time, row counts, active file counts, operation history, and table version. The automated version should emit the same evidence to a run log.

4.7.1 Performance and Reliability

Avoid optimizing blindly. Optimize high-value tables first: large Silver tables, Gold dimensions and facts used by Direct Lake, and tables with frequent appends. Keep Z-Order columns stable

unless query patterns materially change. Coordinate compaction with streaming writers and avoid running expensive maintenance during peak reporting windows.

4.7.2 Security and Governance Checklist

- Restrict who can run destructive commands such as `DELETE`, `TRUNCATE`, and `VACUUM`.
- Record table owners, retention policies, optimization cadence, and recovery contacts.
- Do not shorten retention without understanding legal, audit, and incident-response needs.
- Validate schema evolution in development before allowing new columns into shared Silver or Gold tables.
- Keep optimization notebooks parameterized and free of personal workspace paths.

4.8 Interview Q&A

1. **Q: What are the two major physical components of a Delta table?**
A: Parquet data files store the columnar data, and the Delta transaction log records table versions, add and remove actions, schema metadata, and operation history.
2. **Q: Why does `OPTIMIZE` help a table with many small files?**
A: It compacts many small files into fewer larger files, reducing file-listing overhead, task scheduling overhead, and metadata pressure during reads.
3. **Q: When would you use Z-Ordering?**
A: Use it when important queries repeatedly filter on specific columns and colocating similar values will improve data skipping.
4. **Q: How is V-Order different from Z-Ordering?**
A: V-Order is a Fabric-oriented Parquet layout optimization for analytical reads, while Z-Ordering clusters rows by selected columns to improve selective filtering.
5. **Q: What risk does `VACUUM` introduce?**
A: It can permanently remove old unreferenced files, which may prevent Time Travel or recovery to versions that still need those files.
6. **Q: How should schema evolution be governed?**
A: Treat it as an intentional contract change: review the new field, test downstream impact, document the schema, and avoid silently accepting unplanned drift.

4.9 Further Reading

Review Microsoft documentation for Fabric Lakehouses [53], OneLake [60], Fabric Spark [11], V-Order [22], Delta table maintenance and optimization [41, 52], and Fabric notebooks [27]. For format internals, read the Delta Lake documentation [9], the Delta Lake paper [4], and Apache Parquet documentation [2].

Key Takeaways

- Delta tables combine Parquet data files with an ordered transaction log that defines consistent table versions.
- Fabric optimization is a physical layout practice: V-Order, `OPTIMIZE`, and Z-Ordering should improve reads without changing results.

- The small-file problem is common after frequent appends or streaming micro-batches and should be fixed with compaction plus better write policy.
- Time Travel is a recovery and debugging feature, but it depends on retained log and data files.
- `VACUUM` controls storage growth by deleting obsolete files, but it must respect recovery and audit requirements.
- Schema evolution should be planned and governed because Silver and Gold tables are contracts for downstream users.
- Part I closes with an integrated lakehouse that uses OneLake shortcuts, Spark transformations, Delta tables, V-Order, and a Gold star schema.

4.10 Exercises

1. **(Conceptual)** Explain why a Delta table needs both Parquet files and a transaction log.
2. **(Conceptual)** Compare compaction, V-Order, Z-Ordering, and partitioning. Which problem does each address?
3. **(Hands-on)** Create a training Delta table, run `DESCRIBE HISTORY`, and identify the operation that created the latest version.
4. **(Hands-on)** Run an optimization command on a non-production table and compare row counts before and after.
5. **(Recovery)** Simulate an accidental delete in a training copy and query the previous version with `VERSION AS OF`.
6. **(Operations)** Draft a retention policy that balances 30 days of recovery with storage cleanup.
7. **(Design)** Choose Z-Order columns for a Silver IoT table with filters by event date, device, region, and alert type. Explain your choice.
8. **(Interview)** In five sentences, explain the small-file problem and how you would prevent it in a streaming pipeline.

4.11 Milestone 1 Capstone: Spark-Driven Medallion Lakehouse

Capstone Project

Mission. Close Part I by building a Spark-driven medallion lakehouse in Microsoft Fabric. The solution must create Bronze, Silver, and Gold lakehouses, ingest sample data through OneLake shortcuts, clean and conform data with parameterized Spark notebooks, publish a Gold star schema, and optimize Delta tables with V-Order.

Estimated time. 6–8 hours, depending on tenant permissions, shortcut source availability, and sample-data size.

What you'll practice.

- Creating workspaces and lakehouses aligned to Bronze, Silver, and Gold responsibilities.
- Using OneLake shortcuts to avoid unnecessary source-data copying.
- Writing parameterized PySpark notebooks for repeatable Silver transformations.

- Building Gold facts and dimensions from curated Silver Delta tables.
- Running `OPTIMIZE` with V-Order and validating table history.

Scenario

Contoso Retail wants a Part I reference implementation for Fabric data engineering. Source files include customers, products, stores, orders, and order lines. Some data may remain in external object storage or instructor-provided locations; Fabric should expose it through OneLake shortcuts. The engineering team must land raw evidence in Bronze, clean and standardize records in Silver, and publish Gold star-schema tables for Power BI and SQL consumers.

Requirements

Build a working design with these minimum elements:

- A workspace named `fabric-de-milestone1` or an instructor-approved equivalent.
- Lakehouses named `lh_bronze`, `lh_silver`, and `lh_gold`, or clearly separated layer paths if the tenant limits lakehouse attachments.
- OneLake shortcuts in Bronze for at least two sample sources, such as orders and products.
- A parameterized Silver notebook that accepts `processing_date` and `source_system`.
- Silver Delta tables with typed dates, deduplicated business keys, and required null checks.
- Gold star-schema tables such as `dim_customer`, `dim_product`, `dim_date`, `dim_store`, and `fact_sales`.
- V-Order optimization for the major Silver and Gold Delta tables.
- A short operations note covering retry unit, retention, optimization cadence, and data-quality metrics.

Reference Architecture

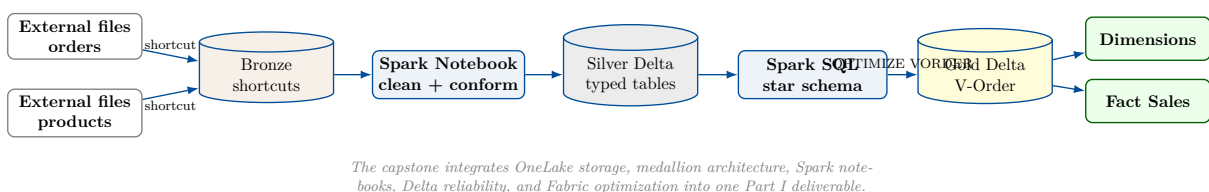


Figure 4.3: Milestone 1 reference architecture from Bronze shortcuts to V-Order optimized Gold tables.

Implementation Steps

1. **Create the workspace.** Create `fabric-de-milestone1` and assign the correct Fabric capacity.
2. **Create lakehouses.** Add `lh_bronze`, `lh_silver`, and `lh_gold`. Document any tenant-specific deviations.
3. **Create shortcuts.** In Bronze, create OneLake shortcuts to sample orders, order lines, customers, products, and stores where available.

4. **Build Silver.** Use PySpark to read raw files or shortcut paths, apply explicit schemas, drop invalid rows, deduplicate keys, and write Delta tables.
5. **Build Gold.** Use Spark SQL or DataFrame logic to create dimensions and `fact_sales` from Silver.
6. **Optimize.** Run OPTIMIZE with V-Order on important Silver and Gold tables.
7. **Validate.** Compare Bronze counts, Silver accepted and rejected counts, and Gold aggregate totals.
8. **Document operations.** Record retry unit, retention policy, optimization cadence, and ownership.

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.types import (
3     StructType, StructField, StringType, IntegerType, DoubleType
4 )
5
6 processing_date = "2026-07-31"
7 source_system = "retail_sample"
8
9 orders_schema = StructType([
10     StructField("order_id", StringType(), True),
11     StructField("customer_id", StringType(), True),
12     StructField("store_id", StringType(), True),
13     StructField("order_timestamp", StringType(), True),
14     StructField("order_status", StringType(), True),
15 ])
16
17 order_lines_schema = StructType([
18     StructField("order_id", StringType(), True),
19     StructField("product_id", StringType(), True),
20     StructField("quantity", IntegerType(), True),
21     StructField("unit_price", DoubleType(), True),
22 ])
23
24 orders_path = f"Files/bronze/shortcuts/orders/order_date={processing_date}/*.csv"
25 lines_path = f"Files/bronze/shortcuts/order_lines/order_date={processing_date}/*.csv"
26
27 orders_df = (
28     spark.read.option("header", "true").schema(orders_schema).csv(orders_path)
29     .withColumn("source_system", F.lit(source_system))
30     .withColumn("ingested_at", F.current_timestamp())
31     .dropDuplicates(["order_id"])
32     .dropna(subset=["order_id", "customer_id", "order_timestamp"])
33     .withColumn("order_timestamp", F.to_timestamp("order_timestamp"))
34     .withColumn("order_date", F.to_date("order_timestamp"))
35     .where(F.col("order_timestamp").isNotNull())
36 )
37
38 lines_df = (
39     spark.read.option("header", "true").schema(order_lines_schema).csv(
40         lines_path)
41     .dropna(subset=["order_id", "product_id"])
42     .withColumn("quantity", F.col("quantity").cast("int"))
43     .withColumn("unit_price", F.col("unit_price").cast("double"))
44     .where((F.col("quantity") > 0) & (F.col("unit_price") >= 0))
45 )
46 orders_df.write.format("delta").mode("overwrite").saveAsTable("silver_orders")

```

```

47 lines_df.write.format("delta").mode("overwrite").saveAsTable("
    silver_order_lines")

```

Listing 4.5: Parameterized Silver transformation for the Milestone 1 capstone.

```

1 CREATE OR REPLACE TABLE dim_customer AS
2 SELECT DISTINCT
3     customer_id,
4     source_system
5 FROM silver_orders
6 WHERE customer_id IS NOT NULL;
7
8 CREATE OR REPLACE TABLE dim_date AS
9 SELECT DISTINCT
10    order_date,
11    YEAR(order_date) AS calendar_year,
12    MONTH(order_date) AS calendar_month,
13    DAY(order_date) AS calendar_day
14 FROM silver_orders
15 WHERE order_date IS NOT NULL;
16
17 CREATE OR REPLACE TABLE fact_sales AS
18 SELECT
19     o.order_id,
20     o.customer_id,
21     o.store_id,
22     l.product_id,
23     o.order_date,
24     l.quantity,
25     l.unit_price,
26     l.quantity * l.unit_price AS sales_amount
27 FROM silver_orders o
28 JOIN silver_order_lines l
29     ON o.order_id = l.order_id;
30
31 OPTIMIZE silver_orders VORDER;
32 OPTIMIZE silver_order_lines VORDER;
33 OPTIMIZE dim_customer VORDER;
34 OPTIMIZE dim_date VORDER;
35 OPTIMIZE fact_sales VORDER;

```

Listing 4.6: Gold star schema and V-Order optimization.

Deliverables

Submit evidence that the Part I milestone is complete:

- Workspace and lakehouse names, including Bronze, Silver, and Gold boundaries.
- Screenshot or exported evidence of OneLake shortcuts in Bronze.
- Notebook parameters and successful Silver transformation output.
- Schemas and row counts for Silver Delta tables.
- Gold star-schema table list with at least one fact and three dimensions.
- DESCRIBE HISTORY evidence showing optimization on key Delta tables.
- A short operations note for retry, retention, optimization cadence, and data-quality checks.

Acceptance Criteria

Use this rubric before declaring Milestone 1 complete:

- ☐ Bronze, Silver, and Gold storage boundaries are visible and documented.
- ☐ At least two Bronze data sources are exposed through OneLake shortcuts or clearly documented substitutes.
- ☐ Silver notebooks use parameters rather than hardcoded personal paths.
- ☐ Silver tables enforce required keys, types, and duplicate rules.
- ☐ Gold contains a star schema with dimensions and **fact_sales**.
- ☐ Major Silver and Gold tables are Delta tables optimized with V-Order where supported.
- ☐ Row-count reconciliation proves that rejected and accepted records are understood.
- ☐ The design states retention, Time Travel, and **VACUUM** expectations.
- ☐ No secrets, real customer identifiers, or personal workspace paths appear in submitted evidence.
- ☐ The project demonstrates Weeks 1–4 as one coherent lakehouse workflow.

Stretch Goals

If you finish early, extend the milestone without changing its core contract:

- Add a quarantine table with rejection reasons for invalid Bronze records.
- Add a product and store dimension with slowly changing attribute notes.
- Add Time Travel validation that compares the latest Gold totals to a previous version.
- Add a data-quality metrics table for each processing date.
- Create a Direct Lake semantic model over the Gold star schema.
- Document how the development workspace would promote to test and production.

Part II

Data Warehousing and Analytics

Synapse Data Warehouse in Fabric

Learning Objectives

- Explain the Fabric Data Warehouse architecture and why the separation of compute from storage changes warehouse scaling and cost behavior.
- Describe the supported T-SQL surface area in Fabric, including `COPY INTO`, and contrast it with a full SQL Server engine.
- Compare the three primary warehouse ingestion patterns—pipelines, T-SQL, and Dataflows Gen2—and select the right one for a given workload.
- Load millions of rows of Parquet data directly from a OneLake path using `COPY INTO`.
- Estimate capacity consumption for a warehouse workload and reason about reserved versus pay-as-you-go billing.
- Recommend a Lakehouse-versus-Warehouse landing pattern for a legacy SQL Server migration that depends on heavy stored procedure logic.

Cross-Cloud Equivalence

The cloud data warehouse—decoupled compute over open or managed storage, SQL as the primary interface—is one of the most portable concepts in data engineering. Fabric’s Synapse Data Warehouse is the Azure-column answer, and its closest relatives are direct one-to-one mappings.

Capability	Fabric Warehouse	AWS	GCP	Snowflake
Warehouse engine	Synapse Data Warehouse	Amazon Redshift	BigQuery	Snowflake virtual warehouses
Bulk load command	<code>COPY INTO</code>	<code>COPY</code>	<code>LOAD DATA</code> / bq load	<code>COPY INTO</code> <table>
Storage model	Delta-Parquet in OneLake	Managed (RA3) or Spectrum over S3	Capacitor columnar storage	Micro-partitions on object storage
Compute scaling	Capacity CUs (F SKUs)	Nodes / serverless workgroups	Slots / on-demand	Warehouse size (XS–6XL)
Cross-db query	Three-part names across items	Redshift Spectrum / federated	Dataset-qualified names	Three-part names, shares
Separation of compute/storage	Yes	Yes (RA3, serverless)	Yes	Yes
Concurrency model	Shared capacity, smoothing	WLM queues / auto-scaling	Slot reservations	Multi-cluster warehouses

Databricks SQL warehouses and Azure Synapse dedicated SQL pools are the in-family equivalents. MongoDB has no warehouse concept; its analytics path runs through Atlas SQL Interface, federated queries, or export into one of the columnar engines above. The durable lesson: every platform separates *where bytes live* from *who computes over them*,

and bills the two differently.

Key Terms

Synapse Data Warehouse: a Fabric item that provides a full transactional SQL warehouse over Delta-Parquet storage in OneLake. **COPY INTO:** the T-SQL statement that bulk-loads files from a storage path into a warehouse table. **Compute-storage separation:** an architecture in which warehouse storage persists in OneLake while query compute is drawn from the shared Fabric capacity. **Three-part naming:** the `database.schema.table` convention that enables cross-database queries between warehouses and lakehouse SQL endpoints in one workspace. **Capacity unit (CU):** the billing and throughput abstraction that warehouses share with every other Fabric workload.

5.1 Week 5 Roadmap

Week 5 opens Part II, the warehousing and business-intelligence arc of the program. Monday covers the Fabric Data Warehouse architecture and the consequences of compute-storage separation. Tuesday builds T-SQL fluency on the supported surface area, with **COPY INTO** as the centerpiece. Wednesday compares the three ingestion patterns—pipelines, T-SQL, and Dataflows Gen2—and introduces idempotent load design. Thursday is the hands-on project: provision a warehouse and load five million rows of Parquet from OneLake with **COPY INTO**. Friday closes with a capacity-cost estimation exercise and an architecture decision for a legacy SQL Server migration.

The warehouse matters because Part I built a lakehouse world optimized for Spark and files, while much of the enterprise still thinks in T-SQL, stored procedures, and star schemas. The Fabric Data Warehouse bridges those worlds: it stores data as open Delta-Parquet in OneLake yet presents a transactional, multi-table SQL surface [58]. Understanding when to land data in a warehouse rather than a lakehouse is one of the highest-leverage judgment calls a Fabric data engineer makes.

Note

The Fabric warehouse is *not* a repackaged SQL Server instance. It is a distributed, SaaS-managed warehouse whose tables are Delta-Parquet files in OneLake. That single fact explains its strengths (open storage, cross-engine reuse, elastic compute) and its limits (a T-SQL surface that is broad but not identical to SQL Server).

5.2 Monday: Warehouse Architecture

A Fabric Data Warehouse consists of two logically separate layers. The *storage layer* persists every table as Delta-Parquet files in OneLake, in a managed area owned by the warehouse item. The *compute layer* is a distributed SQL query engine, provisioned automatically from the workspace's Fabric capacity, that parses T-SQL, plans distributed execution, and reads or writes those files [58].

Definition

Compute-storage separation is an architecture in which durable data lives in object storage independent of any running compute, so compute can scale, pause, or be re-provisioned without moving or re-loading the data.

5.2.1 Why Separation Matters

In a coupled appliance model, scaling compute means redistributing data across nodes, and storage capacity is bounded by the hardware you bought. In the separated model, storage scales with OneLake and compute scales with the capacity SKU. The engineering consequences are practical:

- **Independent scaling.** A capacity increase speeds queries without touching data layout; growing data volume does not force a compute purchase.
- **Open storage.** Because tables are Delta-Parquet in OneLake, Spark notebooks and other warehouses can read the same bytes—no export pipeline is required to hand warehouse data to a data scientist.
- **Failure isolation.** A failed query or a paused capacity never corrupts or loses stored data; the log of table versions lives with the files.
- **Cost clarity.** Storage is billed through OneLake and compute through capacity units, so a workload’s cost drivers can be attributed and optimized separately.

5.2.2 The Warehouse and Its Neighbors

A warehouse does not exist alone. In a typical workspace it sits beside lakehouses and semantic models, and the SQL engine can reach across those boundaries. The warehouse also exposes the same connection string style as the lakehouse SQL analytics endpoint, which means SQL Server Management Studio, Azure Data Studio, and any TDS-capable tool connect to it directly.

Aspect	Fabric Data Warehouse	Lakehouse SQL analytics endpoint
Primary interface	Full transactional T-SQL (DDL, DML, procedures)	Read-only T-SQL over lakehouse Delta tables
Storage	Managed Delta-Parquet in OneLake	Lakehouse Tables area in OneLake
Best fit	T-SQL-centric teams, stored procedures, star schemas	Spark-first pipelines with a SQL consumption layer
Write path	COPY INTO, INSERT, CTAS, pipelines, dataflows	Writes happen through Spark or lakehouse APIs

Table 5.1: The warehouse versus its read-only lakehouse sibling.

Common Pitfall

Do not choose the warehouse purely because the team “knows SQL.” If the transformation logic is Spark-native (feature engineering, ML preprocessing, heavy nested data), land it in a lakehouse and let SQL consumers read it through the SQL analytics endpoint. Choosing the wrong home forces awkward cross-engine orchestration later.

5.3 Tuesday: T-SQL in Fabric

The Fabric warehouse supports a broad, transactional T-SQL surface: DDL such as `CREATE TABLE` and `ALTER TABLE`; DML such as `INSERT`, `UPDATE`, `DELETE`, and `MERGE`; stored procedures, functions, and views; transactions with full ACID semantics; and the `COPY INTO` bulk-load statement. Some SQL Server surface area—CLR, certain index types, and instance-level features—does not apply because there is no instance: the service manages distribution, and tables are physically Delta-Parquet, not B-trees [48].

5.3.1 COPY INTO

`COPY INTO` is the workhorse of warehouse ingestion. It reads Parquet, CSV, or JSON files from a storage path—including OneLake paths and external Azure storage—and loads them into a target table with massively parallel reads [14].

Definition

`COPY INTO` is a T-SQL bulk-load statement that ingests files from a storage location into a warehouse table, with options for file type, credentials, row rejection, and column mapping.

```

1 -- Minimal load from a OneLake Files path exposed via the DFS endpoint
2 COPY INTO dbo.stg_sales
3 FROM 'https://onelake.dfs.fabric.microsoft.com/<workspace>/<lakehouse>/Files/
   stage/sales/2026-07/'
4 WITH (
5     FILE_TYPE = 'PARQUET'
6 );
7
8 -- Load with column mapping and a reject tolerance for dirty CSVs
9 COPY INTO dbo.stg_orders (order_id, order_date, net_amount)
10 FROM 'https://onelake.dfs.fabric.microsoft.com/<workspace>/<lakehouse>/Files/
   stage/orders/'
11 WITH (
12     FILE_TYPE = 'CSV',
13     FIRSTROW = 2,
14     FIELDTERMINATOR = ',',
15     MAXERRORS = 100
16 );

```

Listing 5.1: Core `COPY INTO` patterns against OneLake paths.

Tip

Prefer Parquet over CSV for `COPY INTO` whenever the upstream producer controls the format. Parquet carries schema and column statistics, loads faster, and eliminates the entire class of delimiter, encoding, and type-inference failures that plague CSV bulk loads.

5.3.2 Cross-Database Querying

Within a workspace, the SQL engine resolves three-part names across warehouses and lakehouse SQL analytics endpoints. A query in `dw_sales` can join `lh_gold.dbo.fact_sales` directly, reading the lakehouse’s Delta files in place. This is the technical foundation of the logical data warehouse pattern developed in Chapter 6: physically separate items, logically one queryable surface [17].

Common Pitfall

Three-part cross-item queries are convenient but read the remote item’s Delta files at query time. A dashboard that cross-joins a large lakehouse table on every render will burn capacity and add latency. Materialize hot joins into warehouse tables when the access pattern is interactive and repetitive.

5.4 Wednesday: Ingestion Patterns

Fabric offers three first-class ways to fill a warehouse. Choosing among them is a daily architecture decision, and the choice has cost, latency, and operability consequences.

Pattern		Strengths	Weaknesses	Best for
Pipeline activity	Copy	Visual authoring, 30+ connectors, scheduling, retry, orchestration	Less expressive than SQL for set-based transforms; per-activity authoring overhead	Source-to-landing movement from many heterogeneous systems
T-SQL (COPY INTO, CTAS, procedures)		Set-based power, idempotent by design, versionable as code, no extra items	Requires files already staged in storage; SQL skills required	OneLake-staged files, stored-procedure migrations, code-reviewed ELT
Dataflows Gen2		Low-code Power Query transformations, self-service analysts	Harder to unit-test at scale; visual logic drifts without discipline	Analyst-owned cleansing into staging tables

Table 5.2: The three warehouse ingestion patterns and when each earns its place.

5.4.1 Idempotent Load Design

Whichever pattern lands the data, the load must be rerunnable. Production guidance for the week: stage files under date-partitioned OneLake paths, drive loads from a manifest or watermark table, and design retries so re-running the same batch never produces duplicate rows. On failure, rerun the batch—do not hand-patch partial tables.

```

1 -- Load only files not yet recorded in the manifest, then record the batch.
2 COPY INTO dbo.stg_sales
3 FROM 'https://onelake.dfs.fabric.microsoft.com/<workspace>/<lakehouse>/Files/
   stage/sales/2026-07/'
4 WITH ( FILE_TYPE = 'PARQUET' );
5
6 INSERT cfg.load_manifest (batch_path, rows_loaded, loaded_ts)
7 VALUES ('stage/sales/2026-07/', @@ROWCOUNT, CURRENT_TIMESTAMP);

```

Listing 5.2: Manifest-driven idempotent load: only unprocessed files load, and the manifest records the batch.

Note

A load is idempotent when running it twice produces the same final state as running it once. Manifests, watermarks, MERGE-based upserts, and partition-scoped overwrites are the four standard tools. Hand-editing rows after a partial failure is none of them.

5.5 Thursday: Hands-on Project

The Week 5 lab provisions a warehouse and loads five million rows of Parquet data from a OneLake path using `COPY INTO` issued from the SQL query editor. You will measure load time, validate row counts, and inspect the Delta files the load produced.

5.5.1 Goal and Architecture

Create a workspace named `fabric-de-week05`. Add a lakehouse `lh_stage` to hold the source Parquet files and a warehouse `dw_sales` as the load target. Generate or obtain a synthetic sales dataset of at least five million rows partitioned by month under `Files/stage/sales/`. The architecture is deliberately simple: OneLake files on one side, a warehouse table on the other, and one SQL statement between them.

5.5.2 Step-by-Step Actions

1. Create the workspace and assign it to your Fabric trial capacity.
2. Create the lakehouse `lh_stage` and upload or generate the partitioned Parquet dataset under `Files/stage/sales/yyyy-MM/`.
3. Create the warehouse `dw_sales` in the same workspace.
4. In the warehouse's SQL editor, create the target schema and staging table.
5. Execute `COPY INTO` against one month, record duration and row count.
6. Execute `COPY INTO` for the remaining months, recording the manifest after each.
7. Validate the final row count and aggregate totals against the source files.
8. Inspect the table's underlying Delta files in OneLake to connect the SQL abstraction back to open storage.

```

1 CREATE SCHEMA stg;
2 GO
3
4 CREATE TABLE stg.sales (
5     order_id      BIGINT,
6     customer_id   BIGINT,
7     product_id    INT,
8     store_id      INT,
9     order_date    DATE,
10    quantity      INT,
11    net_amount     DECIMAL(18,2)
12 );
13 GO
14
15 -- Time the load in the query editor.
16 COPY INTO stg.sales
17 FROM 'https://onelake.dfs.fabric.microsoft.com/<workspace-id>/lh_stage/Files/
18      stage/sales/2026-07/'
19 WITH ( FILE_TYPE = 'PARQUET' );
20
21 INSERT cfg.load_manifest (batch_path, rows_loaded, loaded_ts)
22 VALUES ('stage/sales/2026-07/', @@ROWCOUNT, CURRENT_TIMESTAMP);
23 GO
24
25 -- Validation: counts and totals must match the source profile.
26 SELECT COUNT(*) AS loaded_rows,
27        SUM(net_amount) AS total_net_amount,
28        MIN(order_date) AS first_order,
29        MAX(order_date) AS last_order
29 FROM stg.sales;
```

Listing 5.3: Week 5 lab: create the target and load five million rows.

Tip

Find the exact OneLake DFS path for a lakehouse folder by right-clicking the folder in the lakehouse explorer and copying its ABFS or DFS path, then substituting your workspace and item identifiers. A wrong segment in this path is the single most common cause of a failed first `COPY INTO`.

5.5.3 Validation Checklist

- `stg.sales` contains exactly five million rows (or your dataset's documented count).

- Sum and date-range aggregates match a profile computed from the source Parquet files.
- Every loaded batch appears once in `cfg.load_manifest`.
- Re-running the same `COPY INTO` batch is either blocked by the manifest check or proven to duplicate rows—and you can explain which.
- The warehouse table is visible as Delta-Parquet files in OneLake storage.
- Load durations are recorded for the Friday cost discussion.

5.6 Friday: Use Case and Architecture

A regional logistics company is migrating off an on-premises SQL Server data warehouse. The estate contains roughly 400 stored procedures, nightly batch windows, and a finance team that lives in T-SQL. Leadership asks two questions: what will the Fabric capacity cost, and should the target be a lakehouse or a warehouse?

5.6.1 Capacity and Cost Reasoning

Fabric bills compute through capacity units. Four licensing shapes dominate the decision: reserved F-SKU capacity for steady production load, pay-as-you-go F SKUs for bursty or business-hours workloads, legacy Power BI Premium licensing for BI-heavy estates, and the 60-day trial used throughout this book [35, 37].

Option	Billing model	Best for	Watch-outs
Fabric capacity reservation (F2–F64)	Reserved CUs, monthly/annual commit	Steady production workloads with predictable load	Paying for idle CUs off-hours; throttling if under-sized
Fabric pay-as-you-go (F2–F64)	Per-second CU billing; pause/resume any time	Dev/test, bursty or business-hours-only workloads	Easy to forget to pause; un-optimized Spark jobs spike cost
Power BI Premium (P SKU / PPU)	Per-capacity or per-user monthly	BI-heavy estates already on Premium licensing	P SKUs being superseded by F SKUs; PPU limits sharing
Fabric free trial	60-day trial capacity, no cost	Learning, proofs of concept, this syllabus	Expires; unsuitable for production SLAs

Table 5.3: Fabric licensing shapes for warehouse workloads.

The estimation workflow is empirical: run the Thursday lab’s measured load times and the company’s nightly window against the Capacity Metrics app [34], compute CU-seconds per batch, add interactive query load, then size the smallest SKU that absorbs the peak with headroom. Pause pay-as-you-go capacities outside business hours.

5.6.2 Lakehouse or Warehouse for the Migration

The deciding factor is the 400 stored procedures. Rewriting them as Spark notebooks is a multi-quarter rewrite with regression risk; porting them to warehouse T-SQL is mostly a compatibility exercise. The recommended architecture lands data through idempotent `COPY INTO` batches into the warehouse, ports procedures in waves with side-by-side reconciliation against the legacy system, and exposes curated marts through the SQL analytics endpoint or Direct Lake semantic

models. Lakehouses still play a role—Bronze landing of raw extracts and any future data-science workloads—but the transactional transformation heart of this migration belongs in the warehouse.

Common Pitfall

A lift-and-shift that copies stored procedures without idempotency analysis reproduces legacy bugs at cloud speed. Before porting each procedure, document its retry unit: if the nightly job failed halfway, what must be true before it can safely rerun?

5.7 Operational Guidance for Week 5

Treat the warehouse as production infrastructure from day one: name batches, log loads, reconcile counts, and keep load code in version control. The first time a stakeholder asks “why does this number differ from last night’s report,” the manifest table is your answer.

5.7.1 Performance and Reliability

Load during off-peak windows when the capacity is shared with interactive BI. Prefer fewer, larger files for `COPY INTO`; thousands of tiny staged files recreate the small-file problem inside your ingestion layer. Reconcile every batch: row count, aggregate checksum, and manifest entry. Keep stored procedures under source control and deploy them through the same review process as Spark code.

5.7.2 Security and Governance Checklist

- Store no credentials in procedure bodies; use workspace-managed connections and identity-based access to OneLake paths.
- Restrict who can run `COPY INTO` and DDL in production warehouses.
- Record load manifests with batch path, row count, operator or pipeline identity, and timestamp.
- Classify warehouse tables that contain personal data; downstream chapters apply sensitivity labels and RLS.
- Document the retry unit of every stored procedure before it is ported or scheduled.

5.8 Interview Q&A

1. Q: Why does compute–storage separation matter for a warehouse?

A: It lets storage scale with OneLake while compute scales with capacity SKUs, keeps data safe when compute pauses or fails, and opens the same Delta files to other engines without export pipelines.

2. Q: When is `COPY INTO` preferable to a pipeline Copy activity?

A: When files are already staged in storage and the team wants set-based, versionable, idempotent SQL loads—especially for stored-procedure-centric estates migrating to Fabric.

3. Q: What is the unit of Fabric capacity billing?

A: The capacity unit (CU); consumption is metered in CU-seconds across all workloads sharing the capacity.

4. **Q: Why pause a pay-as-you-go capacity outside business hours?**

A: Per-second billing means an idle capacity still accrues cost; pausing dev/test capacities overnight and on weekends removes spend that produces no value.

5. **Q: Which ingestion pattern best fits a stored-procedure-heavy migration?**

A: T-SQL-centric ELT into a warehouse: `COPY INTO` for staged files, procedures for transformation, preserving the team’s existing skills and the procedures’ business logic.

6. **Q: What does the Fabric warehouse store physically?**

A: Delta-Parquet files in OneLake, in a managed area owned by the warehouse item—open storage that Spark and SQL endpoints can also read.

5.9 Further Reading

Review Microsoft documentation for the Fabric Data Warehouse [58], its T-SQL surface area [48], `COPY INTO` [14], and cross-database querying [17]. For capacity economics, study Fabric licensing concepts [35], the Capacity Metrics app [34], and the trial capacity terms [37]. The lakehouse comparison draws on the lakehouse overview [53] and the lakehouse architecture paper [5].

Key Takeaways

- The Fabric Data Warehouse is a transactional T-SQL warehouse whose tables are Delta-Parquet files in OneLake.
- Compute–storage separation makes capacity a scaling dial that never touches stored data, and makes warehouse data readable by Spark and SQL endpoints alike.
- `COPY INTO` is the highest-throughput, most idempotent-friendly way to load staged Parquet files into warehouse tables.
- Pipelines, T-SQL, and Dataflows Gen2 are complementary ingestion patterns; choose by team skill, transformation shape, and operability, not fashion.
- Idempotent loads—manifests, watermarks, and rerunnable batches—are the difference between an operable warehouse and a nightly incident.
- Capacity cost estimation is empirical: measure CU consumption on real loads, then size and pause capacities deliberately.
- Legacy SQL Server migrations with heavy stored procedures usually belong in the warehouse first, with lakehouses serving raw landing and data-science needs.

5.10 Exercises

1. **(Conceptual)** Explain compute–storage separation to a DBA whose entire career has been coupled appliances. What fears does it resolve, and what new responsibilities does it create?
2. **(Conceptual)** Why is the Fabric warehouse’s T-SQL surface “broad but not identical” to SQL Server? Name two features that have no meaning without an instance.
3. **(Hands-on)** Extend the lab: load a second month of Parquet data, then write a query that proves the manifest contains exactly one entry per batch.

4. **(Hands-on)** Convert the lab's `COPY INTO` into a parameterized stored procedure that takes a batch path and refuses to load a batch already present in the manifest.
5. **(Design)** Given a nightly window of 90 minutes and a measured load of 20M rows per hour on an F8, size the capacity for 60M nightly rows with 25% headroom. State your assumptions.
6. **(Design)** Sketch the wave plan for migrating 400 stored procedures: how do you group, port, and reconcile them against the legacy system?
7. **(Interview)** In five sentences, defend choosing the warehouse over the lakehouse for a T-SQL-centric migration.
8. **(Operational)** Write the runbook entry for a failed nightly `COPY INTO`: detection, triage, safe rerun, and escalation.

5.11 Capstone Project: Idempotent Warehouse Load Framework

Capstone Project

Mission. Build a production-shaped warehouse loading framework for a fictional distributor. The framework must stage partitioned Parquet in OneLake, drive `COPY INTO` loads from a manifest table, reject duplicate batches, reconcile every load, and expose a curated mart for downstream SQL consumers. This is the Week 5 Thursday project raised to operational standards.

Estimated time. 4–6 hours, depending on dataset generation and tenant permissions.

What you'll practice.

- Provisioning a warehouse and lakehouse pair in one workspace.
- Designing manifest-driven, idempotent `COPY INTO` batches.
- Writing T-SQL stored procedures that transform staging into a conformed mart.
- Reconciling row counts and aggregates between source, staging, and mart.
- Measuring CU consumption and producing a capacity cost estimate.

Scenario

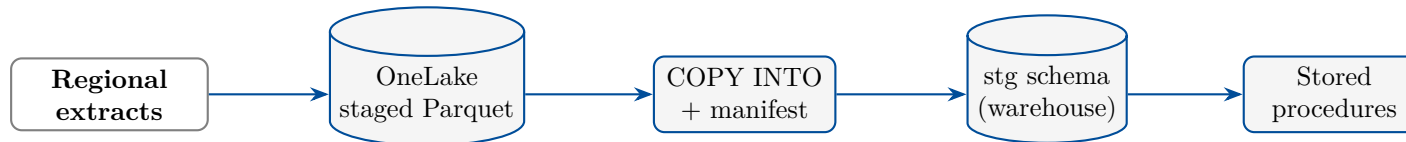
Contoso Distribution receives daily sales extracts from three regional systems as Parquet files dropped into OneLake under date-partitioned paths. The analytics team needs a `mart.daily_sales` table, refreshed nightly, that finance can query with confidence. Extracts occasionally arrive twice after upstream retries; the framework must absorb duplicates without double-counting revenue.

Requirements

- A workspace `fabric-de-ch5-capstone` with lakehouse `lh_stage` and warehouse `dw_distribution`.
- Staged Parquet under `Files/stage/sales/<region>/<yyyy-MM>/` for at least two regions and five days.
- A `cfg.load_manifest` table recording batch path, region, row count, status, and timestamps.
- A stored procedure `cfg.usp_load_batch` that loads one batch only if the manifest has no successful entry for it.

- A stored procedure `mart.usp_build_daily_sales` that rebuilds the mart from staging idempotently.
- Reconciliation queries comparing staged counts, manifest counts, and mart aggregates.
- A one-page capacity estimate built from measured load times.

Reference Architecture



The manifest gates every load: a batch that already succeeded is skipped, so upstream retries cannot double-count revenue.

Figure 5.1: Week 5 capstone: manifest-driven warehouse load framework.

Implementation Steps

1. Create the workspace, lakehouse, and warehouse; generate or upload the staged Parquet extracts.
2. Create `cfg.load_manifest` and the `stg.sales` target table.
3. Implement `cfg.usp_load_batch` with the manifest guard and row-count recording.
4. Implement `mart.usp_build_daily_sales` as a rerunnable rebuild (delete-and-rebuild by batch window or full refresh, your documented choice).
5. Load all batches, including a deliberate duplicate drop, and prove the mart is unaffected.
6. Run reconciliation queries and capture results as evidence.
7. Measure load durations, read the Capacity Metrics app, and write the cost estimate.

```

1 CREATE PROCEDURE cfg.usp_load_batch
2     @batch_path NVARCHAR(500),
3     @region NVARCHAR(50)
4 AS
5 BEGIN
6     IF EXISTS (
7         SELECT 1 FROM cfg.load_manifest
8         WHERE batch_path = @batch_path AND status = 'Succeeded'
9     )
10    BEGIN
11        -- Idempotent: a retried batch is a no-op.
12        RETURN;
13    END
14
15    DECLARE @full_path NVARCHAR(1000) =
16        'https://onelake.dfs.fabric.microsoft.com/<ws>/lh_stage/Files/'
17        + @batch_path + '/';
18
19    COPY INTO stg.sales
20    FROM @full_path
21    WITH ( FILE_TYPE = 'PARQUET' );
  
```

```
22
23     INSERT cfg.load_manifest (batch_path, region, rows_loaded, status,
24     loaded_ts)
25     VALUES (@batch_path, @region, @@ROWCOUNT, 'Succeeded', CURRENT_TIMESTAMP);
26 END
```

Listing 5.4: Capstone manifest guard: skip batches that already succeeded.

Deliverables

- Workspace, lakehouse, and warehouse names with a screenshot or export of the item list.
- The manifest table contents after loading all batches, including the rejected duplicate attempt.
- Both stored procedures under version control.
- Reconciliation query results: staged versus manifest versus mart.
- The capacity cost estimate with measured load times and SKU recommendation.

Acceptance Criteria

- ☐ Every batch is loaded exactly once; a re-dropped extract does not change mart totals.
- ☐ The manifest records path, region, rows, status, and timestamp for every attempt.
- ☐ The mart procedure is rerunnable and produces identical results on consecutive runs.
- ☐ Reconciliation proves staged rows equal manifest rows equal mart contributions.
- ☐ The capacity estimate cites measured durations, not guesses.
- ☐ No credentials, personal paths, or hardcoded workspace IDs appear in submitted code.

Stretch Goals

- Add a `cfg.load_errors` table and route rejected rows from a dirty CSV batch into it.
- Wrap the batch procedure calls in a Fabric pipeline with a `ForEach` over a region list.
- Add a mart-level data-quality test that fails when daily revenue deviates by more than three standard deviations.
- Cross-query the mart from a lakehouse SQL analytics endpoint and document when you would materialize instead.

SQL Analytics Endpoint

Learning Objectives

- Explain what the automatic SQL analytics endpoint is, why it is read-only, and how it reflects lakehouse Delta tables.
- Query lakehouse tables through the endpoint from the web editor and from SQL Server Management Studio.
- Create reusable SQL objects—views, table-valued functions, and stored procedures—over lakehouse Delta tables.
- Enforce object-level security and row-level security in the SQL endpoint.
- Describe what happens when a query crosses a shortcut through the endpoint.
- Design a logical data warehouse that lets analysts query raw lake files in pure T-SQL without duplicating data.

Cross-Cloud Equivalence

The read-only SQL surface over open lake storage is the defining lakehouse pattern, and every cloud has a version of it. Fabric's SQL analytics endpoint is the Azure-native expression; the equivalents differ mostly in how much they hide the files.

Capability	Fabric	AWS	GCP	Snowflake
SQL over lake files	SQL analytics endpoint (read-only, auto-generated)	Athena over S3; Redshift Spectrum	BigQuery external/BigLake tables	External tables; Iceberg tables
Serverless engine	Yes (per-query on capacity)	Athena (per-query/serverless)	Yes (on-demand or slots)	Virtual warehouses
Row-level security	RLS on SQL endpoint objects	Lake Formation row/cell filters	BigQuery row access policies	Row access policies
Column-level security	GRANT/DENY on columns	Lake Formation column filters	Policy tags	Column masking policies
Views/TVFs over lake	Views, TVFs, procedures	Athena views	BigQuery views, TVFs	Views, secure views, UDFs
Open table format	Delta-Parquet in OneLake	Iceberg/Hudi/Delta on S3	Iceberg via BigLake	Iceberg or native

Databricks SQL over Unity Catalog is the closest in-family analog. MongoDB exposes read SQL through the Atlas SQL Interface rather than over lake files. The pattern to carry across clouds: *open files plus a governed SQL layer* delivers warehouse-style access without warehouse-style copies.

Key Terms

SQL analytics endpoint: the automatically generated, read-only T-SQL interface over a lakehouse’s Delta tables. **View:** a saved query that presents filtered, joined, or aggregated lakehouse data as a named relation. **Table-valued function (TVF):** a parameterized view that returns a table. **Object-level security:** grants and denies on schemas, tables, views, and columns. **Row-level security (RLS):** a predicate-based filter that limits which rows a user can see. **Logical data warehouse:** an architecture that queries data in place through SQL objects instead of physically moving it into a warehouse.

6.1 Week 6 Roadmap

Week 6 turns the lakehouse from a Spark artifact into a SQL product. Monday explains the automatic, read-only SQL analytics endpoint that every lakehouse carries. Tuesday builds the reusable SQL object layer—views, TVFs, and stored procedures—over Delta tables. Wednesday covers security: object-level grants and row-level security predicates. Thursday is the hands-on project: connect SSMS to the endpoint and build a view joining a Silver Delta table with a Gold Delta table. Friday designs a full logical data warehouse in which analysts query raw lake files in pure T-SQL without any data duplication.

This week matters because most organizations have far more SQL skills than Spark skills. The SQL analytics endpoint is the bridge that lets the analytics majority consume lakehouse engineering directly [46]. Mastering its object and security model is what separates “a lakehouse the BI team can actually use” from “a pile of Delta files.”

Note

The endpoint is read-only by design: the lakehouse’s write path is Spark (or pipeline/dataflow landing), and the SQL surface is a consumption and governance layer. Trying to use it as an OLTP write surface is the classic Week 6 mistake.

6.2 Monday: The Automatic Endpoint

Every lakehouse in Fabric automatically exposes a SQL analytics endpoint. There is nothing to provision and nothing to size: the endpoint is generated when the lakehouse is created, and it draws compute from the workspace’s capacity per query [46].

Definition

The **SQL analytics endpoint** is a read-only T-SQL interface, generated automatically for every lakehouse, that presents the lakehouse’s Delta tables as queryable SQL tables and keeps them discoverable to T-SQL tools through a standard connection string.

Three properties define its character. First, it is *automatic*: Delta tables written by Spark appear in the endpoint shortly after commit, with no publish step. Second, it is *read-only*: INSERT, UPDATE, and DELETE against lakehouse tables are rejected; writes belong to Spark or ingestion tools. Third, it is *metadata-fresh*: schema changes to Delta tables surface in the endpoint after metadata sync, which is normally near-immediate but can lag briefly under heavy write churn.

Property	SQL analytics endpoint	Synapse Data Warehouse
Provisioning	Automatic with every lakehouse	Explicit item creation
Write support	None over lakehouse tables	Full DML/DDL, transactions
Object model	Views, TVFs, procedures over Delta tables	Native tables plus views, TVFs, procedures
Storage ownership	Lakehouse Tables area in OneLake	Warehouse-managed area in OneLake
Ideal consumer	Analysts reading curated lakehouse data	T-SQL teams owning transformation logic

Table 6.1: The endpoint complements the warehouse; it does not replace it.

Common Pitfall

The endpoint reflects the lakehouse’s **Tables** area, not arbitrary files under **Files**. A CSV sitting in **Files/raw/** is invisible to SQL until something registers it as a Delta table. “My files don’t show up in SSMS” almost always means they were never table-ified.

6.3 Tuesday: Views, TVFs, and Stored Procedures

Although the endpoint cannot write lakehouse tables, it can create SQL objects—views, inline and multi-statement table-valued functions, and stored procedures—that encapsulate query logic over those tables [46]. This is where the endpoint becomes a product surface rather than a raw table dump.

6.3.1 Views as Reusable Joins

A view names a join or filter once so that every report, notebook, and analyst inherits the same definition. When Week 7’s semantic models and Week 8’s Direct Lake reports consume the lakehouse, views are the stabilizing contract that keeps business logic out of individual reports.

```

1 CREATE VIEW gold.v_sales_enriched
2 AS
3 SELECT
4     f.order_id,
5     f.order_date,
6     d.customer_id,
7     d.segment,
8     d.region,
9     f.net_amount
10 FROM gold.fact_sales AS f
11 JOIN gold.dim_customer AS d
12     ON f.customer_key = d.customer_key
13 WHERE d.is_current = 1;
14 GO

```

Listing 6.1: A view joining Silver detail to a Gold dimension.

6.3.2 Table-Valued Functions for Parameterized Access

A TVF is a parameterized view. Use one when the same analytical shape recurs with different filter values—per-region snapshots, as-of-date queries, or tenant-scoped slices.

```

1 CREATE FUNCTION gold.fn_sales_by_region (@region NVARCHAR(50))
2 RETURNS TABLE
3 AS
4 RETURN

```



```

5 (
6     SELECT order_date, customer_id, net_amount
7     FROM gold.v_sales_enriched
8     WHERE region = @region
9 );
10 GO
11
12 -- Usage
13 SELECT * FROM gold.fn_sales_by_region('US');

```

Listing 6.2: An inline TVF for region-scoped sales.

6.3.3 Stored Procedures for Operational Routines

Endpoint stored procedures cannot modify lakehouse tables, but they can orchestrate queries, emit reconciliation results, and encapsulate security-sensitive logic behind `EXECUTE AS` patterns. They are the operational glue of the logical data warehouse.

Tip

Put every multi-line analytical query that two or more consumers share into a view or TVF. The first time a metric definition changes, you will edit one object instead of a dozen reports.

6.4 Wednesday: Security in the SQL Endpoint

The endpoint supports the two security layers that matter most for analytical consumption: object-level security (who can see which tables, views, and columns) and row-level security (which rows within a permitted object a user can see) [46, 44].

6.4.1 Object-Level Security

Standard T-SQL `GRANT`, `DENY`, and `REVOKE` apply to endpoint objects. The pattern for a curated consumption layer is to grant on the Gold schema and views, and deny or simply never grant on Bronze and Silver.

```

1 -- Analysts get the curated view, not the base tables.
2 GRANT SELECT ON gold.v_sales_enriched TO [analysts@contoso.com];
3 DENY SELECT ON silver.customer_pii TO [analysts@contoso.com];
4
5 -- Column-level: hide national ID even from view consumers of base tables.
6 DENY SELECT ON gold.dim_customer (national_id) TO [analysts@contoso.com];

```

Listing 6.3: Object-level and column-level security.

6.4.2 Row-Level Security

RLS attaches a filter predicate to a table through a security policy, so two users running the same query see different rows. The classic implementation uses an inline TVF as the predicate and a mapping table from user to region.

```

1 CREATE TABLE sec.user_region (
2     user_principal NVARCHAR(200),
3     region         NVARCHAR(50)
4 );
5 GO
6
7 CREATE FUNCTION sec.fn_region_predicate (@region NVARCHAR(50))

```

```

8 RETURNS TABLE
9 WITH SCHEMABINDING
10 AS
11 RETURN
12 (
13     SELECT 1 AS allowed
14     FROM sec.user_region
15     WHERE user_principal = USER_NAME()
16           AND region = @region
17 );
18 GO
19
20 CREATE SECURITY POLICY sec.sales_region_policy
21 ADD FILTER PREDICATE sec.fn_region_predicate(region)
22 ON gold.v_sales_enriched_base
23 WITH (STATE = ON);
24 GO

```

Listing 6.4: Row-level security restricting users to their assigned region.

Common Pitfall

RLS filters rows but does not hide the existence of the object, and it composes with object-level security, not instead of it. Grant the object, then filter the rows. Also remember: `USER_NAME()` must resolve to the identity your users actually connect with—test with `EXECUTE AS USER` before declaring victory.

Note

Shortcuts queried through the endpoint execute with the calling identity's access to the target. A shortcut to another lakehouse or to ADLS Gen2 does not bypass permissions; the endpoint enforces them at query time. Design shortcut security as carefully as table security.

6.5 Thursday: Hands-on Project

The Week 6 lab connects SQL Server Management Studio to a lakehouse's SQL analytics endpoint and builds a view that joins a Silver Delta table with a Gold Delta table—the smallest end-to-end proof that lakehouse engineering is consumable in pure T-SQL.

6.5.1 Goal and Architecture

Reuse any lakehouse from Weeks 1–4 that contains a Silver table (for example `silver_orders`) and a Gold table (for example `dim_customer`), or create a fresh workspace `fabric-de-week06` with lakehouse `lh_sales` and seed both tables with a small PySpark script. The deliverable is `gold.v_orders_by_customer`, queried from SSMS.

6.5.2 Step-by-Step Actions

1. Open the lakehouse and copy the SQL analytics endpoint connection string from the lakehouse settings pane.
2. In SSMS, connect with the endpoint string using Microsoft Entra (MFA) authentication.
3. Verify the table list shows the Silver and Gold Delta tables.
4. Create the view joining Silver to Gold, then query it.

5. Add a TVF parameterized by region and test it.
6. Create the `sec.user_region` table, predicate function, and RLS policy from Wednesday's listings; test with `EXECUTE AS USER`.
7. Attempt an `INSERT` into a lakehouse table and record the read-only error—understanding the boundary is part of the lab.

```

1 CREATE VIEW gold.v_orders_by_customer
2 AS
3 SELECT
4     o.order_id,
5     o.order_date,
6     c.customer_id,
7     c.region,
8     o.net_amount
9 FROM silver_orders AS o
10 JOIN dim_customer AS c
11     ON o.customer_id = c.customer_id;
12 GO
13
14 SELECT TOP 20 * FROM gold.v_orders_by_customer ORDER BY order_date DESC;
15
16 -- RLS smoke test: run as a restricted principal
17 EXECUTE AS USER = 'analyst-us@contoso.com';
18 SELECT COUNT(*) AS visible_rows FROM gold.v_orders_by_customer;
19 REVERT;

```

Listing 6.5: Week 6 lab: the Silver-to-Gold view and an RLS smoke test.

6.5.3 Validation Checklist

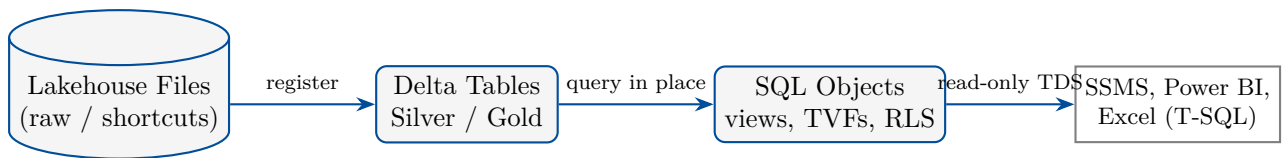
- SSMS connects with Entra authentication and lists the lakehouse tables.
- The view returns joined rows and matches a manual join's row count.
- The TVF filters correctly for at least two region values.
- Under the restricted test user, only US rows are visible; after `REVERT`, all rows return.
- The write attempt fails with a read-only error, and you can explain why that boundary exists.
- The view definition is saved to the team repository as versioned SQL.

6.6 Friday: Use Case and Architecture

An insurance analytics group wants its analysts—fluent in T-SQL, not Spark—to query raw claims files directly, without waiting for a warehouse loading program and without duplicating data into a second store. The answer is a logical data warehouse on the SQL analytics endpoint.

6.6.1 The Pattern

Land raw files in the lakehouse's **Files** area (or shortcut them in place), register curated Delta tables in **Tables**, and expose everything through a layered SQL object model: base tables (Silver/Gold), conformed views, parameterized TVFs, and a security layer of object grants plus RLS. Analysts connect SSMS, Power BI in DirectQuery, or Excel to the endpoint and never touch Spark [46].



No copies: the SQL layer reads the same Delta files Spark wrote.

Figure 6.1: Logical data warehouse: files to governed SQL objects without duplication.

6.6.2 Trade-Offs

The logical pattern eliminates load pipelines and duplication, but it is not free. Queries scan Delta files at query time, so performance depends on the physical tuning of Part I (V-Order, compaction, sensible file sizes). Complex transformations still belong in Spark or a warehouse; the endpoint is a consumption layer, not a transformation engine. And because the endpoint is read-only, any correction workflow must route back through the lakehouse write path.

Common Pitfall

A logical data warehouse fails quietly when the underlying tables are physically unhealthy. If analysts report slow queries, check file counts and compaction before blaming the SQL layer—the endpoint only reads what the lakehouse physically lays down.

6.7 Operational Guidance for Week 6

Treat endpoint objects as a versioned API. Views and TVFs live in source control; grants and RLS policies are reviewed like code; and a change to a view is a contract change for every report above it.

6.7.1 Performance and Reliability

Keep the physical layer healthy (compact, V-Ordered Delta tables), because the endpoint inherits it. Prefer views over repeated ad hoc joins so the optimizer sees stable shapes. Watch metadata sync after heavy Spark writes before demoing to stakeholders. Test RLS with real principals, not assumptions.

6.7.2 Security and Governance Checklist

- Grant on curated Gold objects; never grant directly on Bronze or PII-bearing Silver tables.
- Implement RLS through a predicate function plus a security policy, and test with `EXECUTE AS USER`.
- Deny sensitive columns explicitly where column-level restriction is required.
- Audit who can create or alter views and TVFs in the endpoint.
- Document every view as a data contract: owner, grain, refresh expectations, downstream consumers.
- Verify shortcut targets enforce their own permissions; the endpoint does not weaken them.

6.8 Interview Q&A

1. **Q: Why is the SQL analytics endpoint read-only?**

A: The lakehouse's write path is Spark and ingestion tooling; the endpoint is a consumption and governance layer over the resulting Delta tables, so it exposes queries and objects but rejects DML against the tables themselves.

2. **Q: Which object would you create to reuse a join across reports?**

A: A view—it names the join once so every consumer inherits one definition, and it becomes the contract that stabilizes downstream reports.

3. **Q: How does RLS differ from object-level security?**

A: Object-level security controls *which objects* a user can see; RLS controls *which rows* within a permitted object a user sees, via a filter predicate attached by a security policy.

4. **Q: What happens when you query a shortcut through the endpoint?**

A: The endpoint resolves the shortcut to its target at query time and enforces the caller's permissions on that target—no copy is made and no permission boundary is crossed.

5. **Q: Why connect via SSMS instead of only the web editor?**

A: SSMS provides full T-SQL tooling, source-control-friendly scripts, `EXECUTE AS` security testing, and the connection pattern that external tools and DBAs already use.

6. **Q: What is a logical data warehouse?**

A: An architecture that queries lake data in place through a governed SQL object layer—views, TVFs, RLS—without physically moving data into a separate warehouse store.

6.9 Further Reading

Review Microsoft documentation for the lakehouse SQL analytics endpoint [46], Fabric security fundamentals [44], and cross-database querying [17]. The warehouse comparison draws on the warehouse overview [58] and its T-SQL surface [48]. For the lakehouse storage underneath, revisit the lakehouse overview [53] and Delta Lake documentation [9].

Key Takeaways

- Every lakehouse automatically exposes a read-only SQL analytics endpoint backed by capacity compute.
- The endpoint reflects the **Tables** area—files must be registered as Delta tables before SQL can see them.
- Views, TVFs, and stored procedures turn the endpoint from a table dump into a governed product surface.
- Object-level security and RLS compose: grant the object, then filter the rows, and test with real principals.
- Shortcut queries through the endpoint execute with the caller's permissions on the target; nothing is copied.
- The logical data warehouse lets T-SQL analysts query lake files without duplication, at the cost of inheriting the physical health of the lakehouse.
- Endpoint objects are contracts: version them, review them, and treat changes as downstream-impacting events.

6.10 Exercises

1. **(Conceptual)** Explain to a Spark engineer why the endpoint being read-only is a feature, not a limitation.
2. **(Conceptual)** Compare a view, a TVF, and a stored procedure in the endpoint. Give one scenario where each is the right tool.
3. **(Hands-on)** Add a TVF `gold.fn_customer_orders(@customer_id)` and verify it returns identical results to the equivalent view filter.
4. **(Hands-on)** Extend the RLS lab with a second region and a second test user; prove each user sees only their region.
5. **(Security)** Write the `DENY` statement that hides a salary column from a role that otherwise has `SELECT` on the table, and explain why RLS cannot do that job.
6. **(Design)** Sketch the object layer for the insurance logical data warehouse: which views and TVFs would exist on day one, and who owns each?
7. **(Interview)** In five sentences, defend the logical data warehouse against the charge that it “just moves the bottleneck to query time.”
8. **(Operational)** Write the runbook entry for “analyst reports missing table in SSMS”—metadata sync, table registration, and permission checks.

6.11 Capstone Project: A Governed SQL Consumption Layer

Capstone Project

Mission. Deliver a complete, governed SQL consumption layer over an existing lakehouse: conformed views, parameterized TVFs, object-level security, and tested row-level security—all consumable from SSMS by two analyst personas who must see different rows. This is the Week 6 lab raised to a deployable product.

Estimated time. 3–5 hours on top of an existing Week 1–4 lakehouse.

What you’ll practice.

- Designing a layered SQL object model over Silver and Gold Delta tables.
- Implementing RLS with a mapping table, predicate function, and security policy.
- Testing security as multiple principals from SSMS.
- Versioning endpoint objects as SQL scripts in source control.

Scenario

Contoso’s sales analysts are split into regional teams. Leadership wants one lakehouse, one endpoint, and strict row isolation: a US analyst must never see EU rows, even when running ad hoc queries in SSMS. A shared set of conformed views must give every analyst the same metric definitions.

Requirements

- At least three conformed views over existing Silver/Gold tables, each documented with grain and owner.
- At least one TVF parameterized by region or date.

- A `sec` schema containing the mapping table, predicate function, and an enabled security policy.
- Object grants that expose Gold views to analysts and deny base PII tables.
- Evidence of `EXECUTE AS USER` tests for two personas showing different row counts.
- All objects scripted to `sql/` in the course repository and deployable by re-running the scripts.

Implementation Steps

1. Inventory the lakehouse tables and define the consumption contract (grains, owners, regions).
2. Script the views and TVF; deploy them from SSMS.
3. Build the `sec` schema objects and enable the policy.
4. Grant and deny at the object level.
5. Test as both personas; capture row-count evidence.
6. Commit all scripts and a short deployment note.

Deliverables

- The `sql/` script folder with views, TVF, security objects, and grants.
- A screenshot or transcript of the two-persona row-count test.
- A one-page contract document: view names, grains, owners, consumers.

Acceptance Criteria

- ☐ Both personas connect through SSMS and see only their region's rows in every conformed view.
- ☐ No analyst can select from the base PII table, even directly.
- ☐ All objects redeploy cleanly from scripts into a fresh endpoint.
- ☐ The TVF returns correct results for at least two parameter values.
- ☐ Security objects live in a dedicated schema with documented ownership.
- ☐ The repository contains no credentials or personal connection strings.

Stretch Goals

- Add column-level denies for a salary or national-ID column and prove they survive view access.
- Add a third persona with cross-region access implemented through a second mapping row pattern.
- Script an `ALTER SECURITY POLICY` rollout that adds a second filtered table without downtime.
- Connect Power BI in DirectQuery to the endpoint and show RLS flowing through to report visuals.

Data Modeling for Power BI

Learning Objectives

- Declare the grain of a fact table before adding columns, and distinguish transaction, periodic snapshot, and accumulating snapshot facts.
- Design conformed dimensions shared across fact tables and choose correctly between SCD Type 1 and Type 2 handling.
- Explain the default Power BI semantic model in Fabric workspaces and when to replace it with a custom model.
- Create a custom semantic model, define one-to-many relationships, and write DAX measures.
- Implement an SCD Type 2 upsert against a Gold lakehouse using Delta **MERGE**.
- Resolve circular dependencies in an enterprise model with a bridge-table architecture.

Cross-Cloud Equivalence

Dimensional modeling is platform-independent discipline—Kimball’s method predates every cloud—but each platform has a native place where the model lives. In Fabric it is the Power BI semantic model over OneLake tables.

Capability	Fabric	AWS	GCP	Snowflake
Model home	Power BI semantic model	Redshift tables + QuickSight datasets	BigQuery + Looker semantic layer	Snowflake tables + Snowsight/BI tool models
Star-schema DDL	Warehouse/lakehouse	Redshift (dist/sort keys)	BigQuery (partition/cluster)	Native tables, clustering
SCD2 implementation	Delta MERGE	MERGE / staging upserts	MERGE	MERGE / streams+tasks
Measure language	DAX	SQL / QuickSight calculated fields	LookML / SQL	SQL / BI tool calcs
Denormalization norm	Star over Delta	Star (sort keys)	Often wide nested tables	Star or flattened

Databricks teams model in Delta tables and serve through SQL warehouses or BI tools; MongoDB modeling is document-oriented—embedding versus referencing—which is a different but equally grain-first discipline. The transferable skills are the ones this chapter teaches: declare the grain, conform the dimensions, version the history.

Key Terms

Grain: the precise meaning of one row in a fact table, declared before any column is added. **Conformed dimension:** a dimension with identical keys and meaning shared across multiple fact tables. **SCD Type 1:** overwrite history; old attribute values are lost. **SCD Type 2:** version history with effective dates; every change closes one row and opens another. **Semantic model:** the Power BI layer of tables, relationships, and DAX measures that reports query. **Bridge table:** a resolving table that converts a many-to-many or circular relationship into composable one-to-many paths.

7.1 Week 7 Roadmap

Week 7 moves from storing data to modeling it for humans and measures. Monday is dimensional modeling proper: grain declaration, the three fact table types, conformed dimensions, and SCD Type 1 versus Type 2. Tuesday examines the default semantic model every Fabric workspace generates. Wednesday builds custom semantic models with deliberate relationships. Thursday is the hands-on project: define a Date-to-Sales relationship, write a DAX measure, and implement an SCD Type 2 MERGE against the Gold lakehouse. Friday tackles a classic enterprise modeling failure—circular dependencies—and resolves it with a bridge table.

Modeling is where data engineering meets business meaning. A lakehouse full of clean Delta tables still fails if the model above it double-counts revenue or forgets what a row means. Kimball’s discipline [8] plus Fabric’s semantic model [29] is the combination this week builds.

Note

The single most expensive modeling mistake is skipping the grain declaration. Every double-counting bug, every wrong total, and every “why doesn’t this filter work” traces back to a fact table whose grain was never stated.

7.2 Monday: Kimball Dimensional Modeling

Kimball dimensional modeling organizes analytical data into *facts* (measurements) and *dimensions* (context) [8]. The method begins not with columns but with a sentence: *what does one row of this fact table mean?*

Definition

The **grain** of a fact table is the exact business meaning of a single row—“one row per order line,” “one row per account per day,” “one row per shipment from pickup to delivery.” It is declared first and never silently changed.

7.2.1 The Three Fact Table Types

- **Transaction facts** record one row per business event at the lowest useful grain (order lines, payments, clicks). They are insert-heavy and additive.
- **Periodic snapshot facts** record one row per entity per time period (daily account balances, monthly inventory). They answer “as of” questions.
- **Accumulating snapshot facts** record one row per process instance, updated as milestones complete (order placed, shipped, delivered, returned). Their columns are milestones on a journey.

7.2.2 Conformed Dimensions

A dimension is *conformed* when two fact tables can join the same dimension table and produce comparable results. `dim_date`, `dim_customer`, and `dim_product` shared by sales, returns, and inventory facts are what turn a pile of marts into an enterprise model. Conformance is a governance decision before it is a technical one: someone must own the dimension’s definitions.

7.2.3 Slowly Changing Dimensions

When a customer moves region, what happens to history? **Type 1** overwrites the region—simple, but last year’s “East” sales silently become “West.” **Type 2** closes the current row (`valid_to`, `is_current` = `false`) and inserts a new version—preserving what the world looked like when each fact occurred. Type 2 is required whenever the business asks “as it was” questions or audits decisions made under old attribute values.

Aspect	SCD Type 1	SCD Type 2
History	Lost (overwrite)	Preserved (versioned rows)
Row count	Stable	Grows with changes
Query pattern	Simple joins	Joins must respect <code>is_current</code> or effective dates
Best for	Corrections (typo fixes), irrelevant history	Attribution, audit, “as-was” reporting
Fabric implementation	Delta MERGE update	Two-step MERGE: expire, then insert

Table 7.1: Choosing between SCD Type 1 and Type 2.

Common Pitfall

Mixing Types silently: overwriting `segment` in a dimension that downstream models treat as Type 2 rewrites history without telling anyone. Document the type per attribute, not per table—a dimension often fixes typos with Type 1 on some columns while versioning `region` and `segment` with Type 2.

7.3 Tuesday: The Default Semantic Model

Every lakehouse and warehouse in Fabric automatically exposes a *default semantic model*: Power BI generates a model whose tables mirror the item’s tables, so a report can be built minutes after data lands [29].

The default model is a convenience, not a deliverable. It carries every table, technical columns and all; it has no deliberate relationships beyond auto-detect guesses, no business-named measures, no security roles, and no curated folder structure. Treat it as scaffolding for exploration.

Note

The default semantic model inherits its tables directly from the lakehouse. That is why Part I’s physical discipline—clean Delta tables, sensible names, no `_corrupt_record` columns leaking into Gold—pays off here: the default model can only be as presentable as the tables beneath it.

7.3.1 When the Default Model Is Enough

Exploration, prototyping, and sanity checks. The moment a report goes to a stakeholder, graduate to a custom model: rename tables and columns to business language, hide technical keys, declare relationships deliberately, and move calculations into named DAX measures instead of implicit per-visual aggregations.

7.4 Wednesday: Custom Semantic Models and Relationships

A custom semantic model is a first-class Fabric item: you select the tables, define relationships with explicit cardinality and cross-filter direction, and author measures [29, 18].

7.4.1 Relationships

The healthy default is one-to-many from dimension to fact, single cross-filter direction, active. Bidirectional filters and many-to-many relationships exist for legitimate cases but multiply the ways a filter can silently change a total; reach for them only with a written justification.

7.4.2 DAX Measures

DAX (Data Analysis Expressions) is the formula language of semantic models [18]. Measures are evaluated in filter context at query time, which is what makes them reusable across every visual and slice.

```

1 Total Revenue = SUM ( fact_sales[net_amount] )
2
3 Order Count = DISTINCTCOUNT ( fact_sales[order_id] )
4
5 Average Order Value = DIVIDE ( [Total Revenue], [Order Count] )
6
7 Revenue YTD =
8 CALCULATE ( [Total Revenue], DATESYTD ( dim_date[full_date] ) )

```

Listing 7.1: Foundational DAX measures over the star schema.

Tip

Build measures on top of measures: **Average Order Value** referencing **Total Revenue** and **Order Count** means one definition change propagates everywhere. Implicit aggregations in visuals do the opposite—they fork your business logic into every report.

Common Pitfall

Auto-detected relationships guess cardinality from data profiles and are wrong often enough to matter—especially with SCD2 dimensions, where a **customer_id** appears many times in the dimension. Always define relationships on surrogate keys (**customer_key**), never on business keys in a versioned dimension.

7.5 Thursday: Hands-on Project

The Week 7 lab has two halves: in the semantic model, define a one-to-many relationship between a Date dimension and a Sales fact and write a DAX measure; in the lakehouse, implement the SCD Type 2 MERGE below against **gold.dim_customer** and verify that a changed customer attribute closes the old row and opens a new one.

7.5.1 Goal and Architecture

Work in a workspace `fabric-de-week07` with a lakehouse containing `fact_sales`, `dim_date`, and `dim_customer` (SCD2-modeled with `customer_key`, `customer_id`, `segment`, `region`, `is_current`, `valid_from`, `valid_to`). Part one is modeling; part two is the dimensional history pipeline that keeps the model honest.

7.5.2 Step-by-Step Actions

1. Open the semantic model view in the workspace and create a new custom model over the three tables.
2. Define a one-to-many relationship from `dim_date[date_key]` to `fact_sales[date_key]`; repeat for `dim_customer[customer_key]`.
3. Create the four DAX measures from Wednesday's listing; test `Total Revenue` and `Revenue YTD` in a quick visual.
4. In a notebook, run step 1 of the SCD2 merge: expire current rows whose tracked attributes changed.
5. Run step 2: insert new versions for changed attributes and brand-new customers.
6. Verify with a before/after query: the changed customer now has two rows, exactly one current.
7. Refresh the semantic model and confirm the report now attributes facts to the correct customer version.

```

1 # SCD Type 2 upsert into gold.dim_customer via Delta MERGE (PySpark)
2 from delta.tables import DeltaTable
3
4 tgt = DeltaTable.forName(spark, "gold.dim_customer")
5 src = spark.read.table("silver.customer_daily") # today's snapshot
6
7 # Step 1: expire current rows whose tracked attributes changed
8 (tgt.alias("t").merge(src.alias("s"),
9   "t.customer_id = s.customer_id AND t.is_current = true")
10  .whenMatchedUpdate(
11    condition="t.segment <> s.segment OR t.region <> s.region",
12    set={"is_current": "false", "valid_to": "s.change_date"})
13  .execute())
14
15 # Step 2: insert new versions (changed attributes + brand-new customers)
16 (tgt.alias("t").merge(src.alias("s"),
17   "t.customer_id = s.customer_id AND t.is_current = true"
18   " AND t.segment = s.segment AND t.region = s.region")
19  .whenNotMatchedInsert(values={
20    "customer_id": "s.customer_id", "segment": "s.segment",
21    "region": "s.region", "is_current": "true",
22    "valid_from": "s.change_date", "valid_to": "null"})
23  .execute())

```

Listing 7.2: SCD Type 2 upsert into `gold.dim_customer` via Delta MERGE.

The two-step structure is essential: the first `MERGE` closes the current row, and the second—whose match condition now fails for just-expired rows—inserts the successor version. Running them in the wrong order, or as one combined step, either double-inserts or loses the closing date.

7.5.3 Validation Checklist

- The relationship uses `date_key/customer_key` surrogate keys, one-to-many, single direction.
- `Total Revenue` in a visual equals a direct SQL `SUM` on the fact table.
- A customer with a changed `segment` has exactly two dimension rows; exactly one has `is_current = true`.
- The old row's `valid_to` equals the new row's `valid_from`.
- Facts are attributed to the customer version that was current at the fact's date.
- Re-running the merge against the same snapshot changes nothing (idempotence).

7.6 Friday: Use Case and Architecture

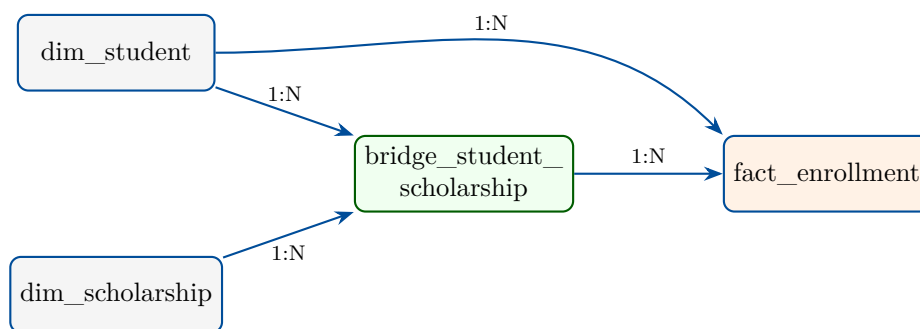
An enterprise model for a university system has collapsed into ambiguity: **Students**, **Enrollments**, **Courses**, and **Scholarships** form a loop. Power BI deactivated one relationship to break the cycle, and now scholarship reports filter enrollments incorrectly. The task is to restore correct filtering with a bridge-table architecture.

7.6.1 Diagnosing the Cycle

Circular dependencies arise when two alternative filter paths exist between the same tables. The engine resolves the ambiguity by deactivating a path, and the deactivated path is always the one someone needed. The fix is never to force both paths active; it is to restructure so that only one unambiguous path remains.

7.6.2 The Bridge Table Pattern

Identify the many-to-many heart of the loop—students hold multiple scholarships; a scholarship covers multiple students—and factor it into a bridge table: `bridge_student_scholarship(student_key, scholarship_key)`. Both dimensions relate one-to-many into the bridge; facts relate through the bridge or directly to dimensions; every filter path is now a composable chain of one-to-many hops.



*After bridging, every filter path is one-to-many;
no ambiguity remains for the engine to “resolve.”*

Figure 7.1: Resolving a circular dependency with a bridge table.

Common Pitfall

A bridge table changes query semantics: filtering by scholarship now flows to students through the bridge, which can surprise report authors expecting a direct relationship. Document the bridge's existence and grain in the model, and hide its keys from report view.

7.7 Operational Guidance for Week 7

Models are contracts. Publish a modeling standard—grain declarations, surrogate-key rules, SCD types per attribute, measure naming—and review every semantic model change against it. A model that quietly redefines **Total Revenue** causes more organizational damage than most outages.

7.7.1 Performance and Reliability

Keep models narrow: hide technical columns, remove unused tables, and prefer integer surrogate keys over wide string keys for relationship columns. Star beats snowflake for Power BI: each extra hop between fact and dimension costs filter-propagation work on every visual. Version the SCD2 merge like any pipeline—it is one, and its failures silently corrupt history.

7.7.2 Security and Governance Checklist

- Declare grain and SCD type per attribute in the model documentation.
- Assign an owner to every conformed dimension.
- Define relationships on surrogate keys with explicit cardinality; justify every bidirectional filter in writing.
- Keep all business calculations in named DAX measures; prohibit per-visual implicit aggregation in certified reports.
- Validate SCD2 merges with a uniqueness test: one current row per business key.
- Record which reports depend on which measures before changing a measure definition.

7.8 Interview Q&A

1. **Q: Why must you declare the grain before adding fact columns?**

A: Every column and every aggregation is only meaningful relative to the grain. Undeclared grain is how two reports produce two different “correct” revenue numbers.

2. **Q: Name the three Kimball fact table types.**

A: Transaction facts (one row per event), periodic snapshots (one row per entity per period), and accumulating snapshots (one row per process instance, updated at milestones).

3. **Q: What makes a dimension conformed?**

A: Shared keys and identical business meaning across fact tables, so sales and inventory sliced by the same customer dimension produce comparable results.

4. **Q: When is SCD Type 2 required instead of Type 1?**

A: When the business must answer “as it was” questions—attribution, audit, or any report that must reflect the attribute values in effect when the fact occurred.

5. **Q: In the MERGE snippet, what closes the current SCD2 row?**

A: Step 1's `whenMatchedUpdate`, which fires when the business key matches, `is_current` is true, and a tracked attribute differs—setting `is_current = false` and stamping `valid_to`.

6. **Q: How does a bridge table resolve a circular dependency?**

A: It factors the many-to-many heart of the loop into its own table, converting every filter path into composable one-to-many hops, so the engine never faces ambiguous alternative paths.

7.9 Further Reading

Read Kimball's dimensional modeling method [8] as the canonical reference. For Fabric specifics, review semantic model documentation [29], the DAX reference [18], and the Direct Lake overview that Week 8 builds upon [24]. The SCD2 merge relies on Delta Lake semantics [9, 4].

Key Takeaways

- Grain is declared before columns; every downstream number inherits it.
- Transaction, periodic snapshot, and accumulating snapshot facts answer different questions; do not blend them.
- Conformed dimensions are a governance achievement as much as a technical one—they are what make cross-mart analysis comparable.
- SCD Type 2 is implemented as two ordered **MERGE** steps: `expire`, then `insert`. Order is correctness.
- The default semantic model is scaffolding; production reporting deserves a custom model with deliberate relationships and named measures.
- Relationships belong on surrogate keys; DAX measures belong in the model, not in visuals.
- Circular dependencies are resolved by restructuring—a bridge table—never by fighting the engine's ambiguity rules.

7.10 Exercises

1. **(Conceptual)** Declare the grain for: a bank account balance table, a shipment tracking table, and a web clickstream table. Which fact type is each?
2. **(Conceptual)** A dimension fixes a typo in `customer_name` (Type 1) but versions `segment` (Type 2). Explain why per-attribute typing is correct and what breaks if you version the typo fix.
3. **(Hands-on)** Add an accumulating snapshot fact `fact_order_journey` with milestone dates; write one DAX measure computing average days from order to delivery.
4. **(Hands-on)** Extend the SCD2 merge to track a third attribute, `loyalty_tier`, and prove idempotence by running it twice.
5. **(Design)** Draw the bridge-table solution for the university model, including which keys live in the bridge and which relationships remain direct.
6. **(Design)** Propose a conformed `dim_date` that serves sales (daily grain), finance (fiscal calendar), and HR (pay periods). What columns must it carry?

7. **(Interview)** In five sentences, explain why Power BI deactivates a relationship when a model contains a loop, and what you do about it.
8. **(Operational)** Write the data-quality test that catches a broken SCD2 merge in production (hint: current-row uniqueness and date-chain contiguity).

7.11 Capstone Project: Conformed Star Schema with SCD2 History

Capstone Project

Mission. Build a conformed star schema for a retail group: `fact_sales` and `fact_returns` sharing `dim_date`, `dim_customer` (SCD2), and `dim_product`; a custom semantic model with deliberate relationships and a named measure library; and a tested SCD2 merge pipeline keeping customer history correct. Prove correctness with reconciliation queries and a uniqueness test.

Estimated time. 5–7 hours.

What you'll practice.

- Declaring grains for multiple facts and conforming shared dimensions.
- Implementing and testing an SCD Type 2 pipeline with Delta `MERGE`.
- Building a custom semantic model with surrogate-key relationships and DAX measures.
- Validating a model with reconciliation queries and uniqueness tests.

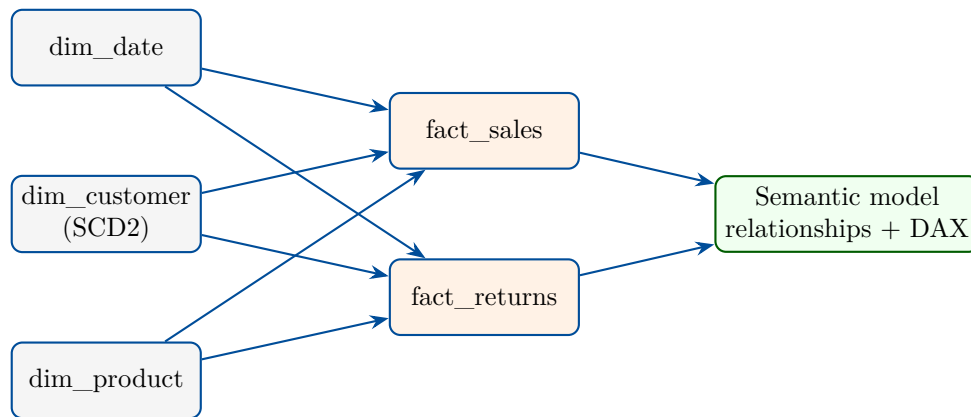
Scenario

Contoso Retail reports sales and returns separately, and executives suspect the two reports disagree because they use different customer and date logic. Your job: one conformed star where both facts share dimensions, customer history is versioned, and `Net Revenue = Sales - Returns` is a single governed measure.

Requirements

- Written grain declarations for both facts, committed to the repository.
- `dim_date`, `dim_customer` (SCD2 columns), and `dim_product` in the Gold lakehouse, shared by both facts.
- The two-step SCD2 merge implemented as a parameterized notebook.
- A custom semantic model: surrogate-key relationships, single-direction filters, hidden technical columns.
- A measure library: `Gross Revenue`, `Return Amount`, `Net Revenue`, `Net Revenue YTD`.
- A uniqueness test proving one current customer row per `customer_id`.
- Reconciliation: model totals equal direct SQL aggregates on the facts.

Reference Architecture



Conformance: both facts join the same dimension tables, so every report slices by identical definitions.

Figure 7.2: Week 7 capstone: conformed star schema feeding one semantic model.

Implementation Steps

1. Write and commit the grain declarations; review them before building anything.
2. Build the three dimensions and two facts in Gold from Silver sources.
3. Implement the SCD2 merge notebook with the uniqueness test as a final assertion.
4. Create the custom semantic model and wire relationships on surrogate keys.
5. Author the measure library; verify **Net Revenue** against SQL.
6. Change a customer's **segment**, rerun the merge, and prove history is preserved and the model still reconciles.

Deliverables

- Grain declarations and the modeling standard excerpt used.
- The SCD2 notebook with test output.
- Screenshots of the model view (relationships) and the measure library.
- Reconciliation evidence: SQL aggregates versus model measures.
- The uniqueness test result before and after a customer change.

Acceptance Criteria

- ☐ Grain declarations exist, are specific, and match the physical fact tables.
- ☐ Both facts share all three dimensions; no per-fact dimension copies exist.
- ☐ The SCD2 merge is idempotent and passes the uniqueness test after every run.
- ☐ All relationships are one-to-many, single-direction, on surrogate keys.
- ☐ **Net Revenue** YTD in a visual equals the equivalent SQL aggregation.

- ☐ A customer attribute change produces a closed old row and a new current row with contiguous validity dates.
- ☐ No business logic lives in report visuals; all calculations are named measures.

Stretch Goals

- Add a degenerate dimension (order number) and document why it lives in the fact.
- Add RLS from Chapter 6 so regional managers see only their region's facts.
- Add a bridge table resolving a many-to-many between customers and promotion campaigns.
- Benchmark the model's query time before and after switching relationships to integer surrogate keys.

Power BI Direct Lake Mode

Learning Objectives

- Explain how Direct Lake reads Delta-Parquet directly into memory, and how it differs from Import and DirectQuery.
- Describe framing and what triggers a Direct Lake fallback to DirectQuery.
- Optimize lakehouse tables for Direct Lake, including the specific role of V-Order.
- Build a Direct Lake report against a Gold lakehouse and benchmark it against DirectQuery.
- Detect and diagnose fallback events with the Capacity Metrics app and model telemetry.
- Architect an executive dashboard with sub-second response over a 100-million-row dataset without nightly refreshes.

Cross-Cloud Equivalence

Direct Lake is Fabric’s distinctive contribution to BI serving: the semantic engine reads the same open Delta-Parquet files that Spark and SQL use, instead of importing copies or delegating queries. Other platforms approximate the goal differently.

Serving pattern	Fabric	AWS	GCP	Snowflake
BI reads open lake files	Direct Lake over OneLake Delta	QuickSight over Athena/Spectrum	Looker/Looker Studio over BigLake	External/Iceberg tables to BI tools
In-memory BI copy	Power BI Import	QuickSight SPICE	Looker in-memory caches	Cached results / extracts in BI tool
Live query to source	DirectQuery (warehouse/endpoint)	QuickSight direct query	Looker live connection	Live connection to Snowflake
Cache/update control	Framing (auto/on-demand)	SPICE refresh schedule	Cache policies	Result cache (24h) + BI caching
Performance lever	V-Order + column pruning	Sort keys + Spectrum tuning	Partition/cluster pruning	Clustering + result cache

Databricks serves BI through SQL warehouses and partner extracts; MongoDB through the Atlas SQL Interface or charts. The shared insight: *copy for speed versus query for freshness* is the universal BI trade-off—Direct Lake is Fabric’s bid to dissolve it by making the lake itself fast enough to serve.

Key Terms

Direct Lake: a Power BI storage mode that loads Delta-Parquet columns from OneLake into memory on demand—no import copy, no per-query round trip. **Framing:** the

process by which a Direct Lake model binds to the current version of its Delta tables. **Fallback:** Direct Lake’s automatic switch to DirectQuery when a query exceeds guardrails or hits unsupported shapes. **V-Order:** Fabric’s write-time Parquet layout optimization that accelerates the column loads Direct Lake performs. **DirectQuery:** a storage mode that translates visual interactions into live SQL against the source.

8.1 Week 8 Roadmap

Week 8 completes the Part II serving stack. Monday explains Direct Lake conceptually: reading Delta-Parquet directly into memory. Tuesday covers the fallback behavior that silently degrades unprepared models to DirectQuery. Wednesday turns to optimization—V-Order, column pruning, and model hygiene. Thursday is the hands-on project: a Direct Lake report over the Gold lakehouse, benchmarked against a DirectQuery twin. Friday architects the executive dashboard: sub-second response on 100 million rows without nightly refresh delays. The chapter closes Part II with the Milestone 2 capstone.

Direct Lake is the payoff of everything since Week 1. OneLake’s open Delta storage, Part I’s physical optimization, Week 6’s governed SQL surface, and Week 7’s star schema converge here into reports that are simultaneously fresh and fast [24].

Note

“No import, no DirectQuery” is the slogan; the engineering reality is *transcoding*: the semantic engine memory-maps and loads Parquet columns from OneLake as queries demand them, so the first query on a cold column pays a load cost and subsequent queries are in-memory fast.

8.2 Monday: The Direct Lake Concept

Import mode copies data into the semantic model on a schedule: fast queries, stale data, duplicated storage. DirectQuery sends every visual interaction to the source as SQL: fresh data, source-dependent latency, query load on the warehouse. Direct Lake threads the needle by reading the Delta-Parquet files in OneLake directly into the engine’s memory on demand [24].

Definition

Direct Lake is a Power BI storage mode in which the semantic model loads columns from OneLake Delta-Parquet files into memory as queries require them, keeping data as fresh as the last table commit without maintaining a separate imported copy.

Property	Import	DirectQuery	Direct Lake
Data freshness	As of last refresh	Live	As of last Delta commit + framing
Query latency	In-memory fast	Source-dependent	In-memory after column load
Storage duplication	Yes (model copy)	No	No (reads OneLake)
Refresh needed	Yes (scheduled)	No	Framing only (lightweight)
Load on source	Refresh-time bursts	Every interaction	Column reads from OneLake
Row guardrails	Capacity memory	Source limits	Fallback thresholds

Table 8.1: The three Power BI storage modes compared.

8.2.1 When Import Still Wins

Import remains the right choice when the source is slow or expensive to query live, when the model needs full DAX query features without fallback risk on enormous tables, or when data arrives from outside OneLake entirely. Direct Lake is the default for OneLake-resident star schemas; it is not a universal replacement.

8.3 Tuesday: Framing and Fallback

Two mechanisms define Direct Lake operations. *Framing* is how the model points at a table version: when Delta tables commit new data, the model re-frames—a metadata operation far cheaper than an import refresh—so visuals see the new files. *Fallback* is the escape hatch: when a query cannot be served from the lake—because a table exceeds row guardrails for the capacity, a column type is unsupported, or a query shape is incompatible—the engine silently routes it to the SQL analytics endpoint as DirectQuery [24, 50].

Definition

Fallback is Direct Lake’s automatic degradation to DirectQuery for a query or table that cannot be served from OneLake. It preserves correctness but sacrifices Direct Lake latency, and it is silent unless you measure it.

8.3.1 What Triggers Fallback

- A table exceeding the capacity’s Direct Lake row-count guardrail.
- Unsupported column types or certain calculated-column and RLS shapes.
- Queries against SQL views rather than physical Delta tables.
- Memory pressure forcing eviction beyond what re-loading can serve.

Common Pitfall

Fallback is silent and per-query: a report can serve 95% of interactions from the lake and fallback only on the executive’s favorite drill path. Monitor fallback events with the Capacity Metrics app and semantic model telemetry—a slow “Direct Lake” report is often a fallback problem or missing V-Order, not a capacity problem [34].

8.4 Wednesday: Optimizing Models for Direct Lake

Direct Lake performance is the compound interest of Part I’s physical discipline. V-Order applies Fabric’s optimized Parquet layout at write time, which directly accelerates the column loads Direct Lake performs [22, 41]. Beyond V-Order:

- **Prune columns.** Direct Lake loads columns, not rows; a 200-column fact costs column-load time for every column the model includes. Model only what reports use.
- **Compact small files.** Chapter 4’s OPTIMIZE work pays off again: many small files multiply the file-open cost of column reads.
- **Prefer integer keys.** Relationship columns load and join fastest as compact integers.
- **Frame deliberately.** Automatic framing keeps models fresh; for tables updated in bursts, on-demand framing after the batch commit avoids mid-load version flapping.

Tip

Run `OPTIMIZE ... VORDER` on Gold tables as the final step of the pipeline that produces them, so every framing binds to already-optimized files. Re-optimizing after reports exist means the next frame picks up the improvement without any report change.

8.5 Thursday: Hands-on Project

The Week 8 lab creates a Power BI report in Direct Lake mode against the Gold lakehouse and benchmarks it against an identical DirectQuery model, producing evidence rather than slogans.

8.5.1 Goal and Architecture

Use a workspace `fabric-de-week08` with a Gold lakehouse containing a star schema—ideally `fact_sales` at several million rows with conformed dimensions from Chapter 7. Build two semantic models over identical tables: `sm_directlake` (Direct Lake) and `sm_directquery` (DirectQuery). Build one report page per model with identical visuals.

8.5.2 Step-by-Step Actions

1. Verify Gold tables are compacted and V-Ordered; record file counts.
2. Create `sm_directlake` and `sm_directquery` over the same tables with identical relationships and measures.
3. Build the benchmark report page: a card (total revenue), a bar chart (revenue by region), and a matrix (revenue by region and month).
4. Time each visual’s first render and repeated interactions in both models; record results.
5. Trigger a Delta commit (append one batch), then measure how quickly each model reflects it.
6. Open the Capacity Metrics app and capture CU consumption and any fallback events for both models.
7. Force a fallback scenario (query a view instead of a table, or exceed a guardrail on a test table) and capture the telemetry signature.

Measurement	Direct Lake	DirectQuery
First render (cold)	<i>record</i>	<i>record</i>
Repeat interaction (warm)	<i>record</i>	<i>record</i>
Freshness after commit	<i>record</i>	<i>record</i>
CU-seconds per session	<i>record</i>	<i>record</i>
Fallback events observed	<i>record</i>	n/a

Table 8.2: Benchmark evidence template: fill with your measured values.

8.5.3 Validation Checklist

- Both models expose identical measures and produce identical totals.
- The benchmark table is complete with real measured values.
- Direct Lake reflects a new commit after framing without an import refresh.
- At least one fallback event was deliberately produced and identified in telemetry.
- V-Order status and file counts of the Gold tables are recorded.
- You can state, with numbers, when DirectQuery would still be the right choice.

8.6 Friday: Use Case and Architecture

The executive team wants a live revenue dashboard: sub-second interactions over a 100-million-row fact, updated within minutes of a sale—no nightly refresh window. This is the scenario Direct Lake was built for, and it succeeds or fails on engineering discipline.

8.6.1 The Architecture

Sales land in Bronze, conform through Silver, and publish to a Gold star schema whose `fact_sales` exceeds 100 million rows. Every Gold table is compacted and V-Ordered at the end of its producing pipeline. A Direct Lake semantic model with automatic framing sits on top; reports query the model. The result: a sale committed to Gold appears in the dashboard after the next frame—seconds to minutes—with in-memory interaction speed.

8.6.2 The Guardrail Conversation

A 100-million-row fact sits near capacity-dependent Direct Lake guardrails. The architecture therefore includes: an F SKU sized so the fact stays within guardrails; column pruning so only reported columns load; monitoring of fallback events with alerting (a sustained fallback rate is a page-the-engineer event, not a shrug); and a rehearsed degradation plan—if the fact outgrows guardrails, the options are aggregation tables, capacity scale-up, or targeted DirectQuery for the largest slice.

Common Pitfall

Designing to exactly the guardrail is designing to fail next quarter. Row guardrails are capacity-dependent and change over releases; leave 30–50% headroom and monitor growth rate, because “it fell back” is discovered by executives as “the dashboard is slow.”

Tip

Pair the Direct Lake dashboard with a Data Activator alert (Chapter 15) on capacity and fallback events. The same real-time muscle that watches IoT temperatures can watch your own platform.

8.7 Operational Guidance for Week 8

Treat fallback as a first-class production signal. The Capacity Metrics app, queried regularly, tells you what users experience before they report it.

8.7.1 Performance and Reliability

Optimize Gold physically before tuning the model. Keep models narrow, keys integer, relationships single-direction. Frame after batch commits rather than mid-pipeline. Benchmark on a schedule—row growth silently changes the performance envelope.

8.7.2 Security and Governance Checklist

- Certify Direct Lake models before executives depend on them; mark exploratory models clearly.
- Apply RLS (Chapters 6 and 21) knowing that some RLS shapes influence fallback behavior—test the combination.
- Restrict who can repoint or re-frame production models.
- Monitor capacity so interactive BI and batch loads do not starve each other.
- Document the degradation plan for guardrail overflow.

8.8 Interview Q&A

1. Q: What triggers a Direct Lake fallback to DirectQuery?

A: Row-count guardrails exceeded for the capacity, unsupported column types or query shapes, queries against views instead of physical tables, or memory pressure—each routes that query to the SQL endpoint silently.

2. Q: Why does V-Order matter specifically for Direct Lake?

A: Direct Lake loads Parquet columns from OneLake at query time; V-Order's write-time layout optimization makes exactly those column reads faster, so cold-query latency drops without any model change.

3. Q: What does framing do in a Direct Lake model?

A: Framing binds the model to the current Delta table versions—a metadata operation that replaces the import refresh, making newly committed data visible without copying it.

4. Q: Which telemetry reveals fallback events in production?

A: The Capacity Metrics app and semantic model telemetry: fallback appears as DirectQuery-mode query load where you expected lake reads.

5. Q: When is Import mode still preferable to Direct Lake?

A: When the source lives outside OneLake, when the source is too slow to serve live column

reads, or when the model needs Import-only DAX features without fallback risk on very large tables.

6. **Q: How does Direct Lake achieve freshness without refresh?**

A: It reads the same Delta files the pipelines write; framing re-points the model at new versions, so a committed sale appears after the next frame rather than the next import window.

8.9 Further Reading

Start with the Direct Lake overview [24] and fallback behavior documentation [50]. The physical prerequisites are V-Order [22] and Delta optimization [41]. Operations rely on the Capacity Metrics app [34] and semantic model fundamentals [29]. The storage foundation remains Delta Lake [9, 4].

Key Takeaways

- Direct Lake loads Delta-Parquet columns from OneLake into memory on demand—Import’s speed trajectory with DirectQuery’s freshness.
- Framing is the lightweight metadata operation that replaces the import refresh.
- Fallback to DirectQuery is silent, per-query, and correct-but-slow; it is the top silent killer of “Direct Lake” performance.
- V-Order, compaction, and column pruning are Direct Lake optimizations applied at the storage layer, before the model exists.
- Benchmark with evidence: identical models, identical visuals, measured latencies, captured telemetry.
- Guardrails are capacity-dependent; design with headroom and a rehearsed degradation plan.
- Import and DirectQuery remain legitimate choices—Direct Lake is the default for OneLake star schemas, not a religion.

8.10 Exercises

1. **(Conceptual)** Explain transcoding to a BI developer who has only ever used Import mode.
2. **(Conceptual)** Why is fallback worse than simply choosing DirectQuery for a table? Consider user experience and diagnostics.
3. **(Hands-on)** Complete the benchmark table with your own measured values and write three sentences of interpretation.
4. **(Hands-on)** Deliberately trigger fallback three different ways (view, oversized table, unsupported shape) and capture each telemetry signature.
5. **(Design)** Size the capacity and model design for a fact growing from 100M to 200M rows over 18 months; include the trigger points for your degradation plan.

6. **(Design)** An acquisition lands: the new subsidiary’s data lives in an on-premises SQL Server with no path to OneLake this quarter. Choose and defend storage modes for a combined report.
7. **(Interview)** In five sentences, explain why Direct Lake dissolves the freshness-versus-speed trade-off, and where it reappears anyway.
8. **(Operational)** Write the alert rule and runbook for a sustained fallback rate on the executive dashboard.

8.11 Milestone 2 Capstone: End-to-End Analytics Workflow

Capstone Project

Mission. Close Part II by building the complete warehousing and BI serving stack: a Synapse Data Warehouse loaded by idempotent `COPY INTO` batches, T-SQL stored procedures that build a star schema from raw OneLake data, a custom semantic model with relationships and DAX measures, and a published Direct Lake Power BI report—with benchmarks, RLS, and a capacity cost estimate as evidence.

Estimated time. 8–10 hours across the milestone’s components.

What you’ll practice.

- Manifest-driven `COPY INTO` loads into warehouse staging.
- Stored procedures building dimensions and facts, with SCD2 where history matters.
- Custom semantic model authoring: surrogate-key relationships, measure library, RLS roles.
- Direct Lake versus DirectQuery benchmarking with fallback telemetry.
- Producing the six rubric artifacts: runnable repo, diagram, tests, monitoring evidence, cost estimate, security review.

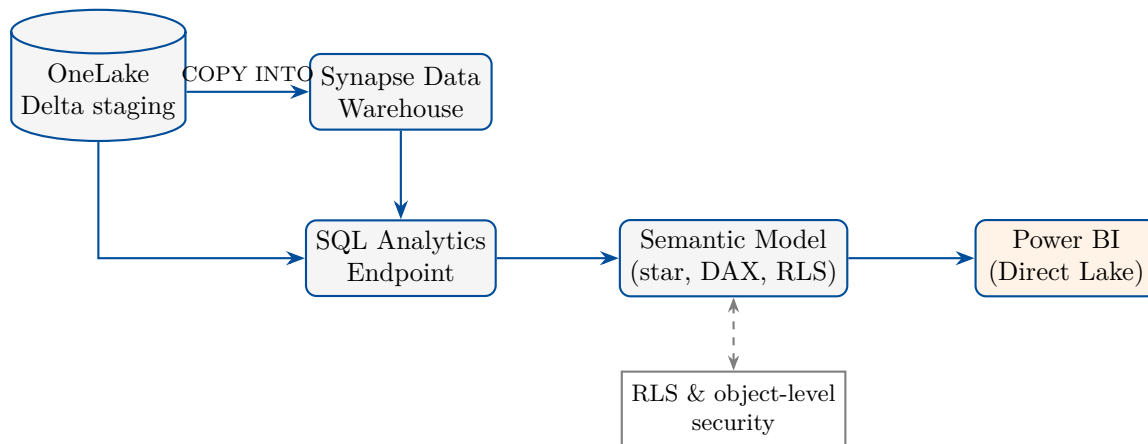
Scenario

Contoso Retail’s analytics program graduates from lakehouse exploration (Part I) to a governed serving layer. Raw order, customer, and product extracts stage in OneLake. Finance requires a warehouse star schema with customer history; executives require the live Direct Lake dashboard from Friday’s use case; security requires regional RLS.

Requirements

- Lakehouse `lh_stage` plus warehouse `dw_retail` in workspace `fabric-de-milestone2`.
- Manifest-gated `COPY INTO` loading staged Parquet into `stg` tables (Chapter 5 pattern).
- Stored procedures building `dim_date`, `dim_customer` (SCD2), `dim_product`, `dim_store`, `fact_sales`.
- Custom semantic model with relationships on surrogate keys, a named measure library, and two RLS roles tested with View-As.
- A published Direct Lake report and a DirectQuery twin with a completed benchmark table.
- Capacity Metrics app evidence: CU consumption and fallback events.
- The cost estimate with two documented optimization decisions.

Reference Architecture



Milestone 2 serving stack: staged files to governed, benchmarked, secured BI.

Figure 8.1: Milestone 2 reference architecture: warehouse-to-Direct Lake serving layer.

Implementation Steps

1. Provision the workspace, lakehouse, and warehouse; stage partitioned Parquet extracts.
2. Implement manifest-gated `COPY INTO` per Chapter 5’s capstone pattern.
3. Write the star-schema stored procedures; implement `dim_customer` as SCD2 per Chapter 7.
4. Build the custom semantic model; author the measure library; configure and test RLS as multiple personas.
5. Publish the Direct Lake report; build the DirectQuery twin; complete the benchmark table.
6. Capture Capacity Metrics evidence, including one deliberate fallback.
7. Write the cost estimate and security review; commit everything with a README that runs end-to-end from a clean environment.

```

1 -- Idempotent load: only files not yet in the manifest
2 COPY INTO dbo.stg_sales
3 FROM 'https://onelake.dfs.fabric.microsoft.com/<ws>/<lh>/Files/stage/sales/2026-07/'
4 WITH ( FILE_TYPE = 'PARQUET' );
5
6 INSERT cfg.load_manifest (batch_path, rows_loaded, loaded_ts)
7 VALUES ('2026-07', @@ROWCOUNT, CURRENT_TIMESTAMP);

```

Listing 8.1: Milestone 2: idempotent warehouse load with manifest recording.

Deliverables

- Git repository that rebuilds the workflow from a clean environment (load → model → report).
- Architecture diagram (TikZ, draw.io, or PNG) showing Warehouse, SQL endpoint, semantic model, and Power BI.

- At least three automated data-quality tests (referential integrity, measure totals, row counts) with a failing-case demonstration.
- Benchmark results and captured fallback events.
- Monthly capacity cost estimate with two documented optimization decisions (for example V-Order and column pruning).
- Security review: RLS demonstrated as multiple test users, least-privilege roles, no hard-coded secrets.

Acceptance Criteria

- ☐ The end-to-end workflow runs from a clean environment following the repository README.
- ☐ Loads are idempotent: re-running a batch leaves the warehouse unchanged.
- ☐ The star schema uses surrogate keys and SCD2 where history matters, verified by uniqueness tests.
- ☐ All measures are named and reconcile to SQL aggregates.
- ☐ The benchmark table is complete, and at least one fallback event was produced and explained.
- ☐ RLS is proven with View-As for every role, not just the developer role.
- ☐ The cost estimate cites measured CU consumption.
- ☐ No secrets or personal paths appear in the repository.

Stretch Goals

- Add an aggregation table for the largest fact and measure its effect on guardrail headroom.
- Automate the benchmark as a repeatable script so regressions surface on every release.
- Wire a Data Activator-style alert on fallback rate using capacity telemetry.
- Extend RLS to a third persona with cross-region access and document the test evidence.

Part III

Data Engineering and ETL Orchestration

Data Factory Pipelines

Learning Objectives

- Build Fabric pipelines using the Copy activity, Lookup, and Get Metadata.
- Compose control flow with ForEach, If Condition, Until, and Switch.
- Parameterize pipelines with parameters, variables, and dynamic expressions.
- Enforce data contracts: validate expected schema before writing to Silver, and classify breaking versus non-breaking changes.
- Build a metadata-driven pipeline that copies a list of tables from Azure SQL Database to OneLake with a Lookup and a ForEach loop.
- Design an ingestion framework that onboards hundreds of tables without manual pipeline creation.

Cross-Cloud Equivalence

The visual, activity-based orchestrator with a rich connector library is a pattern every cloud offers. Fabric Pipelines descends directly from Azure Data Factory, so the mapping is especially tight in the Microsoft column.

Capability	Fabric	AWS	GCP	Snowflake
Visual orchestrator	Data Factory pipelines	Glue Workflows / Step Functions	Cloud Composer (Airflow) / Data Fusion	Tasks / Snowpark orchestration
Bulk copy	Copy activity (90+ connectors)	Glue ETL / DMS	Data Fusion / Datas-tream	COPY INTO / connec-tors
Lookup/enumeration	Lookup, Get Meta-data	DynamoDB/S3 lookups in Step Functions	Airflow operators	Metadata queries in tasks
Control flow	ForEach, If, Until, Switch	Step Functions states	Airflow DAG logic	Task DAGs
Parameterization	Parameters, variables, expressions	Step Functions JSON-Path	Airflow templating (Jinja)	Task parameters
Code-first alterna-tive	Notebooks in pipelines	Lambda / Glue scripts	Composer DAGs as code	Snowpark Python

Databricks orchestrates with Workflows; MongoDB with Atlas Triggers and Functions. The portable skill is the metadata-driven mindset this chapter teaches: config rows enumerate work, and one parameterized pipeline executes all of it—that idea survives any tool migration.

Key Terms

Pipeline: an orchestrated graph of activities that moves and transforms data on a schedule or trigger. **Copy activity:** the managed data-movement activity connecting sources to sinks across dozens of connectors. **Lookup:** an activity that reads a small result set (a config table, a file listing) for use by downstream activities. **ForEach:** a control-flow activity that iterates child activities over an array. **Dynamic expression:** the `@function(...)` expression language that parameterizes activity properties at run time. **Data contract:** a producer-published, consumer-validated specification of schema and semantics for a dataset.

9.1 Week 9 Roadmap

Week 9 opens Part III, the batch-processing and orchestration arc. Monday introduces Fabric pipelines: Copy, Lookup, and Get Metadata. Tuesday covers control flow—ForEach, If Condition, Until, Switch. Wednesday is the conceptual core: parameters, variables, and dynamic expressions, plus data contracts and schema enforcement. Thursday is the hands-on project: a metadata-driven pipeline that reads table names from a configuration source and copies each from Azure SQL Database to OneLake, with an expected-schema validation gate. Friday scales the pattern into an ingestion framework that onboards hundreds of tables automatically.

Pipelines are the circulatory system of a data platform. Chapter 5’s `COPY INTO` loads files already staged; pipelines are how the files get staged, on a schedule, with retries and alerts [42]. The metadata-driven pattern taught this week is the difference between an estate of 300 hand-built pipelines and one parameterized pipeline governed by a config table.

Note

Fabric Pipelines inherits the Azure Data Factory lineage: if you have ADF or Synapse pipeline experience, the concepts transfer directly. What changes in Fabric is the destination gravity—everything lands in OneLake—and the shared capacity billing model.

9.2 Monday: Pipelines, Copy, Lookup, and Get Metadata

A pipeline is a directed graph of *activities*. Three activities dominate ingestion work [42, 19]:

- **Copy activity** moves data between a source and a sink through managed connectors: databases, SaaS applications, file stores, and OneLake itself. It supports full and incremental loads, schema mapping, and fault tolerance settings.
- **Lookup** executes a query or reads a small file and returns its result to the pipeline—classically, “which tables are active in the config table right now?”
- **Get Metadata** inspects a storage location or dataset—file existence, column lists, modified dates—and returns structural facts the pipeline can branch on.

Definition

A **metadata-driven pipeline** is one whose work items come from data—a configuration table or manifest—rather than from activities hardcoded per source. Adding a new source is an `INSERT`, not a deployment.

The mental model: Lookup and Get Metadata *decide what to do*; Copy *does it*; control flow (Tuesday) *shapes the doing*; parameters and expressions (Wednesday) *make it reusable*.

Common Pitfall

Lookup is for small control sets, not data movement. A Lookup that returns a million rows to a ForEach creates a million Copy activities—slow, expensive, and un-debuggable. Enumerate *batches* with Lookup; move *rows* with Copy.

9.3 Tuesday: Control Flow

Control-flow activities shape execution [42]:

Activity	Behavior	Canonical use
ForEach	Iterates child activities over an array; sequential by default, parallel with a batch-size cap	Copy each table returned by a Lookup
If Condition	Branches on a boolean expression	Route breaking schema changes to quarantine
Until	Loops until a condition holds or timeout	Poll an external job's completion status
Switch	Multi-way branch on an expression's value	Dispatch per-source-type loading logic

Table 9.1: Control-flow activities and their canonical uses.

Every activity also carries dependency conditions—*On Success*, *On Failure*, *On Completion*, *On Skip*—that wire activities into a graph rather than a line. Chapter 12 builds the full orchestration and alerting discipline on these edges; this week's discipline is simpler: every Copy inside a loop should have an explicit failure path, even if that path is only “record the failure and continue.”

Tip

ForEach runs sequentially by default. For many-table ingestion, enable parallelism with a bounded batch count (start at 4–8). Unbounded parallelism against a source database is a self-inflicted denial-of-service attack.

9.4 Wednesday: Parameters, Variables, and Data Contracts

9.4.1 Parameters and Variables

Parameters are set at trigger time and immutable during the run—use them for batch dates, environment names, and source-system selectors. *Variables* mutate during the run—use them for watermarks advanced mid-flight, accumulated counts, and control flags. *Dynamic expressions* (`@concat(...)`, `@activity('Lookup').output.value`, `@item().table_name`) wire the two into activity properties at run time.

9.4.2 Data Contracts

A data contract is the producer's published promise about a dataset's schema and semantics, and the consumer's right to validate against it. In Fabric the natural split is: the *producer workspace* publishes and versions the contract; *consumer workspaces* validate incoming batches against it before writing to Silver [42].

Enforcement has two layers. At the storage layer, Delta Lake enforces schema by default—writes with mismatched columns are rejected—while `mergeSchema` and `overwriteSchema` permit con-

trolled evolution [9]. At the pipeline layer, an explicit validation step classifies each incoming change:

Change	Class	Pipeline response
New nullable column added	Non-breaking (additive)	Allow; log contract version bump
Column renamed or dropped	Breaking	Reject batch; quarantine; alert producer
Column type widened compatibly	Borderline	Allow only if contract versioning policy says so
Required column loses nullability	Breaking	Reject batch; alert producer

Table 9.2: Classifying schema changes against a data contract.

Common Pitfall

Delta’s default schema enforcement catches structural violations but not *semantic* ones. A renamed column arriving with the same type as an existing column can pass storage enforcement while corrupting meaning. The contract validation step—an explicit column-list comparison—is what catches it.

9.5 Thursday: Hands-on Project

The Week 9 lab builds the pattern this part of the book is named for: a dynamic pipeline that looks up a list of table names from a configuration source and uses a `ForEach` loop to copy each from an Azure SQL Database to OneLake—plus an expected-schema validation step that rejects breaking changes and allows additive ones.

9.5.1 Goal and Architecture

Create workspace `fabric-de-week09` with lakehouse `lh_ingest`. Stand up (or simulate with a small Azure SQL free tier) a source database containing `cfg.ingest_config` and three sample tables. The pipeline `PL_Metadata_Ingest` reads the config, validates each table’s schema against a recorded expectation, and copies conforming tables to Bronze Delta tables partitioned by batch tag.

9.5.2 Step-by-Step Actions

1. Create `cfg.ingest_config` with columns `schema_name`, `table_name`, `is_active`, `watermark`, and `expected_columns` (a delimited column list).
2. Insert three rows for the sample tables; seed each sample table with a few hundred rows.
3. Build the pipeline: Lookup over active config rows feeding a `ForEach`.
4. Inside the `ForEach`, add a script/lookup step that compares the source’s current column list to `expected_columns`.
5. Branch with an If Condition: additive-only differences proceed to Copy; renamed or dropped columns route to a Set Variable that records the rejection.
6. Copy each valid table to `bronze.<table_name>` with a batch-tag partition.

7. Test: rename a source column (batch rejected), add a nullable column (batch allowed), and confirm both outcomes are recorded.

```

1 {
2   "name": "PL_Metadata_Ingest",
3   "variables": { "vBatchTag": { "type": "String", "defaultValue": "daily" } },
4   "activities": [
5     { "name": "LookupTables", "type": "Lookup",
6       "typeProperties": { "sqlReaderQuery":
7         "SELECT schema_name, table_name FROM cfg.ingest_config WHERE is_active
8         = 1" } },
9     { "name": "CopyEachTable", "type": "ForEach",
10      "dependsOn": [ { "activity": "LookupTables", "dependencyConditions": ["Succeeded"] } ],
11      "typeProperties": {
12        "items": "@activity('LookupTables').output.value",
13        "activities": [ { "name": "CopyToOneLake", "type": "Copy",
14          "typeProperties": {
15            "source": { "sqlReaderQuery":
16              "@concat('SELECT * FROM ', item().schema_name, '.', item().
17              table_name)" },
18            "sink": { "type": "LakehouseTable",
19              "tableOption": "autoCreate",
20              "partitionOption": "@concat(variables('vBatchTag'), '/',
21                item().table_name)" } } } ] } } ]
22 }

```

Listing 9.1: Pipeline JSON (abridged): Lookup feeds ForEach; expressions parameterize the Copy.

```

1 # Notebook invoked per table before the Copy (or as post-landing gate)
2 expected = ["order_id", "order_date", "customer_id", "net_amount"]
3 actual = [f.name for f in spark.read.table("bronze.orders_staging").schema]
4
5 missing = set(expected) - set(actual)    # renamed/dropped -> breaking
6 added   = set(actual) - set(expected)    # new columns       -> additive
7
8 assert not missing, f"BREAKING contract change, missing columns: {missing}"
9 if added:
10    print(f"NON-BREAKING additive columns accepted: {sorted(added)}")

```

Listing 9.2: Contract validation as a notebook gate: breaking changes reject the batch.

9.5.3 Validation Checklist

- The pipeline copies all active config rows with no per-table authoring.
- A renamed source column rejects the batch and records the rejection.
- A new nullable source column passes and is logged as additive.
- Adding a fourth table to the config requires zero pipeline edits.
- Bronze tables carry the batch-tag partition column.
- ForEach parallelism, if enabled, is bounded and documented.

9.6 Friday: Use Case and Architecture

A wholesale platform acquires two companies per year; each arrives with 50–150 operational tables that must flow to OneLake. Hand-building a pipeline per table is a permanent backlog.

The deliverable is a metadata-driven ingestion framework: one parameterized pipeline family governed entirely by configuration.

9.6.1 The Framework Anatomy

- **Configuration store:** `cfg.ingest_config` rows per table—source connection key, query or full-load flag, watermark column, expected schema, target layer, owner, SLA.
- **Orchestrator:** a master pipeline that looks up due tables (by schedule column) and `ForEach`-dispatches to a child pipeline, bounded parallelism.
- **Worker:** a parameterized child pipeline executing contract validation, copy, watermark advancement, and run logging for exactly one table.
- **Run ledger:** `cfg.ingest_run` rows per attempt—status, rows, duration, error—feeding the Monitoring Hub story of Chapter 12.
- **Onboarding path:** a new table is one config row plus a recorded contract, reviewed like code.

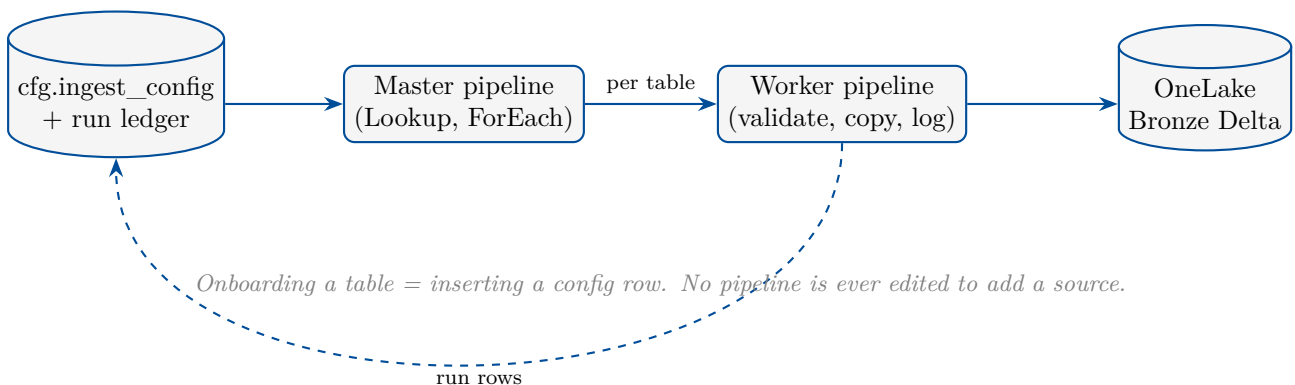


Figure 9.1: Metadata-driven ingestion framework.

Common Pitfall

Metadata-driven frameworks amplify both discipline and sloppiness. One bad default in the worker pipeline now affects every table. Treat the worker’s contract-validation and run-logging behavior as the most reviewed code in the estate, and version the config schema itself.

9.7 Operational Guidance for Week 9

A pipeline you cannot rerun safely is a liability. Every pattern this week—manifests, watermarks, contract gates—exists so that a failed 3 a.m. run can be retried by an on-call engineer following a runbook, not reverse-engineered.

9.7.1 Performance and Reliability

Advance watermarks only on success. Bound `ForEach` parallelism. Prefer incremental queries (`WHERE ts > @watermark`) over full reloads once tables outgrow a few million rows. Keep

Lookup result sets small. Record every run in the ledger—the ledger is what makes “did last night work?” a query rather than an investigation.

9.7.2 Security and Governance Checklist

- Store connection credentials in managed connections or Key Vault references, never in pipeline JSON.
- Restrict who can edit `cfg.ingest_config`—it is executable configuration.
- Version the contract per table; record the owning producer workspace.
- Quarantine rejected batches to a visible location with reasons, not to the void.
- Review worker-pipeline changes with the rigor of a schema migration.
- Document each table’s SLA and retry unit in the config row.

9.8 Interview Q&A

1. **Q: What makes an ingestion framework metadata-driven?**

A: Work items come from configuration data—a table of sources, schemas, and schedules—that a parameterized pipeline reads at run time, so onboarding a source is an `INSERT`, not a new pipeline.

2. **Q: Which activity iterates over a Lookup result set?**

A: `ForEach`, with the Lookup’s `output.value` as its `@items`; parallelism is opt-in and should be bounded.

3. **Q: What distinguishes a breaking from a non-breaking schema change?**

A: Breaking changes remove or rename what consumers depend on (dropped/renamed columns, tightened nullability); non-breaking changes only add—a new nullable column. The contract validation gate rejects the first and allows the second.

4. **Q: Who owns the data contract: producer or consumer workspace?**

A: The producer publishes and versions it; consumers validate against it. Ownership ambiguity is itself a contract failure.

5. **Q: What does `mergeSchema` allow that default Delta writes reject?**

A: Writes whose schema adds columns beyond the table’s current schema; default enforcement rejects any mismatch, which is exactly the protection you want until a reviewed contract bump says otherwise.

6. **Q: Why should watermarks advance only on success?**

A: Because the watermark defines what the next run reads. Advancing on failure silently skips uncommitted data; advancing on success makes rerun safe and duplicates impossible.

9.9 Further Reading

Review the Data Factory in Fabric documentation [19] and pipeline activity reference [42]. The schema-enforcement behavior underneath is Delta Lake’s [9]. For the orchestration layer that monitors these pipelines, Chapter 12 builds on the Monitoring Hub [51]. Metadata-driven patterns generalize the warehouse loading discipline of Chapter 5 [14].

Key Takeaways

- Pipelines compose Copy, Lookup, and Get Metadata with control flow to move and validate data on a schedule.
- Lookup enumerates work; Copy moves data; never reverse the roles at scale.
- Parameters are immutable per run, variables mutate, and dynamic expressions wire both into activities.
- Data contracts make schema changes explicit: additive changes flow, breaking changes are rejected and quarantined.
- Delta’s default schema enforcement is the storage-layer backstop; the pipeline’s validation step is the semantic gate.
- Metadata-driven design turns onboarding into configuration and concentrates risk in one well-reviewed worker pipeline.
- Watermarks advance on success only—that single rule is most of idempotent ingestion.

9.10 Exercises

1. **(Conceptual)** Explain why “enumerate batches, move rows” is the correct division of labor between Lookup and Copy.
2. **(Conceptual)** Classify each as breaking or non-breaking: new nullable column; renamed column; widened INT to BIGINT; column reordered; dropped column.
3. **(Hands-on)** Add an Until activity that polls an external job status every 60 seconds with a 20-minute timeout.
4. **(Hands-on)** Extend the lab pipeline with a run-ledger write after each Copy—status, rows, duration.
5. **(Design)** Design the config schema for a framework spanning Azure SQL, REST APIs, and SFTP sources. Which columns become polymorphic, and how do you keep the worker pipeline sane?
6. **(Design)** Two hundred tables, three source systems, one capacity: propose ForEach parallelism settings and scheduling windows, with reasoning.
7. **(Interview)** In five sentences, explain how a contract gate and Delta schema enforcement complement rather than duplicate each other.
8. **(Operational)** Write the runbook for “config row added but table never lands”—the five checks in order.

9.11 Capstone Project: Config-Driven Multi-Source Ingestion

Capstone Project

Mission. Build a two-tier, metadata-driven ingestion framework: a master pipeline that dispatches due tables to a parameterized worker pipeline performing contract validation, incremental copy, watermark advancement, and run logging. Prove it by onboarding a new table with a single INSERT, and by rejecting a deliberately breaking schema change.

Estimated time. 5–7 hours.

What you'll practice.

- Master/worker pipeline composition with bounded ForEach parallelism.
- Contract validation distinguishing breaking from additive changes.
- Watermark-driven incremental loads with a success-only advance rule.
- Run-ledger design that makes operations queryable.

Scenario

Contoso's analytics platform must absorb three source systems—an Azure SQL ERP, a REST export dropped as files, and a legacy CSV feed—with new tables arriving monthly. The data engineering team is two people. The framework, not headcount, must absorb the growth.

Requirements

- `cfg.ingest_config` and `cfg.ingest_run` tables with the columns this chapter specifies.
- Master pipeline `PL_Master_Ingest` dispatching due config rows to worker `PL_Worker_Ingest` with parallelism capped at four.
- Worker pipeline performing: contract gate → incremental copy `WHERE ts > @watermark` → watermark advance on success → ledger write on every outcome.
- Evidence: one table onboarded by config insert only; one breaking change rejected with quarantine; one additive change flowing through.
- A README section describing the onboarding procedure in five steps.

Implementation Steps

1. Create the config and ledger tables; seed four tables across two source types.
2. Build the worker pipeline end-to-end for one table, manually triggered.
3. Build the master pipeline; verify dispatch and parallelism cap.
4. Execute the three evidence scenarios and capture ledger rows for each.
5. Write the onboarding README and a failure runbook.

Deliverables

- Pipeline JSON exports (master and worker) under version control.
- Config and ledger table contents after the evidence scenarios.
- Screenshot of a bounded-parallelism ForEach run in the Monitoring Hub.
- Onboarding README and failure runbook.

Acceptance Criteria

- ☐ A new table flows end-to-end after one config `INSERT` and no pipeline edits.
- ☐ A renamed source column rejects the batch, writes a quarantine record, and alerts via the ledger.
- ☐ A new nullable source column flows through and is logged as an additive contract change.
- ☐ Watermarks advance only on successful runs; a failed run replays the same slice.

- ☐ Every run—success, rejection, or failure—produces exactly one ledger row.
- ☐ No credentials appear in pipeline JSON or the repository.

Stretch Goals

- Add per-table SLA columns and a ledger query that flags tables breaching their SLA.
- Add a Switch activity dispatching different copy shapes per source type.
- Generate the config row for a new source table automatically from source system metadata.
- Publish a small Power BI operations report over the run ledger.

Dataflows Gen2

Learning Objectives

- Build visual data transformations with Power Query Online in Dataflows Gen2.
- Configure output destinations—lakehouse, warehouse, Azure SQL—and explain staging versus destination behavior.
- Configure incremental refresh and reason about what it avoids recomputing.
- Gate Gold writes with data-quality assertions and quarantine rejected rows with error context.
- Build a Dataflow Gen2 that connects to a web API, pivots and cleans JSON, and lands in a Gold lakehouse table.
- Evaluate Mapping Dataflows, Dataflows Gen2, and Spark notebooks for a team migrating from Excel-based ETL.

Cross-Cloud Equivalence

Low-code, self-service data transformation with a visual query designer appears in every ecosystem, usually powered by the same M engine lineage or a visual DAG builder.

Capability	Fabric	AWS	GCP	Snowflake
Visual/low-code ETL	Dataflows Gen2 (Power Query)	Glue DataBrew / Glue Studio	Data Fusion / Dataprep heritage	Snowsight worksheets, dbt, partner tools
Output destinations	Lakehouse, warehouse, Azure SQL, more	S3, Redshift, databases	BigQuery, Cloud Storage	Snowflake tables
Incremental refresh	Yes (per query, date-column driven)	Job bookmarks (Glue)	Incremental pipelines (Data Fusion)	Streams + tasks, dynamic tables
Transformation engine	Mashup engine (M)	Spark (Glue Studio) / visual recipes	Data Fusion (Dataproc)	SQL pushdown / Snowpark
Data-quality gating	Notebook assertions / pipeline gates	Glue Data Quality	Dataplex data quality	Snowflake data metric functions
Escape hatch to code	M formulas; hand off to Spark notebooks	Glue PySpark	Dataflow/Beam	Snowpark / SQL

Databricks offers lakehouse-native notebooks and Delta Live Tables rather than a visual designer; MongoDB's Compass aggregation builder plays a similar analyst-facing role. The portable idea: *visual tools lower the authoring barrier, but quality gates and destinations are what make their output production-grade.*

Key Terms

Dataflow Gen2: a Fabric item that authors Power Query transformations and lands results into explicit destinations. **Power Query Online:** the browser-based visual transformation designer powered by the M formula language. **Output destination:** the dataflow setting that writes query results into a lakehouse, warehouse, Azure SQL database, or other target. **Staging:** intermediate computation inside the dataflow, separate from what is published to a destination. **Incremental refresh:** refreshing only new or changed partitions based on a date column. **Quarantine:** routing rows that fail quality checks to a rejected-records table with error context instead of dropping or silently accepting them.

10.1 Week 10 Roadmap

Week 10 brings the analysts into the pipeline. Monday covers Power Query Online and visual transformation authoring. Tuesday adds output destinations—the feature that separates Dataflows Gen2 from its Power-BI-only ancestor. Wednesday covers staging versus destinations, incremental refresh, and the data-quality patterns (assertions, quarantine) that make analyst-built flows safe for Gold. Thursday is the hands-on project: a dataflow from a web API through Power Query shaping into a Gold lakehouse table, gated by notebook assertions. Friday evaluates Mapping Dataflows, Dataflows Gen2, and Spark notebooks for a team of Excel-migrating business analysts.

Dataflows Gen2 matters because transformation demand always exceeds data-engineering supply. A governed, destination-aware, quality-gated low-code path lets analysts own the cleansing they understand best, while engineers keep the contracts and the Gold writes [21].

Note

The Week 9 framework governs *which tables move*; Dataflows Gen2 governs *how they are shaped*. They compose: a pipeline can trigger a dataflow, validate its output, and quarantine its rejects—as Thursday’s project demonstrates.

10.2 Monday: Power Query Online

Power Query Online is the authoring surface of Dataflows Gen2: a visual, step-by-step transformation designer in the browser. Every click—filter rows, rename columns, pivot, merge—becomes a step in the Applied Steps pane, and every step compiles to the M formula language, visible and editable in the Advanced Editor [21].

Definition

Power Query is a case-insensitive, functional data-preparation engine whose M language expresses transformations as an ordered pipeline of steps over immutable intermediate tables.

The engineering virtues of the model: each step is inspectable, reorderable, and removable; the step list is effectively a readable diff of intent; and because steps are lazy, the engine folds many operations back to the source (query folding) instead of pulling raw data to the client. The vices: visual logic is harder to unit-test than code, and a talented analyst can build an unreviewable 60-step flow in an afternoon. The week’s discipline: short flows, named steps, code-review by reading the M.

Tip

Name every step in business language (**Remove Test Accounts**, not **Filtered Rows**). When a flow breaks six months later, the step names are the documentation.

10.3 Tuesday: Output Destinations

Dataflows Gen2’s defining advance over classic dataflows is the output destination: each query in the flow can publish its result to a lakehouse table, a warehouse table, an Azure SQL database, or other supported targets [21]. The dataflow stops being a Power BI preprocessing accessory and becomes an ingestion and transformation tool in its own right.

Destination semantics worth memorizing: append versus replace behavior is configurable per query; schema changes at the destination are handled deliberately (new columns can be allowed or rejected, matching Week 9’s contract discipline); and the destination write is where batch identity should be stamped—a load timestamp or batch tag column added as the final step.

Destination	Best for	Watch-outs
Lakehouse table	Analyst-curated Silver/Gold tables consumed by Spark, SQL endpoint, and Direct Lake	Schema drift control; concurrent writes from other tools
Warehouse table	T-SQL-first consumers and stored-procedure estates	Permissions model differs from lakehouse; plan grants
Azure SQL DB	Operational systems of engagement	Throughput limits; not an analytical store

Table 10.1: Dataflow Gen2 output destinations.

Common Pitfall

A dataflow writing directly to a Gold table with append semantics has no quality gate. Anything the flow produces—including a half-transformed batch after a mid-run failure—is live for every consumer. Stage first, gate, then publish (Wednesday’s pattern).

10.4 Wednesday: Staging, Incremental Refresh, and Data Quality

10.4.1 Staging versus Destinations

Within a dataflow, some queries compute intermediate shapes (staging) while others publish to destinations. Keeping the distinction explicit—staging queries are not published, destination queries are thin final projections—makes flows readable and prevents intermediate garbage from materializing into the lakehouse.

10.4.2 Incremental Refresh

Incremental refresh partitions the destination by a date column and refreshes only partitions inside a rolling window. It avoids recomputing (and re-billing) history that cannot change. The prerequisites: a reliable date/datetime column at the destination grain, a window sized to the late-arrival reality of the source, and append-friendly destination semantics.

10.4.3 Data-Quality Gates

The production pattern for analyst-built flows lands output in staging, asserts quality in code, quarantines violations, and only then publishes Gold [21, 7]:

```

1 from pyspark.sql import functions as F
2
3 df = spark.read.table("silver.orders")
4
5 null_pk = df.filter(F.col("order_id").isNull()).count()
6 dup_keys = (df.groupBy("order_id").count()
7             .filter("count > 1").count())
8
9 assert null_pk == 0, f"DQ FAIL: {null_pk} null order_id values"
10 assert dup_keys == 0, f"DQ FAIL: {dup_keys} duplicate order_id keys"
11
12 # Quarantine bad rows instead of failing silently
13 rejected = df.filter(F.col("amount") < 0)
14 if rejected.count() > 0:
15     (rejected.withColumn("reject_reason", F.lit("negative_amount"))
16      .write.format("delta").mode("append")
17      .saveAsTable("gold.rejected_orders"))
18
19 (df.filter(F.col("amount") >= 0)
20  .write.format("delta").mode("overwrite")
21  .saveAsTable("gold.orders"))

```

Listing 10.1: Data-quality assertions gating the Gold write, with quarantine.

Note

Production rule for the week: never fail silently on bad rows. Quarantine records that violate expectations to a rejected Delta table with an error-context column, alert via pipeline notification, and keep the Gold write clean. Failing the entire batch on a handful of bad rows breaks the SLA; ignoring them corrupts every downstream aggregate.

Common Pitfall

Assertions that only run in development are decorations. The gate must execute in the production path—a pipeline step between staging and Gold—or it does not exist as far as consumers are concerned.

10.5 Thursday: Hands-on Project

The Week 10 lab builds a Dataflow Gen2 that connects to a web API, pivots and cleans the JSON response in Power Query, and sets the destination to a Gold lakehouse table—with a notebook data-quality assertion cell gating the Gold write.

10.5.1 Goal and Architecture

Create workspace `fabric-de-week10` with lakehouse `lh_products`. Use a public test API (for example a JSON-placeholder products or exchange-rate endpoint) or an instructor-provided mock. The dataflow lands a staged table; a notebook validates and publishes; a pipeline wires the sequence so the gate is structural, not aspirational.

10.5.2 Step-by-Step Actions

1. Create the dataflow `df_api_products`; add a Web source for the endpoint.

2. In Power Query, expand the JSON records, promote headers, pivot the nested price fields, and set explicit types.
3. Name every step; add a final step stamping `load_ts` with the current timestamp.
4. Set the output destination to `lh_products` table `staging_api_products` (append).
5. In a notebook, implement the assertion cell: primary-key uniqueness, no nulls in required fields, price range sanity.
6. Add quarantine: violations append to `gold.rejected_api_products` with `reject_reason`.
7. Publish passing rows to `gold.api_products`; wrap dataflow → notebook in a pipeline.
8. Break the API response deliberately (instructor mock) and prove the gate rejects and quarantines.

```

1 from pyspark.sql import functions as F
2
3 df = spark.read.table("staging_api_products")
4
5 assert df.filter(F.col("product_id").isNull()).count() == 0, "DQ FAIL: null
   product_id"
6 assert (df.groupBy("product_id").count()
7         .filter("count > 1").count()) == 0, "DQ FAIL: duplicate product_id"
8
9 bad = df.filter((F.col("price") < 0) | (F.col("price") > 100000))
10 (bad.withColumn("reject_reason", F.lit("price_out_of_range"))
11     .write.format("delta").mode("append")
12     .saveAsTable("gold.rejected_api_products"))
13
14 (df.subtract(bad)
15     .write.format("delta").mode("overwrite")
16     .saveAsTable("gold.api_products"))

```

Listing 10.2: Week 10 lab gate: assert, quarantine, publish.

10.5.3 Validation Checklist

- The dataflow refreshes end-to-end from the API to the staging table.
- Every Power Query step has a business-language name.
- The assertion notebook fails loudly on nulls and duplicates.
- Out-of-range prices land in the quarantine table with reasons, not in Gold.
- The pipeline runs dataflow then gate; disabling the gate is impossible without editing the pipeline.
- Gold contains only validated rows, stamped with `load_ts`.

10.6 Friday: Use Case and Architecture

A 40-person analyst team runs the company's reporting on Excel-based ETL: manual extracts, VLOOKUP chains, and heroically maintained workbooks. IT must pick the migration target. The candidates: Mapping Dataflows (the ADF-flavored engine), Dataflows Gen2, and Spark notebooks.

Criterion	Mapping Dataflows	Dataflows Gen2	Spark notebooks
Authoring	Visual DAG (Spark-backed)	Visual Power Query (M)	Code (PySpark/SQL)
Skill match for Excel analysts	Moderate	High (same engine as Excel's Get Data)	Low
Scale-out	Strong (Spark cluster)	Moderate (mashup engine)	Strong
Testability	Moderate	Low-moderate	High (unit tests, CI)
Destinations	Broad	Lakehouse, warehouse, SQL, more	Anything Spark reaches
Governance fit	Pipeline-centric	Workspace-centric with pipeline gates	Full engineering rigor

Table 10.2: Three transformation tools for an Excel-migrating analyst team.

10.6.1 Recommendation

Dataflows Gen2 for the analyst-owned cleansing layer—the M engine is literally the technology many already use inside Excel—behind the Week 9/10 gate pattern: staging destination, notebook assertions, quarantine, Gold publish. Spark notebooks for the heavy, shared, engineered transformations. Mapping Dataflows only where existing ADF estates and Spark-scale visual DAGs already justify it. The architecture is a portfolio, not a winner.

Common Pitfall

Letting each analyst pick their own tool recreates the Excel sprawl in a new medium. The platform team's job is to make the governed path (dataflow → gate → Gold) the *easiest* path, and to measure adoption rather than assume it.

10.7 Operational Guidance for Week 10

Analyst-built flows are production software the moment a destination table feeds a report. Apply engineering discipline proportional to consumption, not authorship.

10.7.1 Performance and Reliability

Prefer query folding; watch for steps that break folding and pull full tables to the mashup engine. Size incremental refresh windows to late-arrival reality. Keep destination writes append-stamped with batch identity. Monitor refresh failures like pipeline failures—they are pipeline failures.

10.7.2 Security and Governance Checklist

- Store API credentials in the dataflow's connection settings, never in M text or notebook cells.
- Require the gate pattern (staging → assertions → Gold) for any flow feeding certified data.
- Review dataflows by reading the M in Advanced Editor, not by clicking through steps.
- Record flow owners; orphaned flows with destinations are unmaintained production code.
- Quarantine tables must have owners, retention, and a consumer (someone reads the rejects).

- Version dataflow definitions in Git where tenant support allows.

10.8 Interview Q&A

1. **Q: What is an output destination in Dataflows Gen2?**

A: The setting that publishes a query’s result into an external target—lakehouse, warehouse, Azure SQL—turning the dataflow from a BI preprocessor into a general ingestion and transformation tool.

2. **Q: Why stage data before writing to a destination?**

A: Staging separates computation from publication so quality gates can run between them; a flow that writes Gold directly has no point at which bad rows can be stopped.

3. **Q: What does incremental refresh avoid recomputing?**

A: Historical partitions outside the rolling refresh window—only partitions containing new or changed dates are reprocessed, saving compute and time.

4. **Q: Why quarantine bad rows instead of failing the whole batch?**

A: A handful of bad rows should not break the SLA for millions of good ones; quarantine preserves the batch, records the violations with context, and keeps Gold clean for consumers.

5. **Q: When are Spark notebooks preferable to Dataflows Gen2?**

A: When transformations need scale-out compute, unit tests, code review, and CI—the engineering rigor a shared, complex transformation deserves.

6. **Q: What does Power Query compile visual steps into?**

A: The M formula language, visible in the Advanced Editor—which is why code review of a dataflow means reading M, not watching clicks.

10.9 Further Reading

Review the Dataflows Gen2 overview [21] and the Power Query documentation behind it [43]. The quality-gating pattern extends the Delta semantics of [9] and borrows expectation vocabulary from Great Expectations [7]. Orchestration of the gate pattern appears in Chapter 12 [51] and pipeline mechanics in Chapter 9 [42].

Key Takeaways

- Dataflows Gen2 = Power Query authoring plus first-class output destinations; it is an ingestion tool, not a BI accessory.
- Every visual step compiles to M; review the M, name the steps, keep flows short.
- Destinations make schema and append/replace semantics explicit—choose them as deliberately as pipeline sinks.
- Incremental refresh reprocesses only the rolling window; size the window to late-arrival reality.
- The production pattern is staging → assertions → quarantine → Gold publish, wired structurally by a pipeline.
- Quarantine with reasons, alert, and keep Gold clean; never fail silently and never nuke the batch for a few bad rows.

- Tool selection for analyst teams is a portfolio decision; governance, not the tool, is what makes output trustworthy.

10.10 Exercises

1. **(Conceptual)** Explain query folding and why a single step can silently disable it for everything downstream.
2. **(Conceptual)** Compare append and replace destination semantics; give one scenario where each is dangerous.
3. **(Hands-on)** Add incremental refresh to the lab dataflow on `order_date` with a 7-day window; prove a historical correction outside the window requires a full refresh.
4. **(Hands-on)** Extend the gate with a third assertion (referential integrity against `dim_product`) and a corresponding quarantine reason.
5. **(Design)** Write the platform policy that decides when an analyst’s dataflow may publish directly to Gold (hint: the honest answer is “never”—defend or amend it).
6. **(Design)** Sketch the migration wave plan for the 40-analyst Excel estate: which workbooks move first, and what evidence proves each wave succeeded?
7. **(Interview)** In five sentences, explain to an Excel power user why their new dataflow cannot write directly to the finance Gold table.
8. **(Operational)** Write the runbook for “quarantine table grew 10× overnight”—detection, triage, producer contact, and consumer communication.

10.11 Capstone Project: Analyst-Grade API Ingestion with Quality Gates

Capstone Project

Mission. Deliver the complete analyst-safe ingestion path: a Dataflow Gen2 from a live web API through staged, named Power Query steps; a notebook gate enforcing three assertions with quarantine; a pipeline wiring the sequence; and an incremental-refresh configuration justified by the source’s late-arrival behavior. Prove it with a deliberately broken payload.

Estimated time. 4–6 hours.

What you’ll practice.

- Production-grade Power Query authoring: named steps, explicit types, batch stamping.
- Destination semantics and incremental refresh windows.
- Assertion-based quality gates with quarantine and reasons.
- Structural orchestration that makes bypassing the gate impossible.

Scenario

Contoso’s pricing analysts need a daily feed of competitor product data from a public API. Their last solution was a spreadsheet refreshed by hand. The platform team will provide the governed path; the analysts will own the dataflow’s transformation logic going forward.

Requirements

- Dataflow `df_competitor_prices` with named steps, explicit types, and `load_ts` stamping, landing in `staging_competitor_prices`.
- Incremental refresh on the destination with a written window justification.
- Gate notebook with three assertions (key uniqueness, required-field nulls, price sanity) and quarantine with `reject_reason`.
- Pipeline `PL_daily_competitor_prices` running dataflow → gate on a schedule.
- Evidence run: a mocked payload violating each assertion, with quarantine rows captured.
- One-page handoff document enabling the analysts to own the flow.

Implementation Steps

1. Build and stage the dataflow; verify typed, stamped output.
2. Configure incremental refresh; document the window rationale.
3. Implement the gate notebook and quarantine table.
4. Wire and schedule the pipeline; verify the gate cannot be skipped.
5. Execute the broken-payload evidence run; capture quarantine evidence.
6. Write the analyst handoff document.

Deliverables

- Exported dataflow definition and gate notebook in the repository.
- Quarantine evidence from the broken-payload run.
- Pipeline run history showing dataflow → gate sequencing.
- The incremental-refresh justification and analyst handoff documents.

Acceptance Criteria

- ☐ The scheduled pipeline runs end-to-end without manual steps.
- ☐ Each of the three assertions demonstrably rejects and quarantines a violating payload.
- ☐ Gold contains only validated rows, each stamped with batch identity.
- ☐ Incremental refresh reprocesses only the configured window.
- ☐ The pipeline fails visibly (not silently) when the gate asserts.
- ☐ No credentials appear in M text, notebooks, or the repository.

Stretch Goals

- Add a Great Expectations suite alongside the notebook assertions and compare the authoring experience.
- Publish a small operations report over the quarantine table (rejects by reason, by day).
- Add a second API source to the same dataflow with a merge step, keeping step names readable.

- Parameterize the gate thresholds through a config table so analysts tune sensitivity without code edits.

Data Integration and Connectivity

Learning Objectives

- Distinguish the on-premises data gateway from the VNet data gateway and select the correct one per scenario.
- Connect Fabric to hybrid sources including SQL Server, Oracle, and SAP.
- Ingest REST APIs and webhooks with sound credential and paging patterns.
- Install and configure an on-premises data gateway and run a daily extraction pipeline through it.
- Design gateway clusters for high availability and load balancing.
- Architect a secure hybrid connectivity pattern for a bank that forbids public endpoints on transactional databases.

Cross-Cloud Equivalence

Hybrid connectivity—reaching data behind corporate networks and inside VNets without exposing it publicly—is a universal enterprise requirement with remarkably similar shapes across clouds.

Capability	Fabric	AWS	GCP	Snowflake
On-prem relay/agent	On-premises data gateway	DMS agents, Data-Sync agents, Storage Gateway	Transfer appliance / Database Migration Service	External access via connectors, Snowpipe through private links
VNet/VPC private path	VNet data gateway	VPC endpoints, PrivateLink	Private Service Connect, VPC peering	PrivateLink / Private Service Connect
Outbound-only pattern	Gateway initiates outbound 443	Agents poll outbound	Agents/connectors outbound	Outbound from customer network
HA model	Gateway clusters (2+ nodes)	Multi-AZ services	Regional redundancy	Multi-AZ by default
API ingestion	Copy/Web activities, eventstreams	API Gateway + Lambda, AppFlow	Cloud Functions / Workflows	External functions, connectors

Databricks reaches private sources through VPC peering and PrivateLink; MongoDB Atlas through private endpoints and VPC peering. The invariant across clouds: *the private side initiates the connection outbound*, so no inbound firewall rule ever exposes the database.

Key Terms

On-premises data gateway: an agent installed inside the corporate network that brokers outbound-only connections between Fabric and local sources. **VNet data gateway:** a Microsoft-managed gateway that lets Fabric reach sources inside an Azure virtual network

without a customer-managed VM. **Gateway cluster:** two or more gateway nodes providing high availability and load distribution. **Webhook:** an HTTP callback a source system invokes to push events, rather than waiting to be polled. **Private endpoint:** a network interface that gives a service a private IP inside a VNet, removing public exposure.

11.1 Week 11 Roadmap

Week 11 connects Fabric to the enterprise as it actually exists: databases behind firewalls, services inside VNets, and APIs that push as well as pull. Monday compares the on-premises data gateway with the VNet data gateway. Tuesday connects the classic hybrid sources—SQL Server, Oracle, SAP. Wednesday covers REST APIs and webhooks. Thursday is the hands-on project: install the on-premises gateway against a mock SQL Server and run a daily extraction to OneLake. Friday architects the bank scenario: strict hybrid connectivity with zero public endpoints.

Connectivity is where elegant cloud architectures meet organizational reality. The pipelines of Chapter 9 assume reachability; this week supplies it, securely, with the operational discipline (clustering, patching, monitoring) that makes a gateway production infrastructure rather than a developer’s side install [56, 54].

Note

Gateways are identity-bearing infrastructure: the gateway’s registered connections carry credentials to your most sensitive systems. Treat a gateway server with the same change control as a database server, because that is effectively what it fronts.

11.2 Monday: On-Premises versus VNet Data Gateways

Both gateway types solve the same problem—Fabric is a SaaS platform; your data is not—but for different network topologies [56, 54].

Aspect	On-premises data gateway	VNet data gateway
Where it runs	Customer-managed Windows machine inside the corporate network	Microsoft-managed compute attached to your Azure VNet
Reaches	SQL Server, Oracle, file shares, SAP—anything the host can route to	Azure services inside the VNet (Azure SQL MI, VMs, private endpoints)
Management burden	You patch, monitor, and cluster the host(s)	Microsoft manages the nodes; you manage the VNet attachment
Best for	True on-premises estates, non-Azure networks	Azure-resident private sources; no VM to maintain

Table 11.1: Choosing between the two gateway types.

Definition

An **on-premises data gateway** is a customer-installed agent that creates outbound-only, encrypted connections to the cloud relay, so cloud services can query on-premises sources without any inbound firewall opening.

The outbound-only property is the security cornerstone: the gateway initiates every connection over TLS to the Azure relay; the corporate firewall sees ordinary outbound 443 traffic; no public IP, no inbound rule, no port forwarding ever touches the database network segment.

Common Pitfall

Installing the gateway on a random analyst workstation is the classic anti-pattern: the machine sleeps, gets patched at whim, and leaves the estate when the analyst does. Gateway hosts are servers—dedicated, patched, monitored, and clustered.

11.3 Tuesday: Connecting Hybrid Sources

11.3.1 SQL Server

The reference case. Create a connection in Fabric bound to the gateway cluster, supplying server, database, and a least-privilege SQL identity. Pipelines, dataflows, and semantic models then reference the connection by name; the gateway handles the wire. For extraction to OneLake, pair the connection with Chapter 9’s metadata-driven Copy pattern.

11.3.2 Oracle and SAP

Oracle connects through the same gateway with the Oracle client or managed provider installed on the gateway host—which makes the host’s software inventory part of the integration contract. SAP arrives through the SAP connectors (for example SAP HANA, SAP BW, or SAP Table/BW Open Hub patterns), typically with additional licensing and driver considerations on the gateway host. In both cases, the driver’s version and bitness on the host are first-class deployment dependencies.

11.3.3 Connection Hygiene

- One logical connection per source per environment (dev/test/prod), named consistently.
- Least-privilege identities: read-only accounts for extraction, scoped to required schemas.
- Credential rotation procedures documented per connection; a gateway connection with a dead password pages someone at 3 a.m.

Tip

Keep a connection inventory table—source, environment, gateway cluster, identity, owner, rotation date—in the same config database as Chapter 9’s ingestion config. Connections are configuration, and configuration belongs in tables, not in portal archaeology.

11.4 Wednesday: REST APIs and Webhooks

Not every source is a database. REST APIs and webhooks move an increasing share of enterprise data, and both have discipline requirements that naive pipelines ignore.

11.4.1 REST APIs

Ingest APIs with the Copy activity’s REST connector or a Web activity followed by processing. The four recurring concerns: *authentication* (OAuth2 client credentials or managed identity—never API keys pasted into pipeline JSON); *pagination* (follow `next` links or page parameters until exhaustion; a pipeline that reads page one is a pipeline that loses data); *rate limits* (honor

429 responses with backoff, or the API will do the throttling for you); and *schema drift* (JSON responses evolve silently—apply Week 9’s contract gate to the parsed result).

11.4.2 Webhooks

Webhooks invert the flow: the source pushes events to an endpoint you expose. In Fabric, eventstreams (Chapter 13) and custom endpoints receive such pushes. The credential pattern matters: validate shared secrets or tokens on receipt, and design the receiver to be idempotent because push sources retry.

Common Pitfall

Webhook receivers without authentication are open write endpoints into your lake. Validate a shared secret or signature on every request, log rejected calls, and rotate the secret on a schedule.

11.5 Thursday: Hands-on Project

The Week 11 lab stands up the full hybrid path: a mock on-premises SQL Server, the on-premises data gateway, and a daily Fabric pipeline extracting through it to OneLake.

11.5.1 Goal and Architecture

Use a local SQL Server (Developer edition, a Docker container, or an instructor VM) seeded with a `sales.orders` table. Install the on-premises data gateway on a suitable host (your machine is acceptable *for the lab*; note explicitly why it would not be in production). Create the Fabric connection, then build pipeline `PL_daily_orders_extract` running the extraction on a schedule.

11.5.2 Step-by-Step Actions

1. Seed the mock SQL Server with `sales.orders` (a few thousand rows) and create a read-only login `fabric_reader`.
2. Download and install the on-premises data gateway; register it to your tenant with a recovery key stored in the team vault.
3. In Fabric, create the connection: gateway, server, database, and the `fabric_reader` identity.
4. Build the pipeline: Copy activity from the gateway connection to `bronze.orders_onprem` in the lakehouse, partitioned by load date.
5. Add the watermark pattern: `WHERE order_ts > @watermark` with the watermark advanced only on success.
6. Schedule the pipeline daily; run it manually once and validate row counts.
7. Simulate failure (stop the gateway service), observe the pipeline failure, restart, and verify the rerun is idempotent.

```

1 CREATE LOGIN fabric_reader WITH PASSWORD = '<from-vault>';
2 CREATE USER fabric_reader FOR LOGIN fabric_reader;
3 EXEC sp_addrolemember 'db_datareader', 'fabric_reader';
4
5 CREATE TABLE sales.orders (
6     order_id INT IDENTITY PRIMARY KEY,
```

```

7      customer_id  INT,
8      order_ts     DATETIME2,
9      net_amount   DECIMAL(18,2)
10 );

```

Listing 11.1: Mock source setup for the Week 11 lab.

11.5.3 Validation Checklist

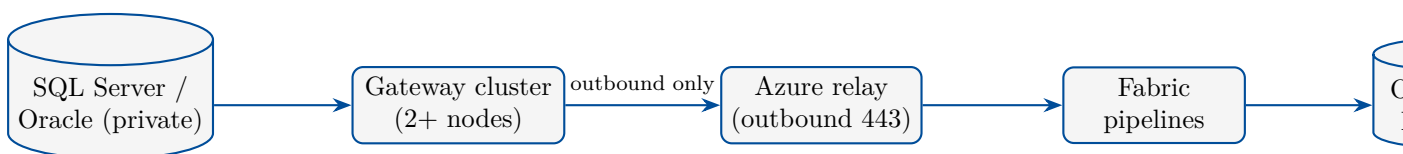
- The gateway registers and shows online in the Fabric portal.
- The connection uses the read-only identity, never a personal or admin account.
- The pipeline extracts incrementally and the watermark advances only on success.
- Gateway-down produces a clear pipeline failure; recovery replays without duplicates.
- The gateway recovery key is vaulted, and the host appears in the connection inventory.
- You can articulate why the same setup on an analyst laptop would be a production incident waiting to happen.

11.6 Friday: Use Case and Architecture

A retail bank’s policy is absolute: transactional databases never have public endpoints—no public IP, no inbound firewall exceptions, no exceptions. Core banking runs on on-premises SQL Server and Oracle. The analytics program needs daily extracts into Fabric.

11.6.1 The Pattern

A clustered on-premises data gateway—two nodes minimum, in the bank’s data-center network segment—initiates outbound-only TLS connections to the Azure relay. Fabric connections bind to the cluster with least-privilege, read-only identities per source. Pipelines extract on schedule to OneLake Bronze; Week 9’s contract gates validate; the run ledger records every attempt. No packet ever flows inbound to the bank’s network; no database listens publicly; the firewall policy sees only outbound 443.



The bank’s perimeter sees only outbound TLS. No inbound rule, no public IP, no exposed listener.

Figure 11.1: Bank-grade hybrid pattern: outbound-only gateway cluster.

11.6.2 Hardening the Pattern

- **Cluster for availability.** A single-node gateway is a single point of failure for every hybrid pipeline; install two or more nodes for high availability and load balancing [56].
- **Patch within one release of current.** Gateway software carries credentials in motion; unpatched gateways are both reliability and security debt.

- **Monitor node health centrally.** Node offline, version drift, and relay connectivity are alertable conditions, not portal trivia.
- **Restrict outbound destinations.** Where policy demands, egress-filter the gateway segment to the documented Azure relay endpoints.
- **Separate duties.** Gateway admins, connection owners, and pipeline developers are distinct roles—Chapter 21 formalizes this.

Common Pitfall

The gateway breaks mid-pipeline more often than teams expect—host reboots, Windows updates, certificate renewals. The lab’s deliberate gateway-down test is not theater: every hybrid pipeline needs a designed answer to “the gateway vanished halfway,” and the answer is watermark-safe idempotent rerun.

11.7 Operational Guidance for Week 11

Gateways and connections are the least visible and most critical infrastructure in a hybrid Fabric estate. Inventory them, cluster them, patch them, and monitor them.

11.7.1 Performance and Reliability

Place gateway hosts near the sources (latency dominates extraction throughput). Size hosts for concurrent query load. Prefer incremental extraction to keep gateway sessions short. Test failover by stopping a node during a run; the cluster should absorb it.

11.7.2 Security and Governance Checklist

- Gateway clusters of two or more nodes for every production estate; single nodes are lab-only.
- Least-privilege, read-only source identities; rotation dates in the connection inventory.
- Recovery keys vaulted at registration time, not after the first rebuild.
- Webhook endpoints authenticated and idempotent.
- Gateway host patching cadence documented and monitored.
- No public endpoints on any source database, verified by network scan, not by assumption.

11.8 Interview Q&A

1. **Q: When do you need a VNet data gateway instead of an on-premises gateway?**

A: When sources live inside an Azure VNet (Azure SQL MI, VMs, private-endpoint services) and you want Microsoft-managed gateway compute without maintaining a Windows host.

2. **Q: Why cluster gateway nodes for production workloads?**

A: A single node is a single point of failure for every hybrid pipeline; clusters provide high availability during patches and failures plus load distribution across concurrent extractions.

3. **Q: Which credential pattern should webhook/API ingestion use?**

A: OAuth2 client credentials or managed identity, stored in managed connections or Key Vault, with webhook receivers validating a shared secret or signature on every call—never static keys embedded in pipeline JSON.

4. **Q: How does a gateway keep data inside the bank’s network perimeter?**

A: The gateway initiates every connection outbound over TLS to the Azure relay; the bank’s firewall sees only outbound 443 traffic, and no inbound rule or public endpoint ever exists on the database segment.

5. **Q: What breaks if the gateway machine goes offline mid-pipeline?**

A: The in-flight activity fails; because extraction is watermark-based and the watermark advances only on success, the rerun replays the same slice idempotently—which is why the failure drill is part of the lab.

6. **Q: Why does driver inventory on the gateway host matter for Oracle and SAP?**

A: The host executes the provider code; driver version and bitness are deployment dependencies, and an unmanaged host upgrade can break every Oracle or SAP connection simultaneously.

11.9 Further Reading

Review the on-premises data gateway documentation [56] and the VNet data gateway documentation [54]. Pipeline mechanics come from Chapter 9 [42]; eventstream-based webhook reception previews Chapter 13 [26]. Security principles here are deepened in Chapter 21 [44].

Key Takeaways

- On-premises gateways are customer-managed agents for corporate networks; VNet gateways are Microsoft-managed attachments for Azure VNets.
- The outbound-only relay pattern is why gateway-based hybrid connectivity passes bank-grade security review.
- Connections are configuration: inventory them, scope them least-privilege, and rotate their credentials on schedule.
- API ingestion demands pagination, rate-limit handling, credential discipline, and schema-drift gates; webhooks demand authentication and idempotent receivers.
- Single-node gateways are lab conveniences and production liabilities; cluster everything that matters.
- Gateway failure mid-pipeline is a designed-for scenario, not an edge case—watermark-safe reruns absorb it.
- The gateway host’s patch level and driver inventory are part of the integration contract.

11.10 Exercises

1. **(Conceptual)** Explain the outbound-only relay pattern to a network engineer skeptical of “cloud access to our database.”

2. **(Conceptual)** Contrast the on-premises and VNet gateways across management burden, reachable sources, and failure domains.
3. **(Hands-on)** Add a second gateway node, form a cluster, and stop one node during an extraction; document the observed behavior.
4. **(Hands-on)** Extend the lab pipeline to call a paginated REST API, following `next` links to exhaustion.
5. **(Design)** Design the connection inventory schema: columns, ownership, rotation tracking, and the query that lists connections overdue for rotation.
6. **(Design)** The bank acquires a subsidiary whose sources sit in AWS. Propose the connectivity pattern and name every trust boundary crossed.
7. **(Interview)** In five sentences, defend gateway clustering as an availability requirement rather than an optimization.
8. **(Operational)** Write the runbook for “gateway node reports unhealthy”—detection, failover verification, patching, and rejoin.

11.11 Capstone Project: Clustered Hybrid Extraction with Failover Evidence

Capstone Project

Mission. Build the bank-grade hybrid pattern end to end: a two-node gateway cluster fronting a mock on-premises SQL Server, a scheduled incremental extraction to OneLake, a connection inventory, and documented failover evidence proving the estate survives a node loss mid-run.

Estimated time. 5–7 hours (a second node may be a VM or a second machine; document your topology either way).

What you’ll practice.

- Registering and clustering gateway nodes with vaulted recovery keys.
- Building watermark-safe incremental extraction through the cluster.
- Executing and documenting a mid-run failover drill.
- Operating connections as inventoried, least-privilege configuration.

Scenario

Contoso Bank’s analytics platform must extract nightly from an on-premises core system under a no-public-endpoint policy. The platform team must demonstrate—to the security review board, with evidence—that extraction is outbound-only, survives a gateway node failure, and never double-counts after a mid-run outage.

Requirements

- Two-node gateway cluster registered to the tenant; recovery keys vaulted.
- Least-privilege read-only source identity; connection recorded in the inventory table.
- Pipeline `PL_nightly_core_extract`: watermark-incremental, success-only advance, ledger row per run.

- Failover drill: stop one node mid-run, capture the outcome, and prove the retry is duplicate-free.
- A network-evidence appendix: firewall or connection logs showing outbound-only flows.
- Security review narrative suitable for the bank's review board.

Implementation Steps

1. Install and register both gateway nodes; form the cluster; vault the recovery keys.
2. Create the least-privilege identity and the Fabric connection bound to the cluster.
3. Build and schedule the incremental extraction pipeline with ledger logging.
4. Run the failover drill; capture pipeline and gateway evidence.
5. Assemble the network evidence and the security narrative.

Deliverables

- Cluster topology diagram and node inventory.
- Pipeline export and ledger contents across the drill.
- Failover evidence: timestamps, affected run, clean retry.
- Network evidence and the review-board narrative.

Acceptance Criteria

- ☐ Both nodes serve the cluster; stopping either one does not lose a night's data.
- ☐ The mid-run failure produces a visible failure and a duplicate-free retry.
- ☐ The source identity is read-only and schema-scoped.
- ☐ Network evidence shows outbound-only flows; no inbound rule exists.
- ☐ The connection inventory row is complete, including rotation date.
- ☐ No secrets appear in the repository, pipeline JSON, or screenshots.

Stretch Goals

- Add node-health alerting so the drill's failure would have paged the on-call.
- Add a second source (Oracle XE or a container) to the same cluster and document the driver dependency.
- Automate the connection-inventory freshness check as a scheduled notebook.
- Extend the security narrative with an egress-filtering design for the gateway segment.

Orchestration and Monitoring

Learning Objectives

- Invoke notebooks and Dataflows Gen2 from pipelines as orchestrated activities.
- Manage dependencies with On Success, On Failure, On Completion, and On Skip paths.
- Monitor runs in the Monitoring Hub and route alerts with runbook links to Teams or Outlook.
- Build a master orchestration pipeline that sequences a dataflow, a notebook, and failure alerting.
- Make every rerun idempotent so failed batches can be retried without data repair.
- Design a self-healing orchestration layer for a 15-system banking reconciliation.

Cross-Cloud Equivalence

Orchestration with dependency graphs, retries, and alerting is the most tool-shaped yet conceptually stable layer of the stack. The concepts—DAGs, trigger rules, compensating actions—transfer verbatim.

Capability	Fabric	AWS	GCP	Snowflake
Orchestrator	Pipelines (master/child)	Step Functions / MWAA (Airflow)	Cloud Composer (Airflow)	Task graphs
Dependency edges	On Success/Failure/Completion/Skip	States, catch/ retry	Airflow trigger rules	AFTER task deps
Cross-workload invoke	Notebook, dataflow, pipeline activities	Lambda, Glue, Batch	Cloud Run, Dataflow	Snowpark, external functions
Monitoring	Monitoring Hub + run history	CloudWatch + Step Functions console	Cloud Monitoring / Airflow UI	Task history, resource monitors
Alerting	Teams/Outlook activities, Activator	SNS / EventBridge	Alerting policies	Notifications / alerts
Lineage hooks	Fabric lineage view	OpenLineage (MWAA)	OpenLineage (Composer)	Access history

Databricks Workflows and MongoDB Atlas Triggers round out the series’ family. The portable discipline: *an alert without a runbook is noise; a retry without idempotence is corruption.*

Key Terms

Orchestration: sequencing and governing dependent workloads—pipelines, notebooks, dataflows—as one operable system. **Dependency condition:** the edge type between

activities: On Success, On Failure, On Completion, On Skip. **Monitoring Hub:** Fabric’s central view of runs across pipelines, notebooks, dataflows, and semantic models. **Compensating action:** the corrective step an On Failure path executes—quarantine, rollback, requeue—beyond mere notification. **Runbook:** the linked operational document an alert carries: rerun steps, backfill procedure, escalation owner.

12.1 Week 12 Roadmap

Week 12 closes Part III by turning the previous three weeks’ artifacts into one operable system. Monday covers invoking notebooks and dataflows from pipelines. Tuesday formalizes dependency management. Wednesday covers the Monitoring Hub, alert routing, and run history. Thursday is the hands-on project: a master pipeline that triggers a dataflow, waits, runs a notebook, and alerts on failure. Friday designs the self-healing orchestration layer for a 15-system banking reconciliation. The chapter closes with the Milestone 3 capstone.

Orchestration is where reliability is won or lost. Chapters 9–11 built movers, shapers, and connectors; this week decides what happens when they fail at 3 a.m. [51, 42].

Note

Production rule for the week: route pipeline failures to Teams/Outlook and attach a runbook link to every alert—expected rerun steps, backfill procedure, and escalation owner. An alert without a runbook creates a 3 a.m. archaeology session. Use On Failure paths for compensating actions, not just notifications.

12.2 Monday: Invoking Notebooks and Dataflows from Pipelines

Pipelines orchestrate the platform’s other workloads as first-class activities [42]. The *Notebook activity* executes a Spark notebook with parameters—this is how Chapter 10’s quality gate and Week 3’s transformation notebooks become production steps. The *Dataflow activity* triggers a Dataflows Gen2 refresh and waits for completion. A pipeline can also invoke another pipeline, which is how master/worker designs (Chapter 9) compose.

Definition

An **orchestration pipeline** is a pipeline whose primary purpose is sequencing other workloads—dataflows, notebooks, child pipelines—with dependency conditions, rather than moving data itself.

Parameter passing is the critical skill: the pipeline computes values (batch date, watermark, config row) and hands them to the notebook as parameters; the notebook reads them through its parameter cell and must never assume defaults in production. The return path matters too: notebooks signal failure by failing (an unhandled assertion, per Chapter 10’s gate), and the pipeline’s dependency edges translate that failure into compensating action.

Tip

Keep notebooks idempotent by contract: same parameters in, same final state out. The pipeline’s job is to call them with the right parameters; the notebook’s job is to be safely callable twice.

12.3 Tuesday: Dependency Management

Every activity edge carries a condition [42]:

Condition	Fires when	Canonical use
On Success	Predecessor succeeded	The happy path: gate passes, Gold publishes
On Failure	Predecessor failed	Compensating action + alert with runbook link
On Completion	Predecessor finished either way	Cleanup that must always run (temp tables, session teardown)
On Skip	Predecessor was skipped	Rare; explicit handling of conditional branches

Table 12.1: Dependency conditions and their canonical uses.

The discipline that separates operable pipelines from fragile ones: **every activity that can fail has an explicit On Failure path**. Minimum viable failure path: write the ledger row, execute the compensating action (quarantine partial output, reset state), and send the alert with its runbook link. “Do nothing and check the portal tomorrow” is not a failure path; it is an outage schedule.

Common Pitfall

On Completion is not On Success. Wiring a downstream step to On Completion because “it should usually work” makes partial failures propagate silently—the classic way corrupted staging becomes published Gold. Publish on success; clean up on completion.

12.4 Wednesday: Monitoring Hub and Alert Routing

The Monitoring Hub is Fabric’s cross-workload run observatory: pipeline runs, notebook executions, dataflow refreshes, and semantic model refreshes appear with status, duration, capacity, and error detail [51]. It answers the daily questions—what ran, what failed, what is slow—without portal archaeology.

Run history per pipeline provides the forensic view: activity-level durations and errors, input and output payloads, and retry trails. Paired with Chapter 9’s run ledger (which is queryable), the hub (which is visual) covers both operations and audit.

Alert routing closes the loop. Pipelines send Teams or Outlook messages from activities on the On Failure path; Data Activator (Chapter 15) can alert on capacity and platform events. Every alert carries: what failed, when, which batch, the runbook link, and the escalation owner.

Note

A good alert is a work order, not a scream. “PL_Master_Ingest failed at 03:12 on batch 2026-07-31; runbook: docs/runbooks/master-ingest.md; owner: DE on-call” lets a half-awake engineer act. “Pipeline failed” starts an investigation.

12.5 Thursday: Hands-on Project

The Week 12 lab builds the master orchestration pipeline: it triggers a Dataflow Gen2, waits for completion, triggers a Spark notebook (the Chapter 10 quality gate), and sends a Teams/Outlook

alert on failure.

12.5.1 Goal and Architecture

Reuse the Week 10 estate: dataflow `df_api_products`, gate notebook, lakehouse `lh_products`. Create pipeline `PL_Master_Daily` in workspace `fabric-de-week12` that sequences them with explicit dependency paths and a ledger write for every outcome.

12.5.2 Step-by-Step Actions

1. Create the pipeline with a `batch_date` parameter (default: UTC today).
2. Add the Dataflow activity; wire its On Success to the Notebook activity.
3. Configure the Notebook activity with the gate notebook, passing `batch_date` as a parameter.
4. Wire the notebook's On Success to a ledger write (status `Succeeded`).
5. Wire On Failure of both activities to a shared failure branch: ledger write (status `Failed`, error detail), then a Teams/Outlook activity with the runbook link.
6. Run the happy path; verify the ledger and the Gold table.
7. Break the gate deliberately (inject a duplicate key into staging); verify the alert fires with runbook link and the ledger records the failure.
8. Fix staging, rerun with the same `batch_date`, and prove the rerun is clean and duplicate-free.

12.5.3 Validation Checklist

- The pipeline sequences dataflow → gate with explicit dependency edges.
- Every activity that can fail has a wired On Failure path.
- The alert message names the pipeline, batch, runbook link, and owner.
- The ledger holds exactly one row per run outcome.
- The rerun after failure is idempotent—no duplicate Gold rows.
- Run history in the Monitoring Hub matches the ledger's account.

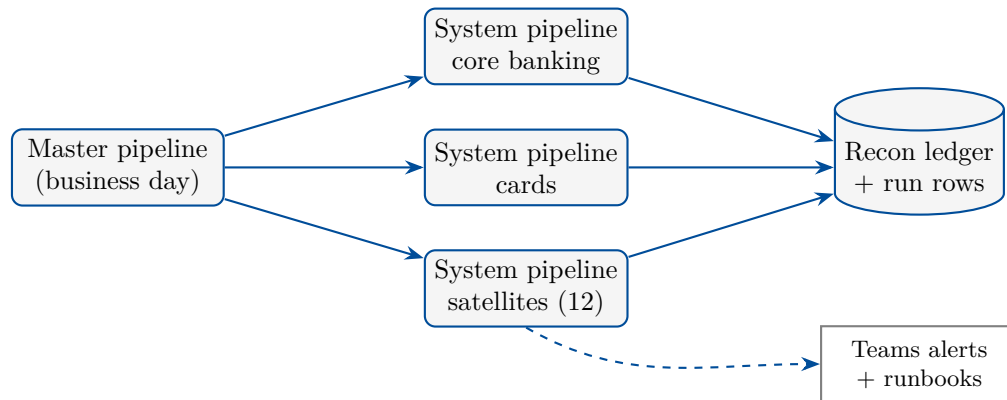
12.6 Friday: Use Case and Architecture

A bank runs a daily reconciliation spanning 15 dependent systems: core banking, cards, payments, general ledger, and eleven satellite feeds. The current estate is a cron-and-hope collection of scripts. The task: a resilient, self-healing orchestration layer in Fabric pipelines.

12.6.1 The Design

- **Layered pipelines.** A master pipeline per business day dispatches to system-level child pipelines, which dispatch to table-level workers (Chapter 9's framework, now fully operationalized).
- **Dependency maps, not chains.** Each system's pipeline declares its upstream dependencies; the master waits on completion conditions, not wall-clock schedules.

- **Compensating actions.** On Failure paths quarantine partial batches, mark the business day **degraded** in the recon ledger, and page with a runbook—they do not merely email.
- **Idempotent everything.** Watermarks advance on success; rerunning any system replays exactly its missing slice; the reconciliation report compares, never assumes.
- **Degraded-mode operation.** When a satellite feed misses its window, the master proceeds with a documented **degraded** flag and auto-backfills on the feed’s recovery—self-healing by design.



Self-healing: degraded-mode proceeds with flags; recovered feeds backfill automatically; every outcome is a ledger row.

Figure 12.1: Self-healing reconciliation orchestration across 15 systems.

Common Pitfall

Wall-clock scheduling (“system B starts at 04:00 because system A usually finishes by 03:50”) fails on the first slow night. Express real dependencies and let completion conditions drive; reserve clocks for windows and deadlines, not sequencing.

12.7 Operational Guidance for Week 12

Orchestration quality is measured on the worst night, not the average one. Drill failure paths monthly: kill a gateway, inject a contract violation, miss a feed window—and watch the estate heal itself or page correctly.

12.7.1 Performance and Reliability

Monitor duration trends, not just failures; a pipeline slowing 10% weekly is a failure in advance. Cap master-pipeline parallelism against capacity reality. Keep run history retention aligned with audit requirements. Prefer few, well-instrumented master pipelines over dozens of overlapping schedulers.

12.7.2 Security and Governance Checklist

- Every failure alert carries a runbook link and an owner; no orphan alerts.
- Compensating actions are least-privilege and auditable—they write to the ledger and quarantine, never to Gold.
- Runbook documents are versioned beside the pipelines they serve.

- Access to retry, skip, and force-run controls is restricted and logged.
- The reconciliation ledger is immutable—corrections are new rows, not edits.
- Scheduling identities are service principals, not personal accounts.

12.8 Interview Q&A

1. **Q: What is the difference between On Failure and On Completion paths?**
A: On Failure fires only when the predecessor failed—the right edge for compensating actions and alerts. On Completion fires either way—the right edge for unconditional cleanup. Publishing on On Completion propagates partial failures; that confusion corrupts data.
2. **Q: What does the Monitoring Hub show per pipeline run?**
A: Status, timing, duration, capacity consumption, and error detail across pipelines, notebooks, dataflows, and refreshes, with activity-level drill-down for forensics.
3. **Q: Why attach a runbook link to every alert?**
A: The alert arrives at 3 a.m. to a half-awake human; the runbook turns it from an investigation into a procedure—rerun steps, backfill path, escalation owner.
4. **Q: How do you make a rerun of a failed batch idempotent?**
A: Watermarks advance only on success; writes are merge- or partition-scoped; the batch re-executes exactly its missing slice, producing the same final state as a single clean run.
5. **Q: What is a compensating action in an On Failure path?**
A: The corrective work beyond notification: quarantining partial output, marking the business day degraded, resetting control state—so the estate is consistent before the alert is even read.
6. **Q: Why are dependencies better than wall-clock schedules?**
A: Clocks encode assumptions about duration that fail exactly on slow nights; completion conditions encode actual readiness, so the graph self-adjusts.

12.9 Further Reading

Review the Monitoring Hub documentation [51] and pipeline dependency and activity references [42, 19]. The ledger patterns extend Chapter 9 [14], and Data Activator-based platform alerting previews Chapter 15 [57]. For lineage as an observability complement, see OpenLineage [62].

Key Takeaways

- Pipelines orchestrate notebooks, dataflows, and child pipelines; parameters flow down, failures flow up through dependency edges.
- On Success publishes, On Failure compensates, On Completion cleans up—confusing them is a corruption vector.
- The Monitoring Hub is the visual observatory; the run ledger is the queryable one; you need both.
- Every alert is a work order: what failed, which batch, runbook link, owner.
- Idempotent reruns make 3 a.m. failures boring; design for the retry, not the apology.

- Self-healing orchestration declares real dependencies, operates degraded with flags, and backfills on recovery.
- Drill the failure paths; orchestration quality is defined by the worst night.

12.10 Exercises

1. **(Conceptual)** Justify “publish on success, clean up on completion” with a concrete corruption scenario for the reverse.
2. **(Conceptual)** Why does the recon ledger forbid edits? What audit property does append-only provide?
3. **(Hands-on)** Add an Until loop to the lab pipeline that waits for a late satellite feed with a documented deadline and degraded-mode exit.
4. **(Hands-on)** Build the duration-trend query over the run ledger: per pipeline, weekly average duration and week-over-week delta.
5. **(Design)** Map the dependency graph for five of the bank’s fifteen systems, marking which can run degraded and which halt the business day.
6. **(Design)** Design the alerting severity ladder: which failures page, which email, which merely log—and who decided?
7. **(Interview)** In five sentences, explain how compensating actions and idempotent reruns together make an orchestration layer “self-healing.”
8. **(Operational)** Write the runbook for the lab pipeline’s most likely failure, then have a teammate execute it cold.

12.11 Milestone 3 Capstone: Automated Batch ELT Pipeline

Capstone Project

Mission. Close Part III by building the complete batch ELT system: a config-table-driven Fabric pipeline family (Lookup → ForEach Copy) ingesting from REST APIs and Azure SQL, Dataflows Gen2 for visual cleansing, Spark notebooks for ML feature engineering, a contract gate with quarantine, and a master orchestration pipeline with failure alerts wired to a runbook—scheduled, monitored, and documented.

Estimated time. 10–12 hours across the milestone’s components.

What you’ll practice.

- Metadata-driven ingestion with watermark-safe incremental loads.
- Contract validation distinguishing breaking from additive schema changes.
- Dataflow → gate → notebook sequencing under one master pipeline.
- Data-quality assertions with quarantine and failing-case evidence.
- Alerting with runbook links, plus the capacity cost estimate and security review rubric artifacts.

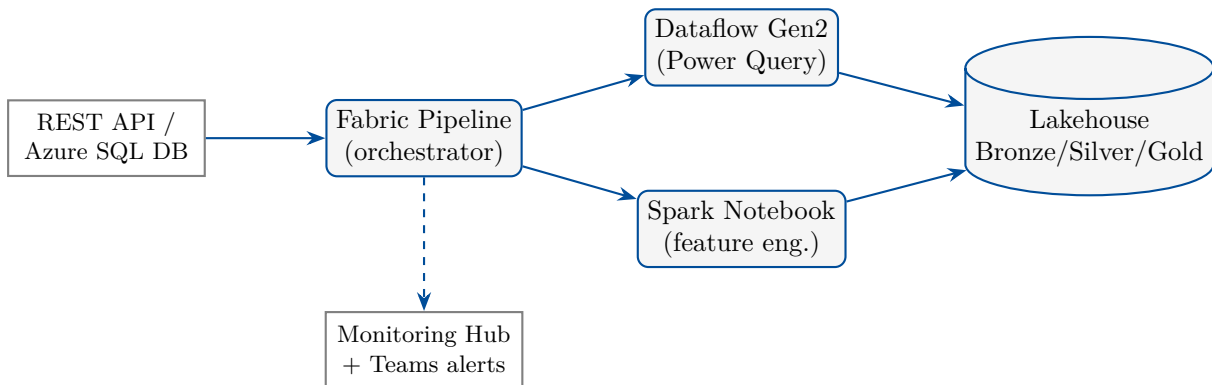
Scenario

Contoso’s operations analytics program must absorb REST API feeds and Azure SQL extracts nightly, cleanse them through analyst-owned dataflows, engineer ML features in Spark, and publish Gold tables that Chapter 8’s Direct Lake reports consume—all self-healing, all observable, all rerun-safe.

Requirements

- `cfg.ingest_config` with watermark and `is_active` columns, and the `cfg.ingest_run` ledger.
- The dynamic pipeline family with a schema-validation gate rejecting breaking contract changes.
- Bronze landing, Dataflow Gen2 cleansing to Silver, notebook feature engineering and DQ assertions with quarantine before Gold.
- Master pipeline wiring: On Failure → Teams/Outlook alert with runbook link; watermarks advance only on success.
- The six rubric artifacts: clean-environment README, architecture diagram, three DQ tests with failing cases, alert evidence, cost estimate, security review.

Reference Architecture



One orchestrator, three workload types, one observable estate.

Figure 12.2: Milestone 3 reference architecture: orchestrated batch ELT.

Implementation Steps

1. Create the config and ledger tables; seed sources including at least one REST feed.
2. Build the worker pipeline with the contract gate; build the master with bounded parallelism.
3. Build the cleansing dataflow with named steps and a staging destination.
4. Implement the feature-engineering notebook and the DQ gate with quarantine.
5. Wire the master pipeline's On Failure branch to the alert with runbook link.
6. Run the deliberate-failure drill; capture the alert and the clean rerun.
7. Produce the cost estimate and security review; finalize the README.

```

1 -- Advance the watermark only after a successful batch
2 UPDATE cfg.ingest_config
3 SET watermark = @max_ts
4 WHERE schema_name = @sch AND table_name = @tbl
5    AND @batch_status = 'Succeeded';
6 -- On rerun, the source query reads WHERE ts > @watermark,

```

```
7 -- so a retried batch never re-copies committed rows.
```

Listing 12.1: Milestone 3: watermarks advance only after a successful batch.

Deliverables

- Repository that runs end-to-end from a clean environment on schedule.
- Architecture diagram showing pipeline, dataflow, notebook, and lakehouse flow.
- Three automated DQ tests including a quarantined-row case, each with failing-case evidence.
- Teams/Outlook alert evidence from the deliberate-failure drill, with the runbook.
- Monthly capacity cost estimate with two documented optimization decisions.
- Security review: gateway credentials vaulted, least-privilege connection identities, no hard-coded secrets.

Acceptance Criteria

- ☐ The scheduled end-to-end run completes from a clean environment using only the README.
- ☐ New tables onboard via config insert; no pipeline edits.
- ☐ Breaking schema changes reject and quarantine; additive changes flow and are logged.
- ☐ Every run—success, rejection, failure—produces exactly one ledger row.
- ☐ The failure drill fires the alert with its runbook link, and the rerun is duplicate-free.
- ☐ Watermarks advance only on success, proven by a killed mid-run retry.
- ☐ No credentials or personal paths appear anywhere in submitted artifacts.

Stretch Goals

- Enable ForEach parallelism with a documented bound and show the duration improvement.
- Add a duration-trend report over the ledger and alert on 30% week-over-week regression.
- Add degraded-mode operation for one satellite feed with automatic backfill on recovery.
- Publish the operations dashboard (run health, quarantine volumes) as a Direct Lake report.

Part IV

Real-Time Intelligence and Streaming

Eventstreams and Real-Time Ingestion

Learning Objectives

- Describe the Real-Time Intelligence architecture in Fabric and where Eventstreams sit within it.
- Configure Eventstream sources: Azure Event Hubs, custom apps, and IoT-style producers.
- Route Eventstream destinations to lakehouses, KQL databases, and Reflex items.
- Send mock IoT telemetry from a Python producer to a custom app endpoint and land it in a KQL database.
- Design idempotent, replay-safe consumers for at-least-once delivery.
- Architect a global telemetry ingestion point for five million mobile games.

Cross-Cloud Equivalence

Managed streaming ingestion—durable event buffers with fan-out to analytical and operational sinks—exists on every cloud. Fabric Eventstreams wraps Azure’s eventing lineage in the SaaS workspace model.

Capability	Fabric	AWS	GCP	Snowflake
Event ingestion	Eventstreams (Event Hubs lineage)	Kinesis / MSK (Kafka)	Pub/Sub	Snowpipe Streaming
Custom producer endpoint	Custom app source	Kinesis PutRecords / Kafka producer	Pub/Sub publisher	Streaming ingest SDK
Stream processing	Eventstream operators, KQL	Kinesis Data Analytics / Flink	Dataflow (Beam)	Streams + tasks / dynamic tables
Analytical sink	KQL database (Eventhouse)	Redshift / Athena over S3	BigQuery streaming	Snowflake tables
Lake sink	Lakehouse Delta tables	S3 via Firehose	Cloud Storage	Managed staging
Alerting sink	Data Activator (Reflex)	EventBridge / Lambda	Eventarc / Cloud Functions	Alerts / notifications
Delivery semantics	At-least-once	At-least-once (Kinesis)	At-least-once (default)	Exactly-once ingest option

Databricks ingests streams with Structured Streaming and Delta Live Tables; MongoDB produces streams through change streams. The portable disciplines are this chapter’s core: unique event IDs at the producer, dedup at the first queryable boundary, and knowing your source’s replay window.

Key Terms

Eventstream: a Fabric item that ingests, optionally transforms, and routes event streams to destinations. **Custom app source:** an Event Hubs-compatible endpoint an Eventstream exposes for producer code. **At-least-once delivery:** the guarantee that events arrive one or more times—never zero—which makes duplicates possible. **Replay:** redelivery of events from the source’s retention window after a downstream failure or reconfiguration. **Idempotent consumer:** a consumer that processes the same event multiple times with the same final effect as processing it once. **Eventhouse:** the Fabric item hosting KQL databases for streaming analytics (Chapter 14).

13.1 Week 13 Roadmap

Week 13 opens Part IV, the real-time arc. Monday maps the Real-Time Intelligence architecture. Tuesday covers Eventstream sources. Wednesday covers destinations—lakehouse, KQL database, Reflex. Thursday is the hands-on project: a Python script sends mock IoT temperature data to a custom app endpoint, routed into a KQL database. Friday architects the global telemetry ingestion point for five million mobile games.

Batch engineering asks “how do we move last night’s data?” Streaming asks “how do we reason about data that never stops arriving?” The shift in mindset—unbounded input, event time versus processing time, at-least-once reality—is the real content of the week [26].

Note

Production rule for the week: design Eventstream consumers for idempotency and replay. Events can be delivered more than once, so stamp every event with a unique ID and dedupe downstream (for example Delta MERGE on `event_id`). Know the source’s replay window (Event Hubs retention) so you can reprocess after a downstream failure without data loss or duplicates.

13.2 Monday: Real-Time Intelligence Architecture

Fabric’s Real-Time Intelligence experience organizes streaming work into a small set of cooperating items [26, 55]:

- **Eventstreams** ingest from sources and route to destinations, with optional in-flight operators (filter, aggregate, union, expand).
- **Eventhouses** host KQL databases—the analytical store for high-volume telemetry, covered in Chapter 14.
- **Data Activator (Reflex)** items evaluate conditions over streams and Power BI visuals and fire actions, covered in Chapter 15.
- **Real-Time Dashboards** visualize KQL over live data, covered in Chapter 16.

Definition

Real-Time Intelligence is Fabric’s streaming workload family: ingestion (Eventstreams), analytical storage and query (Eventhouses/KQL), reaction (Reflex), and visualization (Real-Time Dashboards), all sharing OneLake and the workspace capacity model.

The architectural significance: streaming is no longer a separate platform. The same workspace that holds the batch lakehouse holds the eventstream; the same capacity meters both; the same governance applies. The data engineer’s new obligations are the streaming-specific ones—delivery semantics, replay behavior, and event-time reasoning—not a new estate to learn.

13.3 Tuesday: Eventstream Sources

An eventstream begins at a source [26]:

Source	Use	Notes
Azure Event Hubs	Existing event estates, high-throughput producers	Bring your own namespace; the classic enterprise bridge
Custom app	Producer code you write (this week’s lab)	Exposes an Event Hubs-compatible connection string
IoT-style / device feeds	Telemetry at scale	Via IoT Hub or direct producer patterns
Sample data	Learning and demos	Built-in generator for exploration
CDC feeds	Database change capture	Operational tables become streams

Table 13.1: Eventstream source types.

The custom app source deserves emphasis: it gives any process that can open an AMQP/Kafka connection a managed landing point in Fabric. The lab’s Python producer uses it; so does every home-grown telemetry emitter in the enterprise.

Tip

Treat the custom app connection string as a secret from the first lab: environment variable or Key Vault reference in the producer, never in source. The string is write access to your stream.

13.4 Wednesday: Eventstream Destinations

Destinations determine what the stream becomes [26]:

- **KQL database (Eventhouse):** the analytical destination—indexed, compressed, queryable in KQL within seconds. Choose it for telemetry exploration, time-series analytics, dashboards, and anything Reflex or Real-Time Dashboards consume.
- **Lakehouse:** the durable open destination—Delta tables in OneLake, joining the batch estate. Choose it when stream data must combine with batch history in Spark or feed the medallion layers.
- **Reflex (Data Activator):** the reactive destination—conditions evaluate per event or aggregate, actions fire (Chapter 15).
- **Custom endpoints:** onward routing to systems outside Fabric.

Routing is fan-out: one stream, many destinations. The canonical topology sends raw events to both the KQL database (analytics) and the lakehouse (durability and batch integration), plus a filtered branch to Reflex (reaction).

13.4.1 Delivery Semantics and Idempotency

Every hop is at-least-once: Eventstream to KQL, Eventstream to lakehouse, replay from the Event Hubs retention window after failures. The consequence is absolute: *downstream must dedupe*. Stamp a unique `event_id` at the producer—before the first send, not at any hop—propagate it untouched, and enforce dedup at the first queryable boundary: a KQL materialized view with `arg_max` (Chapter 14) or a Delta `MERGE` on `event_id` for the lakehouse. Pair this with consumer checkpointing—recording the last processed offset or timestamp—so restarts resume instead of replaying.

Common Pitfall

Generating the event ID at a mid-pipeline hop is too late: the producer's retry already created two physical copies of the same logical event before any hop saw them. The unique ID must exist at the producer, in the payload, from birth.

13.5 Thursday: Hands-on Project

The Week 13 lab creates a Fabric Eventstream, writes a Python producer that sends mock IoT temperature readings to the custom app endpoint, and routes the stream to a KQL database.

13.5.1 Goal and Architecture

Create workspace `fabric-de-week13` with eventstream `es_iot_telemetry` and eventhouse `eh_iot` containing KQL database `kqldb_iot`. The producer simulates twenty devices emitting temperature readings every second, each event stamped with a unique `event_id`.

13.5.2 Step-by-Step Actions

1. Create the eventstream; add a *custom app* source; copy its connection string into an environment variable.
2. Add the KQL database destination; create the `IoTTelemetry` table.
3. Write the producer: batch sends, unique event IDs, graceful shutdown.
4. Run the producer for five minutes; verify events appear in KQL.
5. Query: events per device per minute, latest reading per device.
6. Kill and restart the producer; observe that replays or retries produce duplicate `event_ids` if—and only if—the producer reuses IDs incorrectly; fix it if so.
7. Document the replay window of the underlying Event Hubs entity.

```

1 import os, json, time, uuid, random
2 from azure.eventhub import EventHubProducerClient, EventData
3
4 CONNECTION_STR = os.environ["EVENTSTREAM_CONN_STR"] # from the custom app
5               source
6
7 producer = EventHubProducerClient.from_connection_string(CONNECTION_STR)
8 devices = [f"device-{i:02d}" for i in range(20)]
9
10 try:
11     while True:
12         batch = producer.create_batch()
13         for device in devices:
14             event = {

```



```

14         "event_id": str(uuid.uuid4()),           # stamped at the producer
15         "device_id": device,
16         "timestamp": time.time(),
17         "temperature": round(18 + 10 * random.random(), 2),
18         "humidity": round(30 + 40 * random.random(), 2),
19     }
20     batch.add(EventData(json.dumps(event)))
21     producer.send_batch(batch)
22     time.sleep(1)
23 except KeyboardInterrupt:
24     producer.close()

```

Listing 13.1: Mock IoT producer sending telemetry to the Eventstream custom app endpoint.

```

1 -- Events per device per minute (dedup on event_id first)
2 IoTTelemetry
3 | summarize arg_max(ingestion_time(), *) by event_id
4 | summarize events = count() by device_id, bin(todatetime(timestamp), 1m)
5 | order by device_id, bin_todatetime_timestamp asc;
6
7 -- Latest reading per device
8 IoTTelemetry
9 | summarize arg_max(todatetime(timestamp), *) by device_id
10 | project device_id, timestamp, temperature, humidity;

```

Listing 13.2: First KQL verification queries over the landed stream.

13.5.3 Validation Checklist

- The producer sends via the connection string from an environment variable, not source code.
- Every event carries a unique `event_id` generated before the first send.
- Events are queryable in KQL within seconds of production.
- The dedup query and the raw count differ after a replay test—and you can explain why.
- The replay window of the source is recorded in the runbook.
- Graceful shutdown leaves no partial batch unaccounted for.

13.6 Friday: Use Case and Architecture

A games publisher operates titles played by five million concurrent mobile devices emitting session, performance, and crash telemetry. The requirement: one global ingestion point, high throughput, low-latency processing into Fabric.

13.6.1 The Architecture

Regional Event Hubs namespaces absorb producer traffic close to players; Eventstreams attach to them and fan out: the full stream to a lakehouse (durability, batch joins with purchase history), the full stream to an Eventhouse KQL database (live analytics), and a filtered branch—crashes and error events—to Reflex for immediate reaction. Producers stamp `event_id` and session identifiers; the first queryable boundaries dedupe; consumer checkpoints make restarts safe.

Design decision	Choice	Reason
Regional Event Hubs front doors	Yes, per geography	Latency and blast-radius isolation
One eventstream per region	Yes	Independent scaling and failure domains
Lakehouse destination	Full stream	Durability + batch integration
KQL destination	Full stream	Live analytics and dashboards
Reflex branch	Crash/error filter only	Alert volume control
Dedup boundary	KQL materialized view + Delta MERGE	First queryable points per path

Table 13.2: Global telemetry architecture decisions.

Common Pitfall

Five million producers will, at statistical certainty, produce every failure mode you did not design for: clock skew, retries after partial batch sends, duplicate SDK initializations. The dedup contract—unique ID at the producer, enforced at the first queryable boundary—is not a best practice here; it is the only thing standing between you and phantom double-counted revenue events.

13.7 Operational Guidance for Week 13

Streaming systems fail differently than batch: continuously, partially, and at inconvenient times. Design for restart, replay, and dedup from day one.

13.7.1 Performance and Reliability

Batch producer sends (amortize connection and per-event overhead). Monitor ingestion lag as a first-class signal—lag is the streaming equivalent of a failed batch. Size the replay window to your slowest realistic recovery. Keep eventstream operators simple; heavy transformation belongs in KQL or Spark where it is testable.

13.7.2 Security and Governance Checklist

- Connection strings in environment variables or Key Vault, rotated, never in source.
- Producers identified per system; a leaked connection string must be attributable.
- Telemetry containing personal data classified at the stream level, not discovered downstream.
- Replay windows documented; reprocessing is an auditable operation.
- Dedup keys specified in the data contract (Chapter 9) of every stream.
- Destination access scoped: lakehouse and KQL permissions reflect who should see live operational data.

13.8 Interview Q&A

1. Q: Name two Eventstream sources and one destination.

A: Azure Event Hubs and custom app endpoints are sources; KQL databases, lakehouses, and Reflex items are destinations.

2. Q: Why stamp every event with a unique ID?

A: At-least-once delivery means every event may arrive multiple times; the producer-stamped ID is the only trustworthy dedup key, because IDs minted mid-pipeline cannot distinguish a producer retry from a new event.

3. Q: What does the Event Hubs retention (replay) window control?

A: How far back a consumer can reprocess after failure—recovery beyond the window loses data, so the window must exceed your slowest realistic recovery.

4. Q: When route events to a KQL database versus a lakehouse?

A: KQL for live analytics, dashboards, and Reflex consumption—indexed and queryable in seconds; lakehouse for durable open storage and batch integration with the medallion estate. Most architectures fan out to both.

5. Q: What does at-least-once delivery imply for downstream consumers?

A: Every consumer must be idempotent: dedupe on the producer-stamped ID at the first queryable boundary, and checkpoint progress so restarts resume instead of replay.

6. Q: What is the canonical fan-out topology for a production stream?

A: Full stream to KQL (analytics) and lakehouse (durability), plus a filtered branch to Reflex (reaction)—three different questions answered from one ingestion point.

13.9 Further Reading

Review the Eventstreams documentation [26] and the Eventhouse overview [55] that Chapter 14 develops. The reaction destination is introduced by the Data Activator documentation [57] (Chapter 15). The lakehouse destination's table semantics come from Delta Lake [9]; Kafka's design is the conceptual baseline for all of it [1].

Key Takeaways

- Real-Time Intelligence = Eventstreams (ingest) + Eventhouses (analyze) + Reflex (react) + Real-Time Dashboards (visualize).
- The custom app source turns any producer into a managed Fabric ingestion point via an Event Hubs-compatible endpoint.
- Destinations define purpose: KQL for analytics, lakehouse for durability and batch joins, Reflex for reaction.
- At-least-once delivery is not a defect to lament but a contract to honor: unique IDs at the producer, dedup at the first queryable boundary.
- Replay windows bound your recovery; size them to your slowest realistic incident.
- Streaming belongs in the same workspace, capacity, and governance as batch—the disciplines differ, the estate does not.
- At global scale, the dedup contract is the difference between telemetry and fiction.

13.10 Exercises

1. **(Conceptual)** Explain event time versus processing time, and why the distinction matters for a daily aggregate computed over a stream.

2. **(Conceptual)** Why is “exactly-once delivery” the wrong goal and “idempotent consumption” the right one?
3. **(Hands-on)** Extend the producer with a deliberate 1% duplicate-send rate; verify the dedup query stays correct and the raw count inflates.
4. **(Hands-on)** Add a lakehouse destination to the eventstream and implement the Delta MERGE dedup on `event_id`.
5. **(Design)** Design the partitioning strategy for the games publisher: how many partitions, keyed by what, and why?
6. **(Design)** A device fleet goes offline for nine hours and backhauls. Which replay windows and late-arrival policies protect the daily aggregates? (Chapter 14 deepens this.)
7. **(Interview)** In five sentences, walk an interviewer through exactly what happens when a KQL destination fails for an hour and recovers.
8. **(Operational)** Write the runbook for “ingestion lag exceeds five minutes”—detection, triage, replay safety check, escalation.

13.11 Capstone Project: Replay-Safe Telemetry Pipeline

Capstone Project

Mission. Build a production-shaped telemetry pipeline: a Python producer fleet simulator, an eventstream fanning out to a KQL database and a lakehouse, dedup enforced at both boundaries, a consumer checkpoint store, and—the proof—a documented replay drill in which a downstream “outage” is recovered without loss or duplicates.

Estimated time. 5–7 hours.

What you’ll practice.

- Producer discipline: unique IDs, batching, graceful shutdown, secret handling.
- Fan-out routing to analytical and durable destinations.
- Dedup at first queryable boundaries on two different engines.
- Checkpointing and the end-to-end replay drill.

Scenario

Contoso Devices ships a connected thermostat line. Telemetry must be live-queryable for support (KQL), durably stored for the data platform (lakehouse), and provably correct under the failure mode the field team fears most: the analytics path goes down for an hour while thermostats keep reporting.

Requirements

- Producer simulator for 50 devices with unique `event_ids` and a configurable duplicate-injection rate.
- Eventstream `es_thermostat` with KQL and lakehouse destinations.
- Dedup: KQL-side `arg_max` pattern and lakehouse-side Delta MERGE on `event_id`.
- A checkpoint table recording last-processed event time per consumer.
- The replay drill: suspend the KQL destination for 30+ minutes, resume, and prove zero loss and zero duplicates with reconciliation queries.

- A runbook documenting replay windows, dedup keys, and the drill procedure.

Implementation Steps

1. Build the producer with duplicate injection and graceful shutdown.
2. Wire the eventstream with both destinations; create tables.
3. Implement both dedup boundaries and the checkpoint store.
4. Run the pipeline for a baseline; record counts.
5. Execute the replay drill; capture before/during/after evidence.
6. Write reconciliation queries proving loss-free, duplicate-free recovery.

Deliverables

- Producer source, pipeline/eventstream exports, and table DDL in the repository.
- Baseline and post-drill reconciliation query outputs.
- The drill narrative: timeline, observed behavior, reconciliation proof.
- The operational runbook.

Acceptance Criteria

- ☐ Every event carries a producer-stamped unique ID; duplicate injection measurably exercises the dedup path.
- ☐ Both destinations dedupe correctly under injected duplicates.
- ☐ The replay drill recovers with zero loss and zero duplicates, proven by queries.
- ☐ Checkpoints allow a consumer restart without reprocessing.
- ☐ The runbook records replay windows and dedup keys per stream.
- ☐ No connection strings appear in source or screenshots.

Stretch Goals

- Add a Reflex branch firing on temperature spikes (previews Chapter 15).
- Add lag monitoring: emit a metric when ingestion lag exceeds a threshold.
- Simulate a nine-hour device backhaul and document the late-arrival impact on aggregates.
- Compare dedup cost: materialized view versus query-time `arg_max`, at one day of volume.

KQL Databases and Eventhouses

Learning Objectives

- Explain the Eventhouse architecture and how KQL databases store and index streaming data.
- Write core KQL: `search`, `where`, `project`, `summarize`, and time-bounded filters.
- Maintain incremental state with materialized views and deduplicate replayed events with `arg_max`.
- Build time series with `make-series`, rolling windows, and `series_decompose_anomalies`.
- Express stream-to-stream and stream-to-dimension joins with time-windowed keys, and reason about late arrivals.
- Troubleshoot a dashboard whose daily aggregates are wrong because offline devices uploaded data late.

Cross-Cloud Equivalence

Purpose-built time-series/log analytics engines—columnar, append-optimized, query-language-native—exist across the ecosystem. Fabric’s Eventhouse is the Azure Data Explorer (Kusto) lineage delivered inside the SaaS workspace.

Capability	Fabric	AWS	GCP	Snowflake
Telemetry/log engine	Eventhouse (KQL database)	CloudWatch Logs Insights / OpenSearch / Timestream	Cloud Logging / BigQuery streaming analytics	Snowflake tables + time-series functions
Query language	KQL	Logs Insights QL / SQL	SQL / Logging query language	SQL
Time-series functions	<code>make-series</code> , <code>series_*</code> family	Timestream SQL functions	BigQuery window/ML functions	Window functions, ASOF joins
Incremental state	Materialized views	Scheduled queries / rollups	BigQuery materialized views	Materialized views, dynamic tables
Dedup idiom	<code>arg_max</code> by id	Window dedup	<code>ROW_NUMBER</code> dedup	<code>ROW_NUMBER</code> / <code>MERGE</code>
Retention control	Retention policies per table	Log group retention / TTL	Log bucket retention / table TTL	Time Travel + data retention

Databricks approaches telemetry with Structured Streaming plus Delta; MongoDB with time-series collections. The portable skills: think in *append-only streams plus incremental state*, bound your lookback windows, and treat late arrivals as a policy decision, not a surprise.

Key Terms

Eventhouse: the Fabric item that hosts one or more KQL databases for real-time analytics. **KQL (Kusto Query Language):** a pipe-composed, read-optimized query language for telemetry and time-series data. **Materialized view:** a continuously maintained query result—incremental state over an append-only stream. **arg_max:** the KQL aggregate that returns the row with the maximum value of an expression—the dedup idiom. **make-series:** the operator that converts event streams into ordered time-series arrays for analysis. **Late arrival:** an event whose event time precedes its arrival by more than the processing window allows.

14.1 Week 14 Roadmap

Week 14 teaches the query heart of real-time intelligence. Monday covers Eventhouses and KQL databases. Tuesday builds KQL fluency—**search**, **where**, **project**, **summarize**—plus materialized views and the **arg_max** dedup idiom. Wednesday goes advanced: time-series analysis, stream joins with time-windowed keys, late-arrival implications, and why unbounded lookback windows degrade performance. Thursday is the hands-on project: a five-minute rolling average and anomaly detection over the Week 13 IoT stream. Friday runs the troubleshooting scenario—a dashboard wrong because devices went offline and uploaded late—and closes the loop on end-to-end exactly-once reasoning.

KQL is the highest-leverage new language in this program. It reads like a pipeline—data flows through | stages—and it is optimized for exactly the questions streaming systems ask: filter by time, aggregate by window, detect the anomaly, find the latest state [30, 55].

Note

The unifying idea of the week: an append-only event stream plus maintained incremental state (materialized views) reproduces most of what mutable tables give you—without giving up the replay-safety of never updating in place.

14.2 Monday: Eventhouses and KQL Databases

An Eventhouse is the Fabric container for KQL databases: the compute and storage fabric that ingests streams, indexes columns, compresses extents, and serves queries within seconds of ingestion [55]. A KQL database inside it holds tables, materialized views, functions, and policies—retention, caching, row-level security.

Definition

An **Eventhouse** is Fabric’s real-time analytics engine host: it provides the compute and storage under which KQL databases ingest, index, and serve high-volume append-only telemetry with sub-second-to-second query latency.

Three properties shape engineering practice. *Append-first:* tables are optimized for relentless inserts; updates and deletes exist but are exceptions, which is why dedup happens at query/materialization time. *Time-native:* every table has an ingestion timestamp, most have an event timestamp, and the engine indexes time aggressively. *Policy-driven:* retention and caching are table policies, so hot recent data serves from cache while history ages out deliberately.

Common Pitfall

Eventhouse compute meters against your Fabric capacity. An unbounded dashboard query (“all events, all time, every refresh”) is a capacity incident disguised as a visualization. Bound every query with a time filter as a habit, not an optimization.

14.3 Tuesday: KQL Basics, Materialized Views, and Dedup

14.3.1 The Core Verbs

KQL composes operators left to right through the pipe [30]:

```

1 -- search: raw text/expression hunt across columns
2 IoTTelemetry
3 | search "device-07";
4
5 -- where + project: filter and shape
6 IoTTelemetry
7 | where Timestamp > ago(1h) and Temperature > 25
8 | project DeviceId, Timestamp, Temperature;
9
10 -- summarize: aggregate by groups and time bins
11 IoTTelemetry
12 | where Timestamp > ago(24h)
13 | summarize AvgTemp = avg(Temperature), Readings = count()
14   by DeviceId, bin(Timestamp, 5m)
15 | order by DeviceId, bin_Timestamp asc;
```

Listing 14.1: Core KQL verbs over the IoT stream.

14.3.2 Materialized Views as Incremental State

A materialized view maintains an aggregate or projection continuously as data arrives—the streaming answer to “recompute the summary every night.” Over an append-only stream, the view is the state: dashboards and rules read the view, never the raw firehose [32].

14.3.3 Dedup with `arg_max`

Chapter 13’s at-least-once reality lands here. The dedup idiom keeps the latest arrival per logical event:

```

1 IoTTelemetry
2 | summarize arg_max(ingestion_time(), *) by EventId
```

Listing 14.2: Deduplication idiom: latest arrival per event id.

Enforced inside a materialized view (or as the first stage of every consumer query), this makes replay harmless: re-delivered events collapse back to one logical row. The dedup key—`EventId` stamped at the producer—is the contract from Chapter 13 made executable.

14.4 Wednesday: Time Series, Joins, and Late Arrivals

14.4.1 Time-Series Analysis

`make-series` converts events into ordered arrays per entity; the `series_*` function family then operates on the arrays—filters, decomposition, forecasting, anomaly detection [30].

14.4.2 Stream Joins with Time-Windowed Keys

Stream-to-stream joins correlate two unbounded flows; the join key alone is insufficient—a time window bounds which events can match. Stream-to-dimension joins enrich events with the current (or as-of) dimension state. Both patterns appear in Friday’s listing.

14.4.3 Late Arrivals and Window Policy

When a device goes offline and backhauls hours of events, every windowed aggregate over the affected period was computed on incomplete data. The engineering responses, in order of preference: *size windows and materialization* *lookback to the realistic lateness distribution*; *re-aggregate on arrival* where the materialized view semantics allow; and *document the lateness policy* so “wrong” numbers are understood as a stated contract, not a defect. Unbounded lookback—“just re-scan everything always”—degrades both query and materialization performance precisely because it abandons the incremental-state design.

Common Pitfall

A dashboard that silently recomputes with late data shows one number at 09:00 and a different number at 17:00 for the same day, and someone will present both in the same meeting. Decide and publish the lateness policy: how late counts, when numbers stabilize, and how revisions are marked.

14.5 Thursday: Hands-on Project

The Week 14 lab queries the Week 13 IoT data: a five-minute rolling average temperature per device and anomaly detection using built-in series functions.

14.5.1 Goal and Architecture

Reuse `kqldb_iot.IoTTelemetry` from Week 13 (or regenerate with its producer). Deliverables: the rolling-average query, the anomaly-flagged result set, and a materialized view maintaining the per-device five-minute means.

14.5.2 Step-by-Step Actions

1. Verify data is flowing (or restart the producer) and bound every query with `ago()`.
2. Build the `make-series` base: average temperature per device per five-minute step.
3. Apply `series_fir` for the rolling average.
4. Apply `series_decompose_anomalies` with a 1.5 threshold and expand back to rows.
5. Validate: inject a synthetic hot streak from the producer and watch it flag.
6. Create the materialized view for the per-device means and compare its latency against the raw query.

```

1 // 5-minute rolling average + anomaly detection per device
2 IoTTelemetry
3 | where Timestamp > ago(6h)
4 | make-series AvgTemp = avg(Temperature) default=0
5   on Timestamp step 5m by DeviceId
6 | extend RollingAvg = series_fir(AvgTemp, repeat(1, 5))
7 | extend Anomalies = series_decompose_anomalies(RollingAvg, 1.5)
8 | mv-expand Timestamp, RollingAvg, Anomalies
9 | project DeviceId,

```

```

10      Timestamp = todatetime(Timestamp),
11      RollingAvg = todouble(RollingAvg),
12      IsAnomaly = toint(Anomalies) == 1
13 | order by DeviceId, Timestamp

```

Listing 14.3: Week 14 lab: rolling average and anomaly detection per device.

14.5.3 Validation Checklist

- Every query carries a bounded time filter.
- The rolling average smooths a noisy device without hiding the injected hot streak.
- Anomaly flags fire on the synthetic spike and stay quiet on normal noise.
- The materialized view returns per-device means faster than the raw aggregation.
- You can explain what `default=0` does to sparse series—and when that default lies.

14.6 Friday: Use Case and Architecture

A streaming dashboard shows daily aggregations that keep changing: mobile devices went offline overnight and uploaded their buffered events at dawn. Daily totals for “yesterday” grew between the morning standup and the afternoon review. Diagnose and fix.

14.6.1 Diagnosis

The aggregates are computed over arrival-time windows, not event-time windows with a lateness policy. Yesterday’s numbers were computed from whatever had *arrived* by midnight, then silently revised as the backhaul landed. Three compounding issues: no event-time windowing, no stated lateness policy, and—discovered during the fix—duplicates from at-least-once redelivery inflating the revised totals.

14.6.2 The Fix: End-to-End Exactly-Once as a Discipline

Assemble the week’s concepts into the full pattern: Eventstream’s at-least-once delivery; idempotent ingestion into KQL and the lakehouse; the dedup key stamped at the producer and enforced at the first queryable boundary; and consumer checkpointing so restarts resume rather than replay. The query layer implements it:

```

1 // Dedup (keep latest arrival per id), windowed stream join, dimension lookup
2 let Telemetry =
3     IoTTelemetry
4     | where Timestamp > ago(1h) // bounded lookback window
5     | summarize arg_max(ingestion_time(), *) by EventId;
6 let Alerts =
7     DeviceAlerts
8     | where AlertTime > ago(1h)
9     | summarize arg_max(ingestion_time(), *) by AlertId;
10 Telemetry
11 | join kind=inner Alerts on DeviceId
12 | where abs(datetime_diff('second', Timestamp, AlertTime)) <= 300
13 | join kind=inner ( // stream-to-dimension
14     Devices | project DeviceId, Site, Model
15 ) on DeviceId
16 | summarize Readings = count() by Site, Model, bin(Timestamp, 1m)

```

Listing 14.4: Dedup, windowed stream join, and dimension enrichment in one pattern.

For the dashboard itself: aggregate on event time, set the materialization lookback to the measured lateness distribution (for example 24 hours for this fleet), mark values inside the lateness window as *provisional* in the report, and let the materialized view absorb revisions instead of the executives.

Common Pitfall

Eventstream replay is the top source of silent duplicates downstream: after a consumer failure or reconfiguration, the source's retention window redelivers, and every at-least-once hop can double-land rows. Stamp the dedup key at the producer, propagate it untouched, and enforce dedup at the first queryable boundary so every downstream consumer—dashboards, Reflex rules, reverse ETL—reads already-clean data.

14.7 Operational Guidance for Week 14

KQL estates stay healthy through policy: retention, caching, bounded queries, and materialized views doing the heavy lifting dashboards would otherwise do badly.

14.7.1 Performance and Reliability

Bound every query with time. Push repeated aggregations into materialized views. Keep lookback windows honest about lateness. Use `ingestion_time()` deliberately: it measures the pipeline, not the world. Review extent sizes and caching policies monthly against query patterns.

14.7.2 Security and Governance Checklist

- Apply row-level security policies on KQL tables where telemetry carries tenant or personal scoping.
- Retention policies reflect both cost and compliance—telemetry is not free to keep forever.
- Restrict who can alter materialized views; they are production state.
- Classify telemetry tables; device identifiers joined to customers are personal data.
- Record the lateness policy per dashboard as a published contract.
- Audit `arg_max` dedup usage: consumers bypassing the dedup view read dirty data.

14.8 Interview Q&A

1. Q: How does a KQL materialized view maintain incremental state over an append-only stream?

A: It continuously applies its aggregation or projection to newly arrived extents, so consumers read maintained state instead of re-scanning history—the streaming replacement for nightly recompute.

2. Q: What does `arg_max(ingestion_time(), *)` by `EventId` deduplicate?

A: At-least-once redeliveries: all physical copies of one logical event collapse to the latest-arrived row, keyed by the producer-stamped `EventId`.

3. Q: Why do unbounded lookback windows degrade query and materialization performance?

A: They abandon incremental state: every query and every view update re-scans ever-growing history instead of the recent slice, so cost grows with data age rather than with workload.

4. **Q: How do late arrivals affect a time-windowed stream join?**

A: Events arriving after the window closes never match, so joins undercount; the window must reflect the measured lateness distribution, and the policy must state how unmatched events are handled.

5. **Q: Where must the dedup key be generated and enforced for end-to-end exactly-once under replay?**

A: Generated at the producer before the first send, propagated untouched through Eventstream, and enforced at the first queryable boundary—KQL materialized view or Delta MERGE—so every downstream consumer reads clean data.

6. **Q: What does consumer checkpointing protect against?**

A: Restart-induced replay: recording the last processed offset or timestamp lets a resumed consumer continue rather than reprocess, which keeps both cost and duplicates bounded.

14.9 Further Reading

Review the KQL documentation [30], materialized views [32], and the Eventhouse overview [55]. The ingestion path feeding these tables is Chapter 13's Eventstreams [26]; the reaction layer above them is Chapter 15's Data Activator [57].

Key Takeaways

- Eventhouses host KQL databases: append-first, time-native, policy-driven stores for telemetry.
- KQL composes through pipes: `where`, `project`, `summarize`, `bin()`—readable and time-obsessed.
- Materialized views are incremental state; dashboards should read views, not firehoses.
- `arg_max(ingestion_time(), *)` by `EventId` is the dedup contract made executable.
- `make-series` plus the `series_*` family turns event streams into analyzable time series, including anomaly detection.
- Stream joins need time-windowed keys; late arrivals need a published policy, not silent revision.
- Unbounded lookback abandons incremental design—bound every query by time, always.

14.10 Exercises

1. **(Conceptual)** Explain append-first design and why dedup happens at query time rather than ingestion time in a KQL database.
2. **(Conceptual)** Contrast event time, ingestion time, and processing time; give one question each answers correctly.
3. **(Hands-on)** Rewrite the lab's anomaly detection with `series_decompose` and compare baseline/residual behavior to `series_decompose_anomalies`.

4. **(Hands-on)** Build the materialized view version of the dedup pattern and measure the latency difference against query-time dedup at one day of volume.
5. **(Design)** Write the lateness policy for the mobile-fleet dashboard: window sizes, provisional marking, and revision communication.
6. **(Design)** Design the stream-to-dimension “as-of” enrichment when device metadata itself changes over time.
7. **(Interview)** In five sentences, explain how materialized views, `arg_max` dedup, and bounded lookback together deliver exactly-once *effects* over at-least-once delivery.
8. **(Operational)** Write the runbook for “materialized view lagging behind ingestion”—detection, cause analysis, catch-up verification.

14.11 Capstone Project: Windowed Analytics over a Replay-Safe Stream

Capstone Project

Mission. Deliver the analytical layer over Chapter 13’s replay-safe stream: deduplicated base views, materialized per-device aggregates, a rolling-average anomaly detector, a time-windowed alert-correlation join, and a written lateness policy—with evidence that late and duplicated data cannot corrupt the published numbers.

Estimated time. 5–7 hours on top of the Chapter 13 estate.

What you’ll practice.

- Materialized view design for incremental aggregation.
- Anomaly detection with the `series_*` family.
- Time-windowed stream joins and stream-to-dimension enrichment.
- Lateness policy as a published, tested contract.

Scenario

Contoso Devices’ support organization needs a live fleet-health view: per-site rolling temperatures, anomaly flags, and correlation between telemetry spikes and device alerts. The data platform team’s requirement: yesterday’s numbers must be stable by 09:00, with a documented policy covering the fleet’s nine-hour offline backhauls.

Requirements

- Deduped base consumption (view or materialized view) enforcing `arg_max` on `EventId`.
- Materialized view of per-device, per-site five-minute means.
- The rolling-average anomaly query parameterized by threshold.
- The alert-correlation join with a justified time window.
- The lateness policy document: lookback, provisional marking, stabilization time.
- Evidence runs: injected duplicates and a simulated nine-hour backhaul, both absorbed correctly.

Implementation Steps

1. Build the deduped base and the materialized aggregates.

2. Implement and tune the anomaly query against injected spikes.
3. Implement the correlation join; justify the 300-second window or revise it with data.
4. Write the lateness policy; implement provisional marking in the consumption queries.
5. Run both evidence scenarios and capture query outputs.

Deliverables

- All KQL artifacts (views, materialized views, functions) as versioned scripts.
- Anomaly and correlation query results on the evidence data.
- The lateness policy document.
- Evidence outputs proving duplicate and backhaul absorption.

Acceptance Criteria

- ☐ Injected duplicates never appear in any published aggregate.
- ☐ The simulated backhaul revises only provisional windows, never stabilized ones.
- ☐ Every consumption query is time-bounded; none scans full history.
- ☐ The anomaly detector fires on the synthetic spike and stays quiet on noise.
- ☐ The correlation join's window is justified with measured latency data.
- ☐ The lateness policy is written, reviewed, and referenced by the dashboard contract.

Stretch Goals

- Add a forecast (`series_decompose_forecast`) for next-hour site temperature and track its error.
- Implement as-of dimension joins for device firmware version at event time.
- Compare materialized-view cost versus on-demand aggregation at seven days of volume; publish the break-even.
- Feed the anomaly flags to a Reflex item (previews Chapter 15) and measure detection-to-alert latency.

Data Activator (Reflex)

Learning Objectives

- Explain the Data Activator object model: triggers (now “rules”), conditions, and actions.
- Create Reflex items from Power BI visuals and from Eventstream data.
- Configure automated actions: Teams messages, emails, and custom workflows.
- Build a sustained-threshold trigger that alerts only after three consecutive minutes of breach.
- Suppress alert noise with hysteresis, aggregation, and re-alert policy.
- Design a proactive alerting system notifying drivers via custom webhooks when trucks go off route.

Cross-Cloud Equivalence

Event-driven alerting over analytics—conditions evaluated continuously against live data—exists across clouds, though Fabric’s no-code Reflex is unusually approachable.

Capability	Fabric	AWS	GCP	Snowflake
Data-driven alerts	Data Activator (Reflex)	CloudWatch Alarms + SNS	Cloud Monitoring alerts	Snowflake Alerts
Stream-native trigger	Reflex over Eventstream/KQL	Kinesis Analytics + Lambda	Dataflow + Eventarc	Streams + tasks + notifications
BI-visual trigger	Reflex from Power BI visuals	QuickSight threshold alerts	Looker alerts/schedules	Via BI tool
Actions	Teams, email, Power Automate	SNS targets, Lambda	Pub/Sub, Cloud Functions, webhooks	Email, webhooks, stored proc
Custom workflows	Power Automate flows	Step Functions	Workflows	External functions
Evaluation latency	Seconds to minutes	Alarm period granularity	Policy-based	Scheduled evaluation

Databricks alerts evaluate SQL on schedules; MongoDB Atlas triggers fire on change streams. The portable disciplines are this chapter’s heart: *sustained conditions over spikes, actions with owners, and alert noise treated as a reliability bug.*

Key Terms

Reflex item: the Fabric item that watches data and acts on conditions. **Trigger/rule:** the watched data and the identity of what is being monitored. **Condition:** the predicate evaluated as data arrives—thresholds, changes, sustained states. **Action:** what fires when the condition holds—Teams, email, Power Automate workflow. **Hysteresis:**

requiring a condition to hold (or clear) for a sustained period, damping flapping alerts.
Alert noise: the volume of low-value notifications that trains humans to ignore alerts.

15.1 Week 15 Roadmap

Week 15 closes the loop that Chapters 13 and 14 opened: streams are ingested and analyzed; now the platform must *react*. Monday covers the Data Activator object model. Tuesday creates Reflex items from Power BI visuals and Eventstreams. Wednesday covers actions, from Teams messages to custom workflows. Thursday is the hands-on project: an email alert when IoT temperature exceeds a threshold for three consecutive minutes. Friday designs the logistics alerting system with custom app webhooks for off-route trucks.

The week’s engineering theme is restraint. Any system can fire an alert; a well-designed system fires the right alert, once, to the right owner, with context—and stays silent otherwise [57].

Note

Alert latency expectation: Reflex evaluates on the order of seconds to minutes depending on source and condition. It is an operational-reaction layer, not a safety-critical control loop. Do not put it between a furnace and a meltdown; do put it between a KPI and the human who can fix it.

15.2 Monday: Triggers, Conditions, and Actions

Data Activator’s model has three parts, and every sound design names them explicitly [57]:

Definition

A **Reflex rule** binds *data* (a stream column or a Power BI measure), a *condition* (a predicate over that data, possibly sustained or aggregated), and an *action* (a notification or workflow fired when the condition holds).

- **Data and identity:** what is watched (temperature, order count, fraud score) and per-what (per device, per card, per region). The identity granularity defines who can be paged and how many parallel alert streams exist.
- **Conditions:** instant thresholds (temperature > 80), sustained conditions (temperature > 80 for 3 consecutive minutes), and change conditions (value enters/exits a state). Sustained conditions are the primary noise weapon.
- **Actions:** Teams messages to a channel, email to a list, or a Power Automate flow for arbitrary custom workflows—tickets, webhooks, escalations.

Common Pitfall

Single-spike conditions on noisy telemetry produce alert storms, and alert storms produce ignored alerts, and ignored alerts are worse than no alerts. The “3 consecutive minutes” pattern in Thursday’s lab is not a detail; it is the difference between a signal and a siren.

15.3 Tuesday: Reflex from Power BI and Eventstreams

Reflex items originate where the data already lives [57]:

15.3.1 From Power BI Visuals

A visual on a report—a card showing today’s revenue, a gauge showing capacity—can carry a Reflex alert: *set alert* on the visual, define the measure, condition, and cadence, and the rule evaluates as the model refreshes or frames. This path suits KPI-grade reactions: business metrics with business owners.

15.3.2 From Eventstreams and KQL

Over streaming data, Reflex evaluates per event or per aggregation window directly on the stream or the KQL table. This path suits telemetry-grade reactions: device temperatures, error rates, the fraud patterns of the Milestone 4 capstone. The Chapter 14 discipline applies upstream: dedupe before the rule query, because duplicates double-fire alerts.

Tip

Choose the source closest to the truth with the least latency that still reads deduplicated data. For KPIs, that is usually the semantic model; for telemetry, the deduped KQL view—never the raw stream with its replay duplicates.

15.4 Wednesday: Actions and Custom Workflows

Native actions cover the common cases: Teams channel messages, individual Teams messages, and email. The escape hatch is Power Automate: a Reflex action can trigger a flow, and a flow can do essentially anything—create a ticket in an ITSM tool, call a webhook, post to a third-party chat, open an incident.

Action design is contract design. A good action payload includes: what fired (rule name), on what identity (device, region), the observed values, when, the severity, and the runbook link (Chapter 12’s discipline arriving in real time). A payload of “threshold exceeded” is a dead end delivered at speed.

Re-alert policy completes the model: after firing, a rule should cool down—suppress repeats while the condition continues to hold, and optionally fire a distinct “resolved” notification when it clears. Without cooldown, one sustained breach is fifty identical messages.

Common Pitfall

Automating an action that mutates production (restart a service, scale a capacity) from a Reflex rule without a human gate is auto-remediation roulette. Start with notify-only, measure false-positive rate for a month, then graduate specific, reversible actions to automation.

15.5 Thursday: Hands-on Project

The Week 15 lab sets up a Data Activator trigger on the streaming IoT data from Weeks 13–14: an email fires when a device’s temperature exceeds a threshold for three consecutive minutes.

15.5.1 Goal and Architecture

Reuse `es_iot_telemetry` and `kqldb_iot`. Create a Reflex item `reflex_iot_temp` over the deduplicated KQL view (or the eventstream with a dedup-aware aggregation), with per-device identity, a sustained condition, and an email action.

15.5.2 Step-by-Step Actions

1. Create the Reflex item from the eventstream or the KQL table; set the identity to `DeviceId`.
2. Define the condition: `Temperature > 26` sustained for 3 consecutive minutes (per device).
3. Configure the email action with a complete payload: device, observed values, window, runbook link.
4. Configure cooldown: suppress repeats for 15 minutes while the condition holds; add a resolved notification.
5. Inject a synthetic hot streak from the producer (Chapter 14’s validation method) for two minutes—no alert should fire.
6. Extend the streak past three minutes—the alert fires exactly once.
7. Let the streak end—the resolved notification fires.

15.5.3 Validation Checklist

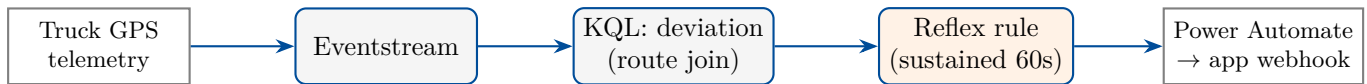
- The rule evaluates per device, not on the fleet aggregate.
- A two-minute breach fires nothing; a sustained breach fires once.
- The payload contains identity, values, window, severity, and runbook link.
- Cooldown suppresses repeats during a continuing breach.
- The resolved notification fires on recovery.
- The rule reads deduplicated data—a replay drill does not double-fire.

15.6 Friday: Use Case and Architecture

A logistics company runs 800 delivery trucks with GPS telemetry flowing into Fabric through the Chapter 13 pipeline. Requirement: when a truck deviates from its planned route, notify the driver instantly via the company’s custom mobile app—which exposes a webhook—and log the event for the operations team.

15.6.1 The Design

Route deviation is computed in KQL: truck position versus planned route corridor (a stream-to-dimension join against the route table, per Chapter 14). A Reflex rule watches the deduplicated deviation signal per `TruckId`, with a sustained condition—off-corridor for 60 seconds—to absorb GPS jitter and tunnel dropouts. The action is a Power Automate flow calling the driver-app webhook with a signed payload, plus a Teams message to the regional operations channel. Cooldown prevents nagging; the resolved notification tells the driver they are back on route; every firing writes an audit row to a KQL table for the weekly operations review.



Per-truck identity, sustained condition, signed webhook payload, audit row per firing.

Figure 15.1: Logistics alerting: telemetry to driver notification in seconds.

Common Pitfall

GPS telemetry lies constantly in small ways—jitter, tunnels, urban canyons. A threshold on raw position error alerts on every overpass. Sustained conditions plus a corridor width tuned to measured GPS accuracy are what separate “driver notified when truly lost” from “driver learned to ignore the app.”

15.7 Operational Guidance for Week 15

An alerting estate is a product with users. Measure it: firings per week, false-positive rate, time-to-acknowledge, and the count of alerts nobody acted on. The last metric is the one that predicts disaster.

15.7.1 Performance and Reliability

Keep rule evaluation on deduplicated, materialized data. Bound evaluation windows. Monitor Reflex item health like any production item. Test actions end-to-end monthly—an alert path that silently broke last release is discovered at the worst moment.

15.7.2 Security and Governance Checklist

- Webhook payloads signed or token-authenticated; secrets vaulted, never in flow definitions.
- Alert destinations (channels, lists) reviewed for information sensitivity—telemetry can leak operational detail.
- Every rule has a named owner; ownerless rules are deleted, not orphaned.
- Automated mutating actions require documented approval and a rollback path.
- Audit rows for every firing kept per the retention policy.
- Rules read deduplicated data; a replay must never re-page a human.

15.8 Interview Q&A

1. Q: Which three parts define a Data Activator trigger?

A: The data being watched (with its identity granularity), the condition evaluated over it (threshold, sustained, or change), and the action fired when the condition holds.

2. Q: Why require “3 consecutive minutes” instead of a single spike?

A: Sustained conditions filter the transient noise that dominates real telemetry; they convert a flapping siren into a trustworthy signal, which is the only kind humans keep answering.

3. **Q: Which actions can Reflex fire natively?**

A: Teams messages and email, plus Power Automate flows as the escape hatch to arbitrary workflows—tickets, webhooks, custom apps.

4. **Q: How do you create an alert from a Power BI visual?**

A: Set alert on the visual, binding the rule to the visual’s measure; the rule evaluates as the semantic model refreshes or frames, which suits KPI-grade business reactions.

5. **Q: What alert latency should you expect from a Reflex item?**

A: Seconds to minutes depending on source and condition cadence—operational reaction time, not hard-real-time control.

6. **Q: Why must Reflex rules read deduplicated data?**

A: Because at-least-once delivery and replay produce duplicate events upstream, and a rule over raw duplicates double-fires—paging humans twice for one event.

15.9 Further Reading

Review the Data Activator documentation [57]. The upstream disciplines are Chapter 13’s eventstream semantics [26] and Chapter 14’s KQL dedup and windowing [30, 32]. Alert-as-work-order design inherits Chapter 12’s runbook discipline [51].

Key Takeaways

- Data Activator binds data, condition, and action; every sound rule names all three plus an owner.
- Sustained conditions are the primary defense against alert storms.
- Reflex evaluates in seconds to minutes: operational reaction, not control loops.
- Actions carry context—identity, values, severity, runbook—or they are noise delivered at speed.
- Cooldown and resolved-notifications complete the alert lifecycle.
- Rules read deduplicated data; replay must never re-page a human.
- Alert estates are products: measure firings, false positives, and ignored alerts, and delete what nobody acts on.

15.10 Exercises

1. **(Conceptual)** Define hysteresis and explain why alerting systems need it, using the truck-corridor example.
2. **(Conceptual)** When is a Power-BI-visual rule preferable to a stream-native rule, and vice versa?
3. **(Hands-on)** Add a second severity tier to the lab rule (warning at 25, critical at 26) with different actions and cooldowns.
4. **(Hands-on)** Build the Power Automate flow posting a signed payload to a test webhook; verify signature validation.
5. **(Design)** Design the escalation ladder for the logistics system: driver, regional ops, national ops—with timing and conditions.

6. **(Design)** Specify the alert-metrics dashboard: which four metrics, over what window, reviewed how often?
7. **(Interview)** In five sentences, explain why ignored alerts are worse than no alerts, and how sustained conditions plus cooldown prevent the slide.
8. **(Operational)** Write the monthly alert-path test procedure for the lab's rule chain.

15.11 Capstone Project: Fleet Alerting with Noise Discipline

Capstone Project

Mission. Build the two-tier fleet alerting system: per-device Reflex rules with sustained conditions and cooldowns, a warning/critical severity split, a signed webhook action through Power Automate, a resolved-notification lifecycle, an audit table of every firing, and a one-month noise report proving the estate is worth answering.

Estimated time. 4–6 hours on top of the Weeks 13–14 estate.

What you'll practice.

- Sustained-threshold rule design with per-identity granularity.
- Severity tiers with differentiated actions.
- Signed custom-webhook delivery via Power Automate.
- Cooldown, resolution, and audit as first-class lifecycle features.
- Measuring alert quality instead of assuming it.

Scenario

Contoso Devices' support team needs fleet temperature alerting they can trust. The previous system (a cron job emailing on every spike) trained everyone to ignore it within a month. The replacement must fire rarely, fire correctly, carry context, and prove its own quality with data.

Requirements

- Warning and critical rules per device with sustained conditions and distinct actions.
- Cooldown and resolved notifications configured and demonstrated.
- The Power Automate webhook action with a validated signature.
- Audit table capturing every firing: rule, device, values, timestamps, resolution.
- Evidence: injected noise produces zero alerts; injected sustained breach produces exactly one per tier.
- The noise report template: firings, false positives, acknowledgments, ignored alerts.

Implementation Steps

1. Build the warning and critical rules over the deduplicated view.
2. Configure actions, cooldowns, and resolution notifications.
3. Build and secure the Power Automate webhook flow.
4. Implement the audit table write.
5. Run the noise and breach evidence scenarios; capture the audit rows.
6. Draft the noise report from the audit table.

Deliverables

- Rule configurations exported or documented with screenshots.
- The secured flow definition and a captured signed payload.
- Audit table contents across the evidence scenarios.
- The noise report template with one populated week (real or simulated).

Acceptance Criteria

- ☐ Injected sub-threshold noise fires nothing.
- ☐ A sustained breach fires exactly once per tier, with full context payloads.
- ☐ Cooldown suppresses repeats; resolution notifies once.
- ☐ Every firing and resolution appears in the audit table.
- ☐ The webhook rejects unsigned payloads in testing.
- ☐ The noise report computes firings, false-positive rate, and ignored-alert count from audit data.

Stretch Goals

- Add fleet-level aggregate rules (site temperature, not device) with separate owners.
- Add a weekly auto-generated noise report delivered to the rules' owners.
- Implement adaptive thresholds from Chapter 14's anomaly detection instead of static values.
- Add a suppression window for known maintenance periods, with audit visibility.

Real-Time Dashboards

Learning Objectives

- Distinguish KQL querysets from Real-Time Dashboards and choose correctly per audience.
- Build auto-refreshing dashboards directly on KQL databases.
- Configure Power BI automatic page refresh and weigh DirectQuery versus KQL-backed freshness.
- Implement reverse ETL: upsert curated Gold data back to Dataverse on stable business keys.
- Reason about sync frequency—near-real-time micro-batches versus nightly syncs—and its CU and API-throttling cost.
- Design the sub-second BI access layer for a manufacturing floor monitoring machine health.

Cross-Cloud Equivalence

Live operational dashboards and the reverse flow of curated data back into operational systems are cross-cloud patterns with different brand names per vendor.

Capability	Fabric	AWS	GCP	Snowflake
Live dashboards	Real-Time Dashboards (KQL)	QuickSight / Grafana on Timestream	Looker / Grafana on BigQuery streaming	Snowsight / connected BI
Auto-refresh BI	Power BI automatic page refresh	QuickSight refresh	Looker persistent derived tables	BI tool refresh
Reverse ETL target	Dataverse / CRM via pipelines	AppFlow to SaaS	Workflows / AppSheet writes	Snowflake reverse-ETL partners
Upsert semantics	Alternate-key upsert to Dataverse	API-level upsert per target	API-level per target	MERGE / external writes
Sync cost driver	CUs + target API throttling	Lambda/AppFlow invocations	Function invocations + API quotas	Warehouse credits + API quotas

Databricks serves live BI from SQL warehouses; MongoDB has Charts and Atlas-native dashboards. The transferable insight of the week: *dashboards close the loop from data to humans; reverse ETL closes it from data back to systems*—and both loops are billed.

Key Terms

KQL queryset: an interactive, shareable collection of KQL queries for exploration.
Real-Time Dashboard: a Fabric item of auto-refreshing tiles, each bound to a KQL query.
Automatic page refresh: Power BI's scheduled re-query of live-connected vi-

suals. **Reverse ETL**: pushing curated analytical data back into operational systems (CRM, Dataverse). **Identity resolution**: matching analytical records to operational records on stable business keys. **Micro-batch sync**: small, frequent reverse-ETL runs trading freshness for throttling and cost pressure.

16.1 Week 16 Roadmap

Week 16 completes Part IV. Monday contrasts KQL querysets with Real-Time Dashboards. Tuesday builds dashboards natively on KQL. Wednesday covers Power BI automatic page refresh and reverse ETL to Dataverse—identity resolution and sync-frequency economics. Thursday is the hands-on project: an auto-refreshing Real-Time Dashboard plus a reverse-ETL pipeline upserting a Gold customer-segment table into Dataverse. Friday designs the manufacturing-floor access layer with sub-second machine-health dashboards. The chapter closes with the Milestone 4 capstone: real-time fraud detection.

This week is where Part IV pays off visibly: the streams, queries, and rules of Weeks 13–15 become the live surfaces humans actually watch [15]—and, through reverse ETL, the analytical work flows back into the operational systems where action happens.

Note

Two audiences, two tools: querysets are for engineers exploring; Real-Time Dashboards are for operators watching. Building a dashboard for exploration, or handing an operator a queryset, are symmetric category errors.

16.2 Monday: KQL Querysets versus Real-Time Dashboards

A **queryset** is a working document: saved, shareable KQL queries with results, meant for iterative exploration and handoff between engineers. A **Real-Time Dashboard** is a production surface: tiles bound to queries, parameterized, auto-refreshing, permissioned, and meant to be glanced at for months without edits [15, 30].

Aspect	KQL queryset	Real-Time Dashboard
Primary user	Engineer, analyst exploring	Operator, executive watching
Content	Queries and result grids	Visual tiles over queries
Refresh	On demand	Automatic, per tile/page
Governance	Shared as code-like artifacts	Shared as monitored products
Failure mode	Wrong analysis, caught by review	Wrong wall monitor, caught by nobody

Table 16.1: Querysets explore; dashboards operate.

The engineering consequence: dashboards deserve production rigor—versioned source queries (exported from the queryset that bred them), bounded time windows, owners, and the lateness policy of Chapter 14 attached as a visible annotation.

16.3 Tuesday: Building Dashboards on KQL

Tiles bind KQL queries to visuals: time charts, bar charts, scorecards, tables. The practices that separate operable dashboards from demo screens [15]:

- **Every query is time-bounded.** A tile scanning all history on every refresh is a capacity leak with a UI.
- **Parameters drive filtering.** A `_device` or `_site` parameter lets one dashboard serve many scopes; hardcoded filters fork dashboards per team.
- **Read materialized views.** Tiles over Chapter 14's maintained state refresh in milliseconds; tiles over raw aggregation recompute the world every thirty seconds.
- **Refresh cadence matches decision cadence.** A floor-safety tile refreshes in seconds; a daily-throughput tile refreshing every ten seconds burns capacity to tell nobody anything new.

Tip

Set tile refresh to the slowest cadence the decision tolerates, not the fastest the platform allows. Freshness beyond the decision cadence is pure CU cost.

16.4 Wednesday: Automatic Page Refresh and Reverse ETL

16.4.1 Power BI Automatic Page Refresh

For Power BI reports over live connections (DirectQuery or KQL), automatic page refresh re-queries visuals on a schedule. The cost model is direct: every refresh is real query load on the capacity—a report open on forty wall monitors at ten-second refresh is four queries per second, forever. Set page refresh deliberately, per page, and prefer the KQL-native dashboard for second-level needs.

16.4.2 Reverse ETL

Analytics earns its keep when it changes operations. Reverse ETL pushes curated Gold data—customer segments, churn scores, recommended actions—back into the systems where people work: Dataverse, the CRM, the support console [42].

Definition

Reverse ETL is the pipeline direction that writes curated analytical outputs from Gold tables back into operational systems, keyed by stable business identities, at a frequency chosen as an explicit cost–freshness trade-off.

Two disciplines define success. **Identity resolution:** match on stable business keys—Dataverse record IDs, CRM-native keys—never on names or emails, which change and collide. Upsert by alternate key, so reruns update rather than duplicate. **Sync-frequency economics:** near-real-time micro-batches (every 15 minutes) maximize freshness and multiply CU and target-API-throttling cost; nightly full syncs minimize both and serve many use cases perfectly well. Choose per entity, in writing, with the business owner.

Common Pitfall

Reverse ETL without idempotent upsert keys slowly forks reality: the CRM accumulates duplicate contacts, sales works phantom records, and the lakehouse's pristine Gold table gets blamed. Upsert on the stable key, reconcile counts both directions weekly, and alert on drift.

16.5 Thursday: Hands-on Project

The Week 16 lab has two halves: a Real-Time Dashboard natively in Fabric with auto-refreshing charts over the IoT KQL data, and a reverse-ETL pipeline upserting a Gold customer-segment table into Dataverse on a stable business key.

16.5.1 Goal and Architecture

Part one: dashboard `rtd_fleet` over `kqldb_iot` with tiles for active device count, current temperature trend per device, and anomaly count. Part two: pipeline `PL_reverse_etl_segments` upserting `gold.customer_segment` to a Dataverse table keyed on `customer_id` as an alternate key. (A trial Dataverse environment or instructor mock suffices.)

16.5.2 Step-by-Step Actions

1. Create the dashboard; add the three tiles, each from a bounded query over the deduplicated view.
2. Set per-tile refresh cadences (10s for active devices, 60s for trends) and justify each in the tile description.
3. Add a `_device` parameter and prove one dashboard serves all devices.
4. Build the reverse-ETL pipeline: read `gold.customer_segment`, upsert to Dataverse by the `customer_id` alternate key.
5. Run twice; verify the second run updates in place—record counts match, no duplicates.
6. Reconcile: Gold row count equals Dataverse segment count; log both to the ledger.
7. Estimate the CU and API-call cost of 15-minute versus nightly sync; write the recommendation.

```

1 // Tile 1: active devices (seen in the last 2 minutes)
2 IoTTelemetry
3 | where ingestion_time() > ago(2m)
4 | summarize arg_max(ingestion_time(), *) by EventId
5 | summarize ActiveDevices = dcount(DeviceId);
6
7 // Tile 2: temperature trend per device (parameterized)
8 IoTTelemetry
9 | where Timestamp > ago(30m)
10 | where DeviceId == _device or _device == "all"
11 | summarize AvgTemp = avg(Temperature) by DeviceId, bin(Timestamp, 1m)
12 | order by bin_Timestamp asc;
```

Listing 16.1: Dashboard tile queries: bounded, deduplicated, parameterized.

16.5.3 Validation Checklist

- Every tile query is time-bounded and reads deduplicated or materialized data.
- Refresh cadences are documented with a decision-cadence justification.
- The dashboard parameter works; no per-device dashboard forks exist.
- Reverse ETL upserts: a second run changes no record count, only updated fields.
- Reconciliation counts match and are ledgered.
- The sync-frequency recommendation cites CU and throttling arithmetic.

16.6 Friday: Use Case and Architecture

A manufacturing plant’s operators monitor machine health—vibration and heat—on floor-mounted displays. Requirements: sub-second to few-second latency, per-line views, and graceful behavior when a sensor gateway drops.

16.6.1 The Design

Machine telemetry lands through eventstreams into the Eventhouse; per-line materialized views maintain rolling vibration and temperature state; Real-Time Dashboards render per-line tiles (parameterized, so one dashboard serves all lines) at a five-second refresh—the decision cadence of a floor operator watching for a hot spindle. Reflex rules (Chapter 15) handle reaction; the dashboards handle awareness. Sensor-gateway drops surface as a “silence” tile: machines whose last heartbeat exceeds a threshold, because on a plant floor *no data is itself data*.

Common Pitfall

A wall dashboard that silently freezes looks exactly like a healthy plant. Every real-time surface needs a freshness indicator—last-event age, tile heartbeat—so operators can distinguish “all quiet” from “all broken.” This is the dashboard form of the lateness policy.

Requirement	Design choice	Chapter discipline
Sub-second/few-second reads	Materialized views, not raw scans	Ch. 14 incremental state
Per-line views	Dashboard parameters	Ch. 16 parameterization
Sensor-drop detection	Silence/heartbeat tile	Ch. 13 ingestion monitoring
Reaction on breach	Reflex rules, sustained	Ch. 15 noise discipline
Cost control	Refresh = decision cadence	Capacity stewardship

Table 16.2: Manufacturing-floor dashboard design, mapped to Part IV disciplines.

16.7 Operational Guidance for Week 16

Dashboards and reverse ETL are the platform’s most visible artifacts—and therefore where operational credibility is won or lost in front of the business.

16.7.1 Performance and Reliability

Audit tile queries monthly for unbounded scans. Track dashboard refresh cost in the Capacity Metrics app [34]. Reconcile reverse-ETL counts both directions on a schedule. Treat freshness indicators as required tile furniture.

16.7.2 Security and Governance Checklist

- Dashboard permissions match the operational audience; telemetry detail can be sensitive.
- Reverse ETL writes to operational systems are least-privilege and fully ledgered.
- Dataverse upsert keys are documented stable business keys, never display fields.
- Sync-frequency decisions are recorded with business-owner sign-off.
- Personal data flowing into a CRM inherits its sensitivity classification (Chapter 22).
- Every dashboard has a named owner and a freshness indicator.

16.8 Interview Q&A

1. **Q: When would you choose a Real-Time Dashboard over a KQL queryset?**
 A: When the audience watches rather than explores—operators and executives need parameterized, auto-refreshing, governed tiles; querysets remain the engineers’ working documents that breed the tiles’ queries.
2. **Q: What does automatic page refresh cost in DirectQuery mode?**
 A: Every refresh is real query load against the capacity, multiplied by every open report instance—forty wall monitors at ten seconds is four queries per second forever, which is why cadence follows decision cadence.
3. **Q: What is reverse ETL, and why push Gold data back to a CRM?**
 A: Reverse ETL writes curated analytical outputs—segments, scores—into operational systems where people act. Analytics that never reaches the system of action is a report nobody operationalizes.
4. **Q: Which identity-resolution key would you sync Dataverse records on?**
 A: A stable business key—the Dataverse record ID or a CRM-native identifier as an alternate key—never a mutable, collidable display field like name or email.
5. **Q: Trade-off between 15-minute and nightly reverse-ETL syncs?**
 A: Fifteen-minute micro-batches buy freshness at multiplied CU consumption and target-API throttling pressure; nightly syncs are cheap and simple. The choice is per entity, in writing, with the business owner.
6. **Q: Why does every real-time dashboard need a freshness indicator?**
 A: A frozen dashboard is indistinguishable from a healthy system; last-event age separates “all quiet” from “all broken” for the humans trusting the wall.

16.9 Further Reading

Review the Real-Time Dashboards documentation [15], the KQL reference behind every tile [30], and the pipeline mechanics that implement reverse ETL [42]. Cost governance relies on the Capacity Metrics app [34]; upstream stream correctness on Chapters 13–14 [26, 55].

Key Takeaways

- Querysets explore; dashboards operate—different audiences, different rigor.
- Every tile query is time-bounded, deduplicated or materialized, and refreshed at decision cadence.
- Automatic page refresh multiplies query load by open viewers; wall monitors are a capacity design input.
- Reverse ETL closes the analytics loop into operational systems, on stable keys, with ledgered reconciliation.
- Sync frequency is an explicit per-entity business decision with CU and throttling arithmetic attached.
- Freshness indicators are required: no-data is itself data, and frozen dashboards lie silently.
- Part IV’s disciplines—dedup, windows, sustained conditions, cadence—compose into surfaces the business can trust.

16.10 Exercises

1. **(Conceptual)** Explain why decision cadence, not platform capability, sets refresh cadence.
2. **(Conceptual)** Why is a mutable field like email unacceptable as a reverse-ETL upsert key, even with deduplication?
3. **(Hands-on)** Add the heartbeat tile (machines silent beyond threshold) to the lab dashboard.
4. **(Hands-on)** Extend reverse ETL with a field-level change log: which Dataverse records changed, which field, old versus new.
5. **(Design)** Cost the wall-monitor estate: twenty displays, three refresh cadence options, one capacity. Show the arithmetic and recommend.
6. **(Design)** Design the reverse-ETL failure mode for a Dataverse API-throttling event: queue, backoff, and reconciliation behavior.
7. **(Interview)** In five sentences, explain how Part IV’s four chapters compose into one trustworthy real-time platform.
8. **(Operational)** Write the runbook for “dashboard shows stale data”—freshness check, view lag check, ingestion lag check, escalation.

16.11 Milestone 4 Capstone: Real-Time Fraud Detection

Capstone Project

Mission. Close Part IV with the canonical real-time system: a Python producer simulating credit-card swipes into a Fabric Eventstream, routing to a KQL database, an impossible-travel fraud query (same card, two cities, ten minutes), a Data Activator rule alerting the security team on Teams, and a Real-Time Dashboard—all dedup-safe, replay-proven, and cost-estimated.

Estimated time. 10–12 hours across the milestone’s components.

What you’ll practice.

- Producer discipline with unique transaction IDs and deliberate fraud scenarios.
- Fan-out routing and dedup at the first queryable boundary.
- Stateful pattern detection in KQL ([prev](#) over serialized partitions).
- Sustained-condition Reflex rules that do not alert-storm.
- The replay drill and the six rubric artifacts, including the capacity estimate and PII security review.

Scenario

Contoso Bank’s card division needs sub-minute fraud reaction. Swipes stream from the authorization network; the platform must detect impossible travel—a card used in two cities within ten minutes—and page the security team, while a live dashboard shows the fraud queue. Duplicates from redelivery must never double-fire an alert.

Requirements

- Producer `card_simulator.py` (faker-based) with unique `txn_id` per swipe and a configurable impossible-travel injection mode.

- Eventstream routing to the KQL database; dedup view on `txn_id`.
- The fraud query using `order by/serialize/prev` over the deduped stream.
- Reflex rule with sustained/aggregated condition and cooldown, firing Teams to the security channel.
- Real-Time Dashboard: fraud queue, swipe rate, freshness indicator.
- The replay drill: simulated downstream outage, recovery, zero duplicate alerts.
- Cost estimate (Eventhouse versus capacity CU burn) and security review (producer credentials in Key Vault, least-privilege KQL access, PII noted).

Reference Architecture

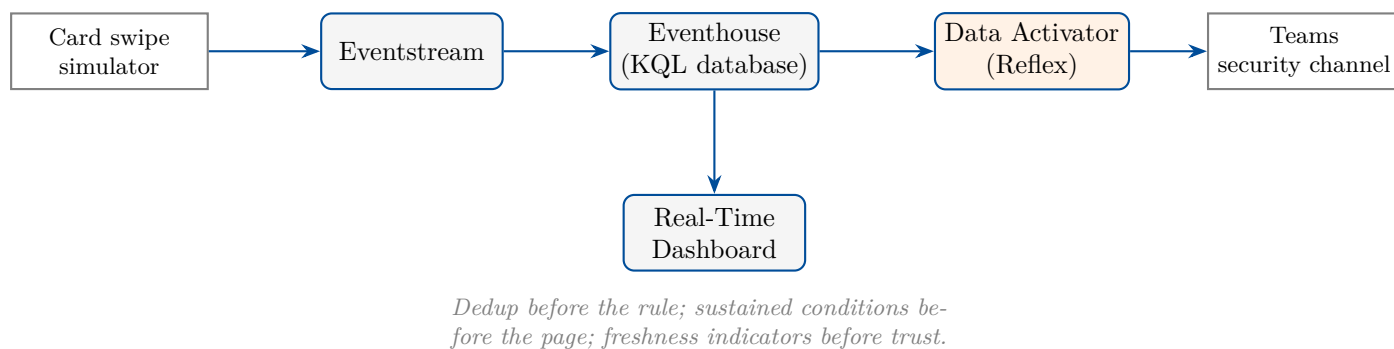


Figure 16.1: Milestone 4 reference architecture: real-time fraud detection.

Implementation Steps

1. Build the simulator with unique IDs and the fraud-injection mode; vault the connection string.
2. Wire the eventstream to KQL; create the deduped view on `txn_id`.
3. Implement and test the impossible-travel query against injected cases.
4. Build the Reflex rule (sustained condition, cooldown, full context payload).
5. Build the dashboard with the freshness indicator.
6. Run the replay drill; capture zero-duplicate-alert evidence.
7. Produce the cost estimate and PII security review; finalize the README.

```

1 // Impossible travel: same card, different cities, <= 10 min apart
2 Txn
3 | summarize arg_max(ingestion_time(), *) by txn_id -- dedup first
4 | order by card_id asc, ts asc
5 | serialize
6 | extend p_card = prev(card_id), p_city = prev(city), p_ts = prev(ts)
7 | where card_id == p_card and city != p_city and ts - p_ts <= 10m
8 | project card_id, ts, city, p_city
  
```

Listing 16.2: Milestone 4: impossible travel over the deduplicated stream.

Deliverables

- Simulator source, item exports, and KQL artifacts in the repository.
- Fraud-detection evidence: injected cases detected, benign travel ignored.
- Teams alert capture from a simulated fraud event, plus replay-drill evidence.
- Dashboard screenshot with the freshness indicator visible.
- Cost estimate with two documented optimization decisions.
- Security review: vaulted producer credentials, least-privilege KQL access, PII classification.

Acceptance Criteria

- ☐ The end-to-end stream runs from a clean environment: producer → Eventstream → KQL → alert.
- ☐ Every swipe carries a unique `txn_id`; the dedup view collapses injected duplicates.
- ☐ Injected impossible-travel cases alert; same-city rapid swipes do not.
- ☐ The replay drill produces zero duplicate alerts and zero lost transactions.
- ☐ The dashboard's freshness indicator demonstrably distinguishes quiet from broken.
- ☐ The cost estimate cites measured CU consumption; the security review names the PII handling.
- ☐ No connection strings or secrets appear in source or evidence.

Stretch Goals

- Add a second rule: velocity (more than five swipes per card per minute) with its own severity tier.
- Add distance-aware impossible travel using real city coordinates and speed thresholds.
- Feed fraud outcomes back to a case-management table via reverse ETL.
- Load-test the pipeline at 10× volume and record where lag first appears.

Part V

Data Science and Machine Learning

Data Science Foundations in Fabric

Learning Objectives

- Navigate the Fabric Data Science experience: notebooks, Data Wrangler, and the lakehouse connection.
- Clean data visually with Data Wrangler and export the generated PySpark code.
- Perform exploratory data analysis on samples before committing to full Spark passes.
- Engineer features that are safe from target and temporal leakage.
- Keep feature sets governed in the lakehouse instead of scattered CSV exports.
- Architect a Spark-native daily inventory forecasting system for 1,000 retail stores.

Cross-Cloud Equivalence

Notebook-centered data science over lake storage is the dominant pattern across platforms; the differences are in how tightly tracking, registry, and storage are integrated.

Capability	Fabric	AWS	GCP	Snowflake
Notebooks	Fabric notebooks (Spark)	SageMaker Studio / EMR notebooks	Vertex AI Workbench / Colab	Snowflake Notebooks
Visual prep	Data Wrangler	Data Wrangler (SageMaker)	Vertex / Colab tooling	Snowsight / worksheets
Compute for EDA	Starter pools + custom pools	SageMaker instances / EMR	Vertex runtimes / Dataproc	Snowpark warehouses
Feature storage	Lakehouse Delta tables	Feature Store (SageMaker)	Vertex AI Feature Store	Feature Store / tables
Experiment tracking	Built-in MLflow	SageMaker Experiments / MLflow	Vertex AI Experiments	MLflow / Snowpark ML
Sample-first EDA	Pandas on Spark samples	SageMaker Processing	BigQuery sampling	SAMPLE clauses

Databricks is the closest relative—Fabric’s data-science experience deliberately mirrors its notebook-first, MLflow-tracked shape. MongoDB feeds these platforms through connectors rather than hosting ML itself. The portable discipline: *explore on samples, engineer features without leakage, and store features as governed tables.*

Key Terms

Data Wrangler: the visual, code-generating data-cleaning tool embedded in Fabric notebooks. **EDA:** exploratory data analysis—profiling distributions, nulls, and relationships before modeling. **Feature:** a model input column derived from raw data. **Data leakage:** information in training features that would not exist at prediction time. **Fea-**

ture set: a governed, versioned Delta table of model-ready features. **Spark-native:** computed where the data lives, in distributed Spark, rather than exported to single-node tools.

17.1 Week 17 Roadmap

Week 17 opens Part V, the data-science and AI arc—written, like the rest of this book, for the data engineer who must make ML work *production-shaped*. Monday covers the Data Science experience: notebooks and Data Wrangler. Tuesday builds EDA discipline with Pandas and PySpark, samples first. Wednesday is the conceptual core: feature engineering and the leakage failure modes that silently invalidate models. Thursday is the hands-on project: visual cleaning in Data Wrangler with generated PySpark code. Friday architects Spark-native daily inventory forecasting for 1,000 stores.

Data engineers do not usually train the models, but they own everything the models stand on: clean inputs, honest features, reproducible pipelines. A model trained on leaked features is a production incident scheduled for later [20].

Note

The week’s recurring trap: a workflow that works beautifully in a notebook on a sample, then fails silently at full scale—either computationally (a Pandas pattern that does not distribute) or statistically (a feature computed with tomorrow’s information). Both are engineering failures, and both are this week’s business.

17.2 Monday: The Data Science Experience

Fabric’s Data Science experience centers on the notebook you already know from Week 3, extended with ML-native items: experiments, models, and the visual Data Wrangler tool [20]. The lakehouse attachment is the constant: notebooks read and write Delta tables in OneLake, so data science inherits the governance of Part I rather than inventing a parallel storage world.

17.2.1 Data Wrangler

Data Wrangler is visual data cleaning inside the notebook: point at a DataFrame, apply operations—fill nulls, drop columns, encode categories, filter outliers—through a UI, and watch the corresponding PySpark code generate live. The generated code is the deliverable: it is reviewable, diffable, and reusable, which a point-and-click session alone is not.

Definition

Data Wrangler is Fabric’s visual data-preparation surface that converts interactive cleaning steps into executable PySpark or Pandas code, merging the speed of visual exploration with the reproducibility of code.

Common Pitfall

Data Wrangler’s output is only as reproducible as you make it. Export the generated code into a parameterized notebook cell, not into a one-off cell you ran by hand once. The visual session is exploration; the exported code is engineering.

17.3 Tuesday: EDA with Pandas and PySpark

EDA answers three questions before any modeling: what is in this data (distributions, types, ranges), what is wrong with it (nulls, duplicates, impossible values), and what relates to what (correlations, group structures) [20].

The scale discipline: **explore on samples, verify at scale**. Pull a 1–5% sample (or a time-bounded slice) into Pandas for fast iteration—histograms, scatter matrices, value counts. Then confirm the sample’s claims with targeted full-set PySpark aggregations: global null rates, cardinalities, min/max bounds. The sample is for thinking speed; the full pass is for truth.

EDA task	Tool	Scale note
Distributions, plots, quick pivots	Pandas on a sample	Fast iteration; mind sampling bias
Null/duplicate/validity rates	PySpark full pass	Cheap aggregations, full truth
Cardinality and joins feasibility	PySpark full pass	<code>approx_count_distinct</code> for huge sets
Correlations on big data	Sample first, verify on full	Correlation on a biased sample lies

Table 17.1: Sample-first EDA: which tool answers which question.

Tip

Sample *stratified*, not just randomly, when the data has structure: a random 1% of click-stream is mostly your noisiest hour. Sample per day or per segment so the exploration matches the population.

17.4 Wednesday: Feature Engineering and Leakage

Feature engineering transforms raw columns into model inputs: ratios, aggregates over windows, encodings, flags. In Fabric, features are computed in Spark and stored as Delta tables in the lakehouse—governed, versioned, and shared—rather than as CSV exports on a laptop [20, 27].

17.4.1 The Leakage Taxonomy

Leakage is information in training that will not exist at prediction time. It produces models that ace evaluation and fail in production. The recurring species:

- **Target leakage:** a feature derived from the outcome—a “churn_reason” column available only after churn.
- **Temporal leakage:** aggregates computed over windows that include the future—a customer’s “total lifetime spend” used to predict their second purchase.
- **Preprocessing leakage:** scalers or encodings fit on the full dataset before the train/test split, so test statistics bleed into training.
- **Group leakage:** the same entity split across train and test—one customer’s sessions in both sets.

Definition

Data leakage is the presence of information in model inputs that would be unavailable—or computed differently—at the time the model must actually make its prediction.

The engineering defenses: fixed training windows (features computed as-of a declared cutoff date, never “as of now”); the same feature-computation code path for training and serving (Chapter 20’s skew battle); group-aware splits; and a feature table with an `as_of_date` column that makes temporality explicit.

Common Pitfall

“As-of-now” features are the quiet killer. A feature pipeline that computes aggregates over all history up to run time trains on the future for every historical row. Every feature needs an explicit as-of timestamp, and the training set must be rebuildable for any historical date.

17.5 Thursday: Hands-on Project

The Week 17 lab uses Data Wrangler in a notebook to visually clean a dataset—missing values, type fixes, outlier handling—and generates the corresponding PySpark code automatically, then hardens that code into a reusable, parameterized cell.

17.5.1 Goal and Architecture

Work in `fabric-de-week17` with lakehouse `lh_retail`. Use a Silver retail table from Part I or a provided messy extract (`silver_store_sales_raw`) containing nulls, string-typed numbers, and outliers. The output is a cleaned Delta table `features_store_sales` plus the exported, parameterized cleaning code.

17.5.2 Step-by-Step Actions

1. Load the raw table into a notebook DataFrame and open Data Wrangler.
2. Clean visually: fill missing `units` with 0, cast `revenue` from string to double, drop rows with negative revenue, cap outliers at the 99th percentile.
3. Export the generated PySpark code into a notebook cell.
4. Parameterize it: input table, output table, and `as_of_date` come from the parameter cell.
5. Run the parameterized cell end-to-end and validate the output schema and counts.
6. Compute one windowed feature—`revenue_7d_avg` per store—with the window ending at `as_of_date`, never after.
7. Write `features_store_sales` as a Delta table with the `as_of_date` column.

```

1 from pyspark.sql import functions as F, Window
2
3 as_of = dbutils.widgets.get("as_of_date") if "dbutils" in dir() else "
      2026-07-31"
4
5 df = spark.read.table("features_store_sales").where(F.col("sale_date") <= as_of
6           )
7 w = (Window.partitionBy("store_id")
8       .orderBy("sale_date")
9       .rowsBetween(-6, 0)) # trailing 7 days, ending at the row's date
10

```

```

11 features = (df.withColumn("revenue_7d_avg", F.avg("revenue").over(w))
12               .withColumn("as_of_date", F.lit(as_of)))
13
14 (features.write.format("delta").mode("overwrite")
15   .option("replaceWhere", f"as_of_date = '{as_of}'))
16   .saveAsTable("gold.features_store_daily"))

```

Listing 17.1: Leakage-safe windowed feature: the window ends at the as-of date.

17.5.3 Validation Checklist

- The exported Data Wrangler code runs unmodified and reproduces the visual result.
- The parameterized version rebuilds the feature table for any historical `as_of_date`.
- No feature uses information dated after its row's `as_of_date`.
- Null, negative, and outlier handling are visible in the exported code, not just the UI.
- The feature table is a governed Delta table, not a CSV.
- A teammate can regenerate yesterday's features exactly.

17.6 Friday: Use Case and Architecture

A retail group must forecast daily inventory needs for 1,000 stores. Constraint: purely Spark-based pipelines—no per-store Python loops on a driver node, no exports to a separate ML estate.

17.6.1 The Architecture

Nightly, a pipeline lands yesterday's sales into the lakehouse; a feature notebook rebuilds `gold.features_store_daily` partition-by-partition (one `replaceWhere` per `as_of_date`); a training notebook fits per-store or per-segment models distributed across the cluster—Spark MLlib, or a pandas-UDF pattern applying a simple model per store group; predictions land in `gold.inventory_forecast`, keyed by store, date, and model version. Everything reads and writes Delta; nothing leaves the lakehouse; Chapter 20 adds the scoring automation and drift detection that complete the loop.



Spark-native: every stage reads and writes Delta in OneLake; no exports, no driver-node loops.

Figure 17.1: Spark-native inventory forecasting for 1,000 stores.

Common Pitfall

The “loop over 1,000 stores in the driver” pattern works on the sample and dies in production: hours of serial compute, one giant point of failure, no parallelism. Grouped map / pandas UDFs or per-segment distributed models keep the work on the executors where the

data already lives.

17.7 Operational Guidance for Week 17

Data science becomes engineering when someone else must rerun it. Parameterize, version, and make every feature reproducible for any historical date.

17.7.1 Performance and Reliability

Explore on samples; verify with targeted full passes. Cache intermediate DataFrames deliberately, not reflexively. Partition feature tables by `as_of_date` so rebuilds are surgical. Keep Data Wrangler exports under review like any transformation code.

17.7.2 Security and Governance Checklist

- Feature tables carry owners, as-of semantics, and sensitivity classifications.
- Personal data in features inherits Part I and Chapter 22 protections.
- Notebook parameters replace all hardcoded paths and dates.
- Sampling methods are documented—stratification choices are modeling decisions.
- Leakage review is a standing agenda item for every new feature set.
- No CSV extracts leave the governed lakehouse for model inputs.

17.8 Interview Q&A

1. **Q: What does Data Wrangler generate from your visual cleaning steps?**
A: Executable PySpark (or Pandas) code—the visual session is exploration, and the exported, parameterized code is the engineering artifact that enters version control.
2. **Q: Why run EDA on a sample before a full PySpark pass?**
A: Iteration speed—plots and pivots return in seconds on a sample—followed by targeted full-set aggregations to confirm the sample’s claims, because biased samples lie.
3. **Q: Name two leakage risks in feature engineering.**
A: Temporal leakage—windows computed past the as-of date—and preprocessing leakage—scalers fit before the train/test split. (Target and group leakage are the other canonical pair.)
4. **Q: Why keep feature sets in the lakehouse instead of CSV exports?**
A: Governance and reproducibility: Delta tables carry schema enforcement, history, owners, and as-of partitions; CSVs carry none of that and fork silently.
5. **Q: What makes a forecasting pipeline Spark-native?**
A: Every stage reads and writes Delta in OneLake and distributes across executors—no driver-node serial loops, no exports to a separate ML estate.
6. **Q: Why must training features be rebuildable for any historical date?**
A: Because a feature computed “as of now” for a historical row trains on the future; explicit as-of dating is what makes backtests honest and leakage testable.

17.9 Further Reading

Review the Fabric Data Science experience [20] and notebook guidance [27]. Feature storage inherits the lakehouse and Delta discipline of Part I [53, 9]. The tracking and operationalization continuations arrive in Chapters 18–20 [39, 10, 49].

Key Takeaways

- Fabric data science runs on the same lakehouse governance as data engineering—there is no separate storage world.
- Data Wrangler accelerates exploration; its exported, parameterized code is the production artifact.
- EDA is sample-first for speed, full-pass for truth—and stratified sampling is a modeling decision.
- Leakage—target, temporal, preprocessing, group—is an engineering defect with statistical symptoms.
- Every feature carries an as-of date, and every training set is rebuildable historically.
- Feature sets are governed Delta tables, never CSV exports.
- Spark-native means distributed compute over lake data; driver-node loops over 1,000 stores are a scalability bug.

17.10 Exercises

1. **(Conceptual)** Give one realistic example of each leakage species from a churn-prediction project.
2. **(Conceptual)** Why does a biased sample corrupt EDA conclusions even when the full data is available later?
3. **(Hands-on)** Rebuild the lab’s feature table for three historical as-of dates and verify partitions are disjoint.
4. **(Hands-on)** Add a leakage test: assert no feature row references data dated after its `as_of_date`.
5. **(Design)** Design the feature-table schema for the inventory forecast: keys, as-of columns, and the rebuild contract.
6. **(Design)** Choose between per-store models and per-segment models for the 1,000-store problem; defend with sparsity and operations arguments.
7. **(Interview)** In five sentences, explain how you would detect temporal leakage in a colleague’s feature pipeline.
8. **(Operational)** Write the runbook for “yesterday’s features disagree with a same-day rebuild”—where do you look first?

17.11 Capstone Project: Leakage-Safe Feature Store Slice

Capstone Project

Mission. Build the feature foundation for the Part V capstones: a governed, as-of-dated feature table for store-day forecasting, produced by a parameterized notebook with Data Wrangler-exported cleaning, a leakage test suite, and historical rebuilds proven for three dates.

Estimated time. 4–6 hours.

What you'll practice.

- Converting visual cleaning into parameterized, reviewed code.
- As-of-dated windowed features with surgical partition rebuilds.
- Automated leakage and schema tests over feature tables.
- Documentation that makes a feature set a governed product.

Scenario

Contoso Retail's data scientists are ready to model daily store demand, but their last attempt trained on leaked features and embarrassed everyone in backtesting. Engineering will deliver the feature table and the proof that it is leak-free; science will build models on it in Chapter 18.

Requirements

- `gold.features_store_daily` partitioned by `as_of_date`, keyed by `(store_id, as_of_date)`.
- At least five features including a trailing-window aggregate and a calendar feature.
- The parameterized build notebook with Data Wrangler-exported cleaning steps.
- A test suite: schema, nulls, uniqueness, and the leakage assertion.
- Rebuild evidence for three historical dates with identical re-run results.
- A feature dictionary: per-feature definition, window, source columns, owner.

Implementation Steps

1. Clean the source in Data Wrangler; export and parameterize the code.
2. Implement the windowed features with as-of discipline.
3. Write the partitioned Delta table with `replaceWhere`.
4. Implement the test suite; prove it fails on an injected leak.
5. Rebuild three historical dates; capture identity evidence.
6. Write the feature dictionary.

Deliverables

- The build notebook and test suite in the repository.
- Test output including the deliberate-leak failure case.
- Rebuild identity evidence (row counts and checksums per date).
- The feature dictionary.

Acceptance Criteria

- ☐ The feature table rebuilds identically for any historical as-of date.
- ☐ The leakage test demonstrably fails when a future-dated input is injected.
- ☐ Keys are unique per (store, as-of date) and the suite proves it.
- ☐ No feature references data after its row's as-of date.
- ☐ The feature dictionary defines every column and its owner.
- ☐ No CSV exports or hardcoded paths exist anywhere in the flow.

Stretch Goals

- Add stratified-sample EDA evidence to the feature dictionary.
- Add a second window length and measure its effect on downstream model skill.
- Automate the rebuild as a pipeline step with the Chapter 12 alerting discipline.
- Publish feature freshness SLAs and the monitor that enforces them.

Model Training and MLflow

Learning Objectives

- Explain Fabric’s machine-learning items: experiments, models, and their MLflow integration.
- Track parameters, metrics, and artifacts with `mlflow.autolog()` and explicit logging.
- Register and version models in the Fabric model registry.
- Train and auto-track a Scikit-Learn Random Forest, registering the best run.
- Compare hundreds of hyperparameter runs efficiently in a shared workspace.
- Attach the metadata—dataset version, code commit—that makes a model auditable and rollback-able.

Cross-Cloud Equivalence

Experiment tracking and model registries are the two pieces of ML infrastructure every platform converged on, and MLflow is the closest thing the industry has to a common denominator—it is native in Fabric and Databricks and deployable everywhere else.

Capability	Fabric	AWS	GCP	Snowflake
Experiment tracking	Built-in MLflow	SageMaker Experiments / MLflow on EC2	Vertex AI Experiments	MLflow (external or integrated)
Model registry	Fabric model registry (MLflow)	SageMaker Model Registry	Vertex AI Model Registry	Snowflake Model Registry
Autologging	<code>mlflow.autolog()</code>	Same library	Same library	Same library
Training compute	Fabric Spark pools	SageMaker Training / EMR	Vertex Training / Dataproc	Snowpark ML / warehouses
Artifact store	OneLake (via MLflow)	S3	Cloud Storage	Stages / tables
Run comparison UI	Experiment item UI	SageMaker Studio	Vertex console	Registry UI / notebooks

Databricks’ managed MLflow is the direct ancestor of Fabric’s integration. MongoDB does not host training; it supplies features. The portable claim of this chapter: *if it is not tracked, it did not happen; if it is not registered, it cannot be governed; if it cannot be rolled back, it is not production.*

Key Terms

Experiment: the MLflow container grouping related training runs. **Run:** one execution of training code with its parameters, metrics, and artifacts. **`mlflow.autolog()`:** automatic capture of framework-specific parameters, metrics, and models per run. **Model**

registry: the governed catalog of versioned, stage-tagged models. **Model version:** an immutable, numbered registration of a trained model. **Alias/stage:** a movable pointer (for example **Production**) over versions enabling rollback.

18.1 Week 18 Roadmap

Week 18 makes machine learning auditable. Monday covers experiments and models as Fabric items. Tuesday is MLflow tracking: parameters, metrics, artifacts, and autolog. Wednesday covers the model registry: registration, versioning, and aliases. Thursday is the hands-on project: train a Random Forest with `mlflow.autolog()`, track it, register the best run. Friday designs the collaborative workflow for hundreds of hyperparameter runs in a shared workspace.

Chapter 17 made features honest; this week makes models honest. The untracked notebook that produced the production model is organizational debt with interest: nobody can reproduce it, improve it, or roll back from it [39, 10].

Note

The minimum metadata contract for every training run: parameters, metrics, dataset version (or as-of date), code commit, and the environment. Autolog gives you the first three cheaply; the last two are discipline.

18.2 Monday: Experiments and Models in Fabric

Fabric surfaces MLflow through two workspace items [39]. An **experiment** item is the tracking container: open it to browse runs, compare metrics, and inspect artifacts. A **model** item is the registry entry: versioned, aliased, and loadable by name from any notebook.

Definition

An **MLflow experiment** groups related training runs under one name; each **run** records the parameters, metrics, tags, and artifacts of one execution, so work is comparable, searchable, and reproducible.

The workspace integration changes the collaboration physics: experiments are visible to the whole workspace (not trapped on one laptop), runs accumulate as organizational memory, and the registry attaches governance—who may promote a model to **Production** is a permission, not a convention.

18.3 Tuesday: Tracking with MLflow

18.3.1 Autolog

One line before training turns a Scikit-Learn, Spark MLlib, or XGBoost fit into a fully tracked run: parameters, metrics, and the model artifact itself [10, 39].

```

1 import mlflow
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import roc_auc_score
5
6 mlflow.set_experiment("churn-model-week18")
7 mlflow.autolog()
8
```

```

9 df = spark.read.table("gold.churn_features").where("as_of_date = '2026-07-31'")
  .toPandas()
10 X = df.drop(columns=["customer_id", "churned", "as_of_date"])
11 y = df["churned"]
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.2, random_state=42, stratify=y)
14
15 with mlflow.start_run():
16     mlflow.set_tag("dataset_as_of", "2026-07-31")
17     mlflow.set_tag("code_commit", "a1b2c3d")           # from your CI or notebook
    env
18     model = RandomForestClassifier(n_estimators=200, max_depth=12, random_state
    =42)
19     model.fit(X_train, y_train)
20     auc = roc_auc_score(y_test, model.predict_proba(X_test)[: , 1])
21     mlflow.log_metric("holdout_auc", auc)

```

Listing 18.1: Autologged training run with explicit metadata tags.

18.3.2 Explicit Logging

Autolog captures the framework’s view; you log the rest: business metrics (expected revenue saved per flagged churning), data slices (AUC by region), and environment facts. A run without its business metric is a science project; with it, a decision input.

Tip

Log one “headline” metric with a stable name across every experiment in the program. Cross-experiment comparison—this month versus last, this model family versus that—requires a metric that means the same thing everywhere.

18.4 Wednesday: The Model Registry

Registration promotes a run’s artifact into the governed catalog: the model gets a name, an immutable version number, and movable aliases such as **Staging** and **Production** [39, 10].

Definition

The **model registry** is the versioned catalog of trained models; an **alias** is a movable pointer over versions, so promoting or rolling back production is a pointer move, not a redeploy.

Why register rather than reference run artifacts directly: the registry is the governance boundary. Versions are immutable (the artifact a version pointed at never changes), aliases decouple consumers from versions (scoring code loads `models:/churn_model/Production` and never hardcodes a version), and the audit trail—who promoted what, when, with which metrics—lives in one place.

Common Pitfall

Scoring code that loads a specific run ID or version number bypasses the entire governance point. Consumers load by alias; humans move aliases under review; rollbacks are alias moves. The day a hardcoded version ships to production, the registry becomes decorative.

18.5 Thursday: Hands-on Project

The Week 18 lab trains a Scikit-Learn Random Forest on the Chapter 17 feature table, tracks the run with `mlflow.autolog()`, and registers the best model.

18.5.1 Goal and Architecture

Reuse `gold.churn_features` (or a provided classification dataset) in workspace `fabric-de-week18`. Run a small hyperparameter grid (six runs), compare runs in the experiment UI, and register the winner as version 1 of `churn_model`.

18.5.2 Step-by-Step Actions

1. Set the experiment and enable autolog.
2. Load one as-of partition of the feature table; split with stratification and a fixed seed.
3. Train six configurations across `n_estimators` and `max_depth`, each in its own run with the metadata tags.
4. Log `holdout_auc` explicitly in every run.
5. In the experiment item, sort by `holdout_auc`; inspect the best run's parameters and artifacts.
6. Register the best run's model as `churn_model` version 1; assign the `Staging` alias.
7. Load `models:/churn_model/Staging` in a fresh notebook and score a sample, proving the registry round-trip.

```

1 import mlflow
2
3 best_run_id = "<run id from the experiment UI>"
4 model_uri = f"runs://{best_run_id}/model"
5
6 registered = mlflow.register_model(model_uri, "churn_model")
7
8 client = mlflow.tracking.MlflowClient()
9 client.set_registered_model_alias("churn_model", "Staging", registered.version)
10
11 # Round-trip proof: load by alias, never by version
12 model = mlflow.pyfunc.load_model("models:/churn_model/Staging")
13 sample = spark.read.table("gold.churn_features").limit(100).toPandas()
14 print(model.predict(sample.drop(columns=["customer_id", "churned", "as_of_date "
  ]))[:5])

```

Listing 18.2: Registering the winning run and loading it back by alias.

18.5.3 Validation Checklist

- Six runs appear in the experiment with parameters, metrics, and artifacts.
- Every run carries `dataset_as_of` and `code_commit` tags.
- The registered model loads by alias in a fresh notebook and scores.
- The experiment UI comparison reproduces your winner selection.
- No run references data past its as-of date (Chapter 17's contract).
- A teammate can find the best run from the experiment UI without asking you.

18.6 Friday: Use Case and Architecture

A data-science guild runs hundreds of hyperparameter tuning runs per week across five practitioners in one workspace. Without conventions, the experiment list becomes an unsearchable swamp of `run-473-final-FINAL`. The task: a collaborative tracking workflow.

18.6.1 The Workflow

- **One experiment per problem**, not per person or per day: `churn-model`, `demand-forecast`. Runs accumulate comparable history.
- **Mandatory tags**: owner, hypothesis (one sentence: “deeper trees capture regional effects”), dataset version, code commit. The hypothesis tag is what separates search from wandering.
- **Stable headline metric**: every run logs `holdout_auc` (or the program’s equivalent), enabling cross-run, cross-week, cross-person sorting.
- **Search discipline**: coarse grid, then narrow; random search over pointless full grids; every configuration tracked, including the failures—negative results are organizational memory too.
- **Registry gates**: promotion to **Staging** requires the metric threshold and a reviewer; **Production** requires the Chapter 20 deployment checklist.

Element	Convention
Experiment naming	<code><problem>-<model-family></code> , one per problem
Required tags	owner, hypothesis, <code>dataset_as_of</code> , <code>code_commit</code>
Headline metric	One stable name per program, logged on every run
Promotion gate	Metric threshold + reviewer for Staging; deployment checklist for Production
Rollback	Alias move to previous version; rehearsed quarterly

Table 18.1: Collaborative tracking conventions.

Common Pitfall

Comparison across runs is only valid when the data and split are constant. Comparing a run on last month’s feature snapshot to one on this month’s is measuring data drift, not model quality. The `dataset_as_of` tag is what makes comparisons honest—filter to a common snapshot before declaring a winner.

18.7 Operational Guidance for Week 18

Tracking is cheap; untracked work is expensive. Make the tracked path the easiest path: notebook templates with autolog and tags pre-wired, experiment naming enforced by convention review, and registry promotion as a permission.

18.7.1 Performance and Reliability

Autolog adds negligible overhead for tabular frameworks. Keep artifact sizes sane (log the model, not the training set). Archive stale experiments per retention policy. Rehearse alias rollback: the first production rollback should not be during an incident.

18.7.2 Security and Governance Checklist

- Registry write and alias-move permissions restricted to designated roles.
- No secrets or personal data in logged parameters, tags, or artifacts.
- Dataset tags reference governed table versions, never laptop paths.
- Promotion requires the metric evidence attached to the run.
- Experiment retention policy documented; organizational memory is curated, not hoarded.
- Model cards or equivalent documentation attached at registration.

18.8 Interview Q&A

1. **Q: What does `mlflow.autolog()` capture per run?**

A: Framework-specific parameters, training and evaluation metrics, and the model artifact itself—automatically—while you add business metrics and metadata tags explicitly.

2. **Q: Why register a model instead of referencing run artifacts directly?**

A: The registry makes versions immutable, decouples consumers through aliases, and centralizes the audit trail of promotions—governance that run-artifact paths cannot provide.

3. **Q: Which metadata should accompany every experiment run?**

A: Parameters, metrics, the dataset version or as-of date, the code commit, and the environment—the minimum contract for reproducibility and honest comparison.

4. **Q: How do you compare hundreds of hyperparameter runs efficiently?**

A: One experiment per problem, one stable headline metric, mandatory tags—then sort and filter in the experiment UI, restricting comparisons to a common dataset snapshot.

5. **Q: What does model versioning enable for rollback?**

A: Aliases over immutable versions: rollback is moving **Production** back to the previous version—instant, reviewed, and reversible, with consumers never changing code.

6. **Q: Why must comparisons filter to a common dataset snapshot?**

A: Runs on different snapshots differ in both model and data; without a fixed snapshot you measure data drift and call it model quality.

18.9 Further Reading

Review MLflow in Fabric [39] and the upstream MLflow documentation [10]. The feature inputs come from Chapter 17’s governed feature tables [20]; the deployment continuation is Chapter 20 [49].

Key Takeaways

- Experiments group runs; runs record parameters, metrics, artifacts, and tags; the workspace makes them organizational memory.
- `mlflow.autolog()` plus explicit business metrics and metadata tags is the complete tracking contract.
- The registry’s immutability and aliases are what make promotion and rollback governance instead of archaeology.

- Consumers load models by alias; humans move aliases under review.
- Collaborative search needs conventions: one experiment per problem, hypothesis tags, a stable headline metric.
- Comparisons are valid only on a common dataset snapshot.
- Untracked models are debt; tracked, registered, alias-governed models are assets.

18.10 Exercises

1. **(Conceptual)** Explain why an immutable version plus a movable alias beats redeployment for rollback.
2. **(Conceptual)** What information does a hypothesis tag preserve that metrics alone cannot?
3. **(Hands-on)** Extend the grid to 24 runs with random search and compare search efficiency against the coarse grid.
4. **(Hands-on)** Add a business metric (expected retained revenue per flagged customer) and re-rank the runs.
5. **(Design)** Write the promotion-gate policy: thresholds, reviewers, and evidence required at Staging and Production.
6. **(Design)** Two teams share one workspace and keep colliding on experiment names. Design the namespace and the enforcement mechanism.
7. **(Interview)** In five sentences, argue to a skeptical engineer that experiment tracking is infrastructure, not bureaucracy.
8. **(Operational)** Rehearse a rollback: move **Production** to the previous version, verify consumers, and document the elapsed time.

18.11 Capstone Project: Governed Model Search and Registration

Capstone Project

Mission. Run a governed model search over the Chapter 17 feature table: a tagged, autologged 20-run random search, cross-run comparison on a fixed snapshot, registry promotion through the Staging gate with evidence, and a rehearsed alias rollback—the complete Week 18 discipline in one artifact set.

Estimated time. 4–6 hours.

What you’ll practice.

- Random search with full tracking and hypothesis tags.
- Snapshot-locked comparison and winner selection.
- Registry promotion with evidence and review.
- Alias-based rollback under a drill.

Scenario

Contoso’s churn program needs its first governed model. The guild’s conventions (Friday’s table) are adopted; you are the first team to execute them end to end, setting the example the

guild will be measured against.

Requirements

- Twenty tracked runs from a random search, each with the four mandatory tags and the headline metric.
- A comparison report (screenshot or export) filtered to one `dataset_as_of` snapshot.
- `churn_model` registered; winner promoted to **Staging** with the metric evidence attached.
- A second version registered and promoted to **Production**, then rolled back under drill, with timings.
- A short model card at registration: purpose, data, metrics, limitations.

Implementation Steps

1. Wire the notebook template: autolog, tags, headline metric.
2. Execute the 20-run search on the fixed snapshot.
3. Compare, select, and register the winner; attach the model card.
4. Promote to **Staging** with evidence; simulate a challenger and promote it to **Production**.
5. Execute the rollback drill; document elapsed time and verification.

Deliverables

- The search notebook and the comparison evidence.
- Registry evidence: versions, aliases, promotion history.
- The model card.
- The rollback drill report.

Acceptance Criteria

- ☐ All 20 runs are tagged, metric-bearing, and snapshot-locked.
- ☐ The winner selection is reproducible from the experiment UI by a reviewer.
- ☐ Promotion to each alias carried its evidence and reviewer.
- ☐ The rollback restored the previous version with consumers untouched.
- ☐ The model card states purpose, data, metrics, and limitations.
- ☐ No run logs secrets, personal data, or laptop paths.

Stretch Goals

- Add a second model family (gradient boosting) to the same experiment and compare families on the fixed snapshot.
- Automate the Staging gate as a notebook asserting the threshold before promotion.
- Add per-region metric logging and include a fairness slice review in the gate.
- Template the whole flow so the next problem starts at step three, not step one.

SynapseML and AI Integration

Learning Objectives

- Explain what SynapseML adds on top of Spark MLlib and when to reach for it.
- Integrate pre-built AI services—Azure OpenAI and Cognitive Services—into Spark pipelines in batch.
- Keep API keys and service credentials out of notebooks through proper secret management.
- Enrich a column of customer reviews with sentiment from Azure OpenAI and publish results to Gold.
- Use Copilot in Fabric to generate and explain code responsibly.
- Design a scalable pipeline that processes terabytes of reviews into BI-consumable sentiment metadata.

Cross-Cloud Equivalence

Calling managed AI services from distributed data pipelines—enrichment at scale—is a cross-cloud pattern with the same shape everywhere: batch the calls, vault the keys, land the results in the analytical store.

Capability	Fabric	AWS	GCP	Snowflake
Distributed ML library	SynapseML on Spark	SageMaker SDK / EMR MLlib	Vertex AI / Dataproc MLlib	Snowpark ML
Managed LLM calls	Azure OpenAI via SynapseML	Bedrock from Lambda/EMR	Vertex AI Gemini APIs	Cortex / external functions
Cognitive AI services	Azure AI Services	Comprehend, Rekognition, Textract	Vision, Natural Language, Document AI	Cortex AI functions
Secret management	Key Vault / workspace identity	Secrets Manager	Secret Manager	External access integrations
AI-assisted coding	Copilot in Fabric	CodeWhisperer/Q Developer	Gemini Code Assist	Copilot integrations
Cost model of enrichment	Service calls + CUs	Per-request pricing	Per-request pricing	Credits + external calls

Databricks runs the same SynapseML library—it originated in the Synapse ecosystem—and Mosaic AI for LLM work; MongoDB Atlas Vector Search occupies the adjacent retrieval niche. The portable rules: *batch instead of row-by-row, vault instead of hardcoded, and land enrichment in Gold where BI can reach it.*

Key Terms

SynapseML: Microsoft’s open-source distributed ML library extending Spark with AI service integrations and additional learners. **Azure OpenAI**: managed access to OpenAI models inside Azure compliance and networking boundaries. **Batch enrichment**: applying an AI service to a DataFrame column distributed across the cluster, rather than row-by-row. **Copilot in Fabric**: the natural-language assistant that generates and explains code in notebooks and other items. **Sentiment enrichment**: appending model-computed sentiment columns to review text at pipeline scale.

19.1 Week 19 Roadmap

Week 19 adds managed AI to the data engineer’s toolbox. Monday introduces SynapseML. Tuesday covers the pre-built AI services—Azure OpenAI and Cognitive Services—and the secret-management discipline they demand. Wednesday covers Copilot in Fabric as a code-generation assistant, with its review obligations. Thursday is the hands-on project: a SynapseML notebook calling Azure OpenAI to append sentiment to customer reviews. Friday scales the pattern to terabytes of reviews feeding a Gold lakehouse for BI.

The data engineer’s role in AI integration is not prompt engineering; it is pipeline engineering around a non-deterministic, rate-limited, per-call-billed dependency [47, 12]. This week teaches that plumbing.

Note

An LLM call inside a Spark pipeline is the most expensive function invocation in your estate by orders of magnitude. Batching, caching, retry policy, and result persistence are not optimizations here—they are the difference between a viable pipeline and a five-figure surprise.

19.2 Monday: Introduction to SynapseML

SynapseML is an open-source library that extends Spark with distributed integrations the base MLlib does not ship: connectors to Azure OpenAI and Cognitive Services, additional learners (for example LightGBM at Spark scale), and utilities for distributed serving [47]. In Fabric it is preinstalled or one environment dependency away, and it runs on the same Spark pools as Chapter 3’s notebooks.

Definition

SynapseML is a Spark-library extension providing distributed transformers for Azure AI services, additional scalable learners, and serving utilities—so AI-service enrichment becomes a DataFrame operation.

The mental model: SynapseML turns “call the API” into a Spark transformer. A column of review text goes in; a column of sentiment comes out; the distribution, batching, and parallelism happen at the executor level. That is precisely what makes it the right tool for terabytes—and what makes its configuration (batch size, concurrency, retry) a production decision rather than a detail.

19.3 Tuesday: Azure OpenAI, Cognitive Services, and Secrets

19.3.1 Pre-Built AI in the Pipeline

The Azure AI services cover the common enrichment cases: language (sentiment, key phrases, entity recognition), vision, document intelligence, and speech. Azure OpenAI adds generative capability: classification with instructions, summarization, extraction into structured columns [12, 47]. The integration choice is between the pre-built task APIs (cheaper, constrained) and the generative endpoint (flexible, pricier)—start with the task API when it fits.

19.3.2 Secret Management

Every one of these calls authenticates. The rule is absolute: **no keys in notebooks**. Options in order of preference: Fabric workspace identity / managed identity where the service accepts it; Key Vault references resolved at run time; environment variables injected by the environment or pipeline. Keys never appear in cell text, pipeline JSON, or—critically—in Git, because Week 23 will sync these notebooks to a repository verbatim.

Common Pitfall

A key pasted into a notebook “temporarily” is a key in Git permanently once Week 23’s Git integration syncs the workspace. Treat any key that touched a cell as compromised: rotate it, then fix the plumbing.

19.4 Wednesday: Copilot in Fabric

Copilot in Fabric generates and explains code from natural language inside notebooks and other items [13]. For the data engineer it is a drafting accelerator: boilerplate PySpark, KQL first drafts, unfamiliar API usage, and explanation of inherited code.

The engineering obligations do not change because the author was an assistant. Copilot-generated code gets the same review as any code: Does it read the table it claims to? Does the join key exist? Does it silently drop rows? Is it leakage-safe (Chapter 17’s tests still apply)? Does it hardcode a path or secret? Copilot drafts; engineers own.

Tip

Copilot is strongest as a first-draft generator against a precise prompt that names tables, columns, and intent—“read `silver.reviews`, filter to 2026, dedupe on `review_id`, write Delta to `gold.reviews_clean`”—and weakest when the prompt is vague enough that you cannot review the output against it.

19.5 Thursday: Hands-on Project

The Week 19 lab uses SynapseML in a Spark notebook to call Azure OpenAI: a column of customer reviews goes in, and a sentiment column comes out, appended and landed for downstream use.

19.5.1 Goal and Architecture

Work in `fabric-de-week19` with a table `silver.reviews` (a few thousand synthetic reviews) and an Azure OpenAI deployment (or the instructor-provided endpoint). The notebook batches calls, persists raw responses, and writes the enriched result to `gold.review_sentiment`.

19.5.2 Step-by-Step Actions

1. Configure the service connection: endpoint and key resolved from Key Vault / environment, never literals.
2. Build the SynapseML transformer for the chat completion with a constrained prompt: return one of **positive**, **neutral**, **negative**.
3. Run on a 50-row sample first; inspect raw outputs before scaling.
4. Set batch size and concurrency deliberately; record the measured cost per thousand rows.
5. Persist raw responses alongside parsed sentiment (debuggability and reprocessing).
6. Write the enriched table to Gold with a schema contract: **review_id**, **sentiment**, **model_version**, **scored_ts**.
7. Validate: null rate, label distribution sanity, and a hand-checked sample of twenty.

```

1 import os
2 from pyspark.sql import functions as F
3 from synapse.ml.services.openai import OpenAICompletion
4
5 reviews = spark.read.table("silver.reviews").where("review_date = '2026-07-31'"
6 )
7 completion = (
8     OpenAICompletion()
9     .setSubscriptionKey(os.environ["AZURE_OPENAI_KEY"])      # from Key Vault/
10     env
11     .setUrl(os.environ["AZURE_OPENAI_ENDPOINT"])
12     .setDeploymentName("gpt-4o-mini")
13     .setPrompt(
14         "Classify the sentiment of this review as exactly one of "
15         "positive, neutral, negative. Review: "
16     )
17     .setBatchSize(20)                                       # batch, never row
18     -by-row
19     .setTextCol("review_text")
20     .setOutputCol("raw_response")
21 )
22 scored = (completion.transform(reviews.limit(50))
23     .withColumn("sentiment", F.lower(F.trim(F.col("raw_response.choices."
24     text"))[0])))
25     .withColumn("model_version", F.lit("gpt-4o-mini-2026-07"))
26     .withColumn("scored_ts", F.current_timestamp()))
27 (scored.select("review_id", "sentiment", "model_version", "scored_ts")
28     .write.format("delta").mode("append")
29     .saveAsTable("gold.review_sentiment"))

```

Listing 19.1: SynapseML batch sentiment over a reviews DataFrame.

19.5.3 Validation Checklist

- No key or endpoint appears as a literal anywhere in the notebook.
- The 50-row sample run is inspected before any scale-up.
- Raw responses are persisted, not just parsed labels.
- Batch size and concurrency are set explicitly with measured cost recorded.
- The Gold table honors its schema contract including **model_version**.

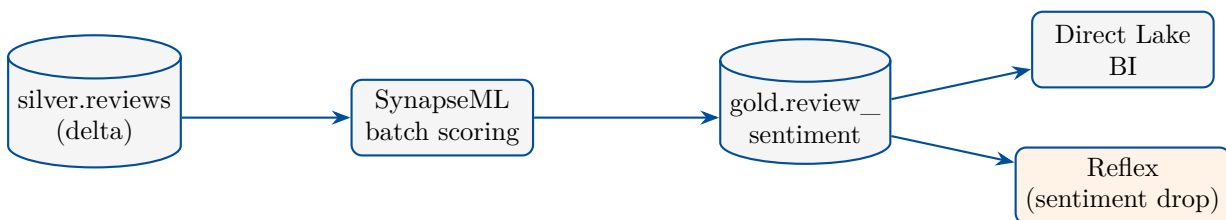
- A hand-checked sample agrees with the model at a documented rate.

19.6 Friday: Use Case and Architecture

A consumer platform accumulates terabytes of user reviews. Product leadership wants sentiment metadata available in the Gold lakehouse for BI—trends by category, alerts on sentiment drops—processed continuously and affordably.

19.6.1 The Pipeline

New reviews land in Silver through the Part III machinery. A scheduled Spark job scores the delta (reviews not yet in `gold.review_sentiment`, joined on `review_id`) with the SynapseML pattern, in bounded batches, within a capacity-friendly window. Results land in Gold with `model_version`; the Chapter 8 Direct Lake model picks them up; a Chapter 15 Reflex rule watches the daily negative-share for a sustained climb. Cost control comes from three levers: scoring only the delta, batching aggressively, and routing the bulk to the cheaper task-appropriate deployment.



Score the delta, batch the calls, version the model, land in Gold—BI and alerts consume, never recompute.

Figure 19.1: Terabyte-scale sentiment enrichment feeding BI and alerting.

Common Pitfall

Re-scoring history on every run is the classic cost explosion: the delta join (“reviews with no sentiment row for the current model version”) is the single most important line in the pipeline. Note the qualifier: a model upgrade legitimately re-scores history, so key the delta on `(review_id, model_version)`.

19.7 Operational Guidance for Week 19

Treat AI services as metered external dependencies inside your pipelines: budgeted, batched, cached where legal, retried with backoff, and always persisted with their raw output.

19.7.1 Performance and Reliability

Score deltas, not history. Batch and bound concurrency to the service’s rate limits. Retry transient failures with backoff; quarantine persistent failures with the Chapter 10 pattern. Record cost per thousand rows and trend it.

19.7.2 Security and Governance Checklist

- Keys resolved from Key Vault or environment; nothing literal, ever.

- Review text containing personal data classified before leaving the tenant boundary; the service’s data-handling terms reviewed.
- Model and deployment versions stamped on every scored row.
- Copilot-generated code reviewed with the same rigor as human code.
- Rate limits and quotas documented; alerts on throttling.
- Raw responses retained per the privacy policy, not indefinitely by default.

19.8 Interview Q&A

1. **Q: What does SynapseML add on top of Spark MLlib?**

A: Distributed transformers for Azure AI services (OpenAI, Cognitive Services), additional scalable learners such as LightGBM, and serving utilities—turning AI-service calls into DataFrame operations.

2. **Q: How do you avoid hardcoding Azure OpenAI keys in a notebook?**

A: Managed identity where supported, otherwise Key Vault references or environment-injected secrets resolved at run time—never literals, because Git integration will eventually sync the notebook verbatim.

3. **Q: Why batch sentiment calls instead of calling the API row-by-row?**

A: Per-call latency and per-request pricing: batching amortizes both across rows, and bounded concurrency respects rate limits without serializing the cluster.

4. **Q: What can Copilot in Fabric generate from natural language?**

A: First-draft code and explanations—PySpark, KQL, API usage—which an engineer then reviews against the same correctness, leakage, and security standards as human code.

5. **Q: Where should sentiment results land to be usable by BI?**

A: In a Gold Delta table with a schema contract and `model_version`, so Direct Lake models and Reflex rules consume them without recomputation.

6. **Q: Why key the scoring delta on (`review_id`, `model_version`)?**

A: New reviews need scoring, and upgraded models legitimately re-score history; the composite key makes both behaviors correct and cheap.

19.9 Further Reading

Review the SynapseML repository and documentation [47] and Azure OpenAI/AI services in Fabric [12]. Copilot capabilities are documented at [13]. The pipeline disciplines reused here come from Chapters 9–12 [42, 51]; the Gold consumption path from Chapter 8 [24].

Key Takeaways

- SynapseML makes AI-service enrichment a distributed DataFrame operation, which is what terabyte-scale requires.
- Secrets come from Key Vault, identity, or environment—never cell text; Git integration makes “temporary” keys permanent.
- Batch, bound concurrency, and persist raw responses: the three plumbing rules of pipeline AI.

- Score deltas keyed on (entity, model version); re-scoring history is a deliberate, budgeted event.
- Copilot drafts, engineers own—review generated code for correctness, leakage, and secrets.
- Sentiment lands in Gold with a schema contract so BI and alerts consume without recompute.
- AI calls are metered dependencies: budget them, measure cost per thousand rows, and trend it.

19.10 Exercises

1. **(Conceptual)** Compare the pre-built task APIs with generative endpoints for sentiment: cost, control, and fit.
2. **(Conceptual)** Why does Git integration change the severity of a hardcoded key from bad habit to incident?
3. **(Hands-on)** Extend the lab to a second language of reviews and document any label-distribution shift.
4. **(Hands-on)** Implement the delta join on (`review_id`, `model_version`) and prove a rerun scores nothing.
5. **(Design)** Design the retry-and-quarantine policy for the enrichment job: which failures retry, which quarantine, which page.
6. **(Design)** Write the Copilot usage policy for the team: what it may draft, what review it must pass, what it may never touch.
7. **(Interview)** In five sentences, explain the cost model of LLM enrichment to a product manager requesting “real-time sentiment on everything.”
8. **(Operational)** Write the runbook for a sudden spike in throttling errors from the AI service.

19.11 Capstone Project: Budgeted AI Enrichment Pipeline

Capstone Project

Mission. Productionize the Week 19 lab: a scheduled, delta-scoring sentiment pipeline over review data with vaulted secrets, bounded batches, quarantined failures, model-versioned Gold output, a cost report per run, and a Reflex alert on sustained negative-share climbs.

Estimated time. 5–7 hours.

What you’ll practice.

- Delta-keyed scoring with model-version awareness.
- Secret resolution and cost instrumentation in a scheduled pipeline.
- Failure quarantine and retry policy for a metered external dependency.
- Closing the loop from enrichment to BI and alerting.

Scenario

Contoso Reviews aggregates product feedback. Engineering owns the sentiment pipeline end to end: nightly delta scoring, monthly cost reporting, and an alert when any category’s negative share climbs for three consecutive days.

Requirements

- Scheduled pipeline scoring only the (`review_id`, `model_version`) delta.
- Secrets resolved at run time; zero literals in code or config.
- Quarantine table for persistently failing rows with error context.
- `gold.review_sentiment` honoring the schema contract.
- A cost report: calls, estimated spend, cost per thousand rows, trend.
- The Reflex rule with sustained condition and cooldown, proven with injected negativity.

Implementation Steps

1. Refactor the lab notebook into a parameterized, secret-resolving scoring notebook.
2. Implement the delta join and quarantine path.
3. Schedule via pipeline with Chapter 12 alerting.
4. Instrument and emit the cost report.
5. Configure and prove the Reflex rule.

Deliverables

- Scoring notebook and pipeline export in the repository.
- Delta-rerun evidence: second run scores zero rows.
- Quarantine evidence from an injected failure.
- Cost report for a week of runs.
- Reflex evidence from the injected negativity scenario.

Acceptance Criteria

- ☐ A rerun of the same batch scores zero rows; a model upgrade re-scores deliberately.
- ☐ No secret appears in any artifact, log, or screenshot.
- ☐ Persistent failures quarantine with context; transient failures retry with backoff.
- ☐ Gold output carries `model_version` and passes the schema contract.
- ☐ The cost report computes per-thousand-row spend from measured calls.
- ☐ The Reflex alert fires once on the sustained scenario and not on noise.

Stretch Goals

- Add a second enrichment (key phrases) sharing the batch infrastructure.
- Compare task-API versus generative-endpoint cost at one million rows; publish the break-even.
- Add a daily Direct Lake sentiment dashboard consuming Gold.
- Add an evaluation harness: a labeled sample re-scored per model version to catch quality regressions.

MLOps and Batch Scoring

Learning Objectives

- Apply registered MLflow models to Delta tables at scale with Spark UDFs and the PREDICT pattern.
- Automate model inference through Fabric pipelines instead of manual notebook runs.
- Design scoring output that honors a downstream schema contract and carries `model_version` for auditability.
- Detect data drift by comparing production feature distributions against training baselines.
- Trigger retraining automatically when drift or quality signals breach thresholds.
- Architect the full MLOps lifecycle for a production recommendation engine.

Cross-Cloud Equivalence

The MLOps loop—train, register, score, monitor, retrain—exists on every platform; what varies is how much of it the platform holds natively.

Capability	Fabric	AWS	GCP	Snowflake
Batch scoring	Spark UDF / PREDICT over Delta	SageMaker Transform	Vertex batch prediction	Snowpark ML / Cortex
Orchestrated inference	Pipelines invoking notebooks	Step Functions	Vertex Pipelines	Tasks
Drift monitoring	Notebook/pipeline distribution checks	SageMaker Monitor	Vertex Model Monitoring	Snowflake model monitors
Retrain trigger	Pipeline on drift signal	EventBridge pipeline	Eventarc + pipeline	Tasks + alerts
Model source of truth	MLflow registry (Ch. 18)	SageMaker registry	Vertex registry	Snowflake registry
Scoring audit	<code>model_version</code> on every row	Job metadata	Prediction metadata	Versioned functions

Databricks Mosaic ML and MLflow cover the same loop natively; MongoDB operationalizes predictions as documents written back by these pipelines. The portable laws: *score with the same features you trained on, stamp every prediction with its model version, and monitor distributions, not just errors.*

Key Terms

MLOps: the operational discipline that keeps models trained, deployed, monitored, and retrained as production systems. **Batch scoring:** applying a model to a table of rows

on a schedule, writing predictions back to storage. **PREDICT**: the SQL/Spark pattern that applies a registered model as a query-time function. **Training-serving skew**: divergence between feature computation at training time and at scoring time. **Data drift**: distributional change in production features or outcomes relative to the training baseline. **Retraining trigger**: the automated signal—drift, quality decay, schedule—that starts a new training run.

20.1 Week 20 Roadmap

Week 20 closes the ML loop and Part V. Monday covers batch scoring with the Spark **PREDICT**/UDF pattern. Tuesday applies registered MLflow models to Delta tables. Wednesday automates inference through Fabric pipelines. Thursday is the hands-on project: load a registered model and score 100,000 rows, writing predictions to OneLake. Friday architects the drift-aware MLOps lifecycle for a recommendation engine. The chapter closes with the Milestone 5 capstone.

Part V’s arc is now complete in outline: honest features (Chapter 17), honest models (Chapter 18), honest enrichment (Chapter 19), and—this week—honest *operations*: scoring that is automated, auditable, and self-aware enough to know when the world has moved on [49, 10].

Note

The two sentences that define the week: predictions without `model_version` cannot be audited or rolled back; and drift unmonitored means the model quietly rots—alert on distribution shift, not just errors.

20.2 Monday: Batch Scoring with PREDICT

Batch scoring applies a model to a table on a schedule and writes predictions back—the dominant production pattern for churn scores, forecasts, recommendations, and risk ratings. In Fabric, the mechanism is a Spark notebook that loads the registered model as a UDF (or uses the **PREDICT** function where the runtime exposes it) and applies it across the cluster [49, 11].

Definition

Batch scoring is scheduled, set-based inference: a pipeline applies a registered model to a Delta table partition and appends predictions—stamped with model version and score time—to a predictions table.

Why batch rather than per-request here: the consumers are analytical (reports, segments, nightly decisions), the volume is large, and the latency budget is generous. Per-request serving is a different architecture with different costs; choosing batch where batch suffices is the first MLOps economy.

The schema contract of the output matters as much as the model: downstream reports, segments, and reverse-ETL pipelines consume the predictions table. Its columns—entity key, score, `model_version`, `scored_ts`—are a published contract (Chapter 9’s discipline, applied to inference).

20.3 Tuesday: Applying Registered Models to Delta Tables

Chapter 18’s registry discipline pays off: scoring code loads by alias, so promotion and rollback require no code change.

```

1 import mlflow
2 from pyspark.sql import functions as F
3
4 predict = mlflow.pyfunc.spark_udf(spark, "models:/churn_model/Production",
5                                   result_type="double")
6
7 df = spark.read.table("gold.churn_features").where("as_of_date = '2026-07-31'")
8
9 (df.withColumn("churn_score", predict(*df.columns))
10  .withColumn("model_version", F.lit("Production"))
11  .withColumn("scored_ts", F.current_timestamp())
12  .write.format("delta").mode("append")
13  .saveAsTable("gold.churn_predictions"))

```

Listing 20.1: Batch scoring with the registered model as a Spark UDF.

The lurking failure is **training-serving skew**: features computed one way in training (Chapter 17’s as-of-dated pipeline) and another way at scoring time (an ad hoc query, a subtly different join). The defense is architectural: scoring reads the *same governed feature table* training read, rebuilt for the scoring as-of date by the *same parameterized code*. There is one feature pipeline; training and serving are two consumers of it.

Common Pitfall

“The model got worse in production” is very often “the features are computed differently in production.” Before retraining anything, diff the training and serving feature definitions—same code path, same window semantics, same as-of discipline—because a skewed feature pipeline retrains into the same skew.

20.4 Wednesday: Automating Inference via Pipelines

Manual notebook runs are how models silently stop scoring: someone is on holiday, the notebook errors, nobody notices until a stakeholder asks why last week’s segment never updated. The fix is Chapter 12’s orchestration applied to inference [42, 51]:

- A scheduled pipeline runs: feature rebuild (as-of today) → scoring notebook (by alias) → validation gate (row counts, null scores, distribution sanity) → ledger write.
- On Failure paths alert with a runbook link, per the Part III discipline.
- The gate blocks publication of anomalous prediction batches—a model that suddenly scores everyone 0.5 is detected by distribution checks before consumers see it.
- Parameters flow as always: `as_of_date` in, batch identity stamped on output.

Tip

Keep the last-known-good batch promotable: write new scores to a versioned partition and flip consumers only after the gate passes. Bad batches then cost an alert, not a downstream incident.

20.5 Thursday: Hands-on Project

The Week 20 lab loads the Chapter 18 registered model and scores a fresh batch of 100,000 rows with the UDF pattern, writing predictions back to OneLake with the full audit contract.

20.5.1 Goal and Architecture

Reuse `fabric-de-week18`'s registry and `fabric-de-week17`'s feature pipeline. Generate a 100,000-row scoring partition of `gold.churn_features` (or use the instructor dataset), then produce `gold.churn_predictions` through a pipeline-orchestrated, gated run.

20.5.2 Step-by-Step Actions

1. Build the pipeline: feature rebuild for `as_of_date` → scoring notebook → gate.
2. Implement scoring by alias with `model_version` and `scored_ts` stamped.
3. Implement the gate: row count equals input, null-score rate zero, score distribution within trained bounds.
4. Run at 100,000 rows; record duration and CU consumption.
5. Re-run the same batch; verify idempotence (partition replace, not duplicate append).
6. Inject a skewed feature (shift one column's scale 10× in a copy) and watch the distribution gate catch it.
7. Promote a challenger model version and repeat: the alias move changes the scored output without code change.

20.5.3 Validation Checklist

- Scoring loads by alias; no version numbers in code.
- Every prediction row carries `model_version` and `scored_ts`.
- The gate catches the injected skew and blocks publication.
- Reruns are partition-idempotent.
- Duration and CU cost at 100k rows are recorded for capacity planning.
- The alias move demonstrably changes output with zero code edits.

20.6 Friday: Use Case and Architecture

A production recommendation engine's click-through rate has decayed 15% over a quarter. Nobody changed the model; the world changed. The task: an MLOps lifecycle that detects drift and automatically triggers retraining.

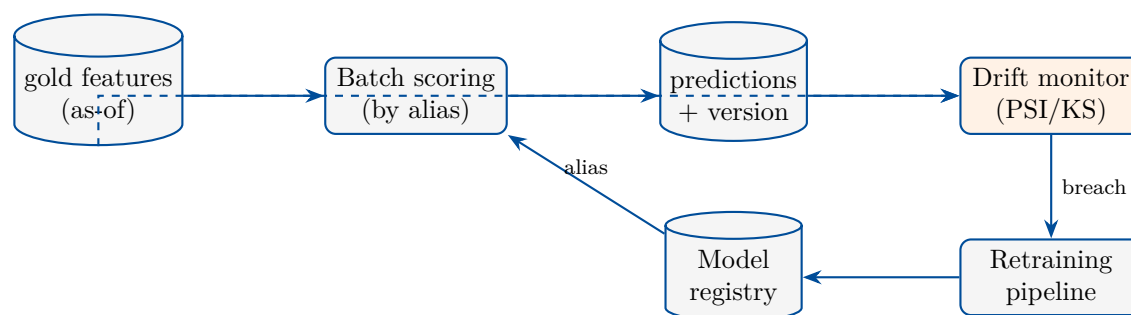
20.6.1 Drift Detection

Compare production feature distributions against the training baselines: population stability index (PSI) or KS statistics per feature, computed on a schedule over the same window semantics. Outcome drift (where labels arrive late—churn takes weeks to observe) is tracked separately from feature drift (visible immediately). Drift metrics land in a monitoring table; thresholds are set per feature from backtest evidence, not vibes.

20.6.2 The Retraining Trigger

A pipeline evaluates drift daily. On breach—say, any headline feature's PSI above 0.25, or business-metric decay beyond tolerance—it triggers the training pipeline (Chapter 18's instrumented flow) automatically: new run, tracked, registered to `Staging`, gated on the metric

threshold, promoted on review. Humans remain in the loop at the promotion gate; machines handle detection and preparation.



The loop closes itself: drift detected → retrain → register → alias → score, with humans gating promotion.

Figure 20.1: The self-aware MLOps lifecycle.

Common Pitfall

Auto-promoting a retrained model without the gate is automating the failure mode: drift triggered by a data-quality bug (a broken upstream feed shifting distributions) retrains the model on garbage and ships it. Drift triggers *retraining and evaluation*; promotion waits for metric evidence and review.

20.7 Operational Guidance for Week 20

MLOps is Chapter 12’s orchestration discipline with statistical content: gates, ledgers, runbooks, and now distributions.

20.7.1 Performance and Reliability

Score by partition with idempotent replace. Measure scoring CU cost per million rows and trend it. Keep baseline distributions versioned beside the model they describe. Alert on gate failures with the prediction-batch runbook.

20.7.2 Security and Governance Checklist

- Scoring and retraining pipelines run under service principals, not personal accounts.
- Prediction tables with personal data inherit classification and RLS (Chapters 21–22).
- Drift thresholds and gate criteria are reviewed configuration, versioned.
- Auto-retraining never auto-promotes; promotion is a human-gated alias move.
- Model artifacts and baselines are access-controlled.
- Every prediction row is attributable: model version, as-of date, batch.

20.8 Interview Q&A

1. Q: What does the Spark PREDICT/UDF pattern do?

A: It applies a registered model as a distributed function over a DataFrame—set-based inference across the cluster, with results stamped and written back to Delta.

2. **Q: Why score in scheduled batch instead of per-request here?**

A: Consumers are analytical and latency-tolerant while volumes are large; batch is dramatically cheaper and simpler, and per-request serving is a different architecture reserved for interactive needs.

3. **Q: Which signal indicates data drift in production?**

A: Distribution shift—PSI or KS statistics on production features versus training baselines—visible immediately, plus slower-arriving outcome/business-metric decay.

4. **Q: Why automate inference via pipelines instead of manual notebook runs?**

A: Manual runs silently stop: holidays, errors, no monitor. A scheduled, gated, ledgered pipeline makes scoring an operated system with alerts, not a habit.

5. **Q: Which schema contract must scoring output honor downstream?**

A: Entity key, score, `model_version`, `scored_ts`—the published columns reports, segments, and reverse ETL depend on; breaking them is a contract breach like any other.

6. **Q: Why does drift trigger retraining but not promotion?**

A: Drift can signal a broken upstream feed as easily as a changed world; retraining and evaluation are automated, but promotion waits for metric evidence and human review so a data bug cannot ship a garbage model.

20.9 Further Reading

Review model training and scoring in Fabric [49], the MLflow model and registry documentation [10, 39], and the Spark compute beneath [11]. Orchestration reuses Chapter 12 [42, 51]; feature honesty comes from Chapter 17 [20].

Key Takeaways

- Batch scoring is scheduled, set-based, gated inference; per-request serving is a different (rarely needed) architecture.
- Load models by alias; promotion and rollback are pointer moves, not deploys.
- One feature pipeline serves training and scoring—the structural cure for training-serving skew.
- Every prediction row carries `model_version` and batch identity; unauditible predictions are technical debt.
- The scoring gate blocks anomalous batches before consumers see them.
- Drift detection compares distributions to baselines; breaches trigger retraining, not promotion.
- MLOps is orchestration plus statistics: pipelines, gates, ledgers—and PSI.

20.10 Exercises

1. **(Conceptual)** Explain training-serving skew with a concrete feature example and its symptom.
2. **(Conceptual)** Why is PSI computed per feature rather than one global drift number?

3. **(Hands-on)** Implement PSI for three features against the training baseline; chart them over two weeks.
4. **(Hands-on)** Make the lab’s scoring partition-idempotent with `replaceWhere` and prove it.
5. **(Design)** Write the drift-response policy: thresholds, actions, and human gates per severity.
6. **(Design)** Design the monitoring table schema: what does one row mean, and who queries it weekly?
7. **(Interview)** In five sentences, describe the full loop from drift signal to safe redeployment.
8. **(Operational)** Write the runbook for “scoring gate failed: distribution out of bounds.”

20.11 Milestone 5 Capstone: End-to-End MLOps Pipeline

Capstone Project

Mission. Close Part V with the complete loop: a Data Factory pipeline orchestrating feature extraction from the lakehouse, a training notebook tracked in MLflow, registry promotion, and a daily batch-scoring notebook populating a Power BI Direct Lake report—with a drift monitor that triggers retraining on distribution shift.

Estimated time. 10–14 hours across the milestone’s components.

What you’ll practice.

- Leakage-safe features with fixed training windows (Chapter 17).
- Autologged, tagged training with registry promotion (Chapter 18).
- Alias-based, partition-idempotent batch scoring with a distribution gate (this chapter).
- Drift monitoring wired to automated retraining.
- The six rubric artifacts, including metric thresholds and PII handling.

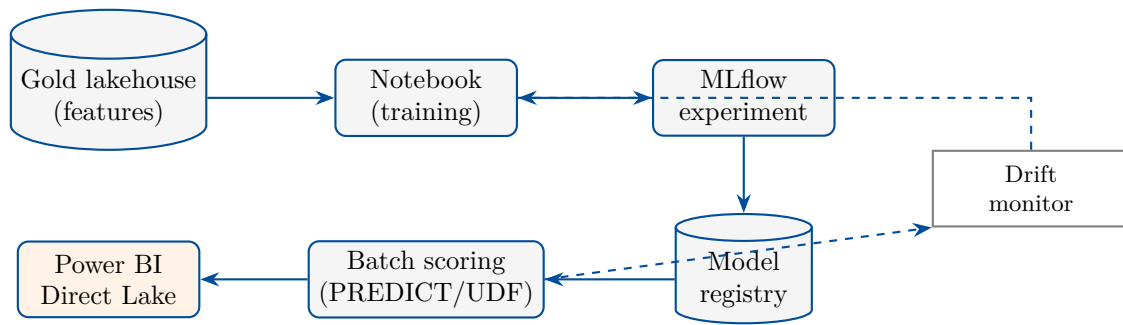
Scenario

Contoso’s retention team will act on churn scores weekly. The platform must produce them daily, auditably, and notice when the world drifts—without a human watching dashboards all day.

Requirements

- Gold feature table with as-of discipline; no post-outcome columns; fixed training windows.
- Training with `mlflow.autolog()`; the best run registered with dataset version and code commit as metadata.
- Pipeline-orchestrated scoring writing predictions with `model_version`, partition-idempotent.
- The distribution gate and the drift check comparing feature distributions against training baselines, triggering retraining on breach.
- A Direct Lake report over the predictions table.
- Rubric artifacts: clean-environment run, diagram, three tests (feature nulls, metric threshold, scoring schema) with failing cases, MLflow and pipeline evidence, cost estimate, security review.

Reference Architecture



Features → train → register → score → report, with drift closing the loop.

Figure 20.2: Milestone 5 reference architecture: the MLflow-driven MLOps lifecycle.

Implementation Steps

1. Build the leakage-safe feature table (reuse Chapter 17’s capstone where possible).
2. Train with autolog; register with metadata; write the model card.
3. Implement scoring by alias with partition-idempotent writes and the gate.
4. Orchestrate: features → score → gate → report refresh, with failure alerts.
5. Implement the drift monitor; wire breach → retraining pipeline (promotion stays human).
6. Run the failing-case drills: null features, sub-threshold model, skewed distribution.
7. Produce cost estimate and security review; finalize the README.

```

1 import mlflow
2 from pyspark.sql import functions as F
3
4 predict = mlflow.pyfunc.spark_udf(spark, "models:/churn_model/Production",
5                                   result_type="double")
6 df = spark.read.table("gold.churn_features")
7
8 (df.withColumn("churn_score", predict(*df.columns))
9  .withColumn("model_version", F.lit("Production"))
10  .write.format("delta").mode("append")
11  .saveAsTable("gold.churn_predictions"))

```

Listing 20.2: Milestone 5: alias-loaded batch scoring with audit columns.

Deliverables

- Repository running the full loop from a clean environment.
- Architecture diagram showing lakehouse, MLflow, registry, and scoring flow.
- The three automated tests with failing-case demonstrations.
- Monitoring evidence: experiment metrics, drift signals, pipeline run history.
- Cost estimate with two documented optimization decisions (for example batch window, pool sizing).

- Security review: model artifacts access-controlled, no API keys in notebooks, PII handling documented.

Acceptance Criteria

- ☐ Features are leakage-safe and rebuildable for historical as-of dates.
- ☐ Every training run is tracked with dataset and commit metadata.
- ☐ Scoring loads by alias and stamps every row with `model_version`.
- ☐ The gate blocks the injected skew; the rerun is partition-idempotent.
- ☐ The drift drill triggers retraining without auto-promotion.
- ☐ The Direct Lake report reflects a new scored batch after framing.
- ☐ No secrets, keys, or personal paths appear in any artifact.

Stretch Goals

- Add champion/challenger scoring: two aliases, split traffic by entity hash, compare business metrics.
- Add per-segment drift thresholds with a fairness review in the promotion gate.
- Automate the metric-threshold check as a pre-promotion pipeline step.
- Feed prediction outcomes back through Chapter 16's reverse ETL into the CRM.

Part VI

Governance, Security, and Exam Prep

Security and Access Control

Learning Objectives

- Assign workspace roles—Admin, Member, Contributor, Viewer—by least privilege.
- Distinguish item-level from workspace-level permissions and prefer the narrower grant.
- Configure OneLake data access roles and SQL row-level security.
- Restrict a user to a single region’s data through RLS on the SQL analytics endpoint.
- Audit a tenant for privilege creep on a schedule.
- Design a healthcare data perimeter with strict separation of duties between engineers and analysts.

Cross-Cloud Equivalence

Layered access control—platform roles, resource permissions, and in-engine row/column security—is structurally identical across clouds; only the names change.

Layer	Fabric	AWS	GCP	Snowflake
Platform container roles	Workspace roles (Admin/Member/Contributor/Viewer)	IAM policies on re-	IAM roles on	Account/database
Item-level grants	Per-item sharing and permissions	Per-resource policies	Per-resource IAM	Object grants
Data-plane scoping	OneLake data access roles	Lake Formation	BigQuery authorized views/dataplex	Row/column policies
Row-level security	SQL RLS predicates	Lake Formation cell filters	Row access policies	Row access policies
Identity	Microsoft Entra ID	IAM / Identity Center	Cloud Identity	IdP via SCIM/OAuth
Audit	Workspace/tenant logs, Purview	CloudTrail	Cloud Audit Logs	Access history

Databricks governs through Unity Catalog’s three-level namespace grants; MongoDB Atlas through database roles and field-level encryption. The portable law: *grant at the narrowest scope that lets the work happen, and audit the drift between policy and reality.*

Key Terms

Workspace role: the broad collaboration tier—Admin, Member, Contributor, Viewer—applied to a workspace. **Item-level permission:** a grant on one item (lakehouse, report) independent of workspace membership. **OneLake data access role:** a role restricting which folders or tables within a lakehouse an identity can read. **Least privilege:** granting the minimum access the task requires. **Privilege creep:** the slow accumulation of broader-than-needed access over time. **Separation of duties:** designing roles so no

single identity can both alter data and approve its use.

21.1 Week 21 Roadmap

Week 21 opens Part VI by securing everything Parts I–V built. Monday covers workspace roles. Tuesday contrasts item-level with workspace-level permissions. Wednesday covers OneLake data access roles and SQL RLS. Thursday is the hands-on project: a workspace with a Contributor role and RLS restricting a user to the US region. Friday designs the healthcare perimeter with separation of duties.

Security in Fabric is layered, and the layers compose: workspace roles set the collaboration ceiling, item permissions narrow it, OneLake roles scope the data plane, and SQL RLS filters the rows [44]. Misunderstanding any layer leaves a hole the others do not cover.

Note

Production rule for the week: grant access at the narrowest scope—prefer item-level permissions and OneLake data access roles over workspace-wide Member/Admin assignments. Audit quarterly: Viewer on a workspace combined with broad item sharing is the most common form of privilege creep in Fabric tenants.

21.2 Monday: Workspace Roles

Four roles define workspace participation [61, 44]:

Role	Can	Give to
Admin	Everything: settings, membership, deletion	Workspace owners only; two or three per workspace
Member	Create/edit items, manage most settings, add members (if allowed)	Core team leads
Contributor	Create and edit items; cannot manage membership or settings	Working data engineers and analysts who build
Viewer	Read and consume items	Consumers, stakeholders, auditors

Table 21.1: Workspace roles and their intended holders.

The distinguishing exam-and-practice question is Contributor versus Viewer: a Contributor *changes* items—edits pipelines, modifies notebooks, writes through them—while a Viewer consumes outputs without altering definitions. Most people who ask for Member actually need Contributor; most who ask for Contributor actually need Viewer.

Common Pitfall

Admin is not “senior user.” An Admin can delete the workspace and everything in it, and can change who else has access. Every unnecessary Admin is an unmonitored disaster path; keep the count at two or three and review it quarterly.

21.3 Tuesday: Item-Level versus Workspace-Level Permissions

Workspace roles grant broad capability across every item in the workspace. Item-level permissions grant access to one item—a specific lakehouse, report, or warehouse—to an identity that

may have no workspace role at all [44].

The decision rule: workspace roles for the team that *builds* the workspace’s products; item-level grants for everyone who *consumes* a specific product. A finance analyst who needs the sales report gets the report item—not Viewer on the workspace, which would expose every draft, experiment, and internal artifact the workspace contains.

Tip

When a stakeholder requests “access to the workspace,” ask which item they actually need. Item-level grants keep the workspace’s internal surface—work in progress, staging tables, experiments—private by default.

Item sharing composes with the deeper layers: sharing a lakehouse item grants the connection, while OneLake roles (Wednesday) scope which data inside it the identity can actually read.

21.4 Wednesday: OneLake Data Access Roles and SQL RLS

21.4.1 OneLake Data Access Roles

OneLake data access roles restrict access to folders or tables *within* a lakehouse: an identity can reach the lakehouse yet read only `Tables/gold/` and not `Files/raw/`, or only the virtual folder for their region [44]. This is the data-plane layer—it applies regardless of which engine (Spark, SQL endpoint, OneLake APIs) the identity uses, which is precisely why it matters: SQL-only security can be bypassed by a Spark notebook unless the storage layer also enforces the rule.

21.4.2 SQL Row-Level Security

Chapter 6 built the mechanism; this week operationalizes it. RLS on the SQL analytics endpoint or warehouse filters rows per user through a predicate function and security policy—the US-region-only analyst is the canonical exercise [46, 44].

```

1 -- Mapping row: the analyst's identity to their region
2 INSERT sec.user_region (user_principal, region)
3 VALUES ('analyst-us@contoso.com', 'US');
4
5 -- Policy already filters gold.v_sales_enriched via sec.fn_region_predicate.
6 -- Verify as the restricted principal:
7 EXECUTE AS USER = 'analyst-us@contoso.com';
8 SELECT DISTINCT region FROM gold.v_sales_enriched; -- must return only 'US'
9 REVERT;
```

Listing 21.1: RLS restricting a principal to US-region rows (Chapter 6 pattern, production framing).

Common Pitfall

Layer-confusion is the recurring security bug: RLS configured on the SQL endpoint does not constrain a Spark notebook reading the same Delta files; a OneLake role does not filter rows inside a permitted table. Each engine’s access path needs its layer—test every path a user actually has, not just the one you intended them to use.

21.5 Thursday: Hands-on Project

The Week 21 lab creates a workspace, assigns a Contributor role, and configures RLS on a SQL analytics endpoint so a test user sees only US-region data.

21.5.1 Goal and Architecture

Create **fabric-de-week21** with a lakehouse containing a regional sales table (reuse any Part I/II estate). Create or use a test identity for the restricted analyst. Deliverables: the role assignment evidence, the RLS objects, and the two-persona test.

21.5.2 Step-by-Step Actions

1. Create the workspace; add the builder identity as Contributor; add the test analyst with item-level access only to the lakehouse.
2. Create `sec.user_region`, the predicate function, and the security policy per Chapter 6.
3. Insert the analyst's mapping row for **US**.
4. As the analyst (or `EXECUTE AS USER`), query the protected view: only **US** rows return.
5. Attempt direct access to the base table: denied.
6. Attempt the same data via a Spark notebook as the analyst: record what the OneLake layer does and does not block; add a data access role if the scenario demands it.
7. Document the access matrix: identity $\times$ path $\times$ result.

21.5.3 Validation Checklist

- The builder holds Contributor, not Member or Admin.
- The analyst holds item-level access, not a workspace role.
- RLS filters to **US** under the analyst identity in SQL.
- Base-table access is denied at the object level.
- The Spark/OneLake path is tested and its result documented.
- The access matrix is written down and reviewed.

21.6 Friday: Use Case and Architecture

A healthcare provider must enforce strict separation of duties: data engineers operate pipelines and infrastructure; data analysts query curated data; neither can silently do the other's job; patient data is visible only as far as each role requires.

21.6.1 The Perimeter Design

- **Workspace split:** engineering workspace (Bronze/Silver, pipelines) versus analytics workspace (Gold, semantic models, reports). Engineers are Contributors in engineering and Viewers—at most—in analytics. Analysts have item-level access to curated Gold items only.
- **Data-plane scoping:** OneLake data access roles confine engineers' operational access to the layers they maintain; raw PII folders are readable only by the ingestion identity and the privacy officer.
- **Row and column security:** analysts reach patient-adjacent data through RLS-filtered views with denied sensitive columns (Chapter 6's column-level pattern).

- **Break-glass:** an emergency-access procedure that is time-boxed, dual-approved, and audit-logged—separation of duties includes a governed exception path, because ungoverned ones emerge anyway.
- **Audit:** quarterly privilege-creep review; every grant attributable to a ticket.

Common Pitfall

The most common separation-of-duties failure is the shared service identity with broad rights used by both teams' automation. Service principals get the same least-privilege treatment as humans—one per pipeline family, scoped to its actual read/write set.

21.7 Operational Guidance for Week 21

Access reviews are operations, not annual theater. Quarterly: list every workspace Admin and Member, every item share, every OneLake role, and reconcile each against a ticket or an owner statement.

21.7.1 Performance and Reliability

RLS predicates add query cost; keep predicate functions simple and indexed where the engine allows. Prefer security by composition (narrow views over RLS'd bases) to baroque per-object grants that nobody can audit.

21.7.2 Security and Governance Checklist

- Admins per workspace: two or three, reviewed quarterly.
- Consumers get item-level grants; builders get Contributor; Member is exceptional.
- OneLake roles cover the Spark path wherever SQL RLS covers the SQL path.
- Every RLS policy is tested as a real restricted principal after any change.
- Service principals are scoped per workload and vaulted.
- Privilege-creep audit is scheduled, evidenced, and acted on.

21.8 Interview Q&A

1. Q: What distinguishes Contributor from Viewer workspace roles?

A: Contributors create and modify items—pipelines, notebooks, models; Viewers consume outputs without altering definitions. The distinction is change authority, not seniority.

2. Q: Why prefer item-level permissions over workspace-level grants?

A: A workspace grant exposes every item—drafts, staging, experiments—to the grantee; item-level grants expose exactly the product the consumer needs and nothing else.

3. Q: What do OneLake data access roles control?

A: Which folders and tables within a lakehouse an identity can read at the storage plane—enforced regardless of engine, closing the Spark-path bypass of SQL-only security.

4. Q: How does SQL RLS restrict a user to US-region rows only?

A: A mapping table links the user principal to their region; an inline predicate function evaluates it per row; a security policy binds the predicate to the table—so identical queries return different rows per user.

5. **Q: What is privilege creep, and how do you audit for it?**

A: The accumulation of broader-than-needed grants over time—classically Viewer-on-workspace plus broad item sharing. Audit quarterly: enumerate grants, map each to a ticket or owner, and revoke what cannot be justified.

6. **Q: Why do service identities need least privilege too?**

A: Automation identities run unattended at scale; a broad shared service principal is an unaudited, always-on privilege that both teams' pipelines inherit—one scoped principal per workload family contains that.

21.9 Further Reading

Review Fabric security fundamentals [44], workspace roles [61], and the SQL endpoint's security model [46]. Governance and compliance continue in Chapter 22 [38]; capacity and deployment controls in Chapter 23 [28].

Key Takeaways

- Workspace roles set the collaboration ceiling: Admin is rare, Member is exceptional, Contributor builds, Viewer consumes.
- Item-level grants serve consumers; workspace roles serve builders.
- OneLake data access roles enforce the data plane across every engine—SQL-only security has a Spark-shaped hole.
- RLS composes with object-level security: grant the object, filter the rows, test as the real principal.
- Every access path a user actually has must be tested, not just the intended one.
- Privilege creep is audited quarterly or it is not audited at all.
- Separation of duties includes service identities and a governed break-glass path.

21.10 Exercises

1. **(Conceptual)** Construct a scenario where Viewer-on-workspace leaks information that item-level sharing would not.
2. **(Conceptual)** Explain why the OneLake layer is necessary even with perfect SQL RLS.
3. **(Hands-on)** Extend the lab with a second region and a manager identity mapped to both; test the union behavior.
4. **(Hands-on)** Build the privilege-creep audit query/export for your workspace: roles, shares, and last review date.
5. **(Design)** Write the break-glass procedure for the healthcare provider: approval, time-boxing, logging, revocation.
6. **(Design)** Design the service-principal map for the Part III pipeline estate: one principal per what, scoped to what?
7. **(Interview)** In five sentences, explain layered security in Fabric and where each layer's blind spot lies.

8. **(Operational)** Write the quarterly access-review procedure with evidence requirements.

21.11 Capstone Project: Layered Access Model with Audit Evidence

Capstone Project

Mission. Implement the healthcare-grade access model on a working estate: split workspaces, least-privilege roles, item-level consumer grants, OneLake data access roles, tested RLS and column denies, a scoped service-principal map, and a completed quarterly-style audit with evidence.

Estimated time. 5–7 hours.

What you'll practice.

- Role assignment by least privilege with an explicit access matrix.
- Composing OneLake, object, column, and row security layers.
- Testing every engine path per persona.
- Producing audit evidence a compliance officer can accept.

Scenario

Contoso Health Analytics must pass a security review: engineers and analysts share one Fabric tenant but must be provably separated, with patient-identifiable columns denied to analytics and every access decision traceable to a ticket.

Requirements

- Engineering and analytics workspaces with role assignments per the access matrix.
- Item-level consumer grants; zero consumer workspace roles.
- OneLake data access roles scoping engineers to their layers.
- RLS plus a column deny on the patient-adjacent table, tested as each persona.
- The service-principal map for all automation.
- The audit export: every grant, its justification, and the review sign-off.

Implementation Steps

1. Write the access matrix (personas $\times$ assets $\times$ layers).
2. Assign roles and item grants; configure OneLake roles.
3. Implement and test RLS and column denies per persona and per engine path.
4. Map and re-scope service principals.
5. Run the audit export; reconcile every grant; obtain sign-off.

Deliverables

- The access matrix and role/share inventories.
- Test evidence per persona per engine path.
- The service-principal map.

- The signed audit export.

Acceptance Criteria

- ☐ No consumer holds any workspace role; no engineer holds analytics-workspace write access.
- ☐ RLS and column denies hold under every tested engine path, including Spark.
- ☐ Every service principal is scoped to its workload with vaulted credentials.
- ☐ Every grant in the audit export has a justification or is revoked.
- ☐ Admin counts are two or three per workspace with named owners.
- ☐ No credentials or PII appear in any submitted evidence.

Stretch Goals

- Automate the audit export as a scheduled notebook with drift detection against the last review.
- Implement the break-glass procedure with time-boxed group membership.
- Add an alerting rule on new Admin assignments.
- Extend the model with a third persona (external auditor) read-only across both workspaces.

Governance, Purview, and GDPR Erasure

Learning Objectives

- Use the Purview hub and tenant administration surfaces for Fabric governance.
- Apply sensitivity labels to Fabric items and reason about downstream inheritance and legal-hold interaction.
- Trace end-to-end lineage and use impact analysis before making changes.
- Execute a GDPR right-to-erasure request across Delta tables, history files, KQL retention, caches, and backups.
- Implement crypto-shredding with per-subject keys in Azure Key Vault, and contrast it with row-level DELETE.
- Architect a five-domain data mesh with centralized Purview governance and per-domain erasure runbooks.

Cross-Cloud Equivalence

Metadata governance—catalog, classification, lineage, and compliance workflows—is a distinct platform layer in every cloud, increasingly unified across the estate rather than per-service.

Capability	Fabric	AWS	GCP	Snowflake
Catalog/governance	Microsoft Purview	DataZone / Glue Catalog + Macie	Dataplex	Horizon Catalog
Sensitivity labels	Microsoft Information Protection labels on items	Macie classifications	Sensitive Data Protection / policy tags	Object tagging, masking policies
Lineage	Fabric lineage view + Purview	Glue/DataZone lineage	Dataplex lineage	Access history / lineage
Endorsement	Promoted / Certified	Business glossary approvals	Data product annotations	Listing certification
Erasure mechanics	Crypto-shredding, Delta VACUUM coordination	S3 object deletion + Glacier waits	TTLs, crypto-shredding (CMEK)	Time Travel/Fail-safe windows
Legal hold	Purview eDiscovery/holds	Legal holds (S3 Object Lock)	Vault holds	Retention + legal processes

Databricks governs through Unity Catalog with Purview or Unity lineage; MongoDB through Atlas auditing and field-level encryption. The hard, portable lesson: *erasure is a propagation problem across caches, history, and backups—row deletes alone never satisfy it.*

Key Terms

Purview hub: Fabric’s integrated governance surface for insights, sensitivity, and lineage. **Sensitivity label:** a classification (for example *Highly Confidential*) applied to an item, traveling downstream. **Endorsement:** Promoted (vouched) or Certified (officially approved) status on a data product. **Lineage:** the item-to-item graph of data flow used for impact analysis. **Crypto-shredding:** erasing a data subject by destroying their encryption key, rendering every encrypted copy unreadable. **Legal hold:** a preservation obligation that overrides erasure until released, recorded as a documented exception.

22.1 Week 22 Roadmap

Week 22 governs the estate. Monday covers the Purview hub and tenant administration. Tuesday covers sensitivity labels and their downstream inheritance—including the legal-hold interaction. Wednesday is the heavyweight: lineage, endorsement, impact analysis, and the GDPR erasure propagation path, comparing crypto-shredding against row-level DELETE. Thursday is the hands-on project: label a lakehouse, trace lineage to a Power BI report, and execute a crypto-shredding erasure. Friday architects the five-domain data mesh with centralized governance.

Everything before this week built capability; this week builds accountability. Regulators, auditors, and—most importantly—data subjects do not care how elegant the pipelines are; they care whether the organization can prove what it holds and delete what it must [38, 31].

Note

The week’s hard truth, stated early: naive DELETE is insufficient for GDPR erasure in Fabric. Delta time travel keeps deleted rows readable in history until VACUUM expires them; KQL data persists per retention; semantic-model caches hold copies; clones and backups re-create erased rows on restore. Erasure is a cross-layer propagation design, not a SQL statement.

22.2 Monday: Purview Hub and Tenant Administration

The Purview hub inside Fabric surfaces governance posture to those who own it: sensitivity-label coverage, endorsement distribution, item inventory, and usage insights across the tenant [38]. Tenant administration (the admin portal) sets the guardrails within which all workspaces operate: which features are enabled, which capacities exist, which export and sharing behaviors are permitted [36].

Definition

The **Purview hub** is Fabric’s integrated governance surface: it aggregates label coverage, endorsement, inventory, and usage signals so governance is observed continuously rather than audited annually.

The data engineer’s Monday skill is reading the hub as a work queue: unlabeled items containing personal data, unendorsed tables feeding executive reports, domains without owners. Governance at scale is not a policy PDF; it is these queues trending toward zero.

22.3 Tuesday: Sensitivity Labels and Inheritance

Sensitivity labels from Microsoft Information Protection apply to Fabric items—lakehouses, warehouses, semantic models, reports—and classify the data they contain [38]. Two behaviors matter most:

Downstream inheritance. When a labeled item feeds another—a labeled lakehouse feeding a semantic model feeding a report—the label flows downstream, so classification follows the data through the serving layers. Inheritance is a default to be audited, not a guarantee to be assumed: verify in the hub that every downstream item of a *Highly Confidential* source carries the label.

Legal-hold interaction. A retention or legal-hold policy overrides an erasure request: data under hold must not be deleted, even for a valid GDPR request. The resolution is procedural, not technical—the hold is recorded as a documented exception in the audit register, the erasure executes when the hold releases, and the register proves both the obligation and its handling [38].

Common Pitfall

Labels classify; they do not by themselves enforce. A *Highly Confidential* label without matching access control (Chapter 21) and without export restrictions is a red sticker on an open door. Label, then verify the enforcement chain behind the label.

22.4 Wednesday: Lineage, Endorsement, and the Erasure Path

22.4.1 Lineage and Impact Analysis

Fabric records item-to-item lineage: this pipeline feeds that lakehouse, which feeds that semantic model, which feeds that report [31]. Lineage’s daily value is *impact analysis*: before changing a table’s schema, renaming a column (Chapter 9’s contract break), or retiring an item, the lineage view lists every downstream consumer who will feel it. Endorsement completes the signal: *Promoted* means the owner vouches; *Certified* means the organization certifies—and certification is what tells an analyst which of the fourteen `sales` tables is the real one.

22.4.2 The GDPR Erasure Propagation Path

A right-to-erasure request must reach every copy [38]:

1. OneLake raw zones (`Files` folders, landed extracts).
2. Bronze/Silver/Gold Delta tables—including *time-travel history files*, which keep deleted rows readable until `VACUUM` expires them.
3. Eventstream/KQL retention windows—telemetry ages out per policy, not per request.
4. Downstream derived tables and aggregates containing the subject’s data.
5. Semantic-model caches holding rows in memory.
6. Clones and backups, which re-create erased rows on restore.

22.4.3 Crypto-Shredding versus Row-Level DELETE

Row-level DELETE removes current rows but leaves history, caches, and backups readable—erasure becomes a multi-week coordination of `VACUUM` schedules, retention windows, and cache

clears, each with its own failure mode. **Crypto-shredding** encrypts each subject’s PII columns with a per-subject key in Azure Key Vault; deleting the key renders every copy—current, historical, cached, backed-up—permanently unreadable in one auditable act [44]. The trade-off: crypto-shredding must be designed in from the start (you cannot shred what you never encrypted per subject), and it makes the key vault a compliance-critical system.

Property	Row-level DELETE	Crypto-shredding
History files	Readable until VACUUM	Unreadable instantly
Caches/backups	Persist until cleared/rotated	Unreadable instantly
Design cost	Low, retrofit-able	High, must be designed in
Auditability	Multi-step evidence chain	One key-deletion event
Key vault criticality	None	Compliance-critical system

Table 22.1: Two erasure strategies, honestly compared.

Common Pitfall

Crypto-shredding without key-vault hygiene is a single point of catastrophic data loss: accidental or malicious bulk key deletion is unrecoverable by design. Key lifecycle—creation, rotation policy, deletion approval workflow, soft-delete windows—is the governance object that makes shredding safe.

22.5 Thursday: Hands-on Project

The Week 22 lab applies a *Highly Confidential* label to a lakehouse, traces lineage from lakehouse through pipeline to Power BI report, and then implements crypto-shredding: per-subject keys in Key Vault, PII encrypted in a notebook, and an erasure request executed by deleting one subject’s key—verifying the data is unrecoverable in current tables and Delta history alike, while a row-level DELETE leaves rows readable via time travel until VACUUM.

22.5.1 Goal and Architecture

Workspace `fabric-de-week22` with lakehouse `lh_pii` containing `customers_pii`, a pipeline and report consuming it, and an Azure Key Vault. The lab proves both erasure strategies and their asymmetry.

22.5.2 Step-by-Step Actions

1. Apply the *Highly Confidential* label to `lh_pii`; verify inheritance to the downstream model and report in the Purview hub.
2. Open the lineage view; capture the path lakehouse → pipeline → model → report.
3. Generate per-subject AES keys in Key Vault for a small synthetic subject set.
4. Encrypt the PII columns per subject in a notebook; store ciphertext in the table, keys only in the vault.
5. Erasure by DELETE: remove subject A’s rows; prove they remain readable via `VERSION AS OF`.
6. Erasure by shredding: delete subject B’s key; prove their ciphertext is undecryptable in current tables *and* history.

7. Record both operations in an audit register, including any legal-hold check performed first.

```

1 from pyspark.sql import functions as F
2 from cryptography.fernet import Fernet
3
4 # Keys live ONLY in Azure Key Vault; resolved per subject at run time.
5 def encrypt_with_subject_key(subject_id: str, plaintext: str) -> str:
6     key = keyvault.get_secret(f"pii-key-{subject_id}") # vault reference,
7     never literal
8     return Fernet(key.encode()).encrypt(plaintext.encode()).decode()
9
10 encrypt_udf = F.udf(encrypt_with_subject_key, "string")
11
12 df = spark.read.table("customers_pii_raw")
13 (df.withColumn("email_enc", encrypt_udf("customer_id", "email"))
14    .drop("email")
15    .write.format("delta").mode("overwrite")
16    .saveAsTable("customers_pii"))
17
18 # Erasure = delete the key in Key Vault: every copy, including
19 # Delta history and backups, becomes permanently undecryptable.

```

Listing 22.1: Per-subject encryption pattern (key resolution from Key Vault only).

22.5.3 Validation Checklist

- The label appears on the lakehouse and inherited downstream items.
- The lineage graph is captured and matches the intended flow.
- DELETE+time-travel demonstrates the residual-history problem.
- Key deletion demonstrates complete, instant erasure including history.
- Keys never appear in code, logs, or tables—only vault references.
- The audit register records both operations and the legal-hold check.

22.6 Friday: Use Case and Architecture

A global conglomerate operates five independent business domains—Retail, Logistics, Finance, Health, Media—each owning its data products in separate Fabric workspaces. The mandate: data-mesh autonomy with centralized governance through Purview.

22.6.1 The Mesh Design

- **Domains as Fabric domains:** each business unit’s workspaces grouped under a domain with a named owner—autonomy with accountability [25].
- **Federated computation, centralized policy:** domains build their own pipelines and products; Purview supplies the shared catalog, label taxonomy, lineage, and endorsement standards.
- **Contracts between domains:** cross-domain data sharing happens through endorsed data products with Week 9-style contracts, never through ad hoc lakehouse grants.
- **The per-domain erasure runbook:** each domain implements the Wednesday propagation path for its own Bronze/Silver/Gold lakehouses and KQL databases—a subject

request fans out to five domains, and each domain's runbook executes the same steps against its own estate, with the central register collecting completion evidence.

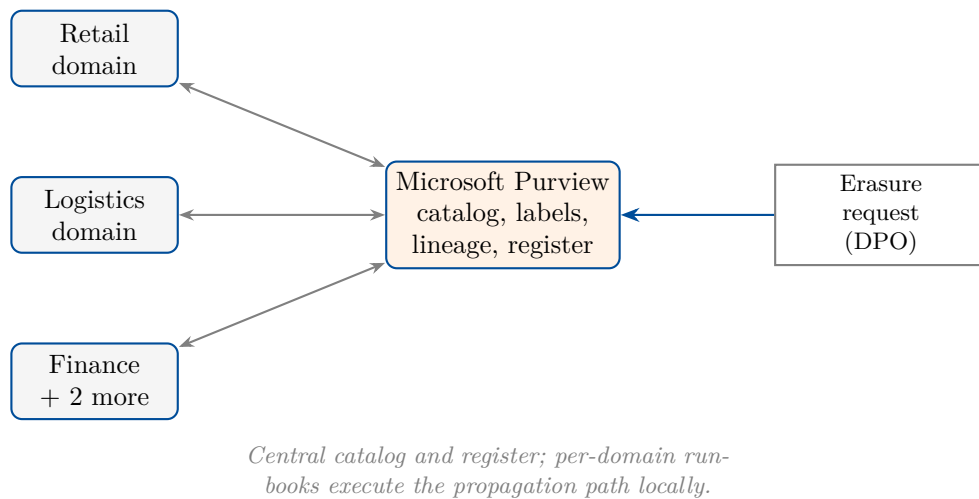


Figure 22.1: Five-domain data mesh with centralized Purview governance.

Common Pitfall

Centralized governance with centralized execution recreates the bottleneck the mesh was designed to break. Purview centralizes *policy and visibility*; domains keep *execution*. The erasure runbook is the test case: centrally defined, domain-executed, centrally evidenced.

22.7 Operational Guidance for Week 22

Governance fails at the edges: unlabeled new items, unowned domains, untested erasure. Schedule the hub review, the inheritance audit, and—annually at minimum—an erasure fire drill on a synthetic subject.

22.7.1 Performance and Reliability

Lineage and labels add negligible runtime cost; their cost is discipline. Erasure coordination with VACUUM schedules must respect Chapter 4's retention rules—shorter retention eases erasure and complicates recovery; the balance is a documented decision.

22.7.2 Security and Governance Checklist

- Label coverage monitored in the hub; inheritance audited, not assumed.
- Legal holds recorded in the register and checked before any erasure.
- Lineage reviewed before any schema-breaking change.
- Certification reserved for products passing the published criteria.
- Erasure drill executed annually with per-domain evidence.
- Key-vault lifecycle (approvals, soft-delete, rotation) documented and tested.

22.8 Interview Q&A

1. **Q: What does a sensitivity label pass on to downstream items?**

A: The classification itself—labels inherit downstream through lineage, so a *Highly Confidential* lakehouse’s models and reports carry the label; inheritance is audited, not assumed.

2. **Q: What is the difference between Promoted and Certified endorsement?**

A: Promoted is the owner’s vouch; Certified is the organization’s official approval against published criteria—the signal that tells consumers which table is the real one.

3. **Q: How does lineage help with impact analysis before a change?**

A: It enumerates every downstream consumer of an item, so a schema change or retirement is reviewed against the actual dependency list rather than discovered by broken reports.

4. **Q: Why is row-level DELETE insufficient for GDPR erasure in Fabric?**

A: Delta history keeps rows readable until VACUUM, KQL retains per policy, caches hold copies, and backups restore them; DELETE addresses only the current table state of one layer.

5. **Q: How does crypto-shredding sidestep all of those?**

A: PII is encrypted per subject with keys in Key Vault; deleting the key renders every copy—current, history, cache, backup—unreadable in one auditable act, with no dependence on per-layer cleanup timing.

6. **Q: What is a legal hold’s interaction with erasure?**

A: A hold overrides the erasure obligation until release; it is recorded as a documented exception in the audit register, and erasure executes on release—procedural resolution, not technical.

22.9 Further Reading

Review Microsoft Purview and Fabric governance [38], lineage [31], sensitivity labels [45], and domains [25]. The Delta retention interactions come from Chapter 4 [52, 9]; access layers from Chapter 21 [44].

Key Takeaways

- The Purview hub turns governance into observable queues: label coverage, endorsement, ownership.
- Labels classify and inherit; enforcement is a separate, verified chain.
- Lineage is the impact-analysis tool: read it before breaking anything.
- Erasure is propagation across raw zones, Delta history, KQL retention, caches, and backups—DELETE alone never suffices.
- Crypto-shredding makes erasure one auditable key-deletion, at the price of designed-in encryption and key-vault criticality.
- Legal holds override erasure and live in the audit register as documented exceptions.
- Data mesh centralizes policy and visibility through Purview while domains execute—including their own erasure runbooks.

22.10 Exercises

1. **(Conceptual)** Explain why a label without enforcement is worse than no label for audit confidence.
2. **(Conceptual)** Argue both sides: when is row-level DELETE an acceptable erasure strategy?
3. **(Hands-on)** Extend the lab: execute VACUUM after the DELETE and show time-travel readability ends.
4. **(Hands-on)** Write the per-domain erasure checklist for one domain, timed step by step.
5. **(Design)** Design the central erasure register: schema, evidence fields, and the query proving a request is fully satisfied.
6. **(Design)** Draft the key-deletion approval workflow that makes crypto-shredding safe against accidents.
7. **(Interview)** In five sentences, walk a DPO through what happens the day an erasure request arrives.
8. **(Operational)** Write the annual erasure fire-drill procedure with pass/fail criteria.

22.11 Capstone Project: Auditable Erasure Pipeline

Capstone Project

Mission. Build the complete erasure capability: labeled and lineage-traced PII estate, per-subject encryption, both erasure strategies demonstrated with their asymmetry proven, a legal-hold check, and a central audit register producing completion evidence for a sample request across two “domains.”

Estimated time. 6–8 hours.

What you’ll practice.

- Label application, inheritance verification, and lineage capture.
- Per-subject encryption with vault-only key resolution.
- Executing and evidencing both erasure strategies.
- The register schema and completion query that satisfy an auditor.

Scenario

Contoso Group’s Data Protection Officer requires a working demonstration—not a policy deck—that a subject erasure request propagates correctly through the Fabric estate, honors legal holds, and produces auditable evidence.

Requirements

- Two workspace “domains,” each with a PII lakehouse, labels applied and inheritance verified.
- Per-subject encryption with keys in Key Vault; no key material in code or logs.
- The propagation checklist executed per domain: raw, tables, history, derived, caches.
- The DELETE-versus-shredding asymmetry demonstrated with time-travel evidence.
- The legal-hold check integrated into the workflow.

- The audit register with a completion query across both domains.

Implementation Steps

1. Stand up both domains' estates with labels and lineage captured.
2. Implement per-subject encryption.
3. Execute the DELETE path with time-travel and VACUUM evidence.
4. Execute the shredding path with history-included unreadability evidence.
5. Run one full request through the register, including the hold check.

Deliverables

- Label and lineage evidence per domain.
- The encryption notebook and vault configuration (references only).
- Both strategies' evidence captures.
- The register schema, rows, and completion query output.
- The per-domain runbook documents.

Acceptance Criteria

- ☐ Labels inherit downstream in both domains, verified in the hub.
- ☐ DELETE leaves history readable until VACUUM, demonstrated.
- ☐ Key deletion renders the subject unreadable everywhere including history, demonstrated.
- ☐ A held subject is skipped and the exception recorded.
- ☐ The register's completion query returns full satisfaction for the executed request.
- ☐ No keys, secrets, or real personal data appear in any artifact.

Stretch Goals

- Add the third domain as a KQL/telemetry estate and extend the checklist to retention windows.
- Automate register fan-out: request in, per-domain tasks created, evidence collected.
- Add the key-deletion approval workflow with dual sign-off.
- Produce the annual fire-drill report template from the register.

Administration and CI/CD

Learning Objectives

- Manage Fabric capacities: metrics, throttling behavior, pause/resume, and scale-up decisions.
- Sync workspaces with Azure DevOps or GitHub through Git integration.
- Promote items through Dev → Test → Prod with deployment pipelines.
- Explain Fabric’s disaster-recovery reality: Git as item recovery, Delta time travel as data recovery, no automatic cross-region failover.
- Deploy zero-downtime schema changes with the expand–migrate–contract pattern and deploy ordering.
- Design an enterprise CI/CD workflow for multiple teams sharing one lakehouse.

Cross-Cloud Equivalence

Infrastructure-as-code discipline, staged promotion, and capacity observability are universal; the Fabric-specific twist is that workspace items are SaaS objects whose source of truth must be externalized to Git.

Capability	Fabric		AWS	GCP	Snowflake
Source control	Workspace integration (ADO/GitHub)	Git	CloudFormation/CDK in Git	Terraform/Deployment Manager in Git	Git integration / Terraform provider
Staged promotion	Deployment pipelines (Dev/Test/Prod)		Multi-account pipelines	Multi-project pipelines	Multi-database/Account promotion
Capacity observability	Capacity Metrics app		CloudWatch	Cloud Monitoring	Account usage views
Throttling model	CU smoothing + rejection		Per-service quotas	Quotas + rate limits	Warehouse queuing/credits
DR for definitions	Git rehydration		IaC redeploy	IaC redeploy	IaC / replication
DR for data	Delta time travel; no auto cross-region		Cross-region replication	Multi-region buckets/datasets	Database replication/failover

Databricks externalizes to Git through Repos and asset bundles; MongoDB Atlas through Terraform and API-driven automation. The portable disciplines: *definitions live in Git, data recovery is a separate design, and deploy order is a correctness property.*

Key Terms

Capacity Metrics app: the per-capacity observability surface for CU consumption and throttling. **Throttling:** capacity protection that smooths or rejects work when CU demand exceeds supply. **Git integration:** bidirectional sync between a workspace's items and an Azure DevOps or GitHub repository. **Deployment pipeline:** the Dev → Test → Prod promotion mechanism with per-stage rules. **Expand–migrate–contract:** the zero-downtime schema-change sequence: add nullable, dual-write and backfill, switch consumers, drop later. **BCDR:** business continuity and disaster recovery design.

23.1 Week 23 Roadmap

Week 23 makes the estate administrable and deployable. Monday covers capacity management: the metrics app, throttling, and pause/resume discipline. Tuesday covers Git integration and its disaster-recovery role. Wednesday covers deployment pipelines, multi-region BCDR, and zero-downtime schema evolution with deploy ordering. Thursday is the hands-on project: connect a workspace to Azure DevOps, commit notebooks and pipelines, and promote through a deployment pipeline. Friday designs the enterprise CI/CD topology for multiple teams sharing a lakehouse.

The week's mindset shift: workspace items are production code, and production code lives in version control with staged promotion. Fabric's SaaS convenience does not exempt it from the discipline—it makes the discipline easier to skip and therefore more important to enforce [28, 23].

Note

Fabric DR reality check, stated once and remembered: OneLake has no traditional backup/restore. Workspace items are recoverable because Git integration plus deployment pipelines rehydrate them; OneLake *data* recovery relies on Delta time travel and soft-delete patterns; multi-region BCDR requires a deliberate dual-workspace design—there is no automatic cross-region failover today.

23.2 Monday: Capacity Management

Capacity is the shared compute pool every chapter has drawn on; Monday teaches its management [35, 34].

23.2.1 The Metrics App and Throttling

The Fabric Capacity Metrics app decomposes CU consumption by item, operation, and time, separating interactive from background usage. Throttling is the enforcement: when demand exceeds supply, the platform first *smooths* (spreads background work forward), then *delays*, then *rejects*—interactive rejection being the user-facing failure. The operational insight: throttling is smoothed and delayed, so by the time users complain, you have already burned hours of interactive capacity. Watch the trend, not the complaint queue [34].

23.2.2 Pause, Resume, and Surge Protection

Pay-as-you-go capacities pause and resume on a schedule—the dev/test economy from Chapter 5. **Surge protection** guards interactive capacity against background storms: configured limits keep a runaway Spark job from consuming the executives' dashboard capacity. Alerting

on CU consumption—the metrics app plus Activator rules on capacity events—turns capacity from a surprise into a signal.

Common Pitfall

Scaling up is not the default answer to throttling. First: find the consumer (the metrics app names it), fix the pathology (unbounded queries, unoptimized Spark, fallback storms), then size. A bigger capacity under a pathological workload is a more expensive pathology.

23.3 Tuesday: Git Integration

Git integration syncs a workspace’s items—notebooks, pipelines, semantic models, reports—bidirectionally with an Azure DevOps or GitHub repository [28]. Commit from the workspace to Git; update the workspace from Git; branch, review, and merge with the team’s normal flow.

Two consequences deserve emphasis. **Disaster recovery:** because Fabric has no traditional backup for workspace metadata, Git integration plus deployment pipelines *are* the recovery mechanism—a deleted workspace is rehydrated by reconnecting its repository to a fresh workspace. **Secret discipline:** items sync verbatim, so a credential pasted into a notebook (Chapter 19’s warning) becomes a credential in the repository. Secret scanning on the repository is not optional.

Tip

Commit from the workspace at every meaningful milestone, with real messages. The Git history is simultaneously your code review record, your change log, and your disaster-recovery point. “Commit when it works” is a recovery strategy; “commit eventually” is not.

23.4 Wednesday: Deployment Pipelines, BCDR, and Zero-Downtime Schema Change

23.4.1 Dev → Test → Prod

Deployment pipelines promote items across stage-bound workspaces, with deployment rules adjusting per-stage specifics (connection strings, parameters, default lakehouses) so the same item definition serves each environment [23]. The promotion path is the only path to production: nothing is hand-built in Prod.

23.4.2 Multi-Region BCDR

OneLake data is not automatically replicated across regions. Deliberate designs only: dual-workspace/dual-capacity layouts with data re-ingestion from source, or Delta clone strategies copying tables between regions on a schedule. Recovery objectives (RPO/RTO) are chosen per workload and priced; the default is no regional failover at all.

23.4.3 Zero-Downtime Schema Evolution

The expand–migrate–contract pattern sequences a breaking schema change so consumers never see a broken intermediate state:

1. **Expand:** add the new nullable column; old and new coexist.

2. **Migrate:** dual-write and backfill the new column; deploy backward-compatible view shims on the SQL analytics endpoint exposing the old shape.
3. **Contract:** switch consumers to the new column; after a full release cycle, drop the old column.

Deploy ordering is the operational crux: schema-change notebooks and view shims deploy *before* the pipelines, notebooks, and semantic models that depend on them. The failure story is canonical: a deployment pipeline promoted a notebook that renamed `customer_segment` before the Gold view shim was deployed; the Direct Lake semantic model still referenced the old column, and every executive dashboard errored at 8 a.m. The fix: view shims are a separate, always-first deployment stage, and old columns drop only after one full cycle of dual-running.

Common Pitfall

Deployment pipelines promote item definitions, not data and not state. A promoted pipeline points at whatever its deployment rules configure; a promoted notebook does not carry its Dev test results. Validate per stage—the artifact that passed Test is the same definition, running against different connections, which is not the same thing as the same outcome.

23.5 Thursday: Hands-on Project

The Week 23 lab connects a workspace to an Azure DevOps repository, commits notebooks and pipelines, and creates a deployment pipeline promoting items to a Prod workspace.

23.5.1 Goal and Architecture

Three workspaces—`fabric-de-w23-dev`, `-test`, `-prod`—one Azure DevOps repository, one deployment pipeline binding them. Content: a lakehouse, a notebook, and a pipeline from any earlier week.

23.5.2 Step-by-Step Actions

1. Connect the Dev workspace to the repository (new branch `main` or `dev`).
2. Commit the lakehouse, notebook, and pipeline; verify the items appear in the repository as JSON definitions.
3. Create the deployment pipeline; assign the three workspaces to stages.
4. Deploy Dev → Test; configure a deployment rule (for example the default lakehouse reference); verify the rule applied.
5. Make a notebook change in Dev; commit; promote through Test to Prod; verify Prod reflects it.
6. Prove the DR property: disconnect and delete nothing destructive, but document—or with an instructor, execute—rehydrating a workspace from the repository.
7. Review the repository's secret-scanning status; remediate anything found.

23.5.3 Validation Checklist

- Item definitions live in Git with meaningful commit messages.
- Promotion flows Dev → Test → Prod only through the pipeline.

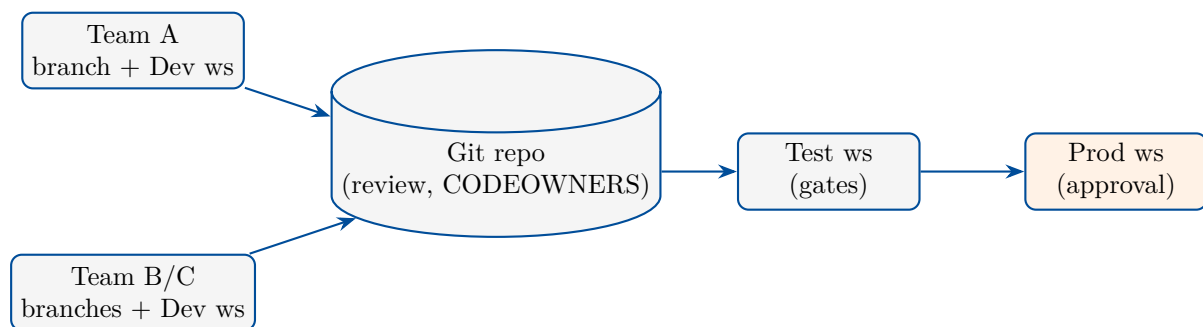
- Deployment rules correctly rewire per-stage references.
- The rehydration path is documented or demonstrated.
- Secret scanning runs on the repository and reports clean.
- Nothing in Prod was hand-edited—the audit trail proves it.

23.6 Friday: Use Case and Architecture

Three data-engineering teams share one lakehouse-backed analytics platform. Requirements: nobody overwrites another team’s code; releases are reviewed; Prod changes only through pipelines; a bad deploy rolls back in minutes.

23.6.1 The Design

- **Branch-per-team, review-to-main:** each team works in a feature branch bound to its own Dev workspace; pull requests gate merge to `main`; CODEOWNERS assigns review by folder (team A’s notebooks, team B’s dataflows).
- **Shared contract surfaces:** the Gold schema and the cross-team table contracts (Chapter 9) live in a folder owned by the platform team; changes there require platform review—the shared lakehouse is protected by review topology, not hope.
- **One deployment pipeline, staged gates:** Test requires automated checks (notebook smoke tests, contract validation); Prod requires approval; rollback is the previous commit re-promoted.
- **Deploy ordering encoded:** the pipeline’s stage order enforces shims-first; the contract stage deploys schema changes before dependent items.



Review topology protects shared surfaces; the pipeline enforces order; rollback is re-promoting the previous commit.

Figure 23.1: Multi-team CI/CD over one shared platform.

Common Pitfall

Three teams sharing one *workspace* (rather than one platform through branches and stages) collides daily: item names clash, edits race, and there is no review boundary. The unit of team isolation is the branch-bound Dev workspace; the unit of integration is the reviewed merge; the unit of release is the pipeline stage.

23.7 Operational Guidance for Week 23

Administration is mostly calendars and drills: weekly capacity review, monthly throttling analysis, quarterly DR rehearsal, per-release deploy-order check.

23.7.1 Performance and Reliability

Trend CU consumption weekly; investigate regressions before they throttle. Pause non-production on schedule. Rehearse workspace rehydration quarterly—an untested recovery plan is a hope, and hope is not a strategy.

23.7.2 Security and Governance Checklist

- Secret scanning enabled on every integrated repository.
- Prod workspace membership minimal; changes flow through the pipeline, not the portal.
- Deployment rules reviewed like code—they rewire production references.
- Capacity alerts configured before the first throttling complaint.
- DR objectives (RPO/RTO) documented per workload with their design implication.
- Deploy-order rule (shims first) encoded in the pipeline, not the wiki alone.

23.8 Interview Q&A

1. Q: Which CU consumption behavior triggers background throttling?

A: Sustained demand beyond capacity supply: the platform first smooths background work forward, then delays, then rejects—interactive rejection being the last and user-visible stage.

2. Q: Why is Git integration the recovery mechanism for workspace items?

A: Fabric has no traditional backup of workspace metadata; the repository holds item definitions, so reconnecting it to a fresh workspace rehydrates the estate.

3. Q: Why is there no automatic cross-region failover for OneLake?

A: OneLake data is not automatically replicated across regions; continuity requires deliberate dual-workspace/dual-capacity designs with re-ingestion or Delta clones—chosen and priced per workload.

4. Q: What does surge protection guard against?

A: Background storms consuming interactive capacity—a runaway job degrading the dashboards executives are watching; limits reserve interactive headroom.

5. Q: Why pause non-production capacities on a schedule?

A: Pay-as-you-go billing is per-second; idle dev/test capacity is pure waste, and schedules remove the human memory from the loop.

6. Q: Walk through expand–migrate–contract for renaming a column.

A: Add the new nullable column (expand); dual-write and backfill, with view shims exposing the old shape (migrate); switch consumers, then drop the old column after a full release cycle (contract)—shims deploying always-first.

23.9 Further Reading

Review Git integration [28], deployment pipelines [23], the Capacity Metrics app [34], and capacity concepts [35]. The schema-change discipline composes Chapter 9’s contracts with Chapter 6’s endpoint shims; governance framing continues in Chapter 22 [38].

Key Takeaways

- Capacity management is proactive: trend CUs, configure surge protection, and scale after—not before—fixing pathologies.
- Git integration externalizes workspace definitions, enabling review, history, and disaster recovery.
- Deployment pipelines are the only path to production; deployment rules rewire per-stage references.
- Fabric DR: Git rehydrates items, Delta time travel covers data, and multi-region is a deliberate dual design—never automatic.
- Expand–migrate–contract plus shims-first ordering makes schema change zero-downtime; reversed ordering makes the 8 a.m. incident.
- Multi-team safety comes from branch isolation, review topology (CODEOWNERS), and staged gates.
- Recovery plans are drilled quarterly or they are fiction.

23.10 Exercises

1. **(Conceptual)** Explain smoothing, delay, and rejection to a manager who asked “why not just buy more capacity?”
2. **(Conceptual)** Why do deployment pipelines promote definitions, not state—and what validation does that imply per stage?
3. **(Hands-on)** Add a Test-stage gate: a smoke-test notebook must pass before Prod approval unlocks.
4. **(Hands-on)** Execute the expand–migrate–contract sequence for a real column rename across two deployments.
5. **(Design)** Write the RPO/RTO table for the estate’s five most critical workloads and the design each implies.
6. **(Design)** Design the CODEOWNERS map for the three-team platform, including the shared-contract folder.
7. **(Interview)** In five sentences, describe Fabric’s DR posture and the three mechanisms that substitute for traditional backup.
8. **(Operational)** Write the quarterly DR rehearsal procedure with pass/fail evidence.

23.11 Capstone Project: Git-Native Staged Delivery

Capstone Project

Mission. Deliver the full Week 23 stack on a real estate: Git-bound Dev workspace, three-stage deployment pipeline with rules and gates, an expand–migrate–contract schema change shipped zero-downtime, capacity alerting, and a documented—or executed—workspace rehydration drill.

Estimated time. 6–8 hours.

What you’ll practice.

- Git discipline: commits, reviews, secret scanning.
- Staged promotion with deployment rules and a Test gate.
- Zero-downtime schema change with shims-first ordering.
- Capacity alerting and the DR drill.

Scenario

Contoso’s platform team formalizes delivery. The current “edit in the workspace and hope” model ends this sprint; your capstone is the reference implementation the other teams will copy.

Requirements

- Dev/Test/Prod workspaces bound to one repository with branch policy and secret scanning.
- The deployment pipeline with per-stage rules and a Test-stage automated gate.
- One real expand–migrate–contract change promoted with the shim deploying first.
- Capacity alerts configured (metrics app + Activator rule).
- The rehydration drill report (executed or tabletop with exact steps).
- A rollback demonstration: previous commit re-promoted to Prod.

Implementation Steps

1. Bind Dev to Git; establish branch policy and CODEOWNERS.
2. Build the pipeline with rules and the Test gate.
3. Ship the schema change in correct order; capture dashboard-stays-green evidence.
4. Configure capacity alerting; fire a test alert.
5. Run the rehydration drill and the rollback demonstration; document both.

Deliverables

- Repository with policies, history, and clean secret scan.
- Pipeline definition and deployment evidence per stage.
- The schema-change evidence: shim first, consumers green throughout.
- Alert configuration and test-fire evidence.
- Drill and rollback reports.

Acceptance Criteria

- ☐ No direct Prod edits; every change arrived through the pipeline.
- ☐ The Test gate demonstrably blocked one failing change.
- ☐ The schema change caused zero consumer-visible errors, evidenced.
- ☐ The rollback restored the prior state in minutes, timed.
- ☐ The rehydration drill produced a working workspace from the repository.
- ☐ Secret scanning is enabled and clean; no credentials in history.

Stretch Goals

- Add PR-triggered validation (lint notebooks, validate contracts) before merge.
- Extend to a second team with its own branch and Dev workspace; run a collision drill.
- Add automated CU-regression detection comparing release-over-release consumption.
- Draft the dual-region BCDR design for one workload with priced RPO/RTO.

DP-600 Certification Preparation

Learning Objectives

- Map the DP-600 (Implementing Analytics Solutions Using Microsoft Fabric) skill domains to this book’s chapters.
- Assess readiness against each domain with evidence, not confidence.
- Reason precisely about Direct Lake versus Import versus DirectQuery—the exam’s favorite trade-off.
- Triage weak domains after a mock exam and remediate with targeted review.
- Sit two full-length mock exams under realistic conditions.
- Close the program with the Milestone 6 capstone: a secure, governed, CI/CD-delivered platform.

Cross-Cloud Equivalence

Certification exists for every platform in this series, and the study discipline is identical even though the services differ.

Element	Fabric	AWS	GCP	Snowflake
Data-engineering cert	DP-600 (Fabric Analytics Engineer)	Data Engineer Associate (DEA-C01)	Professional Data Engineer	SnowPro Advanced: Data Engineer
Adjacent cert	DP-203 lineage / PL-300 (BI)	Data Analytics specialty (retired paths vary)	ML Engineer	SnowPro Core first
Exam style	Scenario MCQ + case studies	Scenario MCQ	Scenario MCQ	Scenario MCQ
Heaviest emphasis	Implement/mode/serve analytics	Ingestion, storage, processing	Pipelines, storage, ML touchpoints	Data movement, performance
Hands-on value	High—labs mirror exam scenarios	High	High	High

Databricks (Data Engineer Associate/Professional) and MongoDB (Developer/DBA tracks) complete the series. The portable meta-skill this chapter trains: *study to the skills outline, measure with mocks, remediate by building—not by rereading.*

Key Terms

DP-600: the Microsoft Certified: Fabric Analytics Engineer Associate exam. **Skills outline:** the published domain-and-weight document that defines what the exam measures. **Mock exam:** a full-length practice attempt under timed, closed-book conditions. **Weak domain:** a skills area whose mock performance falls below the readiness threshold.

Case study: the exam format presenting an architecture scenario with linked questions.
Readiness threshold: the mock score (typically 80%+ on two consecutive attempts) marking exam-day readiness.

24.1 Week 24 Roadmap

Week 24 converts six months of building into a credential—and closes with the capstone that proves the learning was real. Monday reviews the DP-600 objectives against the skills outline. Tuesday is the deep dive on the exam’s favorite topic: Direct Lake versus Import versus DirectQuery. Wednesday is full-length Mock Exam 1, scored by domain. Thursday remediates weak domains with targeted, hands-on review. Friday is Mock Exam 2. The chapter then closes the book with the Milestone 6 capstone: the secure, governed, multi-workspace platform.

A word on method: the exam tests *decision-making in scenarios*, not trivia. Twenty-three chapters of hands-on labs are the preparation; this week is the calibration [33].

Note

Score interpretation discipline: a mock below threshold is not a failure; it is a domain map. The only losing move is rereading everything uniformly. Remediate the weak domains by *building* the relevant lab again—muscle memory outperforms recognition memory on scenario questions.

24.2 Monday: The DP-600 Objectives

The skills outline is the exam’s contract: weighted domains describing exactly what is measured [33]. Read it first, and map it to this book:

Skills area (paraphrased)	This book	Typical weight
Plan, implement, and manage an analytics solution (capacity, workspace, governance, CI/CD)	Chapters 1, 21–23	Largest single share
Prepare and serve data (lakehouse, warehouse, pipelines, dataflows, notebooks)	Chapters 2–5, 9–12	Large
Implement and manage semantic models (modeling, DAX, Direct Lake)	Chapters 6–8	Large
Real-time intelligence (eventstreams, KQL, activator)	Chapters 13–16	Moderate and growing
Data science and ML touchpoints	Chapters 17–20	Smaller but present

Table 24.1: DP-600 domains mapped to chapters. Verify current weights against the live skills outline.

Monday’s deliverable is a personal readiness matrix: per domain, your evidence (which chapter labs you completed, which capstone artifacts exist) and your confidence score. Domains with neither evidence nor confidence are the study plan.

Common Pitfall

Reading the skills outline once, in January-fashion, and never again. The outline changes as Fabric evolves; re-download it in exam week and diff it against your matrix. A domain

you never heard of on exam day is an avoidable own goal.

24.3 Tuesday: Deep Dive—Direct Lake versus Import versus DirectQuery

This trade-off is the exam’s recurring scenario kernel, and Chapter 8 already taught the substance; today is the decision-framework drill. For any scenario, extract four facts: **where the data lives** (OneLake Delta or elsewhere), **freshness requirement** (real-time, minutes, daily), **volume and guardrails** (row counts versus capacity), and **feature needs** (RLS shapes, calculated columns, unsupported types).

Scenario signals	Answer	Why
OneLake star schema, near-real-time, within guardrails	Direct Lake	Freshness without refresh; in-memory after column load
External/slow source, full feature surface, nightly tolerance	Import	No dependency on source at query time; no fallback risk
Massive fact beyond guardrails, or unsupported shapes, live required	DirectQuery (or aggregation + Direct Lake)	Guardrails force it; mitigate with aggregation tables
Mixed estate	Composite models / per-table modes	Mode is per table, not per report

Table 24.2: The storage-mode decision drill.

Drill the follow-ups the exam loves: fallback triggers (guardrails, unsupported shapes, views), framing (metadata re-point, not refresh), V-Order’s role (faster column loads), and the telemetry that reveals fallback (Capacity Metrics app). Each maps to a chapter you have built: 8, 4, and 23 respectively.

Tip

In scenario questions, eliminate answers that violate the stated constraint before comparing the survivors on merit. “Must reflect data within one minute” eliminates every Import answer instantly, no matter how attractively worded.

24.4 Wednesday: Mock Exam 1

Sit Mock Exam 1 under exam conditions: timed, closed-book, no notes, one sitting. The score matters less than the *domain breakdown*:

1. Record score per domain, not just total.
2. For every wrong answer, classify the miss: *knowledge gap* (never learned it), *misread* (knew it, rushed), or *judgment* (two defensible answers, picked the weaker).
3. Knowledge gaps become Thursday’s build list. Misreads become pacing strategy. Judgment misses become re-reads of the relevant chapter’s Friday use case—the architectural reasoning sections.

Common Pitfall

“Reviewing” a mock by reading the correct answers and nodding converts nothing. The remediation unit is the hands-on lab: weak in pipelines? Rebuild Chapter 9’s metadata-driven pipeline. Weak in security? Redo Chapter 21’s RLS lab as a different persona. Building fixes; nodding does not.

24.5 Thursday: Targeted Remediation

Work the build list from Wednesday, weakest domain first. The pattern per domain: re-read the chapter’s roadmap and Key Takeaways (five minutes), redo the Thursday lab (the muscle), redo the Friday use case *from a blank page* (the judgment), then re-answer Wednesday’s missed questions without looking at your notes.

Two domains deserve standing attention regardless of your mock profile, because they are both heavily weighted and easy to under-build: **pipeline parameters and control flow** (Chapter 9—expressions, ForEach semantics, dependency conditions) and **Spark optimization** (Chapters 3–4—partitioning, small files, V-Order, caching decisions). If either felt shaky in the mock, those labs are Thursday.

24.6 Friday: Mock Exam 2 and Readiness

Sit Mock Exam 2 under the same conditions. Readiness for exam day, per this program’s standard: **two consecutive full-length mocks at 80% or above**, no domain below 70%, and—the qualitative check—the ability to explain *why* each wrong option is wrong, not merely which option is right.

Then schedule the real exam within one to two weeks: long enough to polish, short enough that knowledge does not decay. The night before, review the skills outline and your domain matrix, not the labs. Sleep is a study technique.

Readiness signal	Threshold
Full-length mocks	Two consecutive at 80%+
Weakest domain	No domain below 70%
Judgment check	Can explain why wrong options are wrong
Hands-on coverage	Every domain has a completed lab or capstone artifact
Scheduling	Exam within 1–2 weeks of readiness

Table 24.3: The program’s certification-readiness standard.

24.7 Operational Guidance for Week 24

Treat exam preparation like a pipeline: scheduled runs (mocks), measured outputs (domain scores), gated progression (the readiness table), and a rollback plan (remediation loop) rather than hope.

24.7.1 Performance and Reliability

Two mocks under real conditions beat five half-watched video courses. Keep the remediation loop tight—mock, classify, build, re-test—and stop when the readiness table says stop.

24.7.2 Security and Governance Checklist

- Use legitimate practice materials; exam dumps violate the certification agreement and teach the wrong exam.
- Keep your Microsoft Learn profile and certification records accurate.
- Use the trial or a sandbox tenant for labs—never a production tenant for exam experiments.
- Record which tenant and capacity your labs used so cleanup is complete.

24.8 Interview Q&A

1. **Q: Which skill domains does the DP-600 exam weight most heavily?**
A: Planning/managing the analytics solution, preparing and serving data, and semantic modeling carry the largest shares; verify current weights against the live skills outline, because it evolves.
2. **Q: When does Direct Lake beat Import in exam scenarios?**
A: When data lives in OneLake Delta, freshness must be near-real-time without refresh windows, and volumes sit within capacity guardrails—Import wins for external or slow sources and full feature surface needs.
3. **Q: How do you triage weak domains after Mock Exam 1?**
A: Classify every miss as knowledge, misread, or judgment; rebuild the labs for knowledge gaps (weakest domain first), fix pacing for misreads, and re-derive Friday use cases for judgment misses.
4. **Q: Which capacity metric signals Direct Lake fallback?**
A: DirectQuery-mode query load in the Capacity Metrics app where lake reads were expected—fallback is silent in the report but visible in telemetry.
5. **Q: What score thresholds mark you certification-ready in this program?**
A: Two consecutive full-length mocks at 80%+, no domain below 70%, and the judgment check: explaining why wrong options are wrong.
6. **Q: Why remediate by building rather than rereading?**
A: The exam tests scenario decisions; muscle memory from labs produces the reasoning, while recognition memory from rereading produces confidence without capability.

24.9 Further Reading

The authoritative source is the DP-600 certification page and its downloadable skills outline [33]. Revisit the heaviest domains through this book: storage modes [24, 50], pipelines [42], Spark and Delta [11, 9], real-time [26, 30], and governance [44, 38].

Key Takeaways

- The skills outline is the contract; map it to chapters and re-download it before exam day.
- Storage-mode decisions (Direct Lake / Import / DirectQuery) are the exam’s favorite scenario kernel—drill the four facts.
- Mocks are measurement instruments: score by domain, classify every miss.

- Remediate by rebuilding labs, weakest domain first; nodding at answer keys converts nothing.
- Readiness is two consecutive 80%+ mocks, no domain below 70%, plus the why-wrong judgment check.
- Schedule the real exam within one to two weeks of readiness.
- The credential certifies the habit the book taught: measured, evidence-based engineering.

24.10 Exercises

1. **(Conceptual)** Build your personal domain-to-chapter readiness matrix with evidence per cell.
2. **(Conceptual)** Write ten rapid-fire storage-mode decisions from invented scenario signals; check against the drill table.
3. **(Hands-on)** Rebuild the Chapter 9 metadata-driven pipeline from a blank workspace in under ninety minutes.
4. **(Hands-on)** Rebuild Chapter 21's RLS lab for a three-region, three-persona matrix.
5. **(Design)** Write your remediation plan from Mock Exam 1's domain breakdown, with dates.
6. **(Design)** Draft the two-week calendar from readiness to exam day, including the outline diff and sleep plan.
7. **(Interview)** In five sentences, defend Direct Lake to an interviewer whose last gig was all Import mode—then give the two cases where they were right.
8. **(Operational)** Write the post-exam retrospective template: what the exam emphasized versus what you expected.

24.11 Milestone 6 Capstone: Secure, Governed Data Platform

Capstone Project

Mission. Close the book by delivering the platform the whole program implied: a multi-workspace architecture ingesting streaming PII, masking it with Dataflows Gen2 before Gold, governed by sensitivity labels and endorsements, secured by least-privilege roles and RLS, and promoted Dev → Prod through Azure DevOps Git integration and deployment pipelines—with the DR plan documented and the six rubric artifacts produced.

Estimated time. 12–16 hours—the book's largest single deliverable.

What you'll practice.

- Multi-workspace topology with domains and least-privilege access (Chapters 1, 21).
- Streaming PII ingestion with masking before Gold (Chapters 10, 13–14).
- Labels, endorsement, lineage, and the erasure register (Chapter 22).
- Git-native CI/CD with staged promotion and deploy ordering (Chapter 23).
- The complete rubric: clean redeploy, diagram, tests, monitoring, cost, security, DR.

Scenario

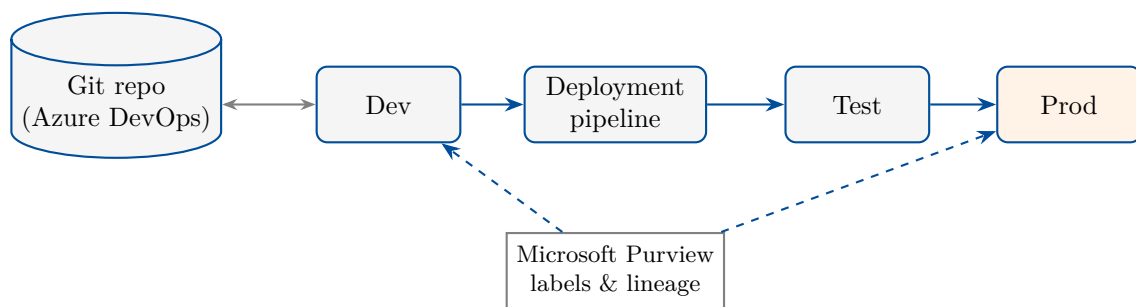
Contoso Health Analytics (the Chapter 21 provider, now grown) must run its full platform under audit: streaming patient-adjacent telemetry, masked before any analyst-visible layer, governed

by Purview, delivered through review-gated pipelines, and redeployable from Git after any disaster.

Requirements

- Dev/Test/Prod workspaces with Git integration and a deployment pipeline with per-stage rules.
- Synthetic PII streaming ingestion; masking (hash emails, redact phones) in a Dataflow Gen2 or notebook before any Gold write.
- Sensitivity labels applied with downstream inheritance verified; endorsement on certified products.
- Least-privilege roles plus SQL RLS; the access matrix from Chapter 21 extended to this estate.
- The DR plan: Git recovery for items, Delta time travel for data, dual-workspace BCDR design for the critical workload.
- Rubric artifacts: clean-environment redeploy, architecture diagram, three DQ tests with failing cases, Capacity Metrics report plus a failure alert with runbook, cost estimate with two optimizations, and the security review.

Reference Architecture



The whole platform redeploys from Git and governs itself by label inheritance.

Figure 24.1: Milestone 6 reference architecture: the governed CI/CD platform.

Implementation Steps

1. Stand up the three workspaces; bind Dev to Azure DevOps; create the deployment pipeline with rules.
2. Ingest the synthetic PII stream in Dev; mask before any Gold write.
3. Apply labels and endorsements; verify inheritance in the Purview hub.
4. Configure least-privilege roles and RLS; extend the access matrix.
5. Promote through the pipeline; run the DQ tests in every environment.
6. Produce monitoring, cost, and security artifacts; write the DR plan.

```

1 # Mask PII before Gold: hash emails, redact phone numbers
2 from pyspark.sql import functions as F
3

```

```

4 masked = (df
5     .withColumn("email", F.sha2(F.col("email"), 256))
6     .withColumn("phone", F.regexp_replace("phone", r"\d", "*")))
7
8 (masked.write.format("delta").mode("overwrite")
9     .saveAsTable("gold.patients_masked"))

```

Listing 24.1: Milestone 6: mask PII before any Gold write.

Deliverables

- The repository that redeploys the platform: Git → Dev → pipeline → Prod.
- The architecture diagram covering workspaces, CI/CD, and Purview governance.
- Three DQ tests with failing-case demonstrations carried through all environments.
- Capacity Metrics report and the failure alert with its runbook.
- The cost estimate with two documented optimizations (for example surge protection, pause schedule).
- The security review and the DR plan document.

Acceptance Criteria

- ☐ The full platform redeploys from a clean environment via Git and the pipeline.
- ☐ PII is masked before Gold; no unmasked personal data reaches an analyst-visible item.
- ☐ Labels inherit downstream, verified; certified products carry endorsement.
- ☐ Roles are least-privilege; RLS is tested per persona; the access matrix is current.
- ☐ No secrets exist in the repository—items sync verbatim, and the scan is clean.
- ☐ The DR plan names the three mechanisms (Git, time travel, dual-workspace BCDR) with owners.
- ☐ Every rubric artifact is present, measured, and reviewable.

Stretch Goals

- Add the Chapter 22 crypto-shredding erasure path to the PII estate.
- Add deploy-order evidence: one expand–migrate–contract change shipped through the pipeline.
- Add capacity alerting wired to Reflex and a monthly cost report.
- Extend to a second domain, proving the governance pattern scales.

This completes the Microsoft Fabric Master Data Engineer program: twenty-four weeks, six milestones, one platform—built, secured, governed, and certified.

Environment Setup and Sample Data

This appendix is the standing setup reference for every lab in the book. Complete it once before Chapter 0 and return whenever a lab assumes a tool, a connection, or a dataset that your machine or tenant does not yet have. Everything here targets Windows with PowerShell, matching the environment used throughout the program.

A.1 Accounts and Tenant

- **Fabric trial capacity.** Create a free 60-day trial capacity from the Fabric portal and enable Fabric in your tenant [37]. The trial covers every lab in the book; it expires, so it is unsuitable for anything with an SLA.
- **Tenant settings.** Where you administer the tenant, review the Fabric admin portal's tenant switches (Git integration, export, sharing) early—several Part VI labs depend on them [36].
- **Azure access.** Some labs (gateways, Key Vault, Azure SQL source, Dataverse) need an Azure subscription or instructor-provided equivalents. A free Azure account suffices for the gateway and SQL labs.

A.2 Local Tooling

Install once, in this order:

1. **Python 3.x** — used by the Week 13 producer, the Week 16 simulator, and synthetic data generation. Verify with `python -version`.
2. **Git** — required from Chapter 0 onward; Week 23 depends on it. Verify with `git -version`.
3. **VS Code** — with the Python extension; the editor for producer scripts and repository work.
4. **Azure CLI (az)** — sign in with `az login`; used for Key Vault, storage, and Appendix B's reference commands.
5. **SQL Server Management Studio (SSMS)** — the T-SQL client for the SQL analytics endpoint and warehouse labs (Chapters 5–6, 21).
6. **Docker Desktop** (optional) — the cleanest way to run the mock on-premises SQL Server for the Chapter 11 gateway lab.

7. **Python packages** — `azure-eventhub` (Week 13), `faker` (Weeks 16, 24), `mlflow` and `scikit-learn` (Weeks 18–20).

```
1 python --version
2 git --version
3 az --version
4 az login
5 pip install azure-eventhub faker mlflow scikit-learn
```

Listing A.1: One-time tool verification (PowerShell).

A.3 Repository and Credential Conventions

Clone or create the course repository with the standard layout—`data/`, `src/`, `tests/`, `infra/`, `docs/`—and observe three credential rules from day one:

- Secrets live in environment variables (for example `AZURE_CLIENT_ID`, `EVENTSTREAM_CONN_STR`) or Azure Key Vault—never in notebooks, pipeline JSON, or Git.
- Enable secret scanning on the course repository before Week 23’s Git integration makes every workspace notebook a committed file.
- Commit generator scripts, never generated data; the datasets below are reproducible by design.

A.4 Workspace Naming Conventions

Labs reference workspaces by predictable names: `fabric-de-weekNN` for weekly labs, `fabric-de-milestoneN` for capstones, and `fabric-de-w23-dev|test|prod` for the CI/CD estate. Lakehouses are `lh_<purpose>`, warehouses `dw_<domain>`, eventstreams `es_<stream>`, KQL databases `kqldb_<domain>`, pipelines `PL_<action>`. Consistency matters because runbooks, alerts, and the run ledger all reference these names.

A.5 Sample Datasets

The book’s labs and capstones standardize on three data sources:

Dataset	Used in	Notes
NYC Taxi (TLC trip records)	Parts I–II (Chapters 1–8)	Public Parquet, mirrored on Azure Open Datasets; 1–5M rows suffice for capstones
Olist Brazilian e-commerce	Parts III and V (Chapters 9–12, 17–20)	~100k orders across 9 related CSVs: orders, items, customers, products, payments, reviews
Synthetic streams (<code>faker</code>)	Parts IV and VI (Chapters 13–16, 21–24)	Credit-card swipes, IoT telemetry, PII-style CRM rows; seed generators for reproducibility

Table A.1: Standard datasets and where they are used.

Volume guidance: Part I medallion work at 1–10M CSV rows; the Part II warehouse star schema at ~5M fact rows; Part III re-ingests the nine Olist tables daily; Part IV streams at 5–20k events per minute during demos; Part V scores ~100k rows per batch.

```

1 from faker import Faker
2 import csv
3
4 fake = Faker()
5 Faker.seed(42) # reproducible
6
7 with open("data/synthetic_customers.csv", "w", newline="") as f:
8     w = csv.writer(f)
9     w.writerow(["customer_id", "name", "email", "phone", "region"])
10    for i in range(100_000):
11        w.writerow([f"C{i:06d}", fake.name(), fake.email(),
12                    fake.phone_number(), fake.random_element(["US", "EU", "APAC
13                    "])])

```

Listing A.2: Seeded synthetic PII generator (commit the script, not the data).

A.6 Troubleshooting Checklist

- **Notebook cannot see the lakehouse:** attach a default lakehouse to the notebook; verify the workspace is on a Fabric capacity, not a shared-only assignment.
- **COPY INTO path errors:** re-copy the OneLake DFS path from the lakehouse explorer; a wrong workspace or item segment is the usual cause (Chapter 5).
- **SSMS cannot connect:** use the SQL analytics endpoint connection string from lakehouse settings, Microsoft Entra MFA authentication, and confirm the endpoint reflects the table (files under **Files/** are invisible until registered as Delta tables).
- **Gateway offline:** check the gateway service on the host, then the cluster view in the portal; the Chapter 11 lab's failure drill covers the recovery pattern.
- **Throttling during labs:** the trial capacity is shared; pause unused capacities and check the Capacity Metrics app before assuming a lab is broken [34].

Tip

When a lab fails, reproduce the failure at the smallest possible scope—one row, one file, one minute of telemetry—before investigating the estate. Every chapter's validation checklist is written to support exactly that reduction.

Appendix B

CLI and API Quick Reference

This appendix collects the commands, endpoints, and idioms the book’s labs use repeatedly, in one lookup-friendly place. It is a reference, not a tutorial: each section points back to the chapter that teaches the concept.

B.1 Azure CLI Essentials

```
1 az login # authenticate (Ch. 0, Appendix A)
2 az account show # confirm subscription context
3 az group create -n rg-fabric-de -l eastus # resource group for lab infra
4
5 # Azure SQL free-tier source for the Week 9/11 labs
6 az sql server create -g rg-fabric-de -n sql-fabric-de-<suffix> -l eastus '
7   -u sqladmin -p '<from-vault>'
8 az sql db create -g rg-fabric-de -s sql-fabric-de-<suffix> -n sourcedb '
9   --edition GeneralPurpose --compute-model Serverless -f Gen5 -c 1
10
11 # Key Vault for Chapters 11, 19, 22 (secrets never in code)
12 az keyvault create -g rg-fabric-de -n kv-fabric-de-<suffix> -l eastus
13 az keyvault secret set --vault-name kv-fabric-de-<suffix> '
14   -n eventstream-conn-str --value '<connection string>'
15 az keyvault secret show --vault-name kv-fabric-de-<suffix> '
16   -n eventstream-conn-str --query value -o tsv
17
18 # Capacity hygiene (Ch. 5, 23): pause dev/test capacities off-hours
19 az fabric capacity resume -g rg-fabric-de -n fcapfabricde
20 az fabric capacity suspend -g rg-fabric-de -n fcapfabricde
```

Listing B.1: Azure CLI commands used across the labs.

B.2 OneLake Paths and Endpoints

Every Fabric item with storage is addressable through the DFS and Blob endpoints. The two forms appear throughout the book:

```
1 # DFS endpoint (used by COPY INTO, ADLS-compatible tools)
2 https://onelake.dfs.fabric.microsoft.com/<workspace>/<item>/Files/<path>
3 https://onelake.dfs.fabric.microsoft.com/<workspace>/<item>/Tables/<table>
4
5 # ABFS form (used by Spark and storage SDKs)
6 abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.Lakehouse/Files/<
  path>
```

Listing B.2: OneLake addressing patterns.

Right-click any folder in the lakehouse explorer to copy these paths exactly; a wrong segment is the most common cause of failed COPY INTO loads (Chapter 5).

B.3 Fabric REST API

The Fabric REST API automates workspace and item management—the operations plane beneath Week 23’s CI/CD. Pattern: acquire a token, call the API, poll long-running operations.

```

1 # Token for the Fabric API (service principal or az CLI context)
2 $token = az account get-access-token '
3     --resource https://api.fabric.microsoft.com --query accessToken -o tsv
4
5 # List workspaces
6 curl -H "Authorization: Bearer $token" '
7     https://api.fabric.microsoft.com/v1/workspaces
8
9 # Create a lakehouse in a workspace
10 curl -X POST -H "Authorization: Bearer $token" -H "Content-Type: application/
11     json" '
12     -d '{"displayName":"lh_lab","description":"Appendix B example"}' '
13     https://api.fabric.microsoft.com/v1/workspaces/<workspace-id>/lakehouses
14
15 # Run a notebook job (long-running; poll the returned Location header)
16 curl -X POST -H "Authorization: Bearer $token" -H "Content-Type: application/
17     json" '
18     -d '{}', '
19     "https://api.fabric.microsoft.com/v1/workspaces/<ws>/items/<notebook-id>/
20     jobs/instances?jobType=RunNotebook"
```

Listing B.3: Fabric REST API essentials.

Common Pitfall

The REST API authenticates as *someone*: a service principal with least-privilege workspace assignment, vaulted secret, and a rotation date. API tokens pasted into scripts reproduce Chapter 19’s key-in-notebook failure at the platform layer.

B.4 T-SQL Quick Reference

The statements the warehousing chapters rely on, in one block:

```

1 -- Bulk load (Ch. 5)
2 COPY INTO dbo.stg_sales
3 FROM 'https://onelake.dfs.fabric.microsoft.com/<ws>/<lh>/Files/stage/sales/'
4 WITH ( FILE_TYPE = 'PARQUET' );
5
6 -- Idempotent manifest recording (Ch. 5, 9)
7 INSERT cfg.load_manifest (batch_path, rows_loaded, loaded_ts)
8 VALUES ('2026-07', @@ROWCOUNT, CURRENT_TIMESTAMP);
9
10 -- Cross-item query (Ch. 5-6): three-part name into a lakehouse endpoint
11 SELECT TOP 10 * FROM lh_gold.dbo.fact_sales;
12
13 -- Object security (Ch. 6, 21)
14 GRANT SELECT ON gold.v_sales_enriched TO [analysts@contoso.com];
15 DENY SELECT ON gold.dim_customer (national_id) TO [analysts@contoso.com];
16
17 -- Row-level security (Ch. 6, 21)
18 CREATE SECURITY POLICY sec.sales_region_policy
19 ADD FILTER PREDICATE sec.fn_region_predicate(region)
```

```

20 ON gold.v_sales_enriched_base
21 WITH (STATE = ON);
22
23 -- Test as a restricted principal (Ch. 6, 21)
24 EXECUTE AS USER = 'analyst-us@contoso.com';
25 SELECT DISTINCT region FROM gold.v_sales_enriched;
26 REVERT;

```

Listing B.4: T-SQL idioms used across Chapters 5–8 and 21.

B.5 KQL Quick Reference

```

1 // Always bound by time (Ch. 14)
2 IoTTelemetry | where Timestamp > ago(1h);
3
4 // Dedup: latest arrival per producer-stamped id (Ch. 13-14)
5 IoTTelemetry | summarize arg_max(ingestion_time(), *) by EventId;
6
7 // Windowed aggregation (Ch. 14)
8 IoTTelemetry
9 | summarize avg(Temperature) by DeviceId, bin(Timestamp, 5m);
10
11 // Rolling average + anomalies (Ch. 14)
12 IoTTelemetry
13 | where Timestamp > ago(6h)
14 | make-series AvgTemp = avg(Temperature) default=0 on Timestamp step 5m by
   DeviceId
15 | extend RollingAvg = series_fir(AvgTemp, repeat(1, 5))
16 | extend Anomalies = series_decompose_anomalies(RollingAvg, 1.5);
17
18 // Impossible travel (Ch. 16)
19 Txn
20 | summarize arg_max(ingestion_time(), *) by txn_id
21 | order by card_id asc, ts asc
22 | serialize
23 | extend p_city = prev(city), p_ts = prev(ts)
24 | where city != p_city and ts - p_ts <= 10m;

```

Listing B.5: KQL idioms used across Chapters 13–16.

B.6 PySpark and Delta Quick Reference

```

1 # Read a managed table; stamp lineage columns (Ch. 3)
2 df = (spark.read.table("bronze.orders")
3       .withColumn("ingested_at", F.current_timestamp()))
4
5 # Idempotent partition-scoped write (Ch. 9, 17, 20)
6 (df.write.format("delta").mode("overwrite")
7   .option("replaceWhere", "as_of_date = '2026-07-31'")
8   .saveAsTable("gold.features_store_daily"))
9
10 # Delta maintenance (Ch. 4)
11 spark.sql("OPTIMIZE silver_orders VORDER")
12 spark.sql("DESCRIBE HISTORY silver_orders")
13
14 # Time travel (Ch. 4, 22)
15 prev = (spark.read.format("delta").option("versionAsOf", 12)
16         .table("silver_sales_orders_training"))
17
18 # Schema evolution (Ch. 9): default enforcement rejects; opt in deliberately

```

```
19 (df.write.format("delta").mode("append")
20     .option("mergeSchema", "true")
21     .saveAsTable("silver.orders"))
22
23 # Registered model as UDF (Ch. 18, 20)
24 predict = mlflow.pyfunc.spark_udf(spark, "models:/churn_model/Production",
25                                     result_type="double")
```

Listing B.6: PySpark/Delta idioms used throughout.

B.7 DAX Quick Reference

```
1 Total Revenue = SUM ( fact_sales[net_amount] )
2
3 Order Count = DISTINCTCOUNT ( fact_sales[order_id] )
4
5 Average Order Value = DIVIDE ( [Total Revenue], [Order Count] )
6
7 Revenue YTD =
8 CALCULATE ( [Total Revenue], DATESYTD ( dim_date[full_date] ) )
```

Listing B.7: DAX idioms used across Chapters 7–8.

Tip

Keep this appendix open during capstones. Every idiom here appears in at least one chapter’s validation checklist; when an acceptance criterion says “idempotent” or “dedup,” these are the exact patterns it refers to.

Lab Solutions and Acceptance Guidance

This appendix provides reference solutions for the weekly Thursday labs and the six milestone capstones. Each entry gives the approach, the critical steps, and the evidence that satisfies the chapter’s acceptance criteria. Use it to unblock yourself after a genuine attempt—not instead of one: the labs are the curriculum, and the DP-600 measures exactly the judgment they build [33].

Note

A solution counts only when the chapter’s acceptance criteria are met with evidence: row counts, screenshots, ledger rows, drill outputs. “It ran once on my machine” is the starting point of each lab, never the end.

C.1 Part I: Core Infrastructure (Weeks 1–4)

C.1.1 Week 1: Workspace, Lakehouse, and Shortcut

Approach. Create the workspace on trial capacity, provision the lakehouse, and shortcut to a public Blob container. **Critical evidence.** The shortcut path read from PySpark with zero bytes uploaded; the screenshot shows the shortcut icon, not a copied folder. **Common failure.** Confusing **Files/** uploads with shortcuts—if the data is physically in OneLake, the lab’s point was missed.

C.1.2 Week 2: Medallion Transformations

Approach. Bronze landing as-is, Silver typed and deduplicated, Gold conformed. **Critical evidence.** Row-count reconciliation Bronze → Silver (accepted + rejected = input), and Gold aggregates matching a direct source computation. **Common failure.** Silent row drops between layers with no quarantine accounting.

C.1.3 Week 3: Spark Notebook Transformations

Approach. Parameterized notebook (`processing_date`, `source_system`), explicit schemas, Delta writes. **Critical evidence.** The notebook reruns with a different date parameter and no code edits. **Common failure.** Hardcoded personal workspace paths.

C.1.4 Week 4: OPTIMIZE, V-Order, and Time Travel

Approach. Baseline query timed, OPTIMIZE . . . VORDER, accidental delete in a training copy, VERSION AS OF recovery. **Critical evidence.** DESCRIBE HISTORY showing optimize and delete; the recovered row count matching the pre-delete baseline. **Common failure.** Running VACUUM before the recovery validation, destroying the versions the lab exists to demonstrate.

C.1.5 Milestone 1 Capstone

The reference flow: Bronze shortcuts → parameterized Silver notebooks → Gold star schema → OPTIMIZE VORDER, with the operations note (retry unit, retention, cadence) as the differentiating artifact. The acceptance criteria's decisive items are the row-count reconciliation and the parameterization; reviewers check those first.

C.2 Part II: Warehousing and BI (Weeks 5–8)

C.2.1 Week 5: COPY INTO Five Million Rows

Approach. Staged Parquet under date-partitioned paths, COPY INTO per month, manifest insert after each batch. **Critical evidence.** Five million rows, aggregates matching the source profile, one manifest row per batch. **Common failure.** A malformed OneLake DFS path—copy it from the lakehouse explorer (Appendix B).

C.2.2 Week 6: SSMS and the Silver-to-Gold View

Approach. Entra-authenticated SSMS connection to the endpoint, the join view, the TVF, the RLS objects, the deliberate (failing) write attempt. **Critical evidence.** Two-persona row counts under EXECUTE AS USER. **Common failure.** Testing RLS as the owner—where every policy passes trivially.

C.2.3 Week 7: Relationship, Measure, and SCD2 Merge

Approach. Surrogate-key one-to-many relationships, four DAX measures, the two-step MERGE. **Critical evidence.** The changed customer shows exactly two rows, one current, with contiguous validity dates; a second merge run changes nothing. **Common failure.** Running the insert step before the expire step—order is correctness.

C.2.4 Week 8: Direct Lake versus DirectQuery Benchmark

Approach. Two identical models, identical visuals, measured timings, one deliberate fallback. **Critical evidence.** The completed benchmark table and a captured fallback event in the Capacity Metrics app. **Common failure.** Benchmarking before V-Order, then concluding the wrong thing.

C.2.5 Milestone 2 Capstone

The reference flow: manifest-gated COPY INTO → stored-procedure star schema (SCD2 customer) → custom semantic model with RLS → benchmarked Direct Lake report. Reviewers check idempotence (re-run changes nothing), the failing DQ test, and RLS proven *as every role*, in that order.

C.3 Part III: ETL Orchestration (Weeks 9–12)

C.3.1 Week 9: Metadata-Driven Pipeline

Approach. Config table → Lookup → ForEach → contract gate → Copy. **Critical evidence.** A renamed column rejected, an additive column flowing, and a fourth table onboarded by INSERT alone. **Common failure.** ForEach unbounded parallelism flattening the source.

C.3.2 Week 10: Dataflow Gen2 with Quality Gate

Approach. Named Power Query steps → staging destination → notebook assertions → quarantine → Gold. **Critical evidence.** The deliberately broken payload quarantined with reasons; Gold untouched. **Common failure.** The gate existing but not being wired into the pipeline—aspirational quality control.

C.3.3 Week 11: Gateway Extraction

Approach. Clustered gateway, least-privilege identity, watermark-incremental pipeline, gateway-down drill. **Critical evidence.** The duplicate-free rerun after the drill. **Common failure.** The gateway on a laptop that sleeps—the lab’s own anti-pattern.

C.3.4 Week 12: Master Orchestration Pipeline

Approach. Dataflow → gate notebook → ledger, On Failure → alert with runbook link. **Critical evidence.** The deliberate-gate-failure alert firing with the link, and one ledger row per outcome. **Common failure.** Wiring On Completion where On Success was meant—partial failures publishing to Gold.

C.3.5 Milestone 3 Capstone

The reference flow: config-driven ingestion → dataflow cleansing → notebook features → gated Gold, under one master pipeline with alerting. The decisive artifacts are the deliberate-failure drill evidence and the watermark discipline (advance on success only).

C.4 Part IV: Real-Time Intelligence (Weeks 13–16)

C.4.1 Week 13: Eventstream and Producer

Approach. Custom app source, Python producer with unique `event_ids`, KQL destination. **Critical evidence.** The dedup query disagreeing with the raw count after the replay test—proving duplicates existed and were handled. **Common failure.** The connection string committed to source.

C.4.2 Week 14: Rolling Average and Anomalies

Approach. `make-series` → `series_fir` → `series_decompose_anomalies`, plus the materialized view. **Critical evidence.** The injected hot streak flagged, normal noise quiet, every query time-bounded. **Common failure.** `default=0` silently inventing readings for offline devices.

C.4.3 Week 15: Sustained-Threshold Alert

Approach. Per-device rule, three-consecutive-minutes condition, cooldown, resolved notification. **Critical evidence.** Two-minute breach fires nothing; sustained breach fires exactly once. **Common failure.** The rule reading the raw stream, double-firing on replay.

C.4.4 Week 16: Dashboard and Reverse ETL

Approach. Bounded parameterized tiles at decision cadence; Dataverse upsert on the alternate key. **Critical evidence.** The second reverse-ETL run changing zero record counts. **Common failure.** Refresh cadence set to platform maximum rather than decision cadence.

C.4.5 Milestone 4 Capstone

The reference flow: faker producer → Eventstream → deduped KQL → impossible-travel query → Reflex → Teams, with the replay drill producing zero duplicate alerts. Reviewers check the dedup-before-rule ordering and the freshness indicator on the dashboard.

C.5 Part V: Data Science and ML (Weeks 17–20)

C.5.1 Week 17: Data Wrangler Export

Approach. Visual cleaning → exported code → parameterized cell → as-of-dated feature table. **Critical evidence.** Rebuilds for three historical dates, identical on rerun. **Common failure.** The windowed feature computed past its as-of date—leakage by construction.

C.5.2 Week 18: Autologged Random Forest

Approach. Six tracked runs, fixed snapshot, registry round-trip by alias. **Critical evidence.** Every run tagged with dataset and commit; the model loading from `models:/.../Staging` in a fresh notebook. **Common failure.** Comparing runs across different data snapshots and calling the difference model quality.

C.5.3 Week 19: SynapseML Sentiment

Approach. Key-Vault-resolved credentials, batched calls, raw responses persisted, Gold with `model_version`. **Critical evidence.** The 50-row sample inspected before scale-up; the measured cost per thousand rows. **Common failure.** A key literal in the notebook—an incident once Week 23 syncs it.

C.5.4 Week 20: Batch Scoring at 100k

Approach. Alias-loaded UDF, partition-idempotent write, distribution gate. **Critical evidence.** The injected skew caught by the gate; the rerun replacing rather than duplicating. **Common failure.** Scoring against ad hoc features instead of the governed feature table—training-serving skew by design.

C.5.5 Milestone 5 Capstone

The reference flow: leakage-safe features → autologged training → registry → gated daily scoring → Direct Lake report, with drift triggering retraining but never promotion. The decisive artifacts are the three failing-case tests and the human-gated promotion evidence.

C.6 Part VI: Governance and Exam Prep (Weeks 21–24)

C.6.1 Week 21: Roles and RLS

Approach. Contributor for the builder, item-level for the analyst, RLS tested per persona per engine path. **Critical evidence.** The access matrix with the Spark path's result documented,

not assumed. **Common failure.** SQL-layer security declared complete while the Spark path reads everything.

C.6.2 Week 22: Labels, Lineage, and Crypto-Shredding

Approach. Label inheritance verified, lineage captured, both erasure strategies demonstrated.

Critical evidence. DELETE rows readable via time travel; shredded subject unreadable everywhere including history. **Common failure.** Keys resolved from anywhere but the vault.

C.6.3 Week 23: Git and Deployment Pipeline

Approach. Dev bound to Git, three-stage pipeline with rules, promotion through stages only. **Critical evidence.** The rehydration path documented or executed; the secret scan clean.

Common failure. Deployment rules left unreviewed while silently rewiring production references.

C.6.4 Week 24: Mock Exams and Remediation

Approach. Two timed, closed-book mocks scored by domain; misses classified; weak domains remediated by rebuilding labs. **Critical evidence.** Two consecutive 80%+ mocks with no domain below 70%.

Common failure. Reviewing answer keys instead of rebuilding labs.

C.6.5 Milestone 6 Capstone

The reference flow: three workspaces, Git-bound Dev, staged pipeline with rules, PII masked before Gold, labels inheriting, RLS tested, DR plan naming its three mechanisms. The decisive artifacts are the clean-environment redeploy and the DR plan—the two things that cannot be faked in a screenshot.

Tip

For portfolio use, each capstone README should lead with the problem statement and architecture diagram, include clean-environment setup, screenshots of the pipeline run, dashboard, and alert, the capacity cost estimate, and a lessons-learned section naming what broke. Interviewers probe trade-offs: be ready to defend Lakehouse versus Warehouse, Import versus Direct Lake, and batch versus streaming at ten times the scale.

Production-Readiness and Governance Checklists

This appendix distills the book’s operational guidance, production notes, and lab acceptance criteria into per-domain checklists. Use them at three moments: when a workload is first promoted to production, at quarterly reviews, and as the rubric when auditing someone else’s estate. Every item traces back to a chapter; the cross-references point to the full reasoning.

Note

A checklist item is not satisfied by intention. Like the lab acceptance criteria in Appendix C, each item requires *evidence*: a screenshot, a query result, a ledger row, a tested failure drill. “We plan to” and “it usually works” are findings, not controls.

D.1 OneLake and Lakehouse

- ☐ Every lakehouse has a named owner, a domain assignment, and a workspace naming convention that encodes environment (dev/test/prod). (Ch. 1)
- ☐ Shortcuts are used only where virtualization is deliberate; each shortcut documents its source URI, authentication method, owner, and update frequency. (Ch. 1)
- ☐ Bronze tables carry ingestion metadata (source path, batch ID, load timestamp) sufficient to replay any batch. (Ch. 2)
- ☐ Silver tables enforce schema, types, deduplication, and key standardization; rejected rows land in quarantine with reasons, never silently dropped. (Ch. 2)
- ☐ Gold tables are the only tables BI consumes; no report points at a raw external shortcut or an uncertified table. (Ch. 1, 2)
- ☐ `OPTIMIZE` (with V-Order where BI reads the table) runs on a schedule for high-churn tables; Z-Ordering is applied only where query filters justify it. (Ch. 4, 8)
- ☐ `VACUUM` retention is set deliberately and never run before pending time-travel recovery needs are validated. (Ch. 4)
- ☐ Schema evolution goes through a reviewed contract change—never silent `mergeSchema` acceptance in production. (Ch. 4, 9)
- ☐ Spark jobs use explicit schemas (no inference on scheduled jobs) and are parameterized by processing date so the retry unit is a batch window. (Ch. 3)

- ☐ Custom pools and pinned environments—not interactive installs—define production Spark dependencies. (Ch. 3)

D.2 Warehouse

- ☐ Bulk loads use `COPY INTO` from staged, date-partitioned paths with one manifest/ledger row per batch. (Ch. 5)
- ☐ Aggregates are reconciled against the source profile after every load; row counts are evidence, not vibes. (Ch. 5)
- ☐ SCD2 merges run expire-then-insert in the correct order and are idempotent: a second run changes nothing. (Ch. 7)
- ☐ Stored procedures and views—not ad hoc queries—own the transformation logic in SQL-centric estates. (Ch. 5, 6)
- ☐ Pay-as-you-go dev/test capacities pause on a schedule outside business hours. (Ch. 5, 23)
- ☐ The choice of Warehouse versus Lakehouse for each Gold product is written down with its reason (SQL-first consumers, finance orientation, or open-format multi-engine access). (Ch. 2)

D.3 Pipelines and Dataflows

- ☐ Ingestion is metadata-driven: onboarding a new source is an `INSERT` into the config table, not a new pipeline. (Ch. 9)
- ☐ ForEach parallelism is bounded; unbounded fan-out against a source is a reviewed exception. (Ch. 9)
- ☐ A contract-validation gate rejects breaking schema changes (renames, drops, tightened nullability) and allows additive ones; both paths are tested. (Ch. 9)
- ☐ Watermarks advance only on success, so reruns replay exactly the missing slice with no duplicates and no gaps. (Ch. 9, 11, 12)
- ☐ Dataflow outputs stage first; quality-gate assertions run between staging and Gold, and the gate is actually wired into the pipeline. (Ch. 10)
- ☐ On Failure paths (not On Completion) carry alerts and compensating actions; publishing waits for unambiguous success. (Ch. 12)
- ☐ Every alert carries a runbook link: rerun steps, backfill path, escalation owner. (Ch. 12)
- ☐ Gateways are clustered (2+ nodes), run on managed hosts with an inventory of driver versions, and authenticate with least-privilege identities. (Ch. 11)
- ☐ The gateway-down drill has been executed and the rerun proven duplicate-free. (Ch. 11)
- ☐ One ledger row per pipeline outcome exists for forensics and audit. (Ch. 12)

D.4 Real-Time Intelligence

- ☐ Producers stamp every event with a unique ID before the first send; the connection string lives in Key Vault, never in code. (Ch. 13)

- ☐ Deduplication (`arg_max` by event ID or equivalent) runs at the first queryable boundary, before any rule or aggregate consumes the stream. (Ch. 13, 14)
- ☐ The replay window of the source exceeds the slowest realistic recovery; the number is documented. (Ch. 13)
- ☐ Every KQL query and materialized view is time-bounded; unbounded lookbacks are rejected in review. (Ch. 14)
- ☐ Late-arrival policy is stated in writing for every windowed join. (Ch. 14)
- ☐ Reflex rules use sustained conditions (not single spikes), read deduped data, and have cooldowns plus a resolved notification. (Ch. 15)
- ☐ Every production dashboard shows a freshness indicator (last-event age). (Ch. 16)
- ☐ Refresh cadence follows decision cadence, not platform maximum; the cost of wall monitors at the chosen cadence is computed. (Ch. 16)
- ☐ Reverse-ETL syncs upsert on a stable alternate key and are idempotent on rerun; freshness versus cost per entity is agreed with the business owner in writing. (Ch. 16)
- ☐ The replay drill produces zero duplicate alerts. (Ch. 13, 15)

D.5 Power BI and Semantic Models

- ☐ The fact table grain is declared before any measure is written; two reports cannot produce two “correct” revenue numbers. (Ch. 7)
- ☐ Dimensions are conformed—shared keys, identical business meaning across facts. (Ch. 7)
- ☐ Storage mode (Import, Direct Lake, DirectQuery) is chosen per table with a recorded rationale; fallback behavior is understood and monitored. (Ch. 8)
- ☐ Direct Lake tables are V-Ordered; fallback events are watched in the Capacity Metrics app, not discovered by users. (Ch. 8)
- ☐ Relationships are surrogate-key one-to-many; circular dependencies are factored through bridge tables. (Ch. 7)
- ☐ RLS is tested with `EXECUTE AS USER` (or View As) for every persona on every engine path—including the Spark path—never only as the owner. (Ch. 6, 21)
- ☐ Certified semantic models are the only sources for executive reporting; endorsement is visible to consumers. (Ch. 7, 22)

D.6 Machine Learning

- ☐ Features live in governed Delta tables with as-of dating; no training on ad hoc CSV extracts. (Ch. 17)
- ☐ Every feature is rebuildable for any historical date; leakage tests (temporal, preprocessing, target) exist and run. (Ch. 17)
- ☐ Every training run is tracked with parameters, metrics, dataset version or as-of date, and code commit. (Ch. 18)
- ☐ Run comparisons filter to a common dataset snapshot; cross-snapshot “improvements” are discarded. (Ch. 18)

- ☐ Models are consumed from the registry by alias; rollback is an alias move, not a code change. (Ch. 18)
- ☐ AI-service credentials resolve from Key Vault or managed identity at run time—never literals in notebooks that Git will sync. (Ch. 19)
- ☐ Enrichment calls are batched, with measured cost per thousand rows before scale-up. (Ch. 19)
- ☐ Every scored row carries `model_version` and `scored_ts`; scoring writes are partition-idempotent. (Ch. 19, 20)
- ☐ A distribution gate (PSI/KS against training baselines) runs on every scoring batch and is proven to catch injected skew. (Ch. 20)
- ☐ Drift triggers retraining and evaluation automatically, but promotion requires metric evidence and human review. (Ch. 20)

D.7 Purview, Security, and Compliance

- ☐ Workspace roles follow least privilege: Contributor for builders, Viewer for consumers; item-level grants expose exactly the products needed. (Ch. 21)
- ☐ OneLake data access roles close the storage-plane bypass so the Spark path enforces what the SQL path enforces. (Ch. 21)
- ☐ Service principals are scoped one-per-workload-family; no broad shared automation identity. (Ch. 21)
- ☐ Privilege-creep audits run quarterly: every grant maps to a ticket or owner, and the unjustified are revoked. (Ch. 21)
- ☐ Sensitivity labels are applied at the source and inheritance downstream is verified, not assumed. (Ch. 22)
- ☐ Lineage is captured and consulted for impact analysis before any schema change or retirement. (Ch. 22)
- ☐ GDPR erasure has a tested strategy per data class: coordinated `DELETE + VACUUM` where history must die, or crypto-shredding with keys in Key Vault where it must die everywhere at once. (Ch. 22)
- ☐ Legal holds are recorded as documented exceptions with erasure executing on release. (Ch. 22)
- ☐ Dev workspaces are Git-bound; promotion flows only through deployment pipelines with reviewed rules. (Ch. 23)
- ☐ The clean-environment rehydration from Git has been executed or its exact steps documented. (Ch. 23)
- ☐ The DR plan names its mechanisms explicitly: Git rehydration for definitions, Delta time travel for recent data, deliberate dual-region design where the business requires it—with per-workload pricing. (Ch. 23)
- ☐ Secret scans run on the connected repository and are clean. (Ch. 23)
- ☐ Capacity throttling behavior (smoothing, delay, rejection) is monitored; surge protection reserves interactive headroom. (Ch. 23)

Tip

Turn these lists into your organization's promotion gate: a workload does not move to production until its domain's checklist is complete *with evidence attached*. The checklist then doubles as the audit rubric and as the onboarding document for the next engineer who inherits the estate.

DP-600 Exam Mapping and Study Guide

This appendix maps the DP-600 *Implementing Analytics Solutions Using Microsoft Fabric* exam to the chapters of this book, then turns that mapping into a concrete study strategy and self-assessment checklist. Chapter 24 runs this process as a guided week; this appendix is the standalone reference version. Always verify domains and weights against the live skills outline—it evolves as Fabric evolves [33].

E.1 Exam Profile

- **Audience.** Analytics engineers who design, build, and operate Fabric analytics solutions: lakehouses, warehouses, pipelines, real-time intelligence, and semantic models.
- **Format.** Scenario-based multiple choice plus case studies; expect “which technology / which setting / what happens next” questions, not definitions.
- **Hands-on value.** High. The exam’s scenarios mirror this book’s labs—Direct Lake fallback triage, gateway placement, RLS verification, capacity throttling—which is why remediation below means *rebuilding*, not rereading.
- **Adjacent certifications.** PL-300 (Power BI Data Analyst) for report-first careers; the DP-203 lineage for Azure-data-engineering context.

E.2 Domain-to-Chapter Map

The skills outline groups objectives into weighted domains. Table E.1 maps each to the chapters that teach it; the paraphrase follows Chapter 24.

Skills area (paraphrased)	This book	Typical weight
Plan, implement, and manage an analytics solution (capacity, workspace, governance, CI/CD)	Chapters 1, 21–23	Largest single share
Prepare and serve data (lakehouse, warehouse, pipelines, dataflows, notebooks)	Chapters 2–5, 9–12	Large
Implement and manage semantic models (modeling, DAX, Direct Lake)	Chapters 6–8	Large
Real-time intelligence (eventstreams, KQL, activator)	Chapters 13–16	Moderate and growing
Data science and ML touchpoints	Chapters 17–20	Smaller but present

Table E.1: DP-600 domains mapped to chapters (from Chapter 24). Verify current weights against the live skills outline [33].

E.3 Objective-Level Mapping

Within each domain, the objectives below are the ones candidates most often under-prepare. Each entry names the chapter section whose lab builds the tested judgment.

E.3.1 Plan, Implement, and Manage an Analytics Solution

- Capacity SKUs, CU metering, smoothing and throttling behavior, surge protection — Ch. 1 (capacities), Ch. 23 (Capacity Metrics app, throttling stages).
- Workspace design, domains, roles, and item-level sharing — Ch. 1, Ch. 21.
- Git integration, deployment pipelines, and deployment rules — Ch. 23.
- Sensitivity labels, endorsement, lineage, and Purview integration — Ch. 22.
- Disaster recovery expectations: Git rehydration, Delta time travel, no automatic cross-region failover — Ch. 23.

E.3.2 Prepare and Serve Data

- Lakehouse versus Warehouse decision criteria — Ch. 2, Ch. 5.
- Shortcuts versus copy/materialization — Ch. 1.
- Spark pools, environments, notebooks, and Delta optimization (OPTIMIZE, V-Order, Z-Order, VACUUM, time travel) — Ch. 3, 4.
- COPY INTO, T-SQL transformation, three-part cross-item queries — Ch. 5.
- Pipeline activities (Lookup, ForEach, If, Until), parameterization, scheduling — Ch. 9.
- Dataflows Gen2 destinations, staging, incremental refresh — Ch. 10.
- On-premises and VNet data gateways — Ch. 11.
- Orchestration patterns, Monitoring Hub, alerting — Ch. 12.

E.3.3 Implement and Manage Semantic Models

- Star-schema modeling, relationships, bridge tables, SCD handling — Ch. 7.
- DAX measures and calculation discipline — Ch. 7.

- Storage modes: Import, DirectQuery, Direct Lake; framing; fallback triggers and their telemetry — Ch. 8.
- SQL analytics endpoint objects: views, TVFs, RLS, column-level security — Ch. 6.

E.3.4 Real-Time Intelligence

- Eventstream sources, destinations, and fan-out topology — Ch. 13.
- Eventhouses, KQL databases, materialized views, dedup idioms — Ch. 14.
- Data Activator (Reflex) triggers from streams and Power BI visuals — Ch. 15.
- Real-Time Dashboards and automatic page refresh — Ch. 16.

E.3.5 Data Science and ML Touchpoints

- Notebooks, Data Wrangler, and feature tables — Ch. 17.
- Built-in MLflow experiments and the model registry — Ch. 18.
- SynapseML and Azure AI service integration — Ch. 19.
- Batch scoring with PREDICT and pipeline-orchestrated inference — Ch. 20.

E.4 Study Strategy

1. **Build the readiness matrix first.** One row per domain, columns for evidence (which chapter labs you completed, which capstone artifacts exist) and confidence. Domains with neither are the plan. (Ch. 24, Monday.)
2. **Study to the outline, not the table of contents.** Re-download the skills outline in exam week and diff it against your matrix; Fabric evolves faster than any book.
3. **Take two timed, closed-book mock exams** scored by domain. Classify every miss as knowledge (rebuild the lab), misread (fix pacing), or judgment (re-derive the Friday use cases). (Ch. 24, Wednesday/Friday.)
4. **Remediate weakest domain first, by building.** Recognition memory from rereading produces confidence without capability; the exam tests scenario decisions.
5. **Memorize the decision tables:** Lakehouse vs. Warehouse (Ch. 2, 5), Import vs. Direct Lake vs. DirectQuery (Ch. 8), gateway types (Ch. 11), Eventstream destinations (Ch. 13), batch vs. streaming (Ch. 13, 20).
6. **Use the readiness threshold:** two consecutive full-length mocks at 80%+ with no domain below 70%, plus the ability to explain why each wrong option is wrong.

E.5 Self-Assessment Checklist

You are ready when you can honestly tick every line—with an artifact or a rebuilt lab as evidence for each.

Platform and Administration

- ☐ I can explain CU metering, smoothing, and the throttling stages, and read them in the Capacity Metrics app. (Ch. 1, 23)

- ☐ I can design a dev/test/prod workspace topology with Git binding and deployment rules. (Ch. 23)
- ☐ I can state what Git rehydration does and does not recover. (Ch. 23)

Data Preparation and Serving

- ☐ I can choose Lakehouse vs. Warehouse for a stated workload and defend the choice. (Ch. 2, 5)
- ☐ I can write and troubleshoot `COPY INTO`, including OneLake path forms. (Ch. 5, Appendix B)
- ☐ I can schedule `OPTIMIZE/V-Order` and explain `VACUUM` retention risk. (Ch. 4)
- ☐ I can build a metadata-driven pipeline with a contract gate. (Ch. 9)
- ☐ I can pick the correct gateway type for an on-prem or VNet source. (Ch. 11)
- ☐ I can wire On Failure alerting with a runbook link. (Ch. 12)

Semantic Models

- ☐ I can model a star schema with declared grain and conformed dimensions. (Ch. 7)
- ☐ I can name Direct Lake fallback triggers and find them in telemetry. (Ch. 8)
- ☐ I can implement and test RLS as each persona. (Ch. 6, 21)

Real-Time Intelligence

- ☐ I can route an eventstream to KQL and lakehouse destinations and justify the fan-out. (Ch. 13)
- ☐ I can write the `arg_max` dedup idiom and a materialized view. (Ch. 14)
- ☐ I can configure a sustained-threshold Reflex alert that fires once. (Ch. 15)

ML Touchpoints

- ☐ I can track an experiment and load a model by registry alias. (Ch. 18)
- ☐ I can explain where SynapseML fits and how credentials are vaulted. (Ch. 19)
- ☐ I can describe batch scoring with `model_version` auditing. (Ch. 20)

Common Pitfall

The most common DP-600 failure pattern is over-weighting the domains you already know. The readiness matrix exists to prevent exactly that: let evidence and confidence—not comfort—set the study order.

Interview Question Bank

This appendix consolidates the Interview Q&A sections of Chapters 1–24 into one categorized bank, organized by domain and difficulty. Answers are abbreviated to the defensible core; the chapter cross-reference after each answer points to the full treatment. Drill them out loud: the exam and the interview both reward the candidate who reasons from trade-offs, not the one who recites features.

Tip

Difficulty legend: **F** = foundational (vocabulary and first principles), **I** = intermediate (design choices and trade-offs), **A** = advanced (failure modes, governance, and judgment under ambiguity).

F.1 Core Infrastructure: OneLake, Medallion, Spark, Delta (Chapters 1–4)

Foundational

- 1. Q (F): Why is OneLake described as OneDrive for Data?**
A: A single logical tenant-wide namespace with shared platform controls, instead of per-team storage accounts. (Ch. 1)
- 2. Q (F): What is the relationship between Parquet and Delta Lake?**
A: Parquet stores columnar files; Delta adds the transaction log and metadata that make them behave like reliable tables for concurrent engines. (Ch. 1)
- 3. Q (F): What are the two major physical components of a Delta table?**
A: Parquet data files plus the `_delta_log` transaction log recording versions, add/remove actions, and schema metadata. (Ch. 4)
- 4. Q (F): Starter pool versus Custom pool in Fabric Spark?**
A: Starter: fast, managed notebook startup with minimal configuration. Custom: explicit control of node family, size, and scale for repeatable workloads. (Ch. 3)
- 5. Q (F): What belongs in Bronze?**
A: Raw source data plus ingestion metadata—paths, batch IDs—sufficient to replay or audit ingestion. (Ch. 2)

Intermediate

1. **Q (I): When would you use a shortcut instead of copying data?**

A: When the source is large, externally owned, or frequently reused; copy when you need curated quality, stable schemas, and certified consumption. (Ch. 1)

2. **Q (I): What makes Silver different from Bronze, and Gold from Silver?**

A: Silver adds schema enforcement, typing, dedup, and quality checks; Gold adds business definitions, aggregations, dimensional models, and certification. (Ch. 2)

3. **Q (I): When should you avoid schema inference?**

A: In large or scheduled jobs—it adds scan cost and lets unexpected files shift inferred types. (Ch. 3)

4. **Q (I): How is V-Order different from Z-Ordering?**

A: V-Order is Fabric's write-time Parquet layout optimization for analytical reads; Z-Ordering clusters rows by chosen columns to improve data skipping on filtered queries. (Ch. 4)

5. **Q (I): Why should a BI report never point directly at a raw shortcut?**

A: Raw data lacks semantic standardization, quality checks, privacy treatment, and performance guarantees—reports consume certified Gold. (Ch. 1)

Advanced

1. **Q (A): What risk does VACUUM introduce?**

A: It permanently removes unreferenced files, destroying the versions time travel or recovery might still need—never vacuum before validating recovery. (Ch. 4)

2. **Q (A): What is the safest retry unit for a daily pipeline?**

A: A specific processing date or batch window: the job rereads the same Bronze files and replaces only the corresponding Silver scope. (Ch. 3)

3. **Q (A): How should schema evolution be governed?**

A: As an intentional contract change—review the new field, test downstream impact, document the schema; never silently accept drift. (Ch. 4)

4. **Q (A): Why might a bank split medallion layers across workspaces?**

A: Stronger access boundaries, limited raw-data exposure, smaller blast radius, and audited promotion from raw logs to curated reports. (Ch. 2)

F.2 Warehousing and BI: Warehouse, SQL Endpoint, Modeling, Direct Lake (Chapters 5–8)

Foundational

1. **Q (F): What is the unit of Fabric capacity billing?**

A: The capacity unit (CU); consumption is metered in CU-seconds across all workloads sharing the capacity. (Ch. 5)

2. **Q (F): What does the Fabric warehouse store physically?**

A: Delta-Parquet files in OneLake, in a managed area owned by the warehouse item—open storage other engines can also read. (Ch. 5)

3. **Q (F): Name the three Kimball fact table types.**

A: Transaction facts, periodic snapshots, and accumulating snapshots. (Ch. 7)

4. **Q (F): How does RLS differ from object-level security?**

A: Object-level controls *which objects* a user sees; RLS controls *which rows* within a permitted object, via a filter predicate. (Ch. 6)

Intermediate1. **Q (I): When is COPY INTO preferable to a pipeline Copy activity?**

A: When files are already staged and the team wants set-based, versionable, idempotent SQL loads—classic for stored-procedure estates. (Ch. 5)

2. **Q (I): When is SCD Type 2 required instead of Type 1?**

A: When the business must answer “as it was” questions—attribution, audit, any report reflecting attribute values in effect when the fact occurred. (Ch. 7)

3. **Q (I): What triggers a Direct Lake fallback to DirectQuery?**

A: Row-count guardrails for the capacity, unsupported column types or query shapes, queries on views, or memory pressure—each silently reroutes that query. (Ch. 8)

4. **Q (I): When is Import mode still preferable to Direct Lake?**

A: When the source lives outside OneLake, is too slow for live column reads, or the model needs Import-only DAX features. (Ch. 8)

5. **Q (I): Which object reuses a join across reports?**

A: A view—it names the join once and becomes the stabilizing contract for every consumer. (Ch. 6)

Advanced1. **Q (A): Why does compute–storage separation matter for a warehouse?**

A: Storage scales with OneLake while compute scales with SKUs; data survives paused compute; the same Delta files open to other engines without export pipelines. (Ch. 5)

2. **Q (A): How does Direct Lake achieve freshness without refresh?**

A: It reads the same Delta files pipelines write; framing re-points the model at new committed versions—freshness after the next frame, not the next import window. (Ch. 8)

3. **Q (A): Which telemetry reveals fallback events in production?**

A: The Capacity Metrics app—DirectQuery-mode query load where lake reads were expected. Fallback is silent in the report but visible in telemetry. (Ch. 8, 24)

4. **Q (A): What is a logical data warehouse?**

A: Querying lake data in place through a governed SQL object layer—views, TVFs, RLS—without physically moving data into a separate warehouse store. (Ch. 6)

5. **Q (A): How does a bridge table resolve a circular dependency?**

A: It factors the many-to-many heart of the loop into its own table, converting every filter path into composable one-to-many hops. (Ch. 7)

F.3 ETL and Orchestration: Pipelines, Dataflows, Connectivity, Monitoring (Chapters 9–12)**Foundational**1. **Q (F): What makes an ingestion framework metadata-driven?**

A: Work items come from a config table a parameterized pipeline reads at run time—onboarding a source is an INSERT, not a new pipeline. (Ch. 9)

2. **Q (F): Which activity iterates over a Lookup result set?**

A: ForEach over the Lookup's `output.value`; parallelism is opt-in and should be bounded. (Ch. 9)

3. **Q (F): What is an output destination in Dataflows Gen2?**

A: The setting that publishes a query's result into lakehouse, warehouse, or Azure SQL—turning the dataflow into a general ingestion and transformation tool. (Ch. 10)

4. **Q (F): Why cluster gateway nodes for production?**

A: A single node is a single point of failure for every hybrid pipeline; clusters add HA during patches plus load distribution. (Ch. 11)

Intermediate

1. **Q (I): What distinguishes a breaking from a non-breaking schema change?**

A: Breaking removes or renames what consumers depend on; non-breaking only adds. The contract gate rejects the first, allows the second. (Ch. 9)

2. **Q (I): Why stage data before writing to a destination?**

A: Staging separates computation from publication so quality gates can run between them; a direct-to-Gold flow has no point at which bad rows can be stopped. (Ch. 10)

3. **Q (I): Why quarantine bad rows instead of failing the batch?**

A: A handful of bad rows should not break the SLA for millions of good ones; quarantine preserves the batch, records violations, keeps Gold clean. (Ch. 10)

4. **Q (I): On Failure versus On Completion paths?**

A: On Failure fires only on predecessor failure (alerts, compensation); On Completion fires either way (unconditional cleanup). Publishing on On Completion propagates partial failures. (Ch. 12)

5. **Q (I): When do you need a VNet gateway instead of an on-premises gateway?**

A: When sources live inside an Azure VNet and you want Microsoft-managed gateway compute without maintaining a Windows host. (Ch. 11)

Advanced

1. **Q (A): Why should watermarks advance only on success?**

A: The watermark defines what the next run reads; advancing on failure silently skips uncommitted data, advancing on success makes rerun safe and duplicates impossible. (Ch. 9)

2. **Q (A): How do you make a rerun of a failed batch idempotent?**

A: Watermarks advance only on success, writes are merge- or partition-scoped, and the batch re-executes exactly its missing slice. (Ch. 12)

3. **Q (A): How does a gateway keep data inside the network perimeter?**

A: It initiates every connection outbound over TLS to the Azure relay—only outbound 443 traffic, no inbound rule or public endpoint. (Ch. 11)

4. **Q (A): Why attach a runbook link to every alert?**

A: The alert reaches a half-awake human at 3 a.m.; the runbook turns investigation into procedure—rerun steps, backfill path, escalation owner. (Ch. 12)

5. **Q (A): Why are dependencies better than wall-clock schedules?**

A: Clocks encode duration assumptions that fail on slow nights; completion conditions encode actual readiness, so the graph self-adjusts. (Ch. 12)

F.4 Real-Time Intelligence: Eventstreams, KQL, Reflex, Dashboards (Chapters 13–16)

Foundational

1. **Q (F): Name two Eventstream sources and one destination.**
A: Azure Event Hubs and custom app endpoints as sources; KQL databases, lakehouses, and Reflex items as destinations. (Ch. 13)
2. **Q (F): Which three parts define a Data Activator trigger?**
A: The data watched (with identity granularity), the condition evaluated, and the action fired. (Ch. 15)
3. **Q (F): What is reverse ETL?**
A: Writing curated analytical outputs—segments, scores—back into operational systems where people act. (Ch. 16)
4. **Q (F): What does the Event Hubs retention (replay) window control?**
A: How far back a consumer can reprocess after failure; recovery beyond the window loses data, so size it beyond your slowest realistic recovery. (Ch. 13)

Intermediate

1. **Q (I): Why stamp every event with a unique ID?**
A: At-least-once delivery means duplicates; the producer-stamped ID is the only trustworthy dedup key—IDs minted mid-pipeline cannot distinguish a retry from a new event. (Ch. 13)
2. **Q (I): When route events to KQL versus a lakehouse?**
A: KQL for live analytics and Reflex (queryable in seconds); lakehouse for durable open storage and batch integration. Most architectures fan out to both. (Ch. 13)
3. **Q (I): Why require “3 consecutive minutes” instead of a single spike?**
A: Sustained conditions filter transient noise, converting a flapping siren into a signal humans keep answering. (Ch. 15)
4. **Q (I): What does `arg_max(ingestion_time(), *) by EventId` do?**
A: Collapses at-least-once redeliveries to the latest-arrived row per logical event, keyed by the producer-stamped ID. (Ch. 14)
5. **Q (I): Why does every real-time dashboard need a freshness indicator?**
A: A frozen dashboard is indistinguishable from a healthy system; last-event age separates “all quiet” from “all broken.” (Ch. 16)

Advanced

1. **Q (A): What does at-least-once delivery imply for consumers?**
A: Every consumer must be idempotent: dedupe on the producer-stamped ID at the first queryable boundary and checkpoint progress so restarts resume instead of replay. (Ch. 13)
2. **Q (A): Why do unbounded lookback windows degrade performance?**
A: They abandon incremental state—every query and view update re-scans ever-growing history, so cost grows with data age rather than workload. (Ch. 14)
3. **Q (A): How do late arrivals affect a time-windowed stream join?**
A: Events arriving after the window closes never match; the window must reflect measured lateness, with a written policy for unmatched events. (Ch. 14)

4. **Q (A): Why must Reflex rules read deduplicated data?**

A: A rule over raw duplicate events double-fires—paging humans twice for one event. (Ch. 15)

5. **Q (A): What does automatic page refresh cost in DirectQuery mode?**

A: Real query load per refresh per open report instance—forty wall monitors at ten seconds is four queries per second forever. Cadence follows decision cadence. (Ch. 16)

F.5 Data Science and ML: Foundations, MLflow, SynapseML, MLOps (Chapters 17–20)

Foundational

1. **Q (F): What does Data Wrangler generate from visual cleaning steps?**

A: Executable PySpark/Pandas code—the visual session is exploration; the exported, parameterized code is the engineering artifact. (Ch. 17)

2. **Q (F): What does `mlflow.autolog()` capture per run?**

A: Framework-specific parameters, metrics, and the model artifact, automatically; you add business metrics and metadata tags explicitly. (Ch. 18)

3. **Q (F): What does SynapseML add on top of Spark MLlib?**

A: Distributed transformers for Azure AI services (OpenAI, Cognitive Services), learners such as LightGBM, and serving utilities. (Ch. 19)

4. **Q (F): What does the Spark PREDICT/UDF pattern do?**

A: Applies a registered model as a distributed function over a DataFrame—set-based inference with results stamped and written back to Delta. (Ch. 20)

Intermediate

1. **Q (I): Name two leakage risks in feature engineering.**

A: Temporal leakage (windows computed past the as-of date) and preprocessing leakage (scalers fit before the train/test split). (Ch. 17)

2. **Q (I): Why register a model instead of referencing run artifacts?**

A: The registry makes versions immutable, decouples consumers through aliases, and centralizes the promotion audit trail. (Ch. 18)

3. **Q (I): Why batch AI-service calls instead of row-by-row?**

A: Per-call latency and per-request pricing amortize across rows; bounded concurrency respects rate limits without serializing the cluster. (Ch. 19)

4. **Q (I): Which signal indicates data drift in production?**

A: Distribution shift—PSI or KS statistics on production features versus training baselines—plus slower-arriving outcome decay. (Ch. 20)

Advanced

1. **Q (A): Why must training features be rebuildable for any historical date?**

A: A feature computed “as of now” for a historical row trains on the future; explicit as-of dating makes backtests honest and leakage testable. (Ch. 17)

2. **Q (A): Why must run comparisons filter to a common dataset snapshot?**

A: Runs on different snapshots differ in model and data; without a fixed snapshot you measure data drift and call it model quality. (Ch. 18)

3. **Q (A): How do you avoid hardcoding Azure OpenAI keys in a notebook?**
A: Managed identity where supported, otherwise Key Vault references resolved at run time—Git integration will eventually sync the notebook verbatim. (Ch. 19)
4. **Q (A): Why does drift trigger retraining but not promotion?**
A: Drift can signal a broken upstream feed as easily as a changed world; retraining is automated, but promotion waits for metric evidence and human review. (Ch. 20)
5. **Q (A): Which schema contract must scoring output honor?**
A: Entity key, score, `model_version`, `scored_ts`—the published columns reports and reverse ETL depend on. (Ch. 20)

F.6 Governance, Security, Administration, and Certification (Chapters 21–24)

Foundational

1. **Q (F): Contributor versus Viewer workspace roles?**
A: Contributors create and modify items; Viewers consume without altering definitions. The distinction is change authority, not seniority. (Ch. 21)
2. **Q (F): Promoted versus Certified endorsement?**
A: Promoted is the owner's vouch; Certified is the organization's official approval against published criteria. (Ch. 22)
3. **Q (F): Why pause non-production capacities on a schedule?**
A: Per-second pay-as-you-go billing makes idle capacity pure waste; schedules remove human memory from the loop. (Ch. 23)
4. **Q (F): Which DP-600 domains weigh most?**
A: Planning/managing the analytics solution, preparing and serving data, and semantic modeling—verify current weights against the live skills outline. (Ch. 24)

Intermediate

1. **Q (I): Why prefer item-level permissions over workspace-level grants?**
A: A workspace grant exposes every item—drafts, staging, experiments; item-level grants expose exactly the product the consumer needs. (Ch. 21)
2. **Q (I): What does a sensitivity label pass downstream?**
A: The classification itself—labels inherit through lineage to models and reports; inheritance is audited, not assumed. (Ch. 22)
3. **Q (I): Why is Git integration the recovery mechanism for workspace items?**
A: Fabric has no traditional backup of workspace metadata; the repository holds item definitions, so reconnecting rehydrates the estate. (Ch. 23)
4. **Q (I): When does Direct Lake beat Import in exam scenarios?**
A: Data in OneLake Delta, near-real-time freshness without refresh windows, volumes within capacity guardrails; Import wins for external/slow sources. (Ch. 24)
5. **Q (I): What does surge protection guard against?**
A: Background storms consuming interactive capacity—a runaway job degrading the dashboards executives watch. (Ch. 23)

Advanced

1. **Q (A): Why is row-level DELETE insufficient for GDPR erasure in Fabric?**
A: Delta history keeps rows until VACUUM, KQL retains per policy, caches hold copies, backups restore them—DELETE touches one layer's current state only. (Ch. 22)
2. **Q (A): How does crypto-shredding sidestep that?**
A: PII encrypted per subject with keys in Key Vault; deleting the key renders every copy—current, history, cache, backup—unreadable in one auditable act. (Ch. 22)
3. **Q (A): What is privilege creep, and how do you audit it?**
A: Accumulation of broader-than-needed grants over time. Quarterly: enumerate grants, map each to a ticket or owner, revoke the unjustified. (Ch. 21)
4. **Q (A): Why no automatic cross-region failover for OneLake?**
A: OneLake data is not automatically replicated; continuity requires deliberate dual-workspace/dual-capacity designs, chosen and priced per workload. (Ch. 23)
5. **Q (A): Walk through expand–migrate–contract for renaming a column.**
A: Add the new nullable column (expand); dual-write and backfill with view shims (migrate); switch consumers, then drop the old column after a full release cycle (contract). (Ch. 23)
6. **Q (A): Why remediate exam weaknesses by building rather than rereading?**
A: The exam tests scenario decisions; muscle memory from labs produces the reasoning, while recognition memory produces confidence without capability. (Ch. 24)

Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the six platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, and Snowflake. The names differ, but the underlying data-engineering concepts—durable object storage, tiered lifecycle economics, immutability, managed relational engines, elastic compute, and disciplined data organization—recur everywhere.

Note

This appendix grows with each chapter. Every new chapter contributes the equivalence tables introduced in its cross-cloud callout, so by the end of the book you will hold a single consolidated Rosetta Stone for moving fluently between clouds. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

G.1 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Re-retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Encryption at rest	SSE-S3 / SSE-KMS	SSE + CMK (Key Vault)	Google-managed / CMEK

The three remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in cloud object storage as immutable, columnar *micro-partitions* that the service manages on your

behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers.

G.2 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

Capability	AWS	Azure	GCP
Managed relational	Amazon RDS	Azure SQL Database	Cloud SQL
Cloud-native engine	Amazon Aurora	SQL Hyperscale	AlloyDB
Managed PostgreSQL	RDS / Aurora PostgreSQL	Azure DB for PostgreSQL	Cloud SQL / AlloyDB
High availability	Multi-AZ	Zone-redundant HA	Regional HA
Read scaling	Read Replicas	Read replicas / geo-replicas	Read Replicas
Global / geo	Aurora Database Global	Active geo-replication	Cross-region replicas
Serverless compute	Aurora Serverless v2	Azure SQL Serverless	AlloyDB (elastic)
Migration service	DMS + SCT	Azure Database Migration Svc	Database Migration Svc

Snowflake, **Fabric**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling.

G.3 Elastic Compute and Analytical Warehouses

Snowflake’s Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

Concept	Snowflake	AWS	Azure / GCP
Compute unit	Virtual Warehouse	Redshift RA3 / Serverless	Synapse pool / Big-Query slots
Elastic concurrency	Multi-cluster warehouse	Concurrency scaling	Autoscale / on-demand
Pause when idle	Auto-suspend / resume	Pause / resume	Auto-pause / serverless
Billing unit	Credits	RPU / node-hours	DWU / slot-hours
Result reuse	Result cache	Result cache	Result / BI cache

Microsoft Fabric meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage.

G.4 Data Organization and the Medallion Pattern

Fabric’s Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

Layer	Fabric	Data (AWS/Azure/GCP)	lake	Snowflake
Bronze (raw)	Bronze Lakehouse	raw/ (S3/ADLS/GCS)	prefix	RAW schema / stage
Silver (validated)	Silver Lakehouse	validated/ prefix		STAGING schema
Gold (curated)	Gold Lakehouse	curated/ prefix		ANALYTICS schema
Storage format	Delta-Parquet	Parquet / Delta / Ice- berg		Micro-partitions

MongoDB realizes the same separation with distinct collections or databases (for example **raw_events**, **validated**, **curated**) governed by schema-design patterns rather than a lake layout.

G.5 Document Data Modeling

MongoDB’s Chapter 2 schema-design patterns generalize to every document and wide-column store.

Concept	MongoDB	Amazon nomoDB	Dy-	Azure Cosmos DB
Primary access unit	Document / collection	Item / table		Item / container
Partitioning	Shard key	Partition key		Partition key
One-table modeling	Embedding + patterns	Single-table design		Embedding + parti- tioning
Time-series grouping	Bucket Pattern	Composite sort keys		Bucketing by key
Optional attributes	Attribute Pattern	Sparse attributes		Sparse properties

Google Cloud’s closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in **VARIANT** (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections.

G.6 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

Concept	DynamoDB	Cosmos DB	Cloud Spanner	MongoDB
Model	Key-value / doc	Multi-model	Distributed SQL	Document
Partitioning	Partition key	Partition key	Split by key range	Shard key
Throughput unit	RCU / WCU	Request Units (RU)	Nodes / processing units	Cluster tier
Secondary indexes	GSI / LSI	Automatic indexing	Secondary indexes	Indexes
Change stream	DynamoDB Streams	Change Feed	Change streams	Change streams
Expiry	TTL	TTL	(manual)	TTL index
Multi-region	Global Tables	Multi-region writes	Multi-region (syn- chronous)	Zones / global clusters
Consistency	Eventual / strong	5 tunable levels	External (True- Time)	Tunable read/write concern

Snowflake and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in **VARIANT**/JSON columns. Google’s dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column).

G.7 Managed Apache Spark

Microsoft Fabric’s Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

Capability	Fabric	AWS	Azure / GCP
Managed Spark	Starter / Custom pools	Glue / EMR / EMR Serverless	Synapse Spark / Databricks; Dataproc
Notebook UX	Fabric Notebooks	Glue Studio / EMR Studio	Synapse / Databricks; Vertex AI
Serverless option	Autoscaling pools	Glue / EMR Serverless	Synapse serverless; Dataproc Serverless
Lake format	Delta on OneLake	Delta / Iceberg / Hudi	Delta; Delta / Iceberg

Snowflake offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector.

G.8 Query Acceleration and Materialized Views

Snowflake’s Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

Feature	Snowflake	AWS Redshift	Azure Synapse / GCP BigQuery
Materialized views	Materialized Views	Materialized views	Materialized views
Point-lookup accel.	Search Optimization Svc	Sort keys / zone maps	(indexing) / search indexes
Result reuse	Result cache	Result cache	Result-set cache / BI Engine
Ephemeral tables	Transient / temporary	Temp tables	Temp tables

Fabric accelerates reads through Direct Lake and its Warehouse engine, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set.

G.9 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

Capability	AWS	Azure	GCP / others
Managed cache	ElastiCache (Redis/Memcached)	Cache for Redis	Memorystore (Redis/Memcached)
Real-time API layer	AppSync / API Gateway	API Management / SignalR	API Gateway / Apigee
Hybrid ingestion	DataSync / Snow family / IoT Core	Data Box / IoT Hub / Arc	Transfer Appliance / IoT Core
Streaming intake	Kinesis / MSK	Event Hubs	Pub/Sub

Snowflake and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache.

G.10 Wide-Column and Key-Value NoSQL

GCP’s Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

Characteristic	GCP Bigtable	AWS	Azure / open source
Data model	Wide-column	DynamoDB (key-value/doc)	Cosmos DB (multi-model)
HBase-compatible	HBase API	Keyspaces (Cassandra)	Managed Instance for Cassandra
Scaling unit	Nodes / clusters	On-demand / provisioned	RU/s (Cosmos)
Best fit	Time-series, IoT, ad-tech	Serverless key-value	Global multi-model

Snowflake and **Fabric** address these workloads analytically rather than as operational key-value stores; **MongoDB** covers the document end of the same NoSQL spectrum.

G.11 Data Protection: Cloning, Time Travel, and Migration

Snowflake’s Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB’s relational-to-document migration map to data-protection and migration tooling across the clouds.

Capability	Snowflake	AWS / Azure	GCP / Fabric
Instant clone	Zero-copy clone	Aurora clone / SQL DB copy	BigQuery table clone; OneLake shortcut
Point-in-time recovery	Time Travel	PITR (RDS/SQL DB backups)	PITR; Delta time travel
Retention safety net	Fail-safe (7 days)	Automated backups	Backups; Delta VACUUM window
Schema migration	—	DMS / SCT	Database Migration Service

MongoDB migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, and **Delta Lake** exposes historical versions through `VERSION AS OF` time-travel queries.

G.12 Cloud Data Warehouses

Chapter 5’s topic—decoupled compute over open or managed storage with SQL as the primary interface—is one of the most portable concepts in data engineering.

Capability	Fabric Warehouse	AWS	GCP	Snowflake
Warehouse engine	Synapse Data Warehouse	Amazon Redshift	BigQuery	Snowflake virtual warehouses
Bulk load command	<code>COPY INTO</code>	<code>COPY</code>	<code>LOAD DATA</code> / bq load	<code>COPY INTO <table></code>
Storage model	Delta-Parquet in OneLake	Managed (RA3) or Spectrum over S3	Capacitor columnar storage	Micro-partitions on object storage
Compute scaling	Capacity CUs (F SKUs)	Nodes / serverless workgroups	Slots / on-demand	Warehouse size (XS–6XL)
Cross-db query	Three-part names across items	Redshift Spectrum / federated	Dataset-qualified names	Three-part names, shares
Concurrency model	Shared capacity, smoothing	WLM queues / auto-scaling	Slot reservations	Multi-cluster warehouses

Databricks answers with SQL warehouses (serverless, pro, or classic) over Unity-Catalog-governed Delta tables, and Azure Synapse dedicated SQL pools are the in-family predecessor. **MongoDB** has no warehouse concept; its analytics path runs through the Atlas SQL Interface or export into a columnar engine. The durable lesson: every platform separates *where bytes live* from *who computes over them*, and bills the two differently.

G.13 SQL over the Open Lake

Chapter 6’s read-only SQL surface over open lake storage is the defining lakehouse pattern, present on every cloud with different degrees of file-hiding.

Capability	Fabric	AWS	GCP	Snowflake
SQL over lake files	SQL analytics endpoint (read-only, auto-generated)	Athena over S3; Redshift Spectrum	BigQuery external/BigLake tables	External tables; Iceberg tables
Serverless engine	Yes (per-query on capacity)	Athena (per-query/serverless)	Yes (on-demand or slots)	Virtual warehouses
Row-level security	RLS on SQL endpoint objects	Lake Formation row/cell filters	BigQuery row access policies	Row access policies
Column-level security	GRANT/DENY on columns	Lake Formation column filters	Policy tags	Column masking policies
Open table format	Delta-Parquet in OneLake	Iceberg/Hudi/Delta on S3	Iceberg via BigLake	Iceberg or native

Databricks SQL over Unity Catalog is the closest in-family analog—the same *open files plus governed SQL layer* idea, with the catalog rather than an auto-generated endpoint carrying the governance. **MongoDB** exposes read SQL through the Atlas SQL Interface rather than over lake files.

G.14 Dimensional Modeling and the Semantic Layer

Kimball’s method (Chapter 7) predates every cloud; what differs per platform is where the model lives and which language expresses its measures.

Capability	Fabric	AWS	GCP	Snowflake
Model home	Power BI semantic model	Redshift tables + QuickSight datasets	BigQuery + Looker semantic layer	Snowflake tables + Snowsight/BI tool models
Star-schema DDL	Warehouse/lakehouse Delta tables	Redshift (dist/sort keys)	BigQuery (partition/cluster)	Native tables, clustering
SCD2 implementation	Delta MERGE	MERGE / staging upserts	MERGE	MERGE / streams+tasks
Measure language	DAX	SQL / QuickSight calculated fields	LookML / SQL	SQL / BI tool calcs

Databricks teams model in Delta tables, implement SCD2 with MERGE INTO, and serve through SQL warehouses or external BI tools. **MongoDB** modeling is document-oriented—embedding versus referencing—a different but equally grain-first discipline.

G.15 BI Serving Modes: Import, Live Query, and Direct Lake

Chapter 8’s Direct Lake is Fabric’s distinctive contribution: the semantic engine reads the same open Delta-Parquet files that Spark and SQL use. Other platforms approximate the goal.

Serving pattern	Fabric	AWS	GCP	Snowflake
BI reads open lake files	Direct Lake over OneLake Delta	QuickSight over Athena/Spectrum	Looker/Looker Studio over BigLake	External/Iceberg tables to BI tools
In-memory BI copy	Power BI Import	QuickSight SPICE	Looker in-memory caches	Cached results / extracts in BI tool
Live query to source	DirectQuery (warehouse/endpoint)	QuickSight direct query	Looker live connection	Live connection to Snowflake
Cache/update control	Framing (auto/on-demand)	SPICE refresh schedule	Cache policies	Result cache (24h) + BI caching
Performance lever	V-Order + column pruning	Sort keys + Spectrum tuning	Partition/cluster pruning	Clustering + result cache

Databricks serves BI through SQL warehouses (with Unity Catalog lineage) and AI/BI dashboards; partner BI tools connect live or via extracts. **MongoDB** serves through the Atlas SQL Interface or Atlas Charts. The shared insight: *copy for speed versus query for freshness* is the universal BI trade-off.

G.16 Visual Orchestration and Bulk Ingestion

Chapter 9’s visual, activity-based orchestrator descends directly from Azure Data Factory; every cloud offers the pattern with a rich connector library.

Capability	Fabric	AWS	GCP	Snowflake
Visual orchestrator	Data Factory pipelines	Glue Workflows / Step Functions	Cloud Composer (Airflow) / Data Fusion	Tasks / Snowpark orchestration
Bulk copy	Copy activity (90+ connectors)	Glue ETL / DMS	Data Fusion / Datastream	COPY INTO / connectors
Control flow	ForEach, If, Until, Switch	Step Functions states	Airflow DAG logic	Task DAGs
Parameterization	Parameters, variables, expressions	Step Functions JSON-Path	Airflow templating (Jinja)	Task parameters
Code-first alternative	Notebooks in pipelines	Lambda / Glue scripts	Composer DAGs as code	Snowpark Python

Databricks orchestrates with Workflows (Lakeflow), parameterizing jobs and notebook tasks; **MongoDB** uses Atlas Triggers and Functions. The portable skill is the metadata-driven mindset: config rows enumerate work, and one parameterized pipeline executes it.

G.17 Low-Code Transformation

Chapter 10’s visual query designer (Power Query / M lineage) has an analyst-facing counterpart in every ecosystem.

Capability	Fabric	AWS	GCP	Snowflake
Visual/low-code ETL	Dataflows Gen2 (Power Query)	Glue DataBrew / Glue Studio	Data Fusion / Dataprep heritage	Snowsight worksheets, dbt, partner tools
Output destinations	Lakehouse, warehouse, Azure SQL, more	S3, Redshift, databases	BigQuery, Cloud Storage	Snowflake tables
Incremental refresh	Yes (per query, date-column driven)	Job bookmarks (Glue)	Incremental pipelines (Data Fusion)	Streams + tasks, dynamic tables
Transformation engine	Mashup engine (M)	Spark (Glue Studio) / visual recipes	Data Fusion (Dataproc)	SQL pushdown / Snowpark
Data-quality gating	Notebook assertions / pipeline gates	Glue Data Quality	Dataplex data quality	Snowflake data metric functions

Databricks offers lakehouse-native notebooks and declarative Delta Live Tables rather than a visual designer; **MongoDB**'s Compass aggregation builder plays the analyst-facing role. The portable idea: visual tools lower the authoring barrier, but quality gates and destinations are what make their output production-grade.

G.18 Hybrid Connectivity

Chapter 11's gateways answer a universal enterprise requirement: reach data behind corporate networks without exposing it publicly.

Capability		Fabric		AWS		GCP		Snowflake
On-prem lay/agent	re-	On-premises gateway	data	DMS agents, Data-Sync agents, Storage Gateway		Transfer appliance / Database Migration Service		External access via connectors
VNet/VPC path	private	VNet data gateway		VPC endpoints, PrivateLink		Private Service Connect, VPC peering		PrivateLink / Private Service Connect
Outbound-only pattern		Gateway initiates outbound 443		Agents poll outbound		Agents/connectors outbound		Outbound from customer network
HA model		Gateway clusters (2+ nodes)		Multi-AZ services		Regional redundancy		Multi-AZ by default

Databricks reaches private sources through VPC peering and PrivateLink; **MongoDB Atlas** through private endpoints and VPC peering. The invariant: *the private side initiates the connection outbound*, so no inbound firewall rule ever exposes the database.

G.19 Orchestration Discipline and Monitoring

Chapter 12's dependency graphs, retries, and alerting are the most tool-shaped yet conceptually stable layer of the stack.

Capability		Fabric		AWS		GCP		Snowflake
Orchestrator		Pipelines (master/child)		Step Functions / MWAA (Airflow)		Cloud Composer (Airflow)		Task graphs
Dependency edges		On Success/Failure/Completion/Skip		States, catch/retry		Airflow trigger rules		AFTER task deps
Monitoring		Monitoring Hub + run history		CloudWatch + Step Functions console		Cloud Monitoring / Airflow UI		Task history, resource monitors
Alerting		Teams/Outlook activities, Activator		SNS / EventBridge		Alerting policies		Notifications / alerts
Lineage hooks		Fabric lineage view		OpenLineage (MWAA)		OpenLineage (Composer)		Access history

Databricks Workflows and **MongoDB Atlas Triggers** round out the family. The portable discipline: *an alert without a runbook is noise; a retry without idempotence is corruption*.

G.20 Streaming Ingestion

Chapter 13's durable event buffers with fan-out to analytical and operational sinks exist on every cloud.

Capability	Fabric	AWS	GCP	Snowflake
Event ingestion	Eventstreams (Event Hubs lineage)	Kinesis / MSK (Kafka)	Pub/Sub	Snowpipe Streaming
Custom producer endpoint	Custom app source	Kinesis PutRecords / Kafka producer	Pub/Sub publisher	Streaming ingest SDK
Stream processing	Eventstream operators, KQL	Kinesis Data Analytics / Flink	Dataflow (Beam)	Streams + tasks / dynamic tables
Analytical sink	KQL database (Eventhouse)	Redshift / Athena over S3	BigQuery streaming	Snowflake tables
Lake sink	Lakehouse Delta tables	S3 via Firehose	Cloud Storage	Managed staging
Delivery semantics	At-least-once	At-least-once (Kinesis)	At-least-once (default)	Exactly-once ingest option

Databricks ingests streams with Structured Streaming, Auto Loader, and Delta Live Tables; **MongoDB** produces streams through change streams. The portable disciplines: unique event IDs at the producer, dedup at the first queryable boundary, and knowing your source’s replay window.

G.21 Time-Series and Log Analytics Engines

Chapter 14’s Eventhouse is the Azure Data Explorer (Kusto) lineage inside the SaaS workspace; columnar, append-optimized telemetry engines exist across the ecosystem.

Capability	Fabric	AWS	GCP	Snowflake
Telemetry/log engine	Eventhouse (KQL database)	CloudWatch Logs Insights / OpenSearch / Timestream	Cloud Logging / BigQuery streaming analytics	Snowflake tables + time-series functions
Query language	KQL	Logs Insights QL / SQL	SQL / Logging query language	SQL
Time-series functions	make-series , series_* family	Timestream SQL functions	BigQuery window/ML functions	Window functions, ASOF joins
Incremental state	Materialized views	Scheduled queries / rollups	BigQuery materialized views	Materialized views, dynamic tables
Dedup idiom	arg_max by id	Window dedup	ROW_NUMBER dedup	ROW_NUMBER / MERGE
Retention control	Retention policies per table	Log group retention / TTL	Log bucket retention / table TTL	Time Travel + data retention

Databricks approaches telemetry with Structured Streaming plus Delta; **MongoDB** with native time-series collections. The portable skills: think in *append-only streams plus incremental state*, bound your lookback windows, and treat late arrivals as a policy decision, not a surprise.

G.22 Data-Driven Alerting

Chapter 15’s continuous condition evaluation over live data exists across clouds, though Fabric’s no-code Reflex is unusually approachable.

Capability	Fabric	AWS	GCP	Snowflake
Data-driven alerts	Data Activator (Reflex)	CloudWatch Alarms + SNS	Cloud Monitoring alerts	Snowflake Alerts
Stream-native trigger	Reflex over Eventstream/KQL	Kinesis Analytics + Lambda	Dataflow + Eventarc	Streams + tasks + notifications
BI-visual trigger	Reflex from Power BI visuals	QuickSight threshold alerts	Looker alerts/schedules	Via BI tool
Actions	Teams, email, Power Automate	SNS targets, Lambda	Pub/Sub, Cloud Functions, webhooks	Email, webhook, stored proc
Evaluation latency	Seconds to minutes	Alarm period granularity	Policy-based	Scheduled evaluation

Databricks alerts evaluate SQL on schedules; **MongoDB** Atlas triggers fire on change streams. The portable disciplines: *sustained conditions over spikes, actions with owners, and alert noise treated as a reliability bug.*

G.23 Live Dashboards and Reverse ETL

Chapter 16 closes the loop twice: data to humans (live dashboards) and data back to systems (reverse ETL).

Capability	Fabric	AWS	GCP	Snowflake
Live dashboards	Real-Time Dashboards (KQL)	QuickSight / Grafana on Timestream	Looker / Grafana on BigQuery streaming	Snowsight / connected BI
Auto-refresh BI	Power BI automatic page refresh	QuickSight refresh	Looker persistent derived tables	BI tool refresh
Reverse ETL target	Dataverse / CRM via pipelines	AppFlow to SaaS	Workflows / AppSheet writes	Snowflake reverse-ETL partners
Upsert semantics	Alternate-key upsert to Dataverse	API-level upsert per target	API-level per target	MERGE / external writes
Sync cost driver	CUs + target API throttling	Lambda/AppFlow invocations	Function invocations + API quotas	Warehouse credits + API quotas

Databricks serves live BI from SQL warehouses and AI/BI dashboards; **MongoDB** has Charts and Atlas-native dashboards. The transferable insight: both loops are billed—dashboards at refresh cadence, reverse ETL per API call.

G.24 Data Science Environments

Chapter 17’s notebook-centered data science over lake storage is the dominant pattern across platforms; the differences are in how tightly tracking, registry, and storage integrate.

Capability	Fabric	AWS	GCP	Snowflake
Notebooks	Fabric notebooks (Spark)	SageMaker Studio / EMR notebooks	Vertex AI Workbench / Colab	Snowflake Notebooks
Visual prep	Data Wrangler	Data Wrangler (SageMaker)	Vertex / Colab tooling	Snowsight / worksheets
Compute for EDA	Starter pools + custom pools	SageMaker instances / EMR	Vertex runtimes / Dataproc	Snowpark warehouses
Feature storage	Lakehouse Delta tables	Feature Store (SageMaker)	Vertex AI Feature Store	Feature Store / tables
Experiment tracking	Built-in MLflow	SageMaker Experiments / MLflow	Vertex AI Experiments	MLflow / Snowpark ML

Databricks is the closest relative—Fabric’s data-science experience deliberately mirrors its

notebook-first, MLflow-tracked shape, including a Feature Store and AutoML. **MongoDB** feeds these platforms through connectors rather than hosting ML. The portable discipline: *explore on samples, engineer features without leakage, store features as governed tables*.

G.25 Experiment Tracking and Model Registries

Chapter 18’s two pieces of ML infrastructure—tracking and registry—converged on MLflow as the industry’s common denominator: native in Fabric and Databricks, deployable everywhere else.

Capability	Fabric	AWS	GCP	Snowflake
Experiment tracking	Built-in MLflow	SageMaker Experiments / MLflow on EC2	Vertex AI Experiments	MLflow (external or integrated)
Model registry	Fabric model registry (MLflow)	SageMaker Model Registry	Vertex AI Model Registry	Snowflake Model Registry
Autologging	<code>mlflow.autolog()</code>	Same library	Same library	Same library
Training compute	Fabric Spark pools	SageMaker Training / EMR	Vertex Training / Dataproc	Snowpark ML / warehouses
Artifact store	OneLake (via MLflow)	S3	Cloud Storage	Stages / tables

Databricks’ managed MLflow—with Unity-Catalog-backed registry—is the direct ancestor of Fabric’s integration. **MongoDB** does not host training; it supplies features. The portable claim: *if it is not tracked, it did not happen; if it is not registered, it cannot be governed; if it cannot be rolled back, it is not production*.

G.26 AI Enrichment at Scale

Chapter 19’s managed-AI calls from distributed pipelines have the same shape everywhere: batch the calls, vault the keys, land the results in the analytical store.

Capability	Fabric	AWS	GCP	Snowflake
Distributed ML library	SynapseML on Spark	SageMaker SDK / EMR MLlib	Vertex AI / Dataproc MLlib	Snowpark ML
Managed LLM calls	Azure OpenAI via SynapseML	Bedrock from Lambda/EMR	Vertex AI Gemini APIs	Cortex / external functions
Cognitive AI services	Azure AI Services	Comprehend, Rekognition, Textract	Vision, Natural Language, Document AI	Cortex AI functions
Secret management	Key Vault / workspace identity	Secrets Manager	Secret Manager	External access integrations
AI-assisted coding	Copilot in Fabric	CodeWhisperer/Q Developer	Gemini Code Assist	Copilot integrations

Databricks runs the same SynapseML library—it originated in the Synapse ecosystem—and offers Mosaic AI for LLM and agent work; **MongoDB** Atlas Vector Search occupies the adjacent retrieval niche (as does Mosaic AI Vector Search). The portable rules: *batch instead of row-by-row, vault instead of hardcoded, land enrichment in Gold*.

G.27 The MLOps Loop

Chapter 20’s train–register–score–monitor–retrain loop exists on every platform; what varies is how much of it the platform holds natively.

Capability	Fabric	AWS	GCP	Snowflake
Batch scoring	Spark UDF / PREDICT over Delta	SageMaker Batch Transform	Vertex batch prediction	Snowpark ML / Cortex
Orchestrated inference	Pipelines invoking notebooks	Step Functions + SageMaker	Vertex Pipelines	Tasks
Drift monitoring	Notebook/pipeline distribution checks	SageMaker Model Monitor	Vertex Model Monitoring	Snowflake model monitors
Retrain trigger	Pipeline on drift signal	EventBridge + pipeline	Eventarc + pipeline	Tasks + alerts
Model source of truth	MLflow registry (Ch. 18)	SageMaker registry	Vertex registry	Snowflake registry
Scoring audit	model_version on every row	Job metadata	Prediction metadata	Versioned functions

Databricks Mosaic ML and MLflow cover the same loop natively, adding Lakehouse Monitoring for drift; **MongoDB** operationalizes predictions as documents written back by these pipelines. The portable laws: *score with the same features you trained on, stamp every prediction with its model version, monitor distributions not just errors.*

G.28 Layered Access Control

Chapter 21’s three layers—platform roles, resource permissions, in-engine row/column security—are structurally identical across clouds; only the names change.

Layer	Fabric	AWS	GCP	Snowflake
Platform container roles	Workspace roles (Admin/Member/Contributor/Viewer)	IAM policies on resources	IAM roles on projects/folders	Account/database roles
Item-level grants	Per-item sharing and permissions	Per-resource policies	Per-resource IAM	Object grants
Data-plane scoping	OneLake data access roles	Lake Formation	BigQuery authorized views/Dataplex	Row/column policies
Row-level security	SQL RLS predicates	Lake Formation cell filters	Row access policies	Row access policies
Identity Audit	Microsoft Entra ID Workspace/tenant logs, Purview	IAM / Identity Center CloudTrail	Cloud Identity Cloud Audit Logs	IdP via SCIM/OAuth Access history

Databricks governs through Unity Catalog’s three-level namespace (catalog/schema/table) with row filters and column masks; **MongoDB** Atlas through database roles and client-side field-level encryption. The portable law: *grant at the narrowest scope that lets the work happen, and audit the drift between policy and reality.*

G.29 Metadata Governance and Erasure

Chapter 22’s catalog, classification, lineage, and compliance workflows form a distinct platform layer in every cloud, increasingly unified across the estate.

Capability	Fabric	AWS	GCP	Snowflake
Catalog/governance	Microsoft Purview	DataZone / Glue Catalog + Macie	Dataplex	Horizon Catalog
Sensitivity labels	Microsoft Information Protection labels on items	Macie classifications	Sensitive Data Protection / policy tags	Object tagging, masking policies
Lineage	Fabric lineage view + Purview	Glue/DataZone lineage	Dataplex lineage	Access history / lineage
Endorsement	Promoted / Certified	Business glossary approvals	Data product annotations	Listing certification
Erasure mechanics	Crypto-shredding, Delta VACUUM coordination	S3 object deletion + Glacier waits	TTLs, crypto-shredding (CMEK)	Time Travel/Fail-safe windows
Legal hold	Purview eDiscovery/holds	Legal holds (S3 Object Lock)	Vault holds	Retention + legal processes

Databricks governs through Unity Catalog (with Purview or Unity lineage integrations); **MongoDB** through Atlas auditing and field-level encryption. The hard, portable lesson: *erasure is a propagation problem across caches, history, and backups—row deletes alone never satisfy it.*

G.30 CI/CD, Capacity, and Disaster Recovery

Chapter 23’s infrastructure-as-code discipline, staged promotion, and capacity observability are universal; Fabric’s twist is that workspace items are SaaS objects whose source of truth must be externalized to Git.

Capability	Fabric	AWS	GCP	Snowflake
Source control	Workspace integration (ADO/GitHub)	Git	Terraform/Deployment Manager in Git	Git integration / Terraform provider
Staged promotion	Deployment pipelines (Dev/Test/Prod)	Multi-account pipelines	Multi-project pipelines	Multi-database/account promotion
Capacity observability	Capacity Metrics app	CloudWatch	Cloud Monitoring	Account usage views
Throttling model	CU smoothing + rejection	Per-service quotas	Quotas + rate limits	Warehouse queuing/credits
DR for definitions	Git rehydration	IaC redeploy	IaC redeploy	IaC / replication
DR for data	Delta time travel; no auto cross-region	Cross-region replication	Multi-region buckets/datasets	Database replication/failover

Databricks externalizes to Git through Repos and Databricks Asset Bundles (with Terraform for workspace infrastructure); **MongoDB** Atlas through Terraform and API-driven automation. The portable disciplines: *definitions live in Git, data recovery is a separate design, deploy order is a correctness property.*

G.31 Certification Landscape

Chapter 24’s study discipline transfers even though the services differ: every platform in the series has a data-engineering certification.

Element	Fabric	AWS	GCP	Snowflake
Data-engineering cert	DP-600 (Fabric Analytics Engineer)	Data Engineer Associate (DEA-C01)	Professional Data Engineer	SnowPro Advanced: Data Engineer
Adjacent cert	DP-203 lineage / PL-300 (BI)	Data Analytics specialty (paths vary)	ML Engineer	SnowPro Core first
Exam style	Scenario MCQ + case studies	Scenario MCQ	Scenario MCQ	Scenario MCQ
Heaviest emphasis	Implement/model/serve analytics	Ingestion, storage, processing	Pipelines, storage, ML touchpoints	Data movement, performance
Hands-on value	High—labs mirror exam scenarios	High	High	High

Databricks certifies through the Data Engineer Associate and Professional exams; **MongoDB** through Developer and DBA tracks. The portable meta-skill: *study to the skills outline, measure with mocks, remediate by building—not by rereading.*

G.32 The Databricks Column: A Consolidated Map

Because Fabric’s Spark, Delta, and MLflow lineage runs directly through the Databricks ecosystem, the two platforms deserve their own consolidated mapping—the densest one-to-one correspondence in this appendix.

Domain	Microsoft Fabric	Databricks
Lakehouse storage	OneLake Delta-Parquet tables	Delta Lake on cloud object storage, Unity-Catalog-managed
Compute	Fabric Spark starter/custom pools on F-SKU capacity	Classic clusters, serverless compute; billed in DBUs
Notebooks	Fabric Notebooks with lakehouse attachment	Databricks Notebooks with cluster/serverless attachment
SQL warehousing	Synapse Data Warehouse; SQL analytics endpoint	Databricks SQL warehouses
Declarative pipelines	Dataflows Gen2; pipeline activities	Delta Live Tables (Lakeflow declarative pipelines)
Orchestration	Data Factory pipelines (master/child)	Databricks Workflows (Lakeflow jobs)
Streaming	Eventstreams + Eventhouse (KQL)	Structured Streaming + Auto Loader + Delta Live Tables
Experiment tracking	Built-in MLflow (experiments, models)	Managed MLflow with Unity Catalog registry
AI services	SynapseML + Azure OpenAI	Mosaic AI; SynapseML also runs on Databricks
Governance catalog	Microsoft Purview + workspace/item permissions	Unity Catalog (catalog/schema/table)
Source control / CI	Workspace Git integration + deployment pipelines	Repos + Databricks Asset Bundles + Terraform
BI serving	Power BI with Direct Lake	AI/BI dashboards and Genie; partner BI via SQL warehouses
Certification	DP-600 (Fabric Analytics Engineer)	Data Engineer Associate / Professional

Tip

Use this appendix as a study aid for the book’s lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other six would express the same idea. That habit turns seven separate vocabularies into one mental model.

Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

Access tier A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

AlloyDB Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

Apache Spark A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

Aurora Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

Availability SLA The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

Bigtable Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

Bucket (storage) The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

Bucket Pattern A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

Capped collection A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

Change Feed / Streams An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

Checkpoint In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

Clustering key A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

CMEK / CMK Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

Cloud Spanner Google Cloud’s horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

- Consistency level** A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).
- Cosmos DB** Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).
- Cross-Region Replication (CRR)** Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure’s equivalent user-configured feature is *object replication*.
- DynamoDB** Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.
- Durability** The probability that stored data is not lost, commonly quoted as “eleven nines” (99.999999999%) for cloud object storage.
- ElastiCache** Amazon’s managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure’s analog is Cache for Redis and GCP’s is Memorystore.
- Embedding vs referencing** The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).
- Fail-safe** Snowflake’s non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.
- Global Secondary Index (GSI)** A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table’s partition key.
- Hierarchical namespace (HNS)** The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.
- Journal (WAL)** The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB’s WiredTiger engine.
- JSON Schema validation** MongoDB’s rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.
- Interleaved table** A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.
- Lakehouse** An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.
- Lifecycle policy** A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.
- Medallion architecture** A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.
- Materialized view** A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).
- Memorystore** Google Cloud’s managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.

- Micro-partition** Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.
- Multi-AZ** An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.
- Multi-cluster warehouse** A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.
- Object Lock** The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.
- OneLake** Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.
- Read replica** A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.
- Relational Migrator** MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.
- Request Unit (RU)** Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.
- Search Optimization Service (SOS)** A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.
- Shortcut (Fabric)** A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.
- Storage class** The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).
- SSE-KMS** Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.
- Time Travel** Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF / TIMESTAMP AS OF` capability.
- Transient table** A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.
- TrueTime** Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.
- TTL (Time to Live)** An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).
- Versioning** Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

Virtual warehouse Snowflake's unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

Wide-column store A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

WiredTiger MongoDB's default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

WORM Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

Z-ordering A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

Zero-copy clone Snowflake's instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

Bibliography

- [1] Apache Software Foundation. Apache kafka documentation. <https://kafka.apache.org/documentation/>, 2026. Apache Kafka documentation.
- [2] Apache Software Foundation. Apache parquet documentation. <https://parquet.apache.org/docs/>, 2026. Open file format documentation.
- [3] Apache Software Foundation. Apache spark overview. <https://spark.apache.org/docs/latest/>, 2026. Apache Spark documentation.
- [4] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Luszczak, et al. Delta lake: High-performance acid table storage over cloud object stores. In *Proceedings of the VLDB Endowment*, volume 13, pages 3411–3424, 2020.
- [5] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2021.
- [6] Databricks. What is the medallion lakehouse architecture? <https://www.databricks.com/glossary/medallion-architecture>, 2025. Architecture documentation.
- [7] Great Expectations Open Source Community. Great expectations: Always know what to expect from your data. https://github.com/great-expectations/great_expectations, 2026. Open-source data quality framework.
- [8] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.
- [9] Linux Foundation Delta Lake Project. Delta lake documentation. <https://docs.delta.io/>, 2026. Open table format documentation.
- [10] Linux Foundation MLflow Project. Mlflow documentation. <https://mlflow.org/docs/latest/index.html>, 2026. Open-source ML lifecycle documentation.
- [11] Microsoft. Apache spark compute in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/spark-compute>, 2026. Microsoft Learn documentation.
- [12] Microsoft. Azure ai services in microsoft fabric. <https://learn.microsoft.com/fabric/data-science/ai-services/overview>, 2026. Microsoft Learn documentation.
- [13] Microsoft. Copilot in microsoft fabric overview. <https://learn.microsoft.com/fabric/get-started/copilot-fabric-overview>, 2026. Microsoft Learn documentation.
- [14] Microsoft. Copy into (transact-sql) in microsoft fabric. <https://learn.microsoft.com/sql/t-sql/statements/copy-into-transact-sql>, 2026. Microsoft Learn documentation.

- [15] Microsoft. Create a real-time dashboard in microsoft fabric. <https://learn.microsoft.com/fabric/real-time-intelligence/dashboard-real-time-create>, 2026. Microsoft Learn documentation.
- [16] Microsoft. Create, configure, and use an environment in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/create-and-use-environment>, 2026. Microsoft Learn documentation.
- [17] Microsoft. Cross-database queries in microsoft fabric. <https://learn.microsoft.com/fabric/data-warehouse/cross-database-queries>, 2026. Microsoft Learn documentation.
- [18] Microsoft. Data analysis expressions (dax) reference. <https://learn.microsoft.com/dax/>, 2026. Microsoft Learn documentation.
- [19] Microsoft. Data factory in microsoft fabric. <https://learn.microsoft.com/fabric/data-factory/>, 2026. Microsoft Learn documentation.
- [20] Microsoft. Data science in microsoft fabric. <https://learn.microsoft.com/fabric/data-science/>, 2026. Microsoft Learn documentation.
- [21] Microsoft. Dataflows gen2 overview. <https://learn.microsoft.com/fabric/data-factory/dataflows-gen2-overview>, 2026. Microsoft Learn documentation.
- [22] Microsoft. Delta lake table optimization and v-order in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/delta-optimization-and-v-order>, 2026. Microsoft Learn documentation.
- [23] Microsoft. Deployment pipelines in microsoft fabric. <https://learn.microsoft.com/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines>, 2026. Microsoft Learn documentation.
- [24] Microsoft. Direct lake in power bi and microsoft fabric. <https://learn.microsoft.com/fabric/get-started/direct-lake-overview>, 2026. Microsoft Learn documentation.
- [25] Microsoft. Domains in fabric. <https://learn.microsoft.com/fabric/governance/domains>, 2026. Microsoft Learn documentation.
- [26] Microsoft. Eventstreams in microsoft fabric real-time intelligence. <https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/overview>, 2026. Microsoft Learn documentation.
- [27] Microsoft. How to use microsoft fabric notebooks. <https://learn.microsoft.com/fabric/data-engineering/how-to-use-notebook>, 2026. Microsoft Learn documentation.
- [28] Microsoft. Introduction to git integration in microsoft fabric. <https://learn.microsoft.com/fabric/cicd/git-integration/intro-to-git-integration>, 2026. Microsoft Learn documentation.
- [29] Microsoft. Introduction to semantic models in power bi and microsoft fabric. <https://learn.microsoft.com/power-bi/connect-data/service-datasets-understand>, 2026. Microsoft Learn documentation.
- [30] Microsoft. Kusto query language (kql) overview. <https://learn.microsoft.com/azure/data-explorer/kusto/query/>, 2026. Microsoft Learn documentation.
- [31] Microsoft. Lineage in microsoft fabric. <https://learn.microsoft.com/fabric/governance/lineage>, 2026. Microsoft Learn documentation.

- [32] Microsoft. Materialized views in kusto. <https://learn.microsoft.com/azure/data-explorer/kusto/management/materialized-views/materialized-view-overview>, 2026. Microsoft Learn documentation.
- [33] Microsoft. Microsoft certified: Fabric analytics engineer associate (dp-600). <https://learn.microsoft.com/credentials/certifications/fabric-analytics-engineer-associate/>, 2026. Certification page and skills outline.
- [34] Microsoft. Microsoft fabric capacity metrics app. <https://learn.microsoft.com/fabric/enterprise/metrics-app>, 2026. Microsoft Learn documentation.
- [35] Microsoft. Microsoft fabric concepts and licenses. <https://learn.microsoft.com/fabric/enterprise/licenses>, 2026. Microsoft Learn documentation.
- [36] Microsoft. Microsoft fabric tenant settings. <https://learn.microsoft.com/fabric/admin/fabric-switch>, 2026. Microsoft Learn documentation.
- [37] Microsoft. Microsoft fabric trial capacity. <https://learn.microsoft.com/fabric/get-started/fabric-trial>, 2026. Microsoft Learn documentation.
- [38] Microsoft. Microsoft purview and microsoft fabric. <https://learn.microsoft.com/fabric/governance/microsoft-purview-fabric>, 2026. Microsoft Learn documentation.
- [39] Microsoft. Mlflow in microsoft fabric. <https://learn.microsoft.com/fabric/data-science/mlflow-overview>, 2026. Microsoft Learn documentation.
- [40] Microsoft. Onelake shortcuts. <https://learn.microsoft.com/fabric/onelake/onelake-shortcuts>, 2026. Microsoft Learn documentation.
- [41] Microsoft. Optimize delta lake tables in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/lakehouse-table-maintenance>, 2026. Microsoft Learn documentation.
- [42] Microsoft. Pipelines in microsoft fabric. <https://learn.microsoft.com/fabric/data-factory/pipeline-overview>, 2026. Microsoft Learn documentation.
- [43] Microsoft. Power query documentation. <https://learn.microsoft.com/power-query/>, 2026. Microsoft Learn documentation.
- [44] Microsoft. Security in microsoft fabric. <https://learn.microsoft.com/fabric/security/>, 2026. Microsoft Learn documentation.
- [45] Microsoft. Sensitivity labels in microsoft fabric. <https://learn.microsoft.com/fabric/governance/sensitivity-labels-overview>, 2026. Microsoft Learn documentation.
- [46] Microsoft. Sql analytics endpoint for a lakehouse in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/lakehouse-sql-analytics-endpoint>, 2026. Microsoft Learn documentation.
- [47] Microsoft. Synapseml: Simple and distributed machine learning. <https://github.com/microsoft/SynapseML>, 2026. Open-source distributed ML library.
- [48] Microsoft. T-sql surface area in microsoft fabric data warehouse. <https://learn.microsoft.com/fabric/data-warehouse/tsql-surface-area>, 2026. Microsoft Learn documentation.
- [49] Microsoft. Train and score machine learning models in microsoft fabric. <https://learn.microsoft.com/fabric/data-science/model-training-overview>, 2026. Microsoft Learn documentation.

- [50] Microsoft. Understand direct lake storage mode and fallback behavior. <https://learn.microsoft.com/fabric/get-started/direct-lake-overview>, 2026. Microsoft Learn documentation.
- [51] Microsoft. Use the monitoring hub in microsoft fabric. <https://learn.microsoft.com/fabric/admin/monitoring-hub>, 2026. Microsoft Learn documentation.
- [52] Microsoft. Vacuum delta tables in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/lakehouse-table-maintenance>, 2026. Microsoft Learn documentation.
- [53] Microsoft. What is a lakehouse in microsoft fabric? <https://learn.microsoft.com/fabric/data-engineering/lakehouse-overview>, 2026. Microsoft Learn documentation.
- [54] Microsoft. What is a virtual network (vnet) data gateway? <https://learn.microsoft.com/data-integration/vnet/overview>, 2026. Microsoft Learn documentation.
- [55] Microsoft. What is an eventhouse in microsoft fabric? <https://learn.microsoft.com/fabric/real-time-intelligence/eventhouse>, 2026. Microsoft Learn documentation.
- [56] Microsoft. What is an on-premises data gateway? <https://learn.microsoft.com/data-integration/gateway/>, 2026. Microsoft Learn documentation.
- [57] Microsoft. What is data activator (reflex)? <https://learn.microsoft.com/fabric/real-time-intelligence/data-activator/activator-introduction>, 2026. Microsoft Learn documentation.
- [58] Microsoft. What is data warehousing in microsoft fabric? <https://learn.microsoft.com/fabric/data-warehouse/data-warehousing>, 2026. Microsoft Learn documentation.
- [59] Microsoft. What is microsoft fabric? <https://learn.microsoft.com/fabric/get-started/microsoft-fabric-overview>, 2026. Microsoft Learn documentation.
- [60] Microsoft. What is onelake? <https://learn.microsoft.com/fabric/onelake/onelake-overview>, 2026. Microsoft Learn documentation.
- [61] Microsoft. Workspaces in microsoft fabric. <https://learn.microsoft.com/fabric/fundamentals/workspaces>, 2026. Microsoft Learn documentation.
- [62] OpenLineage Project. Openlineage: An open standard for data lineage. <https://github.com/OpenLineage/OpenLineage>, 2026. Open lineage metadata standard.

Index

- 100 GB daily processing, 59
- acceptance criteria, 279
- access control
 - equivalence, 312
- ACID, 21
- actions, 185
- ADLS Gen2, 22
- Admin portal, 5
- Admin role, 6
- AI enrichment
 - equivalence, 311
- alert noise, 185
 - capstone, 191
- alerting, 157, 159
 - equivalence, 309
 - logistics, 188
 - Reflex, 185
- aliases, 214
- Amazon S3, 22, 26
- anomaly detection, 176
 - hands-on, 179
- Apache Spark, 52
 - equivalence, 304
- API key management, 220, 222
- arg_max, 176
 - dedup, 178
- at-least-once delivery, 167
- automatic page refresh, 193, 195
- az CLI
 - essentials, 275
- Azure CLI, 272, 275
- Azure OpenAI, 220
 - integration, 222
- banking reconciliation, 160
- batch enrichment, 220
 - architecture, 224
- batch scoring, 228, 229
 - equivalence, 311
 - hands-on, 230
- BCDR, 255, 257
- BI serving
 - equivalence, 306
- Bigtable
 - equivalence, 305
- bridge table, 106, 111
- Bronze layer, 35, 47
 - CSV, 57
 - JSON, 63
 - raw, 37
- bursting, 8
- caching
 - equivalence, 304
- capacity, 19
 - trial, 1, 4
- capacity admin, 5
- capacity estimation, 90
- capacity management, 255, 256
 - equivalence, 313
- capacity metrics, 8
- Capacity Metrics app, 255, 256
- capacity unit, 19
- capacity units, 4, 16
- capstone
 - solutions, 279
- capstone project, 29, 47, 63, 93, 104, 114, 135, 145, 155, 174, 183, 191, 210, 218, 226, 244, 253, 262
 - Part I, 78
 - Part II, 124
 - Part III, 163
 - Part IV, 199
 - Part V, 234
 - Part VI, 269
- case studies, 13
- certification, 264
 - equivalence, 313
 - study guide, 289
- checklists, 284
- churn prediction, 234
- CI/CD
 - capstone, 269
 - equivalence, 313
- circular dependency, 111

- cloning
 - equivalence, 305
- Cognitive Services, 220, 222
- column pruning, 120
- column-level security, 99
- community resources, 13
- compensating actions, 159
- compute and storage separation, 84, 85
- conditions, 185
- conformed dimensions, 106, 107
- Contributor role, 6
- control flow, 128
- Copilot in Fabric, 220, 222
- Copy activity, 128, 129
- COPY INTO, 84, 86
 - capstone, 93, 124
 - five-million-row load, 88
- cost management, 8
- credentials
 - conventions, 273
- cross-cloud equivalence, 301
- cross-database querying, 84, 86
- crypto-shredding, 246, 248
 - capstone, 253
 - hands-on, 249
- CSV ingestion, 41
- curated reports, 43
- custom app endpoint, 169
- Custom pools, 52, 54
- Data Activator, 185
 - equivalence, 309
 - object model, 186
- data contracts, 128, 130
 - capstone, 135
- data curation, 35
- data drift, 228
 - detection, 231
- Data Factory, 128
- data leakage, 203, 205
 - capstone, 210
- data mesh, 246, 250
 - capstone, 269
- data modeling, 106
- data perimeter, 241
- data quality, 138, 140
 - capstone, 145
- data science
 - equivalence, 310
- Data Science experience, 203, 204
- data skipping, 71
- data virtualization, 16, 22, 101
- Data Warehouse, 35, 84
 - architecture, 85
 - dataflow destination, 140
 - Fabric, 38
- data warehouse
 - equivalence, 305
- Data Wrangler, 203
 - hands-on, 206
 - overview, 204
- Databricks
 - equivalence map, 314
- dataflow activity, 158
- Dataflows
 - checklist, 285
 - equivalence, 307
- Dataflows Gen2, 138
 - capstone, 145
 - evaluation, 142
 - warehouse, 87
 - web API, 141
- Dataverse, 22, 26, 193, 195
- DAX, 106, 109
 - hands-on, 109
 - quick reference, 278
- Delta Lake, 16, 21, 35, 68
 - compaction, 75
 - history, 72
 - internals, 70
 - optimization, 68, 71
 - partitioning, 63
 - quick reference, 277
 - remediation, 75
 - retention, 72
 - Spark, 52
 - transaction log, 68, 70
- Delta-Parquet, 20
- dependency management, 157, 159
- deploy ordering, 257
- deployment pipelines, 255, 257
 - capstone, 262
 - hands-on, 258
- dimensional modeling
 - equivalence, 306
- Direct Lake, 38, 117
 - benchmark, 120
 - capstone, 124
 - concept, 118
 - equivalence, 306
 - exam, 264
 - executive dashboard, 121
 - versus Import, 266

- DirectQuery, 117
 - versus Import, 266
- disaster recovery, 255
 - equivalence, 313
 - workspace items, 257
- distributed databases
 - equivalence, 303
- document modeling
 - equivalence, 303
- domain, 20, 40
- DP-600, 264
 - domain map, 289
 - equivalents, 313
 - exam mapping, 289
 - objectives, 265
 - objectives map, 290
 - profile, 289
 - self-assessment, 291
 - study strategy, 291
- dynamic expressions, 128, 130
- elastic compute
 - equivalence, 302
- endorsement, 246, 248
- enterprise CI/CD, 259
- environment promotion, 40
- Event Hubs, 167
 - source, 169
- event streaming
 - equivalence, 308
- event_id, 167
- Eventhouse, 176
 - architecture, 177
 - checklist, 285
- Eventstreams, 167
 - capstone, 174
 - destinations, 169
 - hands-on, 170
 - sources, 169
- exactly-once, 180
- exam domains, 264
- expand-contract, 255, 257
- experiment governance, 216
- experiments, 212
 - Fabric items, 213
- exploratory data analysis, 203, 205
- F SKU, 4, 19
- Fabric capacity, 16
- Fabric Notebooks, 52, 55
- Fabric portal, 7
- Fabric REST API, 276
- Fabric trial, 1, 272
 - setup, 272
 - starting, 4
- fact table, 106
 - types, 107
- faker, 273
- fallback, 117, 119
- feature engineering, 203, 205
 - capstone, 210
- financial institution, 35, 43
- financial medallion, 47
- ForEach, 128, 130
- forecasting
 - inventory, 207
- framing, 117, 119
- fraud detection, 199
- gateway
 - equivalence, 308
- gateway cluster, 148
- GDPR, 246
 - capstone, 253
 - checklist, 287
 - erasure path, 248
- GDPR erasure
 - equivalence, 312
- Get Metadata, 129
- Git
 - installation, 272
- Git integration, 255, 257
 - capstone, 262
 - hands-on, 258
- glossary, 315
- Gold layer, 35, 47
 - curated, 37
 - star schema, 78
- governance, 246
 - checklists, 284
 - equivalence, 312
 - federated, 250
- grain, 107
- hands-on lab, 23, 41, 57, 73, 88, 100, 109, 120, 131, 141, 151, 159, 170, 179, 187, 196, 206, 215, 222, 230, 240, 249, 258
- hybrid connectivity, 148
 - bank, 152
 - capstone, 155
 - equivalence, 308
- hyperparameter tuning, 212
 - collaborative, 216

- idempotent consumer, 167
 - capstone, 174
- idempotent loads, 87
 - capstone, 93
- idempotent rerun, 157
- identity resolution, 193, 195
- If Condition, 130
- Import mode, 117
- impossible travel, 199
- in-memory cache
 - equivalence, 304
- incremental refresh, 138, 140
- ingestion framework, 132
- ingestion patterns, 84, 87
- interview questions, 293
- item-level permissions, 238, 239
- JSON logs, 59
- JSON transaction log, 70
- Kimball dimensional modeling, 106
- KQL, 176
 - basics, 178
 - capstone, 183
 - equivalence, 309
 - quick reference, 277
 - reference, 275
- KQL database, 177
 - destination, 169
- KQL queryset, 193, 194
- Kusto Query Language, 176
- lab solutions, 279
- label inheritance, 248
- Lakehouse, 16, 35
 - dataflow destination, 140
 - destination, 169
 - SQL endpoint, 96
 - versus warehouse, 38
- lakehouse
 - checklist, 284
 - creation, 1, 23
 - organization, 40
 - SQL equivalence, 306
- landing zone, 29
- late-arriving data, 176, 178
 - troubleshooting, 180
- least privilege, 238
 - capstone, 244
- legacy migration, 90
- legal hold, 246, 248
- lineage, 246, 248
- load manifest, 93
- logical data warehouse, 96, 101
 - capstone, 104
- Lookup activity, 128, 129
- low-code ETL
 - equivalence, 307
- M language, 139
- machine learning
 - checklist, 286
- make-series, 176, 178
- manufacturing dashboards, 197
- Mapping Dataflows, 142
- materialized views, 176, 178
 - capstone, 183
 - equivalence, 304
- medallion architecture, 16, 21, 35
 - capstone, 78
 - equivalence, 302
 - principles, 37
- Member role, 6
- metadata-driven ingestion, 128
 - capstone, 135, 163
 - framework, 132
 - hands-on, 131
- micro-batch, 193
- Microsoft 365 Developer Program, 3
- Microsoft Entra ID, 3
- Microsoft Fabric, 16
 - Apache Spark, 52
 - Data Warehouse, 84
 - Delta Lake, 68
 - getting started, 1
 - overview, 18
- Milestone 1 capstone, 78
- Milestone 2 capstone, 124
- Milestone 3 capstone, 163
- Milestone 4 capstone, 199
- Milestone 5 capstone, 234
- Milestone 6 capstone, 269
- MLflow, 212
 - capstone, 218
 - equivalence, 311
 - hands-on, 215
 - scoring, 229
 - tracking, 213
- mlflow.autolog, 212, 213
- MLOps, 228
 - capstone, 234
 - checklist, 286
 - equivalence, 311
- mobile games, 171

- mock exam, 264
 - first, 266
 - second, 267
- mock IoT producer, 170
- model inference, 228
 - automation, 230
- model registry, 212, 214
 - capstone, 218
 - equivalence, 311
- model versioning, 212, 214
- model_version, 228
- models
 - Fabric items, 213
- monitoring
 - equivalence, 308
- Monitoring Hub, 157, 159
- multi-cloud, 301
- multi-cloud architecture, 26
- multi-cloud retail, 29
- multi-team collaboration, 259
- NoSQL
 - equivalence, 303
- notebook
 - verification, 9
- notebook activity, 158
- notebooks, 7
 - equivalence, 310
- null handling, 57
- object storage
 - equivalence, 301
- object-level security, 96, 99
- On Failure, 157, 159
- on-premises data gateway, 148
 - capstone, 155
 - installation, 151
 - versus VNet, 149
- OneDrive for Data, 20
- OneLake, 16
 - checklist, 284
 - concepts, 20
 - Files, 55
 - medallion architecture, 35
 - paths, 41, 275
 - shortcuts, 22, 29
 - Spark access, 52
 - warehouse storage, 85
- OneLake data access roles, 238, 240
- OneLake File Explorer, 7
- OneLake shortcuts
 - capstone, 78
- OPTIMIZE, 68, 71
 - V-Order, 73
- Oracle, 148
 - gateway, 150
- orchestration, 157
 - equivalence, 307
 - hands-on, 159
- organizational account, 1
- output destinations, 138, 140
- pagination, 150
- Pandas, 203
 - versus PySpark, 205
- parameters, 128, 130
- Parquet, 21, 68, 70
- PII masking, 269
- pipeline
 - Copy activity, 87
 - master, 157
 - scoring, 230
- pipelines, 128
 - checklist, 285
 - equivalence, 307
- point-in-time recovery
 - equivalence, 305
- portfolio projects, 13
- Power Automate, 187
- Power BI, 1, 16, 117
 - checklist, 286
- Power BI Desktop, 7
- Power BI free license, 3
- Power Query, 138
- Power Query Online, 139
- PREDICT, 228, 229
- privilege creep, 238
- production readiness, 284
- Purview, 246
 - checklist, 287
 - equivalence, 312
- Purview hub, 247
- PySpark, 7, 23, 52
 - basics, 55
 - deduplication, 57
 - Delta writes, 41
 - EDA, 203
 - quick reference, 277
 - reference, 275
 - Spark SQL integration, 56
- quarantine, 138, 140
 - capstone, 163
- query acceleration

- equivalence, 304
- question bank, 293
- quick reference, 275
- Random Forest, 212
- raw logs, 43
- read-only endpoint, 97
- real-time
 - checklist, 285
- real-time BI, 121
- Real-Time Dashboards, 193
 - building, 194
 - hands-on, 196
 - versus querysets, 194
- real-time dashboards
 - equivalence, 310
- Real-Time Intelligence, 167
 - architecture, 168
- recommendation engine, 231
- Reflex, 185
 - actions, 187
 - capstone, 191
 - destination, 169
 - from Eventstream, 186
 - from Power BI, 186
 - hands-on, 187
- regulatory reporting, 43
- relational databases
 - equivalence, 302
- relationships, 109
- remediation, 267
- replay, 167
- REST API, 7, 275
- REST API ingestion, 148, 150
- retail analytics, 26
- retraining pipeline, 228, 231
- reverse ETL, 193, 195
 - equivalence, 310
 - hands-on, 196
- right to erasure, 246
- row-level security, 96, 99, 238, 240
 - capstone, 104
 - equivalence, 312
 - hands-on, 240
- run history, 159
- runbook, 157
- sample data, 272
 - NYC Taxi, 273
 - Olist, 273
- sample file upload, 9
- sampling, 205
- SAP, 148
 - gateway, 150
- SCD Type 2
 - capstone, 114
 - MERGE, 109
- schema enforcement, 130
- Schema Evolution, 68, 72
- Scikit-Learn, 212
- security, 47, 238
 - checklist, 287
 - network perimeter, 152
- self-healing orchestration, 160
- semantic model, 106
 - checklist, 286
 - custom, 109
 - default, 108
 - Direct Lake, 117
 - equivalence, 306
- sensitivity labels, 246, 248
- sentiment analysis, 220
 - at scale, 224
 - capstone, 226
 - hands-on, 222
- separation of duties, 238
 - capstone, 244
 - healthcare, 241
- series_decompose_anomalies, 178
- service mapping, 301
- setup, 272
- setup verification, 9
- shortcut, 16
- Silver Delta table, 29
- Silver layer, 35
 - Delta, 57, 63
 - optimization, 73
 - validated, 37
- skills outline, 265
- slowly changing dimensions, 106, 107
- small-file problem, 68, 75
- software as a service, 18
- Spark environments, 52, 54
- Spark Notebooks
 - capstone, 63
 - ingestion, 59
- Spark notebooks
 - versus dataflows, 142
- Spark pools, 52
 - Fabric, 54
- Spark SQL, 52, 56
- Spark-native pipelines, 207
- SQL analytics endpoint, 38, 96

- automatic, 97
- equivalence, 306
- SQL Server
 - gateway, 150
 - on-premises, 148
- SSMS, 96, 100, 272
- staging, 138, 140
- Star Schema, 78
- star schema, 106
 - capstone, 114, 124
- Starter pools, 52, 54
- storage classes
 - equivalence, 301
- stored procedures, 90
 - endpoint, 98
- stream joins, 176, 178
- streaming ingestion, 75, 167
 - equivalence, 308
- study strategy, 264
- sub-second latency, 197
- surge protection, 255
- Switch, 130
- Synapse Data Warehouse, 84
- SynapseML, 220
 - capstone, 226
 - equivalence, 311
 - overview, 221
- T-SQL, 84
 - quick reference, 276
 - reference, 275
 - surface area, 86
- table-valued functions, 96, 98
- Teams notifications, 185, 187
- telemetry ingestion, 59
 - capstone, 63
 - global, 171
- temporary views, 56
- tenant, 3
- tenant administration, 247
- tenant setting, 1
- tenant settings, 5
- throttling, 8, 255, 256
- time series, 176, 178
- Time Travel, 68, 72
 - accidental deletion, 73
- time travel
 - equivalence, 305
- time-series
 - equivalence, 309
- tools
 - installation, 272
 - training-serving skew, 205, 228, 229
 - transcoding, 118
 - trial expiry, 8
 - triggers, 185
 - troubleshooting
 - setup, 274
- Until, 130
- Users can create Fabric items, 5
- V-Order, 68, 71
 - capstone, 78
 - Direct Lake, 117, 120
- VACUUM, 68, 72
- variables, 130
- Viewer role, 6
- views, 96, 98
 - hands-on, 100
- virtual warehouse
 - equivalence, 302
- VNet data gateway, 148
- VS Code, 272
- warehouse
 - checklist, 285
- weak domains, 267
- webhooks, 148, 150
 - driver notifications, 188
- wide-column
 - equivalence, 305
- work or school account, 1
- workloads, 18
- workspace, 20
 - capacity assignment, 6
 - creation, 1, 23
 - medallion organization, 40
 - naming, 273
 - roles, 6
- workspace conventions, 272
- workspace isolation, 35, 43, 47
- workspace roles, 238, 239
- Z-Ordering, 68, 71